

Noms : Christine ANTON (21108059) & Elise NICOLAS (21103771)

TME 01 Estimation de densité

Imports

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import pickle
import pandas as pd
from typing import *
from sklearn.model_selection import train_test_split
```

Extraction des données

```
In [2]: POI_FILENAME = "../data/poi-paris.pkl"
parismap = mpimg.imread('../data/paris-48.806-2.23--48.916-2.48.jpg')

# Coordonnées GPS de la carte
xmin, xmax = 2.23, 2.48 # coord_x min et max
ymin, ymax = 48.806, 48.916 # coord_y min et max
coords = [xmin, xmax, ymin, ymax]
```

Fonctions Globales

```
In [3]: def get_density2D(f, data, steps=100) :
    """
    Calcule la densité en chaque case d'une grille steps x steps dont les bornes
    """
    xmin, xmax = data[:,0].min(), data[:,0].max()
    ymin, ymax = data[:,1].min(), data[:,1].max()
    xlin,ylin = np.linspace(xmin,xmax,steps), np.linspace(ymin,ymax,steps)
    xx, yy = np.meshgrid(xlin,ylin)
    grid = np.c_[xx.ravel(), yy.ravel()]
    res = f.predict(grid).reshape(steps, steps)
    return res, xlin, ylin

def show_density(f, data, steps=100, log=False) :
    """
    Dessine la densité f et ses courbes de niveau sur une grille 2D calculée à p
    """
    res, xlin, ylin = get_density2D(f, data, steps)
    xx, yy = np.meshgrid(xlin, ylin)
    plt.figure()
    show_img()
    if log:
```

```

        res = np.log(res+1e-10)
    plt.scatter(data[:, 0], data[:, 1], alpha=0.8, s=3)
    show_img(res)
    plt.colorbar()
    plt.contour(xx, yy, res, 20)

def show_img(img=parismap) :
    """
    Affiche une matrice ou une image selon les coordonnées de la carte de Paris.
    """
    origin = "lower" if len(img.shape) == 2 else "upper"
    alpha = 0.3 if len(img.shape) == 2 else 1.
    plt.imshow(img, extent=coords, aspect=1.5, origin=origin, alpha=alpha)
    ## extent pour controler l'échelle du plan

def load_poi(typepoi, fn=POI_FILENAME) :
    """
    Dictionnaire POI, clé : type de POI, valeur : dictionnaire des POIs de ce typ

    Liste des POIs : furniture_store, laundry, bakery, cafe, home_goods_store, c
    """
    poidata = pickle.load(open(fn, "rb"))
    data = np.array([[v[1][0][1], v[1][0][0]] for v in sorted(poidata[typepoi].i
    note = np.array([v[1][1] for v in sorted(poidata[typepoi].items())])
    return data, note

```

Affichage des données

```

In [4]: # Extraction des données
with open(POI_FILENAME, 'rb') as poi :
    data=pickle.load(poi)

plt.ion()

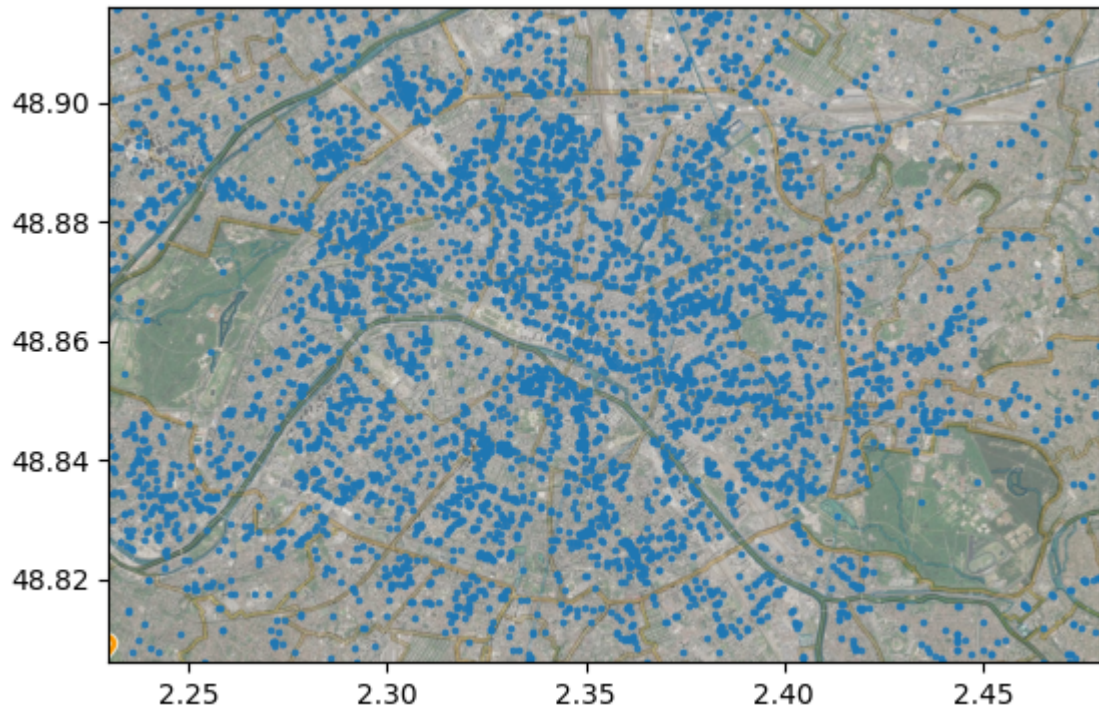
# Liste des POIs : furniture_store, laundry, bakery, cafe, home_goods_store, clo
# La fonction charge la localisation des POIs dans geo_mat et leur note.
geo_mat, notes = load_poi("bar")

# Affiche la carte de Paris
show_img()

# Affiche les POIs
plt.scatter(geo_mat[:,0],geo_mat[:,1],alpha=0.8,s=3)

```

Out[4]: <matplotlib.collections.PathCollection at 0x2435e6b7470>



Classe Density

```
In [5]: class Density(object):
def fit(self, data):
    """
    Apprend un estimateur sur les données passées en argument
    """
    pass

def predict(self, data):
    """
    Prédit la densité pour chaque point de données
    """
    pass

def score(self, data):
    """
    Retourne la log-vraisemblance des données

    Parameters :
        - self : Density object, instance de la classe Density
        - data : np.ndarray, coordonnées [x, y] de chaque point de données
    Returns :
        - LL : float, log-vraisemblance des données
    """
    L = self.predict(data)
    L = np.where(L == 0, 1e-10, L)
    LL = np.log(L).sum()
    return LL
```

Ajouter une très petite valeur (ici, 1e-10) aux densités nulles permet d'éviter le calcul de $\log(0)$, qui n'est pas défini. Cela permet donc de réaliser le calcul du score (log-vraisemblance des données data) de l'estimateur sans problème.

Méthode par histogramme

Classe Histogramme

```
In [6]: class Histogramme(Density):
    def __init__(self, steps=10):
        Density.__init__(self)
        self.steps = steps
        self.density = None
        self.edges = None

    def fit(self, x):
        """
        Apprend l'histogramme de densité sur x

        Parameters :
            - x : np.ndarray, coordonnées [x, y] de chaque point de données
        Returns :
            - d : np.ndarray de taille (steps,steps), densités de x
        """
        self.density, self.edges = np.histogramdd(x, bins=[self.steps]*x.shape[-1])
        return self.density

    def predict(self, x):
        """
        Retourne la densité estimée pour chaque point de x

        Parameters :
            - x : np.ndarray, coordonnées [x, y] de chaque point de données
        Returns :
            - pred : np.ndarray de taille (steps,steps), densités estimées de x
        """
        def to_bin(x):
            """
            Associe chaque point de x à un bin dans l'histogramme

            Parameters :
                - x : np.ndarray, coordonnées [x, y] de chaque point de données

            Returns :
                - indices : List[Tuple[int, int]], indices des bins associés aux points de x
            """
            indices = []

            # Extraction des bornes min et max à partir de self.edges
            min_bounds = [self.edges[i][0] for i in range(x.shape[1])]
            max_bounds = [self.edges[i][-1] for i in range(x.shape[1])]

            for pt in x :
                bin_idx = tuple(
                    max(0, min(int((pt[i] - min_bounds[i]) / (max_bounds[i] - min_bounds[i])),
                                (self.steps - 1)))
                        for i in range(x.shape[1]))
                )
                indices.append(bin_idx)

            return indices
```



```

pred = []
for coord_bin in to_bin(x) :
    pred.append(self.density[coord_bin])

return np.array(pred)

```

Tests Méthode par Histogramme

```

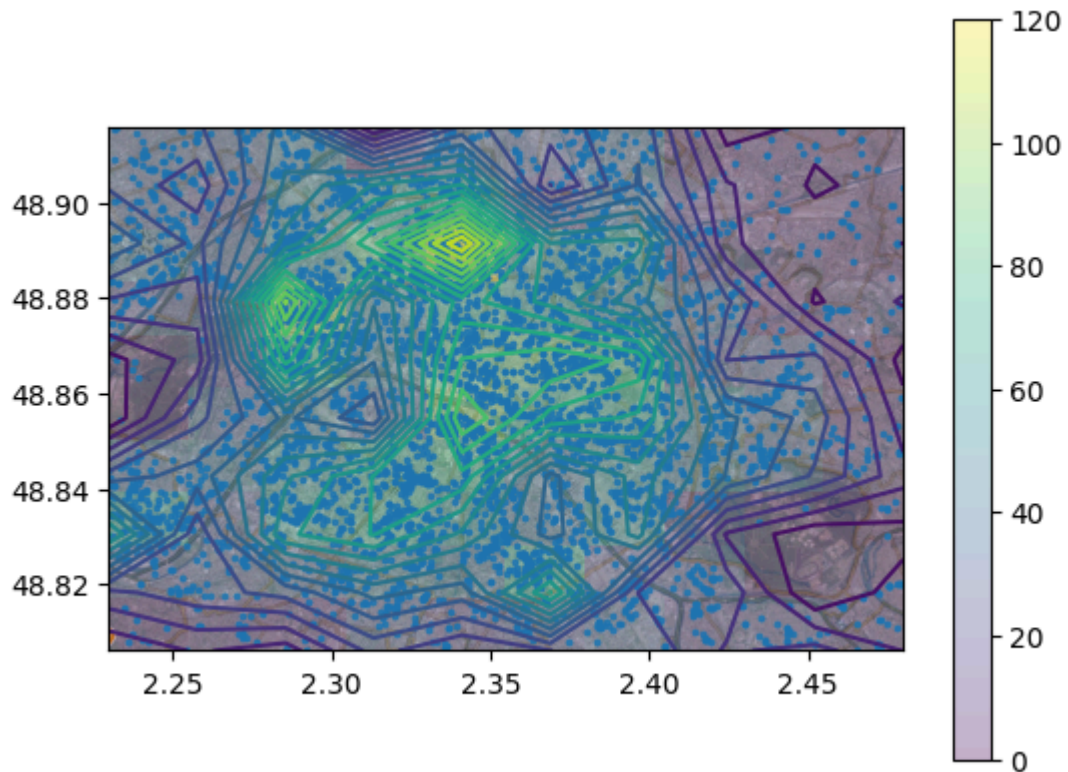
In [7]: # Estimation de La loi de densité géographique pour Le POI "bar" pour un pas de

coordonnees, notes = load_poi('bar')

H = Histogramme() # Par défaut, steps = 10
H.fit(coordonnees)
H.predict(coordonnees)
show_density(H, coordonnees, H.steps)
print(H.score(coordonnees))

```

18906.496305439192



```

In [8]: # Vérification Loi de densité avec get_density2D

# Estimation de La densité sur La grille
density_grid, xlin, ylin = get_density2D(H, coordonnees, steps=10)

# Vérification de La non-négarivité de La densité
assert np.all(density_grid >= 0), "La densité ne doit pas être négative."

# Calcul de L'intégrale de La densité sur La grille
dx = xlin[1] - xlin[0]
dy = ylin[1] - ylin[0]
sum_density = np.sum(density_grid) * dx * dy

print(f"Intégrale de la densité sur la grille : {sum_density:.4f}")

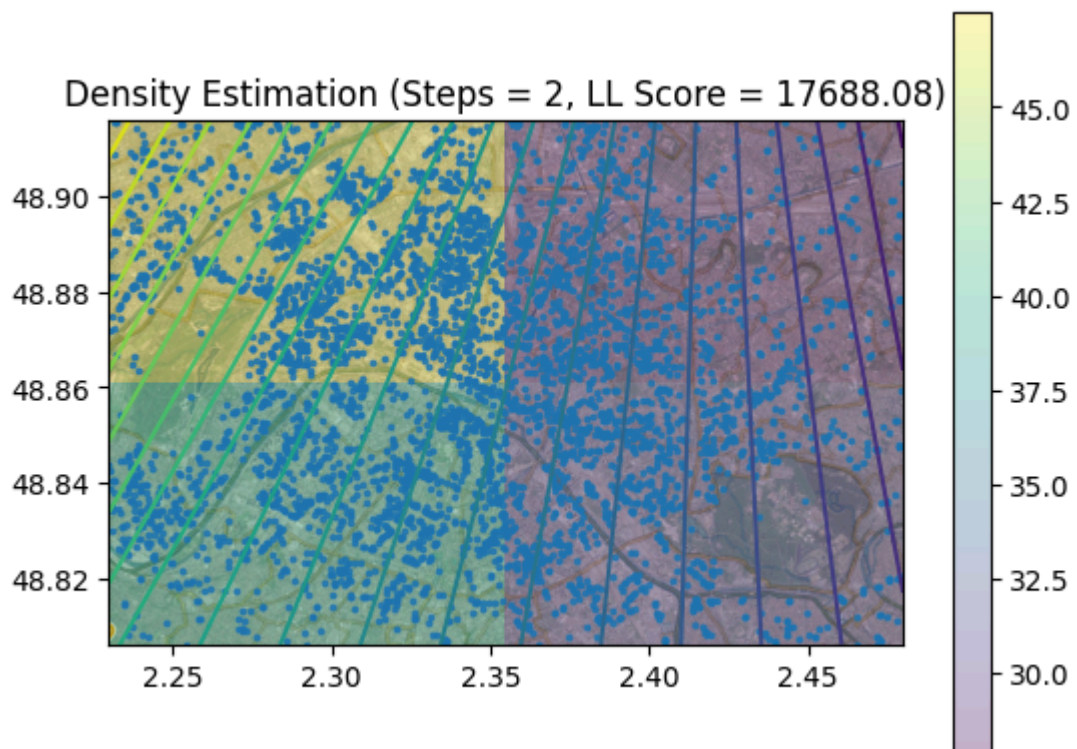
```

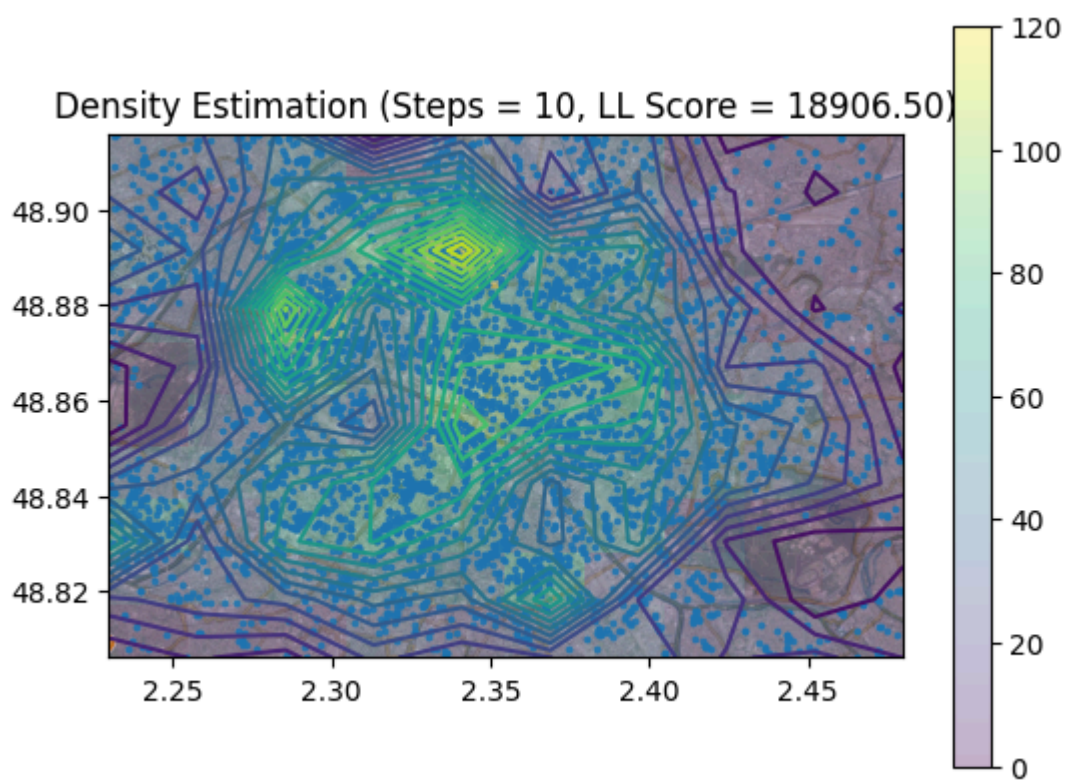
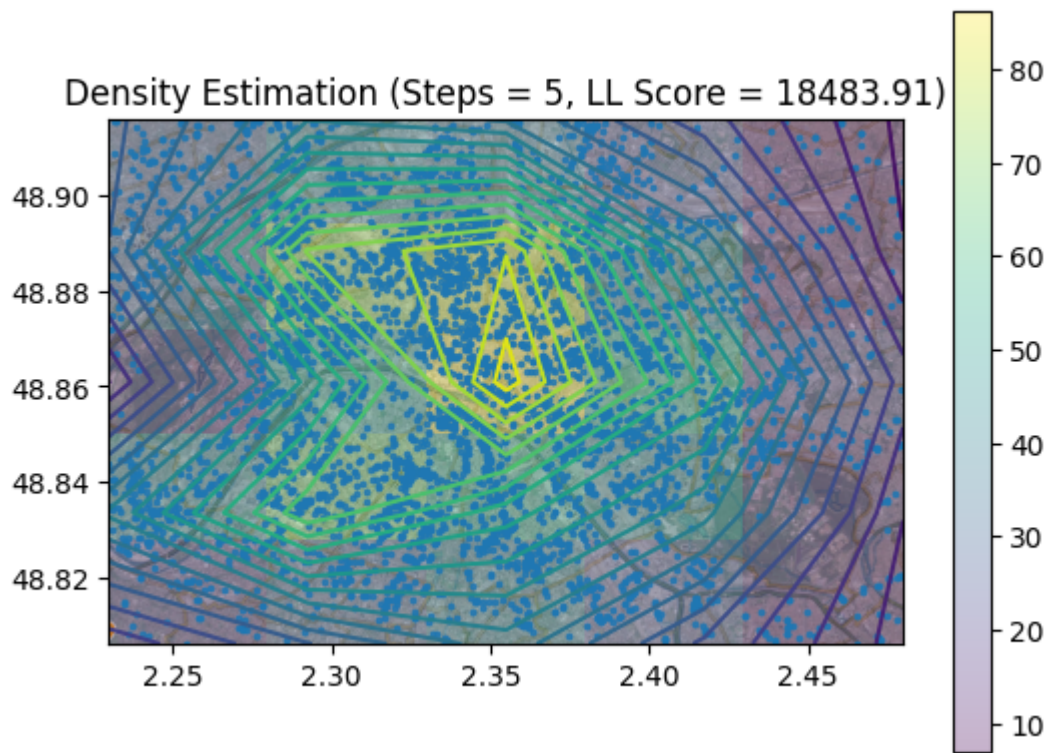
Intégrale de la densité sur la grille : 1.2346

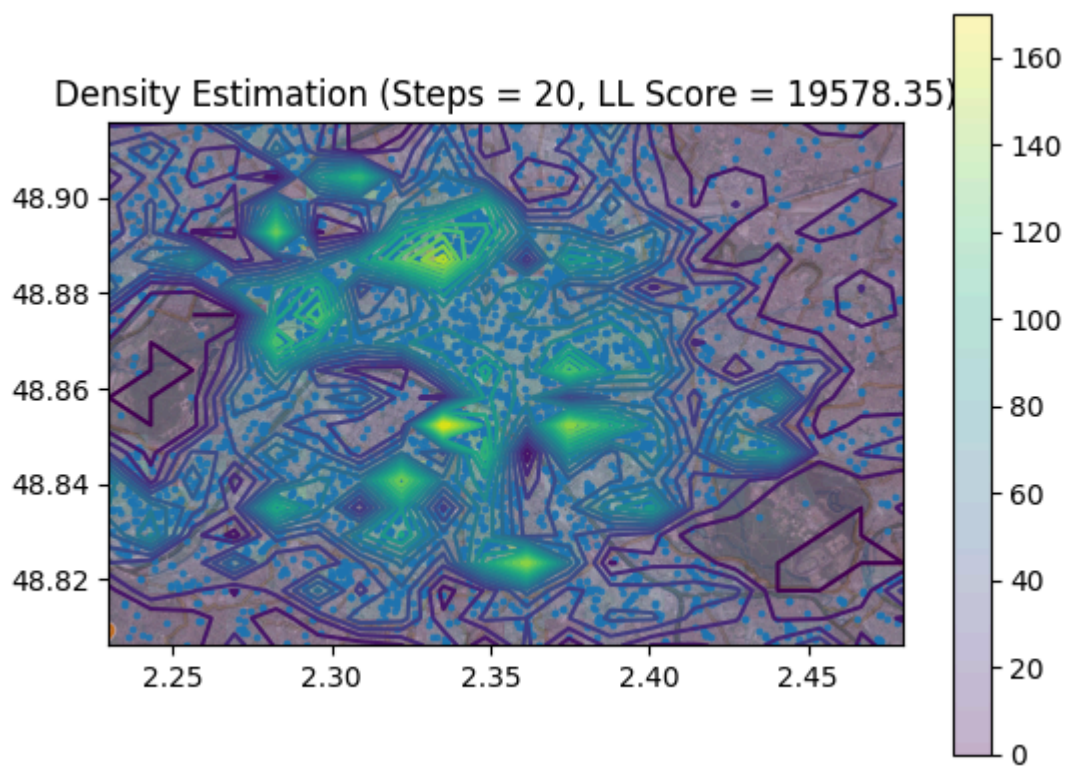
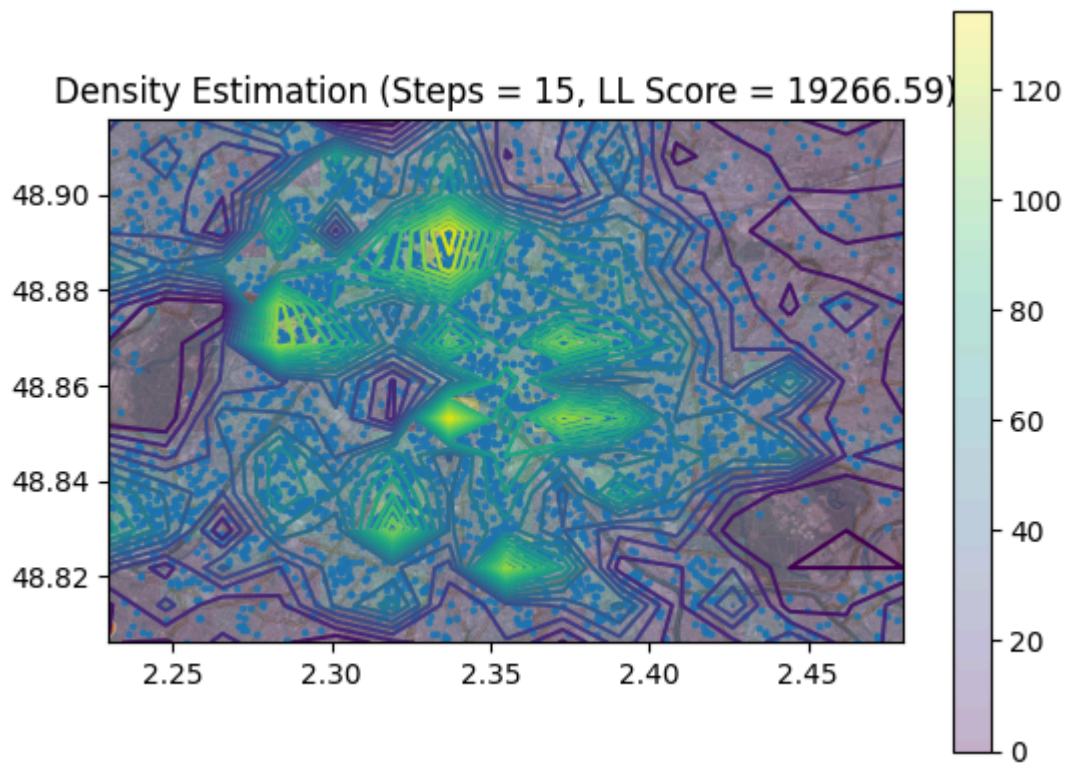
S'il n'y a pas d'erreur et si l'estimateur représente bien une loi de densité (non-négativité et intégrale égale à 1), l'intégrale de la densité estimée sur la grille doit être égale à 1. Ici, on observe une intégrale de 1.23. Ce léger dépassement peut être dû aux bords de la grille, mais étant donné que cela reste proche de 1, cela reste acceptable.

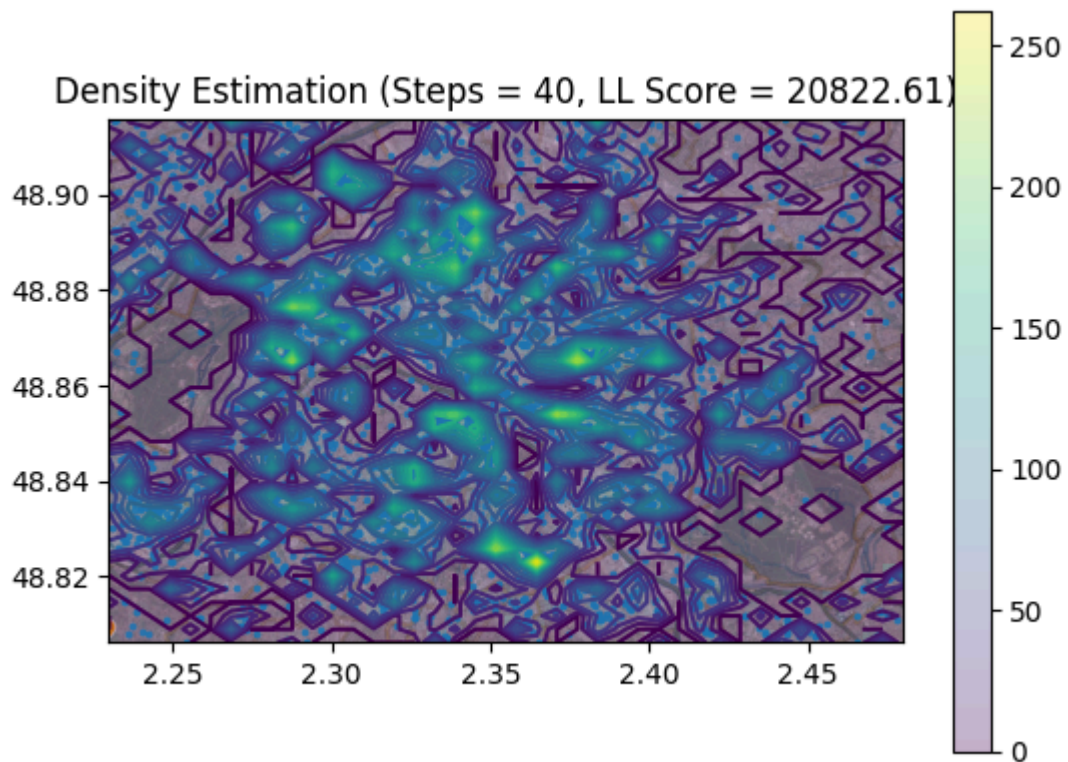
```
In [9]: # Estimation de la loi de densité géographique pour le POI "bar" pour différents
        coordonnees, notes = load_poi('bar')

        for steps in [2, 5, 10, 15, 20, 40]:
            H = Histogramme(steps)
            H.fit(coordonnees)
            H.predict(coordonnees)
            show_density(H, coordonnees, steps)
            plt.gca().set_title(f'Density Estimation (Steps = {steps}, LL Score = {H.sco
```









On observe que plus le nombre d'intervalles de discrétisation est élevé, plus la log-vraisemblance des données d'apprentissage augmente. Cela s'explique par le fait qu'un pas plus fin permet de mieux adapter l'histogramme aux données en capturant plus de détails. Cependant, cette amélioration peut refléter un sur-apprentissage : le modèle colle trop aux données d'apprentissage et perd sa capacité de généralisation. Il est donc important de choisir un pas ni trop petit, pour éviter une approximation grossière, ni trop grand, pour éviter le sur-apprentissage. Ici, on aurait tendance à dire qu'une discrétisation moyenne, autour de 10 à 15 pas, semble être la meilleure car les scores de log-vraisemblance sont relativement similaires pour ces valeurs.

```
In [42]: # Recherche du meilleur pas de discrétisation pour le POI "bar" (split en ensemb
coordonnees, notes = load_poi('bar')
train_data, test_data = train_test_split(coordonnees, test_size=0.3, random_stat
results = []

for steps in range(2, 40):
    H = Histogramme(steps)
    H.fit(train_data)
    ll_train = H.score(train_data)
    ll_test = H.score(test_data)
    results.append((steps, ll_train, ll_test))

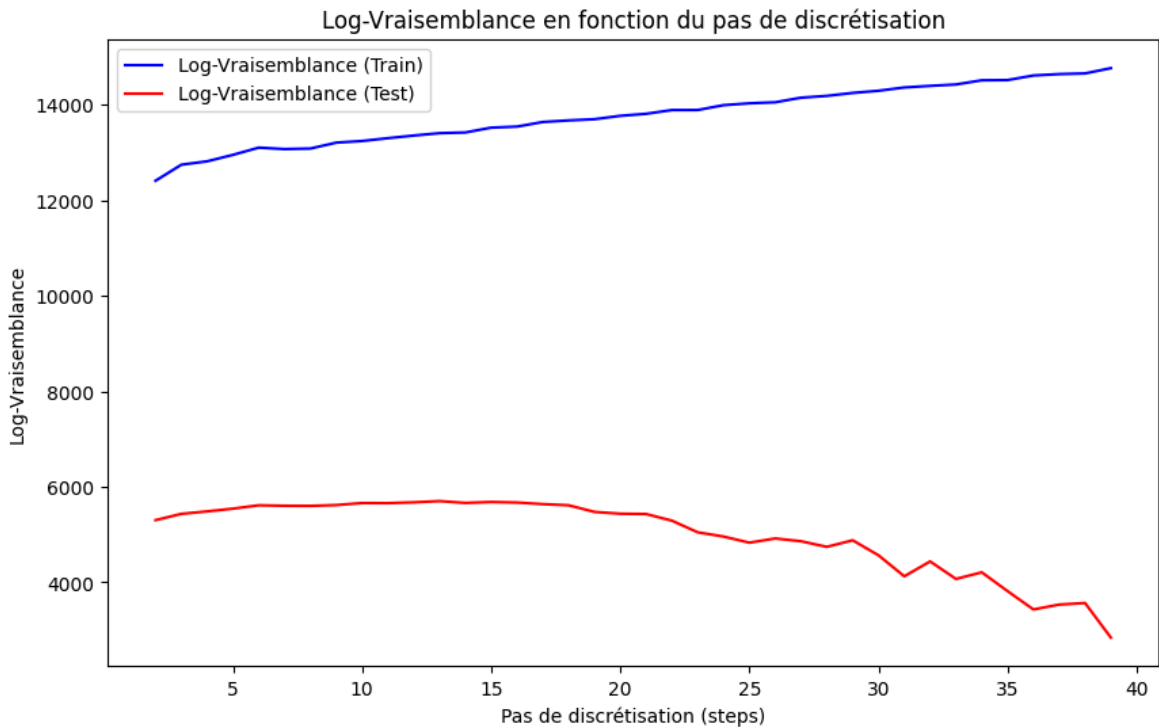
best = max(results, key=lambda x: x[2])
print(f"Meilleur pas de discrétisation : {best[0]} (LL Test = {best[2]:.2f})")

ll_train_values = [x[1] for x in results]
ll_test_values = [x[2] for x in results]

plt.figure(figsize=(10, 6))
plt.plot(range(2, 40), ll_train_values, label='Log-Vraisemblance (Train)', color
```

```
plt.plot(range(2, 40), ll_test_values, label='Log-Vraisemblance (Test)', color='red')
plt.xlabel('Pas de discrétisation (steps)')
plt.ylabel('Log-Vraisemblance')
plt.title('Log-Vraisemblance en fonction du pas de discrétisation')
plt.legend()
plt.show()
```

Meilleur pas de discrétisation : 13 (LL Test = 5700.33)



```
In [43]: # Recherche du meilleur pas de discrétisation pour le POI "night_club" (split en
coordonnees, notes = load_poi('night_club')
train_data, test_data = train_test_split(coordonnees, test_size=0.3, random_stat

results = []

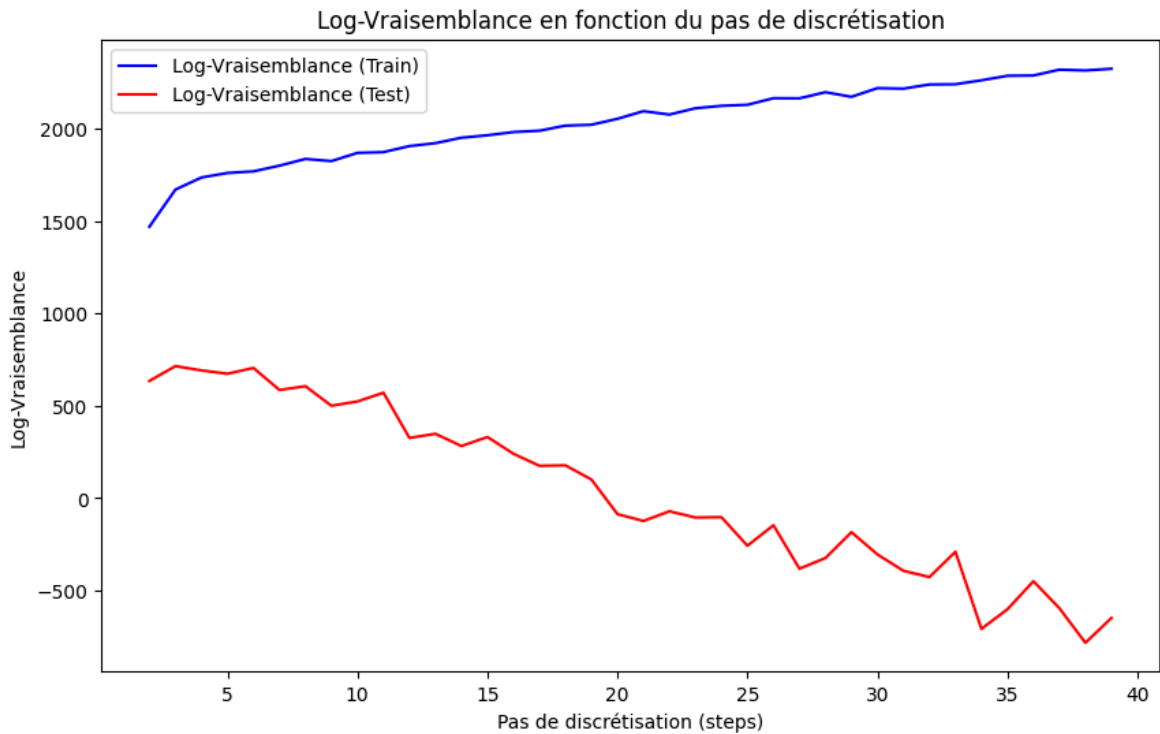
for steps in range(2, 40):
    H = Histogramme(steps)
    H.fit(train_data)
    ll_train = H.score(train_data)
    ll_test = H.score(test_data)
    results.append((steps, ll_train, ll_test))

best = max(results, key=lambda x: x[2])
print(f"Meilleur pas de discrétisation : {best[0]} (LL Test = {best[2]:.2f})")

ll_train_values = [x[1] for x in results]
ll_test_values = [x[2] for x in results]

plt.figure(figsize=(10, 6))
plt.plot(range(2, 40), ll_train_values, label='Log-Vraisemblance (Train)', color='blue')
plt.plot(range(2, 40), ll_test_values, label='Log-Vraisemblance (Test)', color='red')
plt.xlabel('Pas de discrétisation (steps)')
plt.ylabel('Log-Vraisemblance')
plt.title('Log-Vraisemblance en fonction du pas de discrétisation')
plt.legend()
plt.show()
```

Meilleur pas de discrétisation : 3 (LL Test = 714.72)



Pour le POI "bar", le meilleur pas de discrétisation est 13, c'est celui qui donne la log-vraisemblance maximale sur le test. Pour le POI "night_club" (POI plus rare), le meilleur pas de discrétisation est 3. Quand les données sont rares, un pas de discrétisation trop grand risque de créer des zones de densité nulle, ce qui fait chuter la log-vraisemblance sur le test. C'est donc logique que le meilleur pas de discrétisation pour des données rares soit plus petit que pour des données fréquentes.

Méthode à Noyaux

Implémentation des noyaux uniforme et gaussien

```
In [27]: def kernel_uniform(X):
    """
    Calcule le noyau uniforme pour chaque donnée de X

    Parameters :
        - X : np.ndarray, matrice de données

    Returns :
        - np.ndarray : 1 si x_i dans [-1/2, 1/2], 0 sinon
    """
    return np.all(np.abs(X) <= 0.5, axis=1).astype(int)
```

```
In [28]: def kernel_gaussian(X):
    """
    Calcule le noyau gaussien pour chaque donnée de X

    Parameters :
        - X : np.ndarray, matrice de données

    Returns :
        - np.ndarray : matrice contenant les valeurs du noyau pour chaque donnée
```



```

"""
return (2 * np.pi) ** (-X.shape[1] / 2) * np.exp(-0.5 * np.sum(X**2, axis=1))

```

```

In [29]: # Test des deux fonctions

X = np.array([[0.1, 0.2], [0.4, 0.5], [0.6, 0.7], [-0.3, 0.2], [0.5, -0.3], [-0.
print("Résultats du noyau uniforme : ", kernel_uniform(X))

print("Résultats du noyau gaussien : ", kernel_gaussian(X))

```

```

Résultats du noyau uniforme : [1 1 0 1 1 0]
Résultats du noyau gaussien : [0.15522539 0.12965515 0.10405069 0.14913892 0.134
27343 0.12271667]

```

Classe KernelDensity

```

In [33]: class KernelDensity(Density):
def __init__(self, kernel=None, sigma=0.1):
    Density.__init__(self)
    self.kernel = kernel
    self.sigma = sigma

def fit(self, x):
    """
    Apprend la densité à partir des données x en utilisant le noyau spécifié

    Parameters :
        - x : np.ndarray, coordonnées [x, y] de chaque point de données
    """
    self.x = x

def predict(self, data):
    """
    Retourne la densité estimée pour chaque point de données en utilisant le

    Parameters :
        - data : np.ndarray, coordonnées des points

    Returns :
        - pred : np.ndarray, densités estimées pour chaque point de données
    """
    pred = []
    for pt in data :
        density = self.kernel((np.vstack([pt]*self.x.shape[0]) - self.x)/sel
        pred.append(density/(self.x.shape[0]*self.sigma**self.x.shape[1]))
    return np.array(pred)

```

Tests Méthode à Noyaux

```

In [37]: # Vérification loi de densité

coordonnees, notes = load_poi('bar')
K = KernelDensity(kernel=kernel_gaussian)
K.fit(coordonnees)

pred = K.predict(coordonnees)

```

```

# Vérification de la non-négativité de la densité
assert np.all(pred >= 0), "La densité ne doit pas être négative."

xmin, ymin = K.x.min(axis=0) - 3 * K.sigma
xmax, ymax = K.x.max(axis=0) + 3 * K.sigma
grid_x, grid_y = np.mgrid[xmin:xmax:100j, ymin:ymax:100j]
grid = np.vstack([grid_x.ravel(), grid_y.ravel()]).T

# Prédiction de la densité sur la grille
pred = K.predict(grid)

# Estimation de l'intégrale (approx. par somme)
dx = (xmax - xmin) / 100
dy = (ymax - ymin) / 100
integral = np.sum(pred) * dx * dy

print(f"Approximation de l'intégrale : {integral:.4f}")

```

Approximation de l'intégrale : 0.9794

L'intégrale est proche de 1, il s'agit donc bien d'une densité.

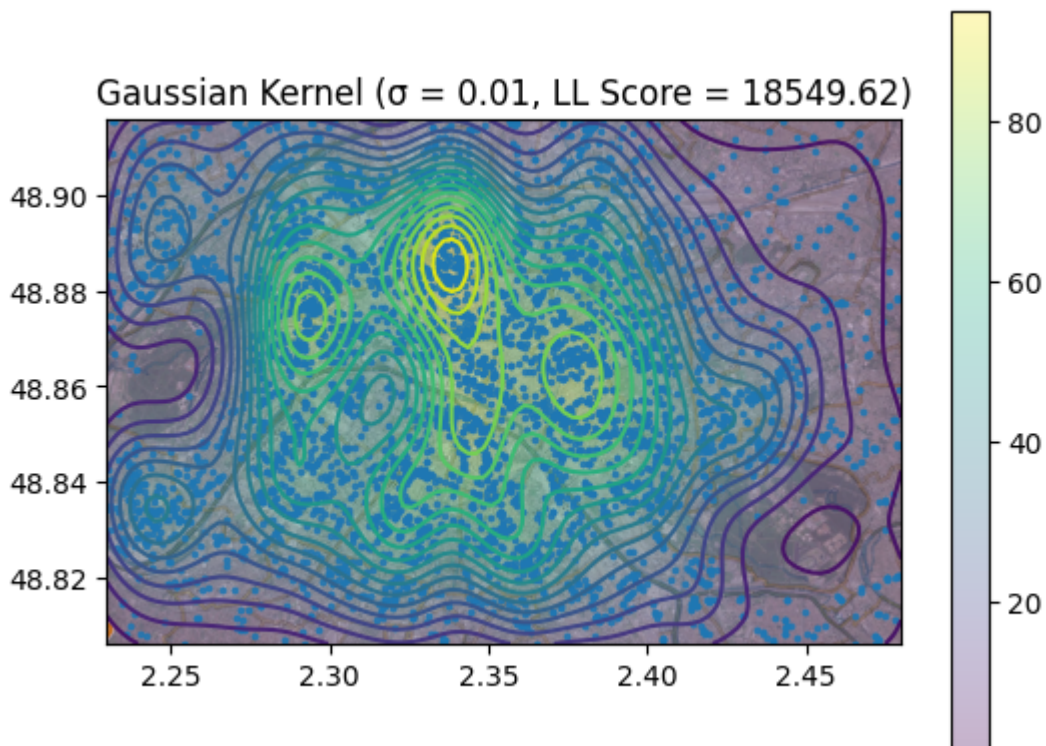
```

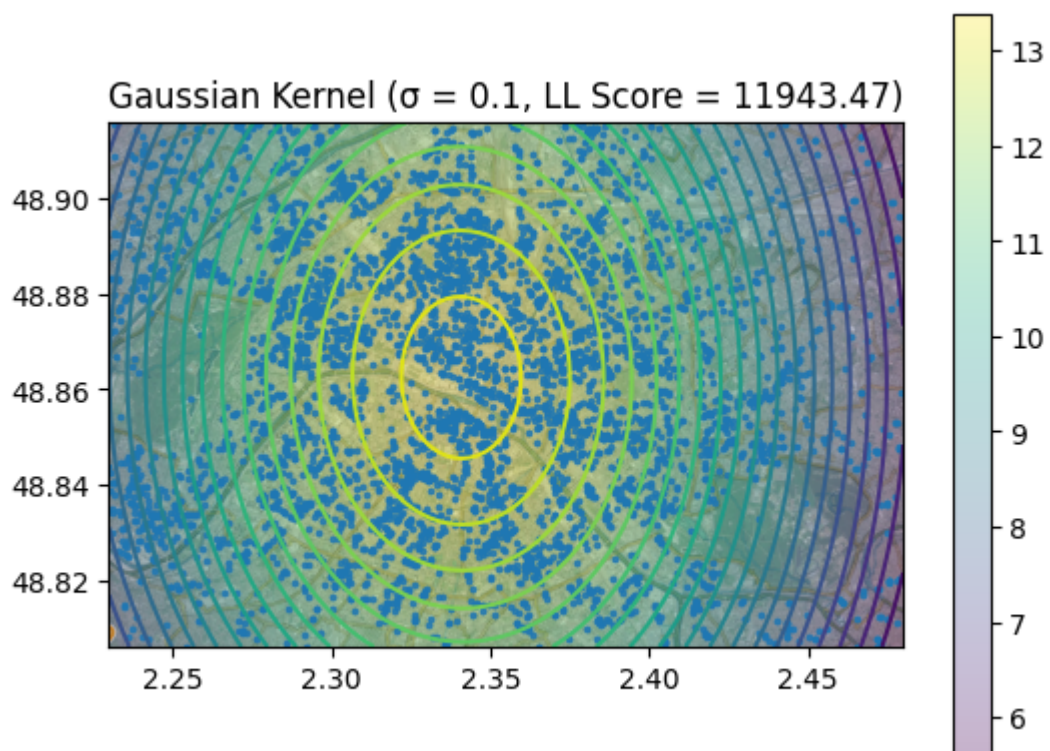
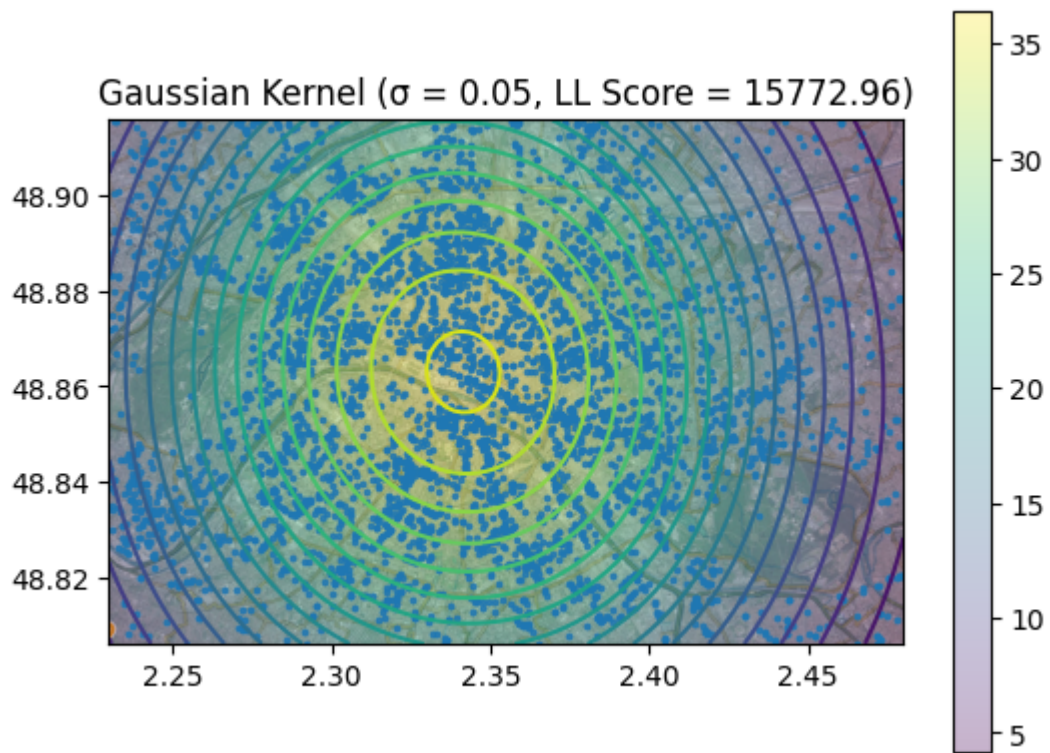
In [39]: # Estimation de la loi de densité géographique pour le POI "bar" pour différents
          coordonnees, notes = load_poi('bar')

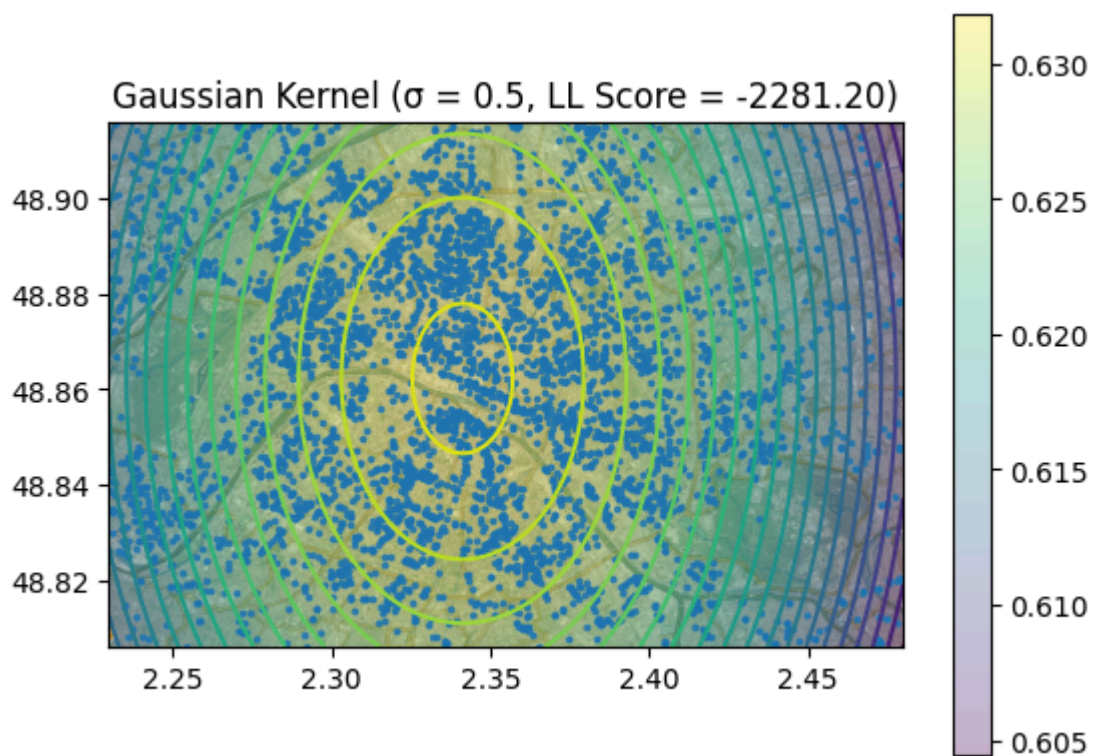
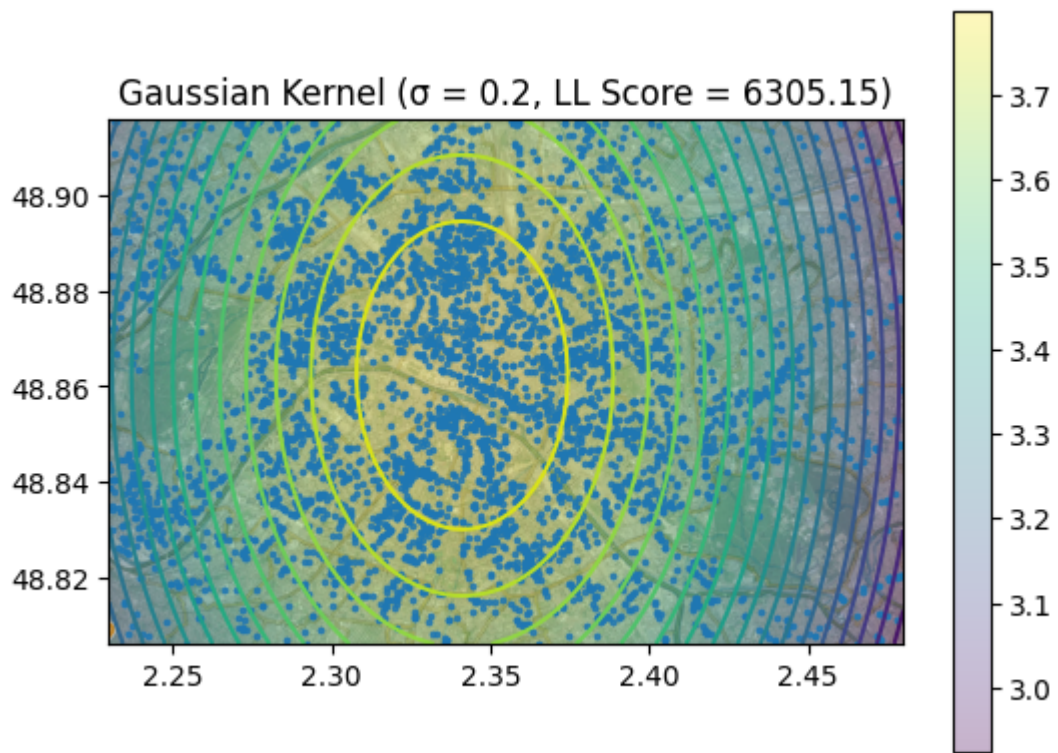
          kernels = [("Gaussian", kernel_gaussian), ("Uniform", kernel_uniform)]

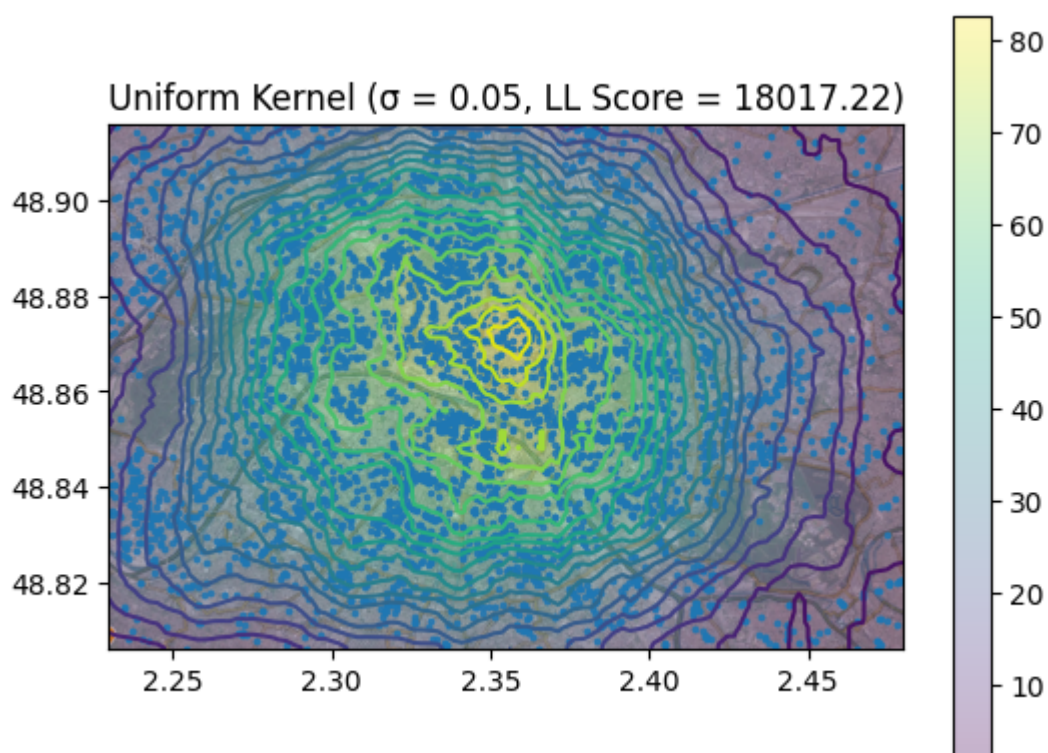
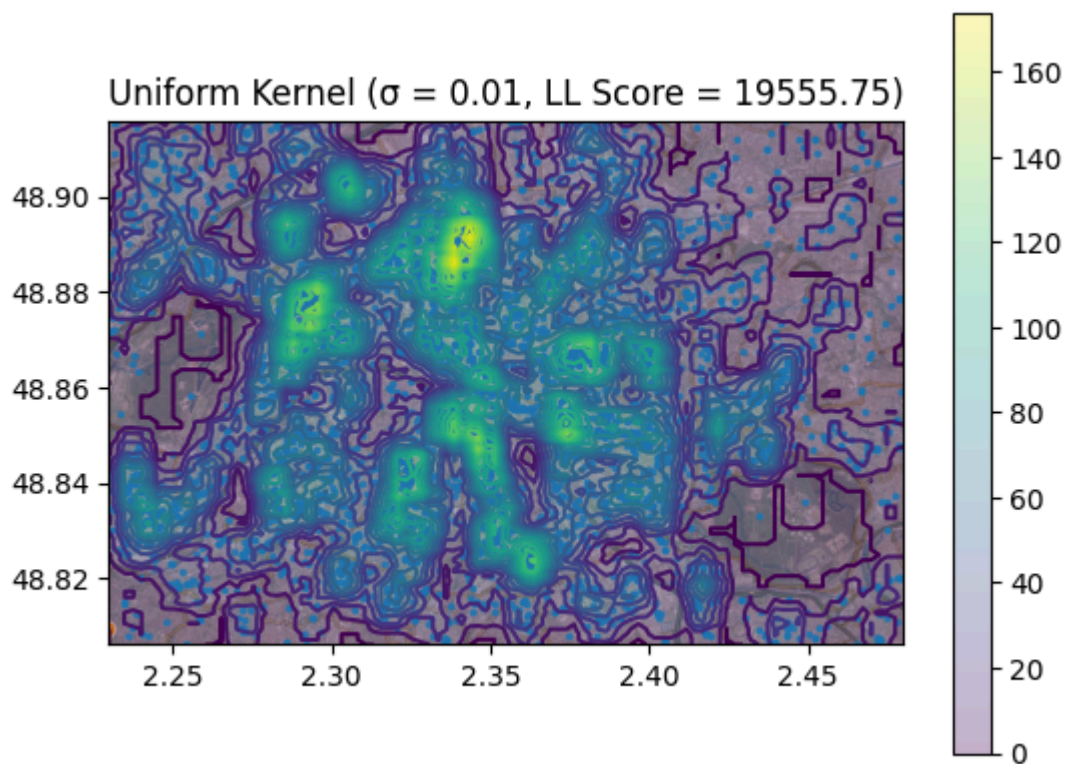
          for kernel_name, kernel in kernels :
              for sigma in [0.01, 0.05, 0.1, 0.2, 0.5] :
                  K = KernelDensity(kernel=kernel, sigma=sigma)
                  K.fit(coordonnees)
                  K.predict(coordonnees)
                  show_density(K, coordonnees)
                  plt.gca().set_title(f'{kernel_name} Kernel ( $\sigma$  = {sigma}, LL Score = {K.s

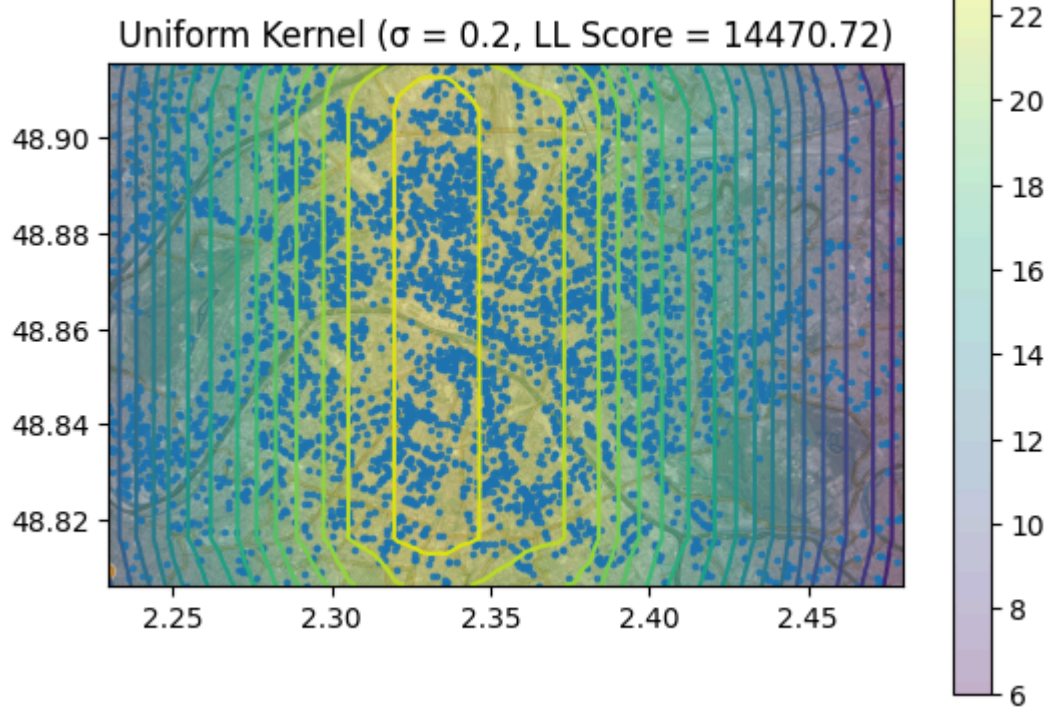
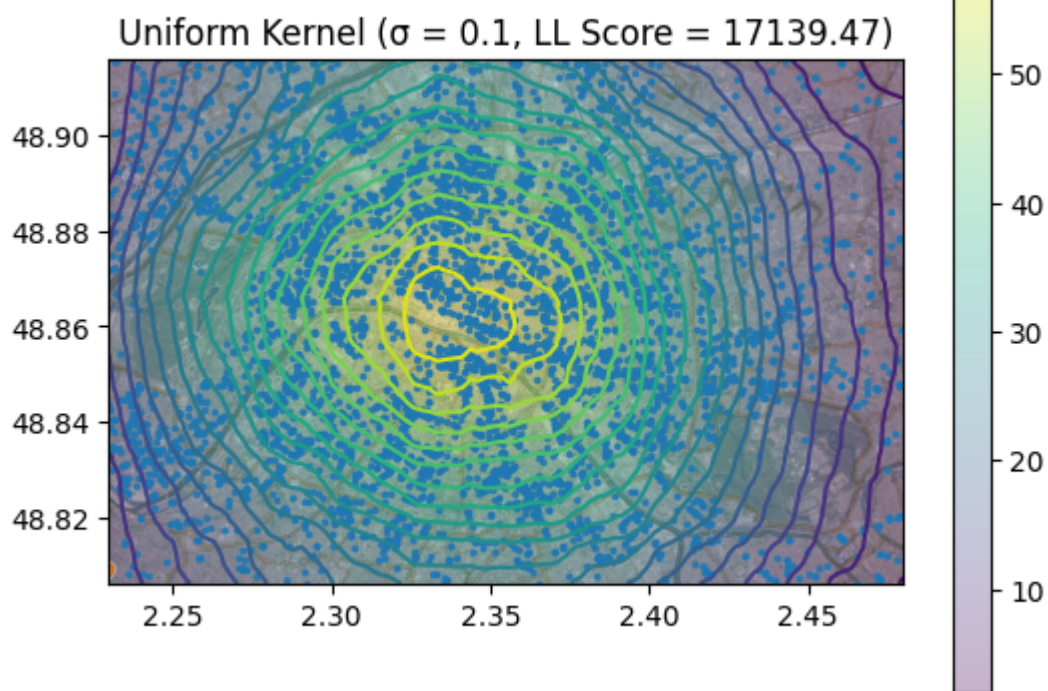
```

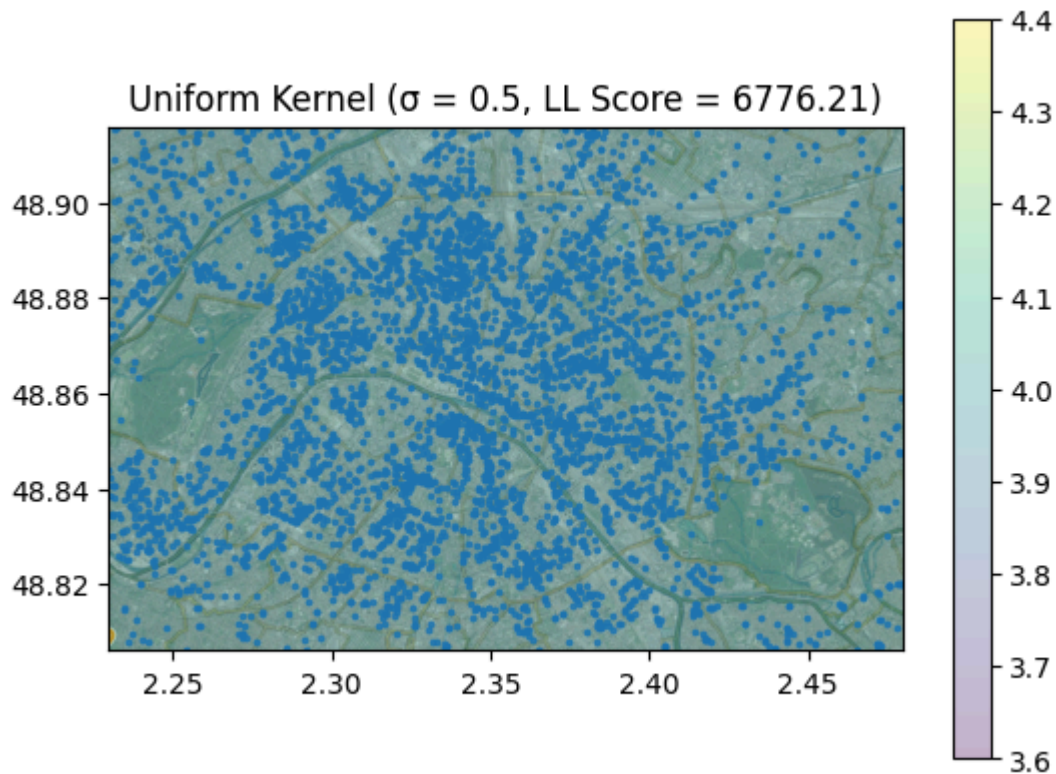












On observe que, pour les deux noyaux (uniforme et gaussien), plus sigma est élevé, plus la log-vraisemblance des données d'apprentissage diminue. Cela s'explique par le fait qu'un sigma plus grand correspond à une estimation de densité plus lissée, qui s'adapte moins précisément aux données. Il y a alors de l'underfitting, ce qui fait baisser la log-vraisemblance.

```
In [40]: # Recherche des meilleurs hyper-paramètres pour Le POI "bar" (split en ensemble
coordonnees, notes = load_poi('bar')
train_data, test_data = train_test_split(coordonnees, test_size=0.3, random_stat

results = []

kernels = [("Gaussian", kernel_gaussian), ("Uniform", kernel_uniform)]
sigmas = [0.01, 0.02, 0.05, 0.1, 0.2, 0.5, 1.0]

plt.figure(figsize=(12, 6))

for kernel_name, kernel in kernels:
    ll_train_values = []
    ll_test_values = []

    for sigma in sigmas:
        K = KernelDensity(kernel=kernel, sigma=sigma)
        K.fit(train_data)
        ll_train = K.score(train_data)
        ll_test = K.score(test_data)
        ll_train_values.append(ll_train)
        ll_test_values.append(ll_test)
        results.append((kernel_name, sigma, ll_train, ll_test))

plt.plot(sigmas, ll_train_values, label=f'Train - {kernel_name}')
plt.plot(sigmas, ll_test_values, label=f'Test - {kernel_name}')
```

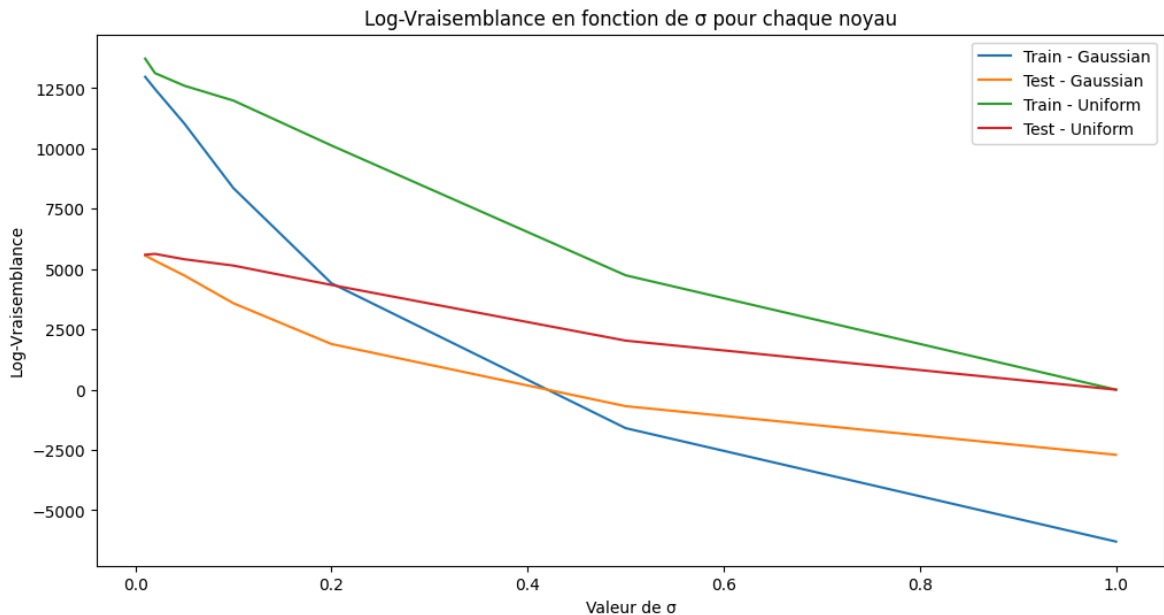
```

best = max(results, key=lambda x: x[3])
print(f"Meilleur : noyau={best[0]}, sigma={best[1]} (LL Test = {best[3]:.2f})")

plt.xlabel('Valeur de  $\sigma$ ')
plt.ylabel('Log-Vraisemblance')
plt.title('Log-Vraisemblance en fonction de  $\sigma$  pour chaque noyau')
plt.legend()
plt.show()

```

Meilleur : noyau=Uniform, sigma=0.02 (LL Test = 5630.43)



```

In [41]: # Recherche des meilleurs hyper-paramètres pour le POI "night_club" (split en en
coordonnees, notes = load_poi('night_club')
train_data, test_data = train_test_split(coordonnees, test_size=0.3, random_stat

results = []

kernels = [("Gaussian", kernel_gaussian), ("Uniform", kernel_uniform)]
sigmas = [0.01, 0.02, 0.05, 0.1, 0.2, 0.5, 1.0]

plt.figure(figsize=(12, 6))

for kernel_name, kernel in kernels:
    ll_train_values = []
    ll_test_values = []

    for sigma in sigmas:
        K = KernelDensity(kernel=kernel, sigma=sigma)
        K.fit(train_data)
        ll_train = K.score(train_data)
        ll_test = K.score(test_data)
        ll_train_values.append(ll_train)
        ll_test_values.append(ll_test)
        results.append((kernel_name, sigma, ll_train, ll_test))

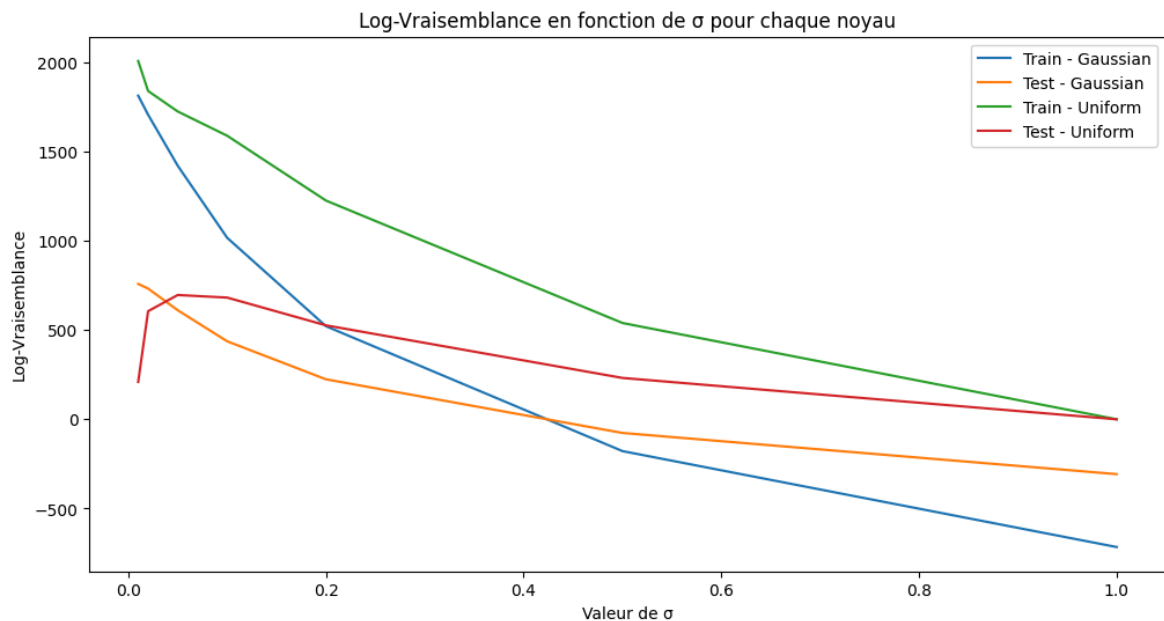
    plt.plot(sigmas, ll_train_values, label=f'Train - {kernel_name}')
    plt.plot(sigmas, ll_test_values, label=f'Test - {kernel_name}')

best = max(results, key=lambda x: x[3])
print(f"Meilleur : noyau={best[0]}, sigma={best[1]} (LL Test = {best[3]:.2f})")

```

```
plt.xlabel('Valeur de  $\sigma$ ')
plt.ylabel('Log-Vraisemblance')
plt.title('Log-Vraisemblance en fonction de  $\sigma$  pour chaque noyau')
plt.legend()
plt.show()
```

Meilleur : noyau=Gaussian, sigma=0.01 (LL Test = 757.96)



Comme pour la méthode par histogramme, les hyperparamètres optimaux dépendent de la quantité de données présentes.

Régression par Nadaraya-Watson

Classe Nadaraya

```
In [53]: class Nadaraya:
def __init__(self, kernel, sigma=0.1):
    self.kernel = kernel
    self.sigma = sigma

def fit(self, x, y):
    """
    Apprend l'estimateur sur les données

    Parameters:
        - x : np.ndarray, coordonnées [x, y] de chaque point de données
        - y : np.ndarray, notes de chaque point de données
    """
    self.x = x
    self.y = y

def predict(self, data):
    """
    Prédit la note pour chaque point de données.

    Parameters:
        - data : np.ndarray, coordonnées [x, y] de chaque point de données
```



```

Returns:
- pred : np.ndarray, notes prédites
"""
pred = []
for pt in data :
    num = np.sum(self.kernel((pt - self.x)/self.sigma) * self.y)
    denom = np.sum(self.kernel((pt - self.x)/self.sigma))
    pred.append(num / denom if denom != 0 else 0)
return np.array(pred)

```

Tests Régression par Nadaraya-Watson

```

In [54]: # Prédiction de La note du POI "bar" en fonction de son emplacement

coordonnees, notes = load_poi('bar')
train_data_x, test_data_x, train_data_y, test_data_y = train_test_split(coordonnees, notes)

kernels = [("Gaussian", kernel_gaussian), ("Uniform", kernel_uniform)]
sigmas = [0.01, 0.05, 0.1, 0.2, 0.5]

for kernel_name, kernel in kernels :
    for sigma in sigmas :
        N = Nadaraya(kernel=kernel, sigma=sigma)
        N.fit(train_data_x, train_data_y)
        pred = N.predict(test_data_x)
        mse = np.mean((pred - test_data_y) ** 2)
        print(f"Kernel: {kernel_name}  $\sigma$  = {sigma:.2f} --> MSE = {mse:.4f} Prediction: {pred}")

```

Kernel: Gaussian σ = 0.01 --> MSE = 3.6098 Prediction: [2.7509604 2.64056119 2.99347308 ... 2.72850463 2.38436244 3.69930149]

Kernel: Gaussian σ = 0.05 --> MSE = 3.7304 Prediction: [3.07282013 3.03658158 3.12712368 ... 3.17791412 2.96674621 3.26320466]

Kernel: Gaussian σ = 0.10 --> MSE = 3.7686 Prediction: [3.09428698 3.08823684 3.1515855 ... 3.11921595 3.0735838 3.14156215]

Kernel: Gaussian σ = 0.20 --> MSE = 3.7764 Prediction: [3.09105725 3.090119 3.1117039 ... 3.09655893 3.08742258 3.10234715]

Kernel: Gaussian σ = 0.50 --> MSE = 3.7781 Prediction: [3.08899518 3.08887664 3.09260847 ... 3.08983444 3.08850094 3.09077257]

Kernel: Uniform σ = 0.01 --> MSE = 3.9155 Prediction: [2.57714286 2.61428571 3.02727273 ... 2.65333333 1.65384615 4.00666667]

Kernel: Uniform σ = 0.05 --> MSE = 3.6323 Prediction: [2.96993318 2.65709677 2.99030303 ... 3.11314985 2.67037037 3.6003268]

Kernel: Uniform σ = 0.10 --> MSE = 3.7083 Prediction: [2.95993355 2.94891407 2.99243542 ... 3.25226891 2.81 3.37244565]

Kernel: Uniform σ = 0.20 --> MSE = 3.7759 Prediction: [3.11491488 3.09530004 3.14164115 ... 3.10764855 3.0946506 3.1137931]

Kernel: Uniform σ = 0.50 --> MSE = 3.7783 Prediction: [3.08854136 3.08854136 3.08854136 ... 3.08854136 3.08854136 3.08854136]

On observe que la MSE reste globalement entre 3.6 et 3.9, quelle que soit la combinaison de noyau et de paramètre σ . La MSE minimale pour le noyau gaussien est obtenue avec $\sigma = 0.01$, et pour le noyau uniforme avec $\sigma = 0.05$. Pour cet ensemble de données, ce sont donc ces valeurs de σ qui permettent le meilleur compromis entre biais et variance.

TME 02 Descente de gradient

Imports

```
In [44]: import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm
from mltools import plot_data, plot_frontiere, make_grid, gen_arti
```

Fonctions de coût

```
In [19]: def mse(w, x, y):
    return np.square((np.dot(x, w) - y))

def mse_grad(w, x, y):
    return 2 * x * (x @ w - y)

def reglog(w, x, y):
    return np.log(1 + np.exp(-y * (x @ w)))

def reglog_grad(w, x, y):
    return (-y / (1 + np.exp(y * (x @ w)))) * x
```

```
In [21]: def check_fonctions():
    # On fixe la seed de l'aléatoire pour vérifier les fonctions
    np.random.seed(0)
    datax, datay = gen_arti(epsilon=0.1)
    wrandom = np.random.randn(datax.shape[1],1)
    assert(np.isclose(mse(wrandom, datax, datay).mean(), 0.54731, rtol=1e-4))
    assert(np.isclose(reglog(wrandom, datax, datay).mean(), 0.57053, rtol=1e-4))
    assert(np.isclose(mse_grad(wrandom, datax, datay).mean(), -1.43120, rtol=1e-4))
    assert(np.isclose(reglog_grad(wrandom, datax, datay).mean(), -0.42714, rtol=1e-4))
    np.random.seed()

check_fonctions()
```

Descente de gradient

```
In [33]: def descente_gradient(datax, datay, f_loss, f_grad, eps=0.01, iter=20):
    _, d = datax.shape
    w = np.zeros((d, 1))
    list_w = [w.copy()]
    losses = [np.mean(f_loss(w, datax, datay))]

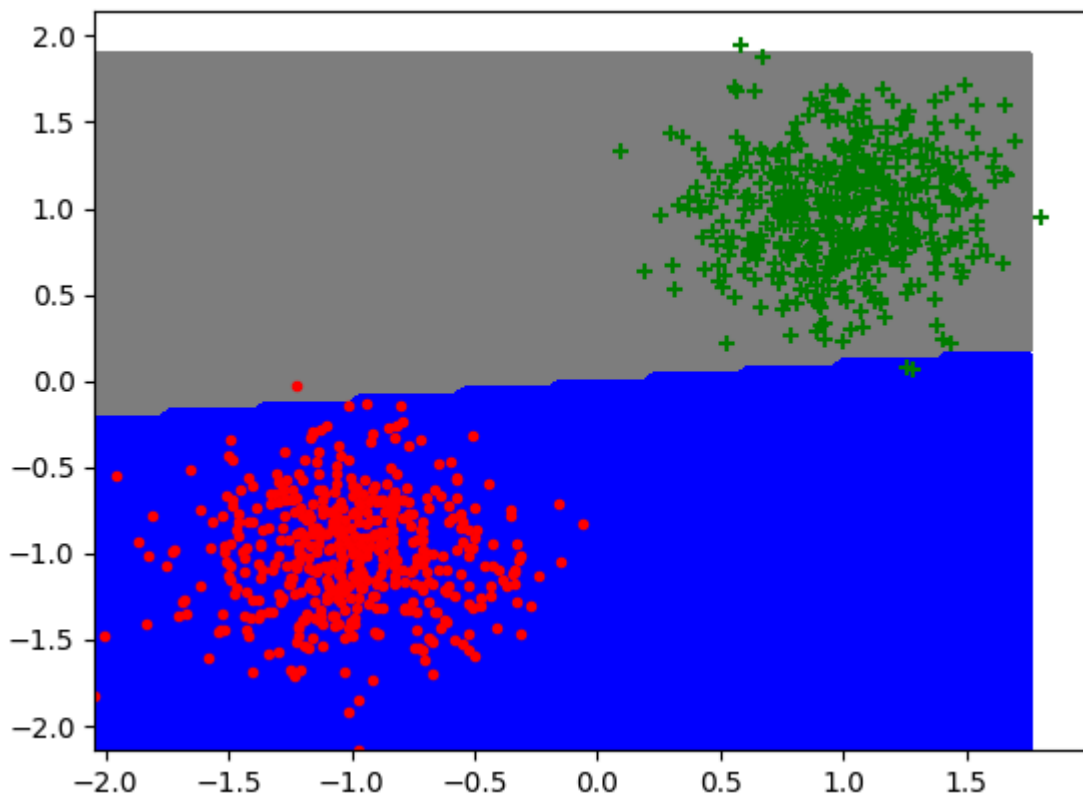
    for i in range(iter):
        grad = np.mean(f_grad(w, datax, datay), axis=0, keepdims=True)
        w = w - eps * grad.T
        list_w.append(w.copy())
        losses.append(np.mean(f_loss(w, datax, datay)))

    return w, list_w, losses
```

Expérimentations

```
In [48]: # Génération de données artificielles selon 2 gaussiennes
datax, datay = gen_arti(data_type=0)

# Frontière de décision avec un w aléatoire (avant apprentissage)
w_random = np.random.randn(datax.shape[1], 1)
plot_frontiere(datax, lambda x: np.sign(x @ w_random), step=100)
plot_data(datax, datay)
```



```
In [49]: # Descente de gradient pour la régression linéaire
w_lin, list_w_lin, losses_lin = descente_gradient(datax, datay, mse, mse_grad, e

# Descente de gradient pour la régression logistique
w_log, list_w_log, losses_log = descente_gradient(datax, datay, reglog, reglog_g

# Fonction de prédiction
def f_lin(x): return np.sign(x @ w_lin)
```

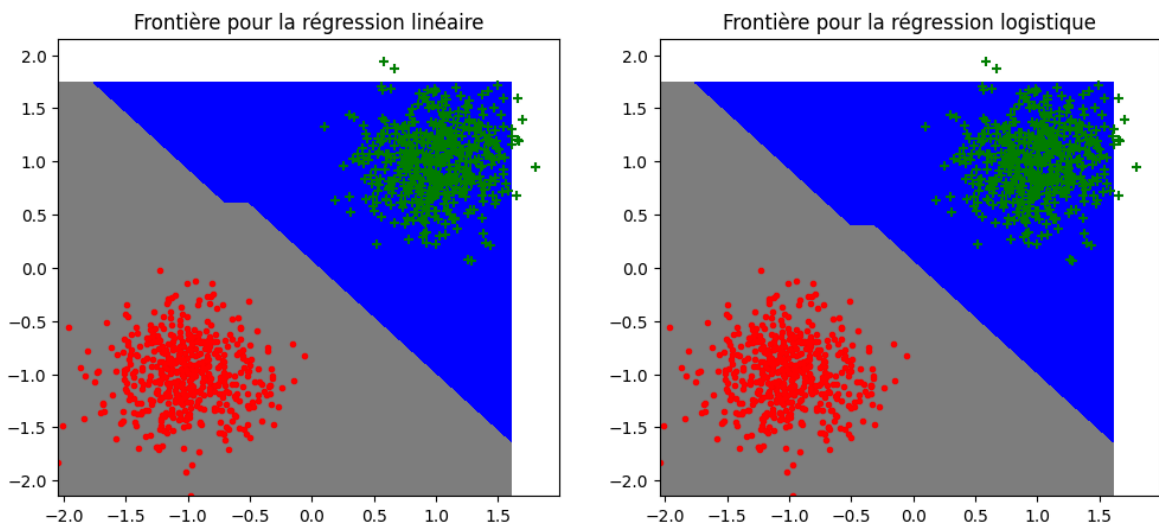
```
def f_log(x): return np.sign(x @ w_log)

# Visualisation des frontières
plt.figure(figsize=(12, 5))

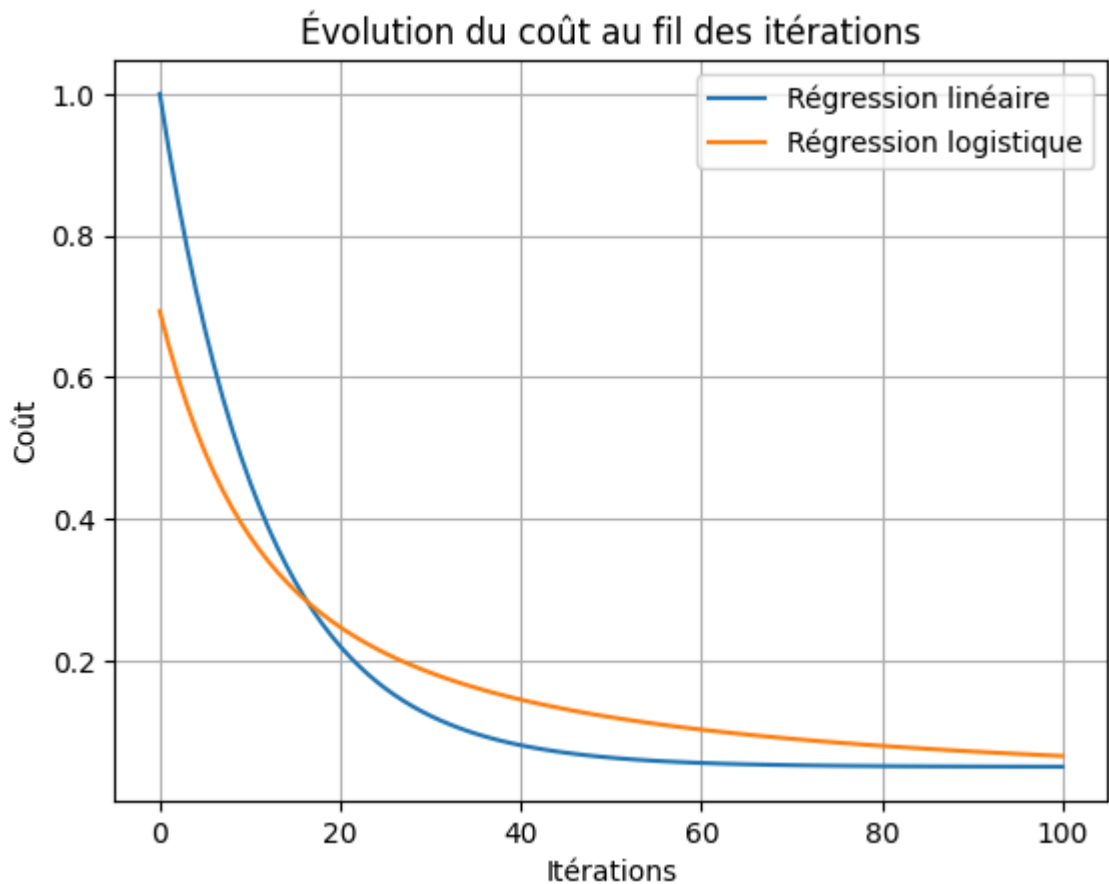
plt.subplot(1, 2, 1)
plot_frontiere(datax, f_lin)
plot_data(datax, datay)
plt.title("Frontière pour la régression linéaire")

plt.subplot(1, 2, 2)
plot_frontiere(datax, f_log)
plot_data(datax, datay)
plt.title("Frontière pour la régression logistique")

plt.show()
```



```
In [50]: # Evolution du coût au fil des itérations
plt.figure()
plt.plot(losses_lin, label="Régression linéaire")
plt.plot(losses_log, label="Régression logistique")
plt.xlabel("Itérations")
plt.ylabel("Coût")
plt.title("Évolution du coût au fil des itérations")
plt.legend()
plt.grid(True)
plt.show()
```

La convergence de l'algorithme dépend fortement du pas de gradient. Un pas trop grand peut provoquer des oscillations ou une divergence (sauts au-dessus de l'optimum), surtout dans les problèmes non séparables. En revanche, un pas trop petit assure une convergence plus stable, mais plus lente. De plus, dans les problèmes non séparables avec du bruit, un petit pas évite les oscillations mais peut bloquer l'algorithme dans un minimum local.

```
In [52]: # Visualisation de la fonction de coût en 2D pour la régression linéaire et logi
grid, x_grid, y_grid = make_grid(datax, step=100)

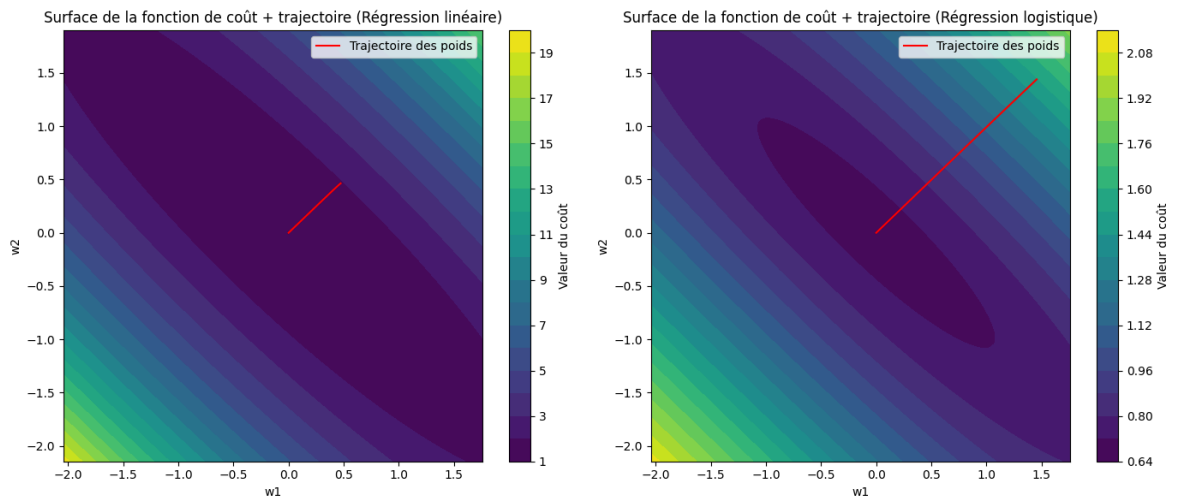
plt.figure(figsize=(14, 6))

# Régression linéaire
plt.subplot(1, 2, 1)
plt.contourf(x_grid, y_grid, np.array([mse(w, datax, datay).mean() for w in grid]
plt.colorbar(label="Valeur du coût")
w_lin_array = np.array(list_w_lin).squeeze()
plt.plot(w_lin_array[:, 0], w_lin_array[:, 1], color='red', markersize=2, label=
plt.xlabel("w1")
plt.ylabel("w2")
plt.title("Surface de la fonction de coût + trajectoire (Régression linéaire)")
plt.legend()

# Régression logistique
plt.subplot(1, 2, 2)
plt.contourf(x_grid, y_grid, np.array([reglog(w, datax, datay).mean() for w in g
plt.colorbar(label="Valeur du coût")
w_log_array = np.array(list_w_log).squeeze()
plt.plot(w_log_array[:, 0], w_log_array[:, 1], color='red', markersize=2, label=
plt.xlabel("w1")
```

```
plt.ylabel("w2")
plt.title("Surface de la fonction de coût + trajectoire (Régression logistique)")
plt.legend()

plt.tight_layout()
plt.show()
```



```
In [57]: # Génération de données artificielles selon 4 gaussiennes
datax, datay = gen_arti(centerx=2, centery=2, sigma=0.2, nbex=1000, data_type=0,

# Frontière de décision avec un w aléatoire (avant apprentissage)
w_random = np.random.randn(datax.shape[1], 1)
plot_frontiere(datax, lambda x: np.sign(x @ w_random), step=100)
plot_data(datax, datay)

# Descente de gradient pour la régression linéaire
w_lin, list_w_lin, losses_lin = descente_gradient(datax, datay, mse, mse_grad, e

# Descente de gradient pour la régression logistique
w_log, list_w_log, losses_log = descente_gradient(datax, datay, reglog, reglog_g

# Fonction de prédiction
def f_lin(x): return np.sign(x @ w_lin)
def f_log(x): return np.sign(x @ w_log)

# Visualisation des frontières
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plot_frontiere(datax, f_lin)
plot_data(datax, datay)
plt.title("Frontière pour la régression linéaire")

plt.subplot(1, 2, 2)
plot_frontiere(datax, f_log)
plot_data(datax, datay)
plt.title("Frontière pour la régression logistique")

plt.show()

# Evolution du coût au fil des itérations
plt.figure()
plt.plot(losses_lin, label="Régression linéaire")
plt.plot(losses_log, label="Régression logistique")
```

```

plt.xlabel("Itérations")
plt.ylabel("Coût")
plt.title("Évolution du coût au fil des itérations")
plt.legend()
plt.grid(True)
plt.show()

# Visualisation de la fonction de coût en 2D pour la régression Linéaire et Logi
grid, x_grid, y_grid = make_grid(datax, step=100)

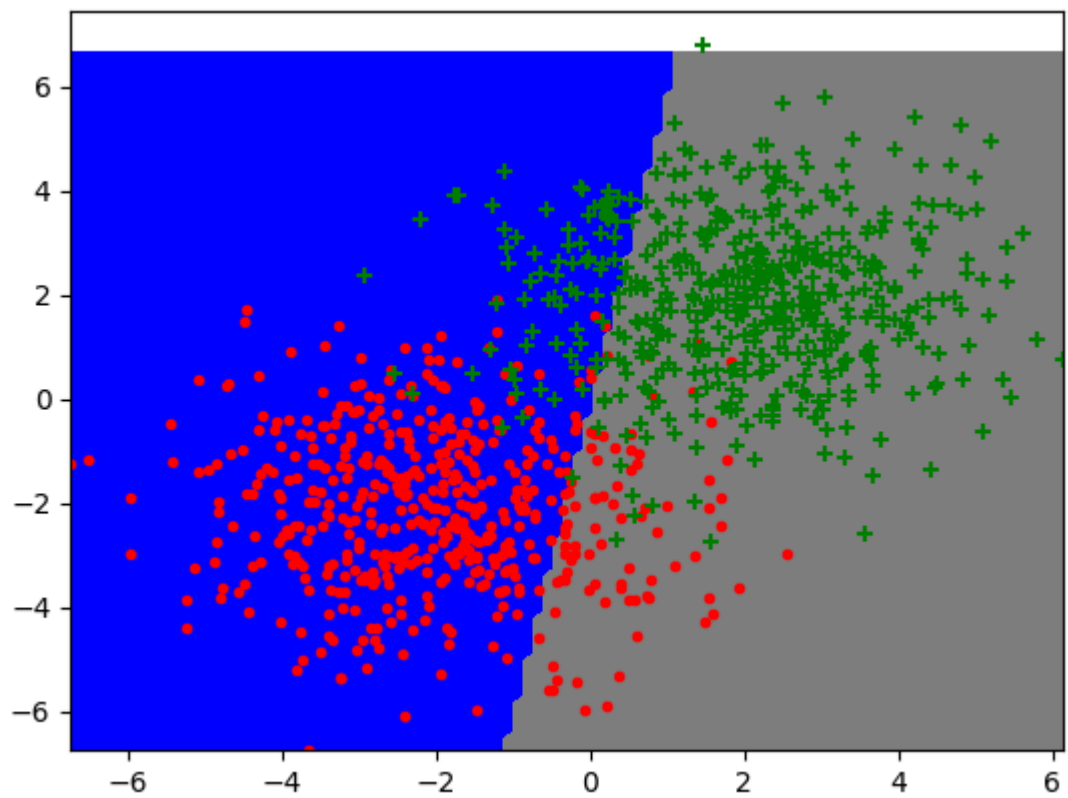
plt.figure(figsize=(14, 6))

# Régression Linéaire
plt.subplot(1, 2, 1)
plt.contourf(x_grid, y_grid, np.array([mse(w, datax, datay).mean() for w in grid]
plt.colorbar(label="Valeur du coût")
w_lin_array = np.array(list_w_lin).squeeze()
plt.plot(w_lin_array[:, 0], w_lin_array[:, 1], color='red', markersize=2, label=
plt.xlabel("w1")
plt.ylabel("w2")
plt.title("Surface de la fonction de coût + trajectoire (Régression linéaire)")
plt.legend()

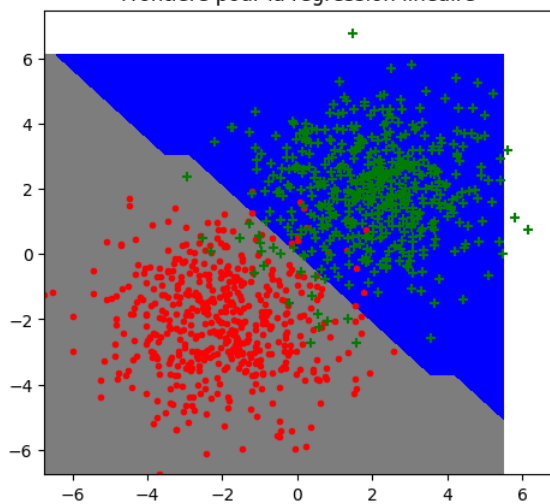
# Régression Logistique
plt.subplot(1, 2, 2)
plt.contourf(x_grid, y_grid, np.array([reglog(w, datax, datay).mean() for w in g
plt.colorbar(label="Valeur du coût")
w_log_array = np.array(list_w_log).squeeze()
plt.plot(w_log_array[:, 0], w_log_array[:, 1], color='red', markersize=2, label=
plt.xlabel("w1")
plt.ylabel("w2")
plt.title("Surface de la fonction de coût + trajectoire (Régression logistique)"
plt.legend()

plt.tight_layout()
plt.show()

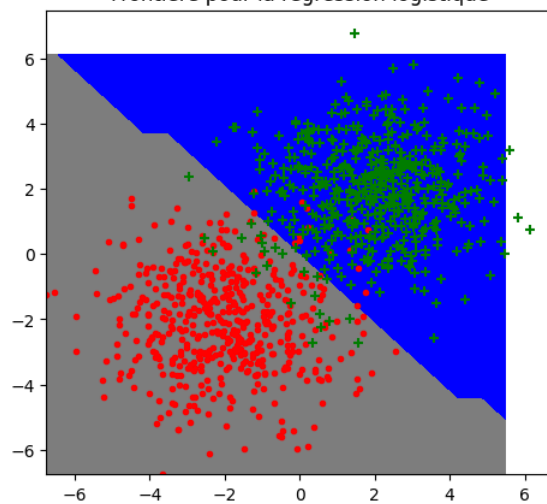
```



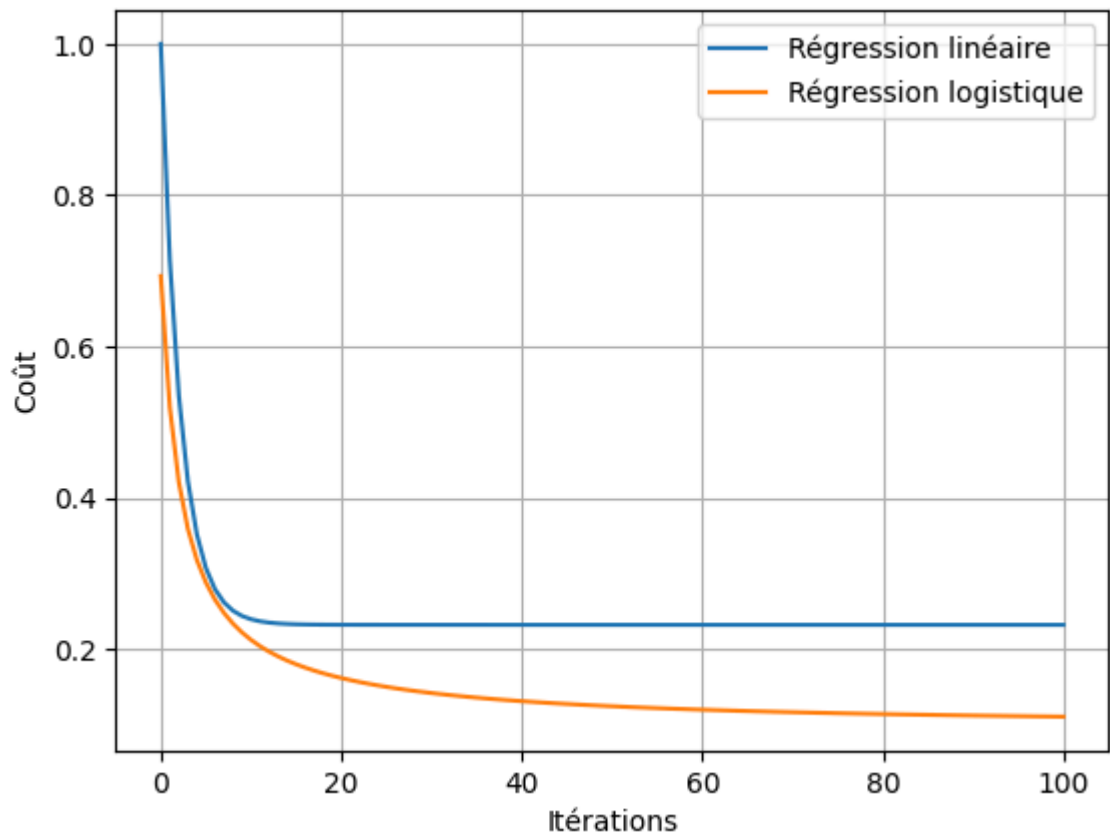
Frontière pour la régression linéaire



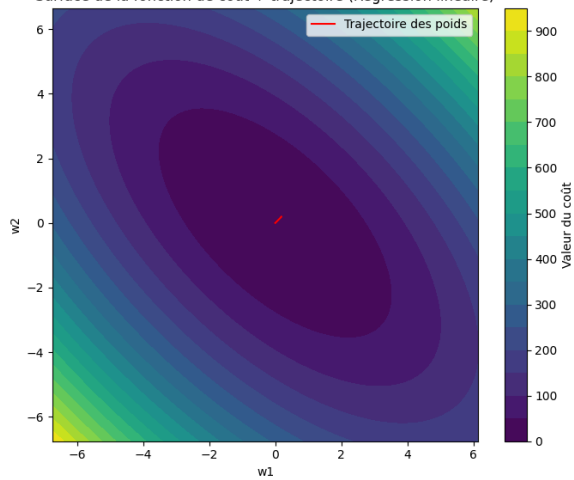
Frontière pour la régression logistique



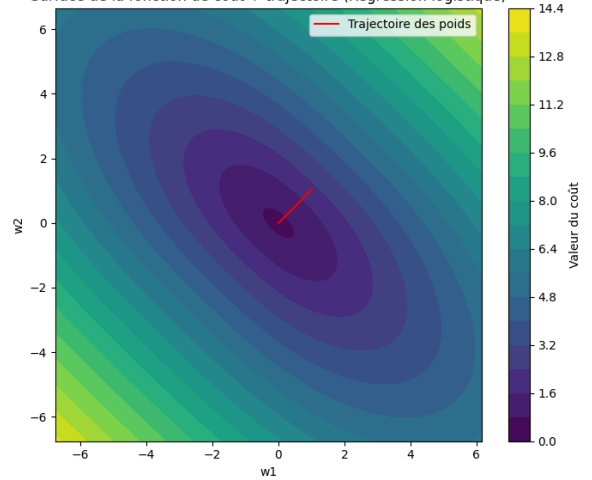
Évolution du coût au fil des itérations



Surface de la fonction de coût + trajectoire (Régression linéaire)



Surface de la fonction de coût + trajectoire (Régression logistique)



TME 05 Perceptron et SVMs

Imports

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from mltools import *
from sklearn.model_selection import train_test_split
from functools import partial
import seaborn as sns
```

I. Perceptron et classe Linéaire (batch)

1) Fonctions d'apprentissage

```
In [ ]: def perceptron_loss(w, x, y):
    """
    Compute the perceptron loss for a set of predictions.

    Params:
        w (np.ndarray): weight vector
        x (np.ndarray): input features
        y (np.ndarray): labels (+1 or -1)

    Returns:
        float: average loss over all samples
    """
    margins = -y * (x @ w)
    loss = np.maximum(0, margins).mean()
    return loss
```

```
In [ ]: def perceptron_grad(w, x, y):
    """
    Compute the gradient of the perceptron loss.

    Params:
        w (np.ndarray): weight vector
        x (np.ndarray): input features
        y (np.ndarray): labels (+1 or -1)

    Returns:
        np.ndarray: gradient of the loss w.r.t. weights
    """
    margins = -y * (x @ w)
    mask = margins > 0
    grad = -(x[mask].T @ y[mask]) / x.shape[0] #seulement exemple mal classés ca
    return grad
```

```
In [ ]: class Lineaire:
    def __init__(self, loss, loss_g, max_iter=100, eps=0.01):
        """
```

Linear classifier trained with custom loss and gradient.

Params:

loss (callable): loss function (e.g. perceptron_loss)
loss_g (callable): gradient of the loss function
max_iter (int): number of training iterations
eps (float): learning rate

"""

```
self.loss = loss #perceptron Loss
self.loss_g = loss_g # perceptron grad
self.max_iter = max_iter
self.eps = eps
self.w = None
self.loss_history = []
```

```
def predict(self, datax):
```

"""

Predict class labels for given input data.

Params:

datax (np.ndarray): input features

Returns:

np.ndarray: predicted labels (+1 or -1)

"""

```
return np.sign(datax @ self.w)
```

```
def score(self, datax, datay):
```

"""

Compute accuracy score of the model.

Params:

datax (np.ndarray): input features

datay (np.ndarray): true labels

Returns:

float: mean accuracy

"""

```
y_pred = self.predict(datax)
return (y_pred == datay).mean()
```

```
def fit(self, datax, datay, testx=None, testy=None):
```

"""

Train the linear model using gradient descent.

Params:

datax (np.ndarray): training input features

datay (np.ndarray): training labels

testx (np.ndarray, optional): test input features

testy (np.ndarray, optional): test labels

Side Effects:

Updates model weights, loss history and error metrics

"""

```
n, d = datax.shape
self.w = np.ones(d)
```

```

self.loss_history = []

self.train_errors = []
self.test_errors = []

for i in range(self.max_iter):
    loss_val = self.loss(self.w, datax, datay) #compute loss
    self.loss_history.append(loss_val) #stock it
    grad = self.loss_g(self.w, datax, datay) #compute grad w
    new_w = self.w - self.eps * grad
    self.w = new_w #update w
    if testx is not None and testy is not None:
        train_error = 1 - self.score(datax, datay) # calcul de l'erreur
        test_error = 1 - self.score(testx, testy)
        self.train_errors.append(train_error) #stock it
        self.test_errors.append(test_error)

```

```

In [ ]: def display_error(train_errors, test_errors) :
        """
        Display the learning curve showing training and test errors.

        Params:
            train_errors (list): list of training errors
            test_errors (list): list of test errors
        """

        plt.figure(figsize=(10,6))
        plt.plot(train_errors, label='Erreur entraînement')
        plt.plot(test_errors, label='Erreur test')
        plt.xlabel("Itérations")
        plt.ylabel("Erreur")
        plt.legend()
        plt.title("Courbe d'apprentissage (erreur par itérations)")
        plt.show()

```

2) Chargement des données USPS

```

In [ ]: def load_usps(fn):
        """
        Load the USPS dataset from a file.

        Params:
            fn (str): file path to the dataset

        Returns:
            tuple: input features (np.ndarray) and labels (np.ndarray)
        """

        with open(fn, "r") as f:
            f.readline()
            data = [[float(x) for x in l.split()] for l in f if len(l.split())>2]
            tmp=np.array(data)
            return tmp[:,1:],tmp[:,0].astype(int)

        def get_usps(l, datax, datay):
            """
            Get the subset of data corresponding to specific labels.

```

```

Params:
    l (int or list of ints): label(s) to retrieve
    datax (np.ndarray): input features
    datay (np.ndarray): labels

Returns:
    tuple: subset of input features and corresponding labels
"""

if type(l)!=list:
    resx = datax[datay==l,:]
    resy = datay[datay==l]
    return resx,resy
tmp = list(zip(*[get_usps(i,datax,datay) for i in l]))
tmpx,tmpy = np.vstack(tmp[0]),np.hstack(tmp[1])
return tmpx,tmpy

def show_usps(data):
    """
    Display an image from the USPS dataset.

    Params:
        data (np.ndarray): flattened image data (size 256)
    """

    plt.imshow(data.reshape((16,16)),interpolation="nearest",cmap="gray")

```

```

In [ ]: def add_noise(data, eps=0.1):
    """
    Add Gaussian noise to the data.

    Params:
        data (np.ndarray): input data
        eps (float): standard deviation of the noise

    Returns:
        np.ndarray: noisy data, clipped between 0 and 1
    """

    noise = np.random.normal(0, eps, data.shape)
    noisy_data = data + noise
    noisy_data = np.clip(noisy_data, 0, 1)
    return noisy_data

```

```

In [8]: # Retrieve data
uspsdatatrain = "../data/USPS_train.txt"
uspsdatatest = "../data/USPS_test.txt"
alltrainx,alltrainy = load_usps(uspsdatatrain)
alltestx,alltesty = load_usps(uspsdatatest)

#filterer le dataset
neg = 5
pos = 6
datax,datay = get_usps([neg,pos],alltrainx,alltrainy)
testx,testy = get_usps([neg,pos],alltestx,alltesty)

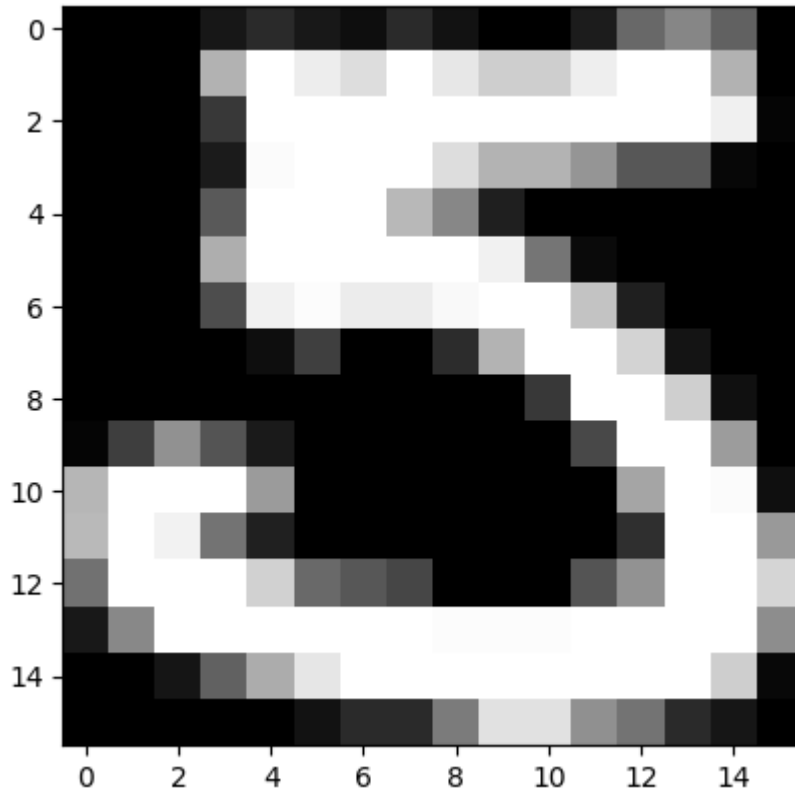
# Affichage de l'image
show_usps(datax[0])
print(f"Shape of x data : {datax.shape}, y data : {datay.shape}")

```

```
plt.show()
print(f"L'image affichée correspond au label : {datay[0]}")

#transformer les labels des données 5, 6 en -1, 1
datay = np.where(datay == pos, 1, -1)
testy = np.where(testy == pos, 1, -1)
print(f"Après modification, l'image affichée correspond au label : {datay[0]}")
```

Shape of x data : (1220, 256), y data : (1220,)



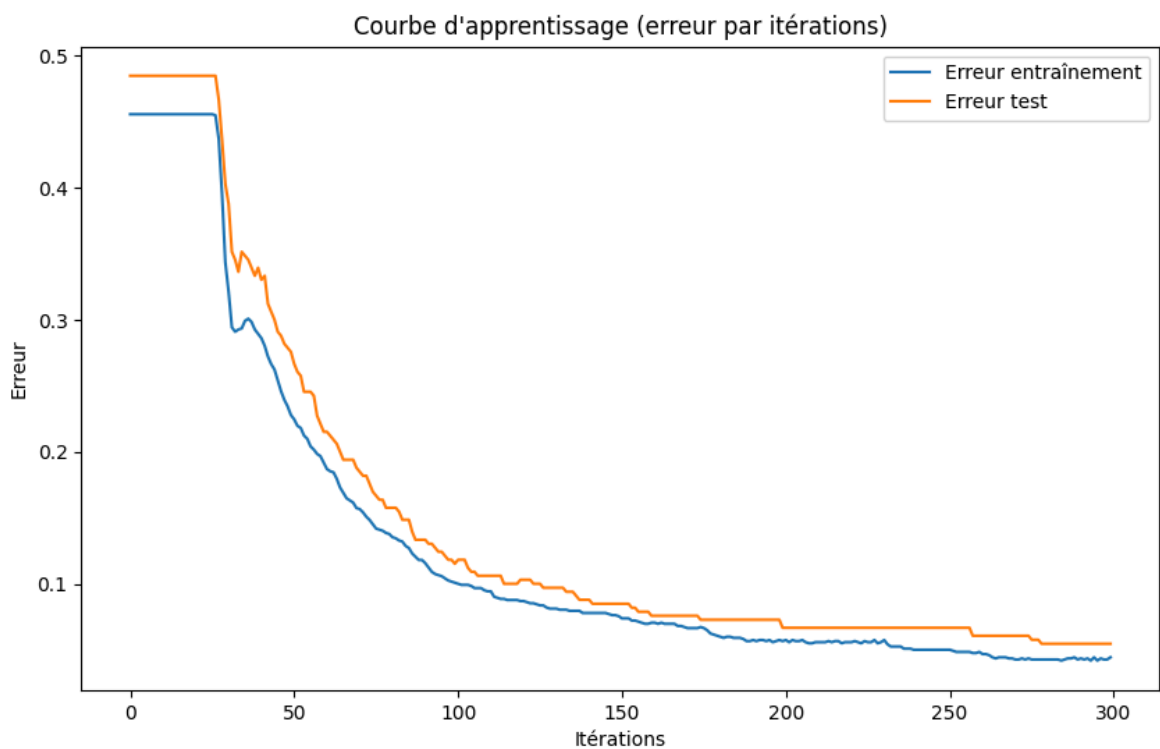
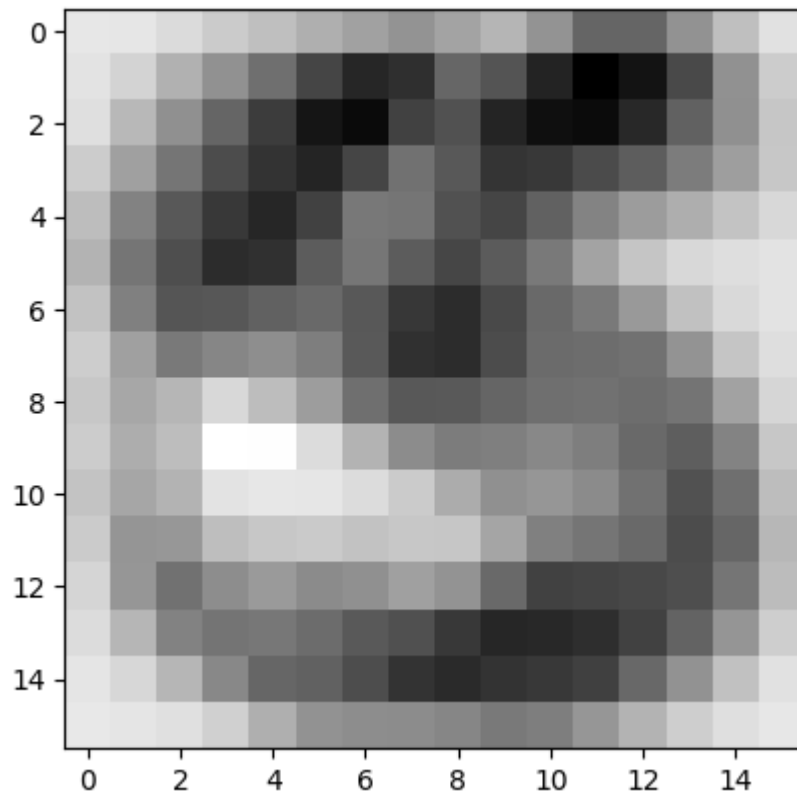
L'image affichée correspond au label : 5

Après modification, l'image affichée correspond au label : -1

3) Tests et résultats

```
In [8]: lin = Lineaire(loss=perceptron_loss, loss_g=perceptron_grad, max_iter=300, eps=0
lin.fit(datax, datay, testx, testy)
print("matrice de poids w après apprentissage : ")
show_usps(lin.w.reshape(16, 16))
display_error(lin.train_errors, lin.test_errors)
```

matrice de poids w après apprentissage :



II. Mini-batch et descente stochastique

1) Fonctions d'apprentissage

```
In [ ]: class Lineaire2:
def __init__(self, loss, loss_g, n_epoch=100, eps=0.01):
"""
Linear classifier trained with custom loss and gradient, supporting batch
```

```

    Params:
        loss (callable): loss function (e.g., perceptron_loss)
        loss_g (callable): gradient of the loss function
        n_epoch (int): number of training epochs
        eps (float): learning rate
    """

    self.loss = loss #perceptron Loss
    self.loss_g = loss_g # perceptron grad
    self.n_epoch = n_epoch
    self.eps = eps
    self.w = None
    self.loss_history = []

def predict(self, datax):
    """
    Predict class labels for given input data.

    Params:
        datax (np.ndarray): input features

    Returns:
        np.ndarray: predicted labels (+1 or -1)
    """

    return np.sign(datax @ self.w)

def score(self, datax, datay):
    """
    Compute accuracy score of the model.

    Params:
        datax (np.ndarray): input features
        datay (np.ndarray): true labels

    Returns:
        float: mean accuracy
    """

    y_pred = self.predict(datax)
    return (y_pred == datay).mean()

def fit_batch(self, datax, datay, testx=None, testy=None):
    """
    Train the linear model using batch gradient descent.

    Params:
        datax (np.ndarray): training input features
        datay (np.ndarray): training labels
        testx (np.ndarray, optional): test input features
        testy (np.ndarray, optional): test labels

    Side Effects:
        Updates model weights, loss history and error metrics
    """

    n, d = datax.shape
    self.w = np.ones(d)
    self.loss_history = []

```

```

self.train_errors = []
self.test_errors = []

for i in range(self.n_epoch):
    loss_val = self.loss(self.w, datax, datay) #compute loss
    self.loss_history.append(loss_val) #stock it
    grad = self.loss_g(self.w, datax, datay) #compute grad w
    new_w = self.w - self.eps * grad
    self.w = new_w #update w
    if testx is not None and testy is not None:
        train_error = 1 - self.score(datax, datay) # calcul de l'erreur
        test_error = 1 - self.score(testx, testy)
        self.train_errors.append(train_error) #stock it
        self.test_errors.append(test_error)

def fit_stochastic(self, datax, datay, testx=None, testy=None):
    """
    Train the linear model using stochastic gradient descent.

    Params:
        datax (np.ndarray): training input features
        datay (np.ndarray): training labels
        testx (np.ndarray, optional): test input features
        testy (np.ndarray, optional): test labels

    Side Effects:
        Updates model weights, loss history and error metrics
    """

    n, d = datax.shape
    self.w = np.ones(d)
    self.loss_history = []

    self.train_errors = []
    self.test_errors = []

    for epoch in range(self.n_epoch):
        indices = np.random.permutation(n)
        datax_shuffled = datax[indices]
        datay_shuffled = datay[indices]

        for i in range(0, n):
            xi = datax_shuffled[i:i+1]
            yi = datay_shuffled[i:i+1]
            grad = self.loss_g(self.w, xi, yi)
            self.w -= self.eps * grad

        train_error = self.loss(self.w, datax, datay)
        self.train_errors.append(train_error)

        if testx is not None and testy is not None:
            test_error = self.loss(self.w, testx, testy)
            self.test_errors.append(test_error)

def fit_minibatch(self, datax, datay, testx=None, testy=None, batch_size=32)
    """
    Train the linear model using minibatch gradient descent.

```

```

Params:
    datax (np.ndarray): training input features
    datay (np.ndarray): training labels
    testx (np.ndarray, optional): test input features
    testy (np.ndarray, optional): test labels
    batch_size (int): size of the minibatches

Side Effects:
    Updates model weights, loss history and error metrics
"""

n, d = datax.shape
self.w = np.ones(d)
self.loss_history = []

self.train_errors = []
self.test_errors = []

for epoch in range(self.n_epoch):
    indices = np.random.permutation(n)
    datax_shuffled = datax[indices]
    datay_shuffled = datay[indices]

    for i in range(0, n, batch_size):
        x_batch = datax_shuffled[i:i + batch_size]
        y_batch = datay_shuffled[i:i + batch_size]
        grad = self.loss_g(self.w, x_batch, y_batch)
        self.w -= self.eps * grad

    # Ajout de l'erreur train
    train_error = 1 - self.score(datax, datay)
    self.train_errors.append(train_error)

    # Ajout de l'erreur test si dispo
    if testx is not None and testy is not None:
        test_error = 1 - self.score(testx, testy)
        self.test_errors.append(test_error)

```

2) Tests et résultats

a) Apprentissage par batch

```

In [ ]: # données non bruitées
lin_batch = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch=300,
lin_batch.fit_batch(datax, datay, testx, testy)
print("matrice de poids w après apprentissage par batch: ")
show_usps(lin_batch.w.reshape(16, 16))
display_error(lin_batch.train_errors, lin_batch.test_errors)

# données bruitées 10%
lin_batch_noised10 = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad, n_e
noisy_x_train = add_noise(datax, eps=0.10)
lin_batch_noised10.fit_batch(noisy_x_train, datay, testx, testy)
print("matrice de poids w après apprentissage par batch pour données bruitées (1
show_usps(lin_batch_noised10.w.reshape(16, 16))
display_error(lin_batch_noised10.train_errors, lin_batch_noised10.test_errors)

# données bruitées 50%

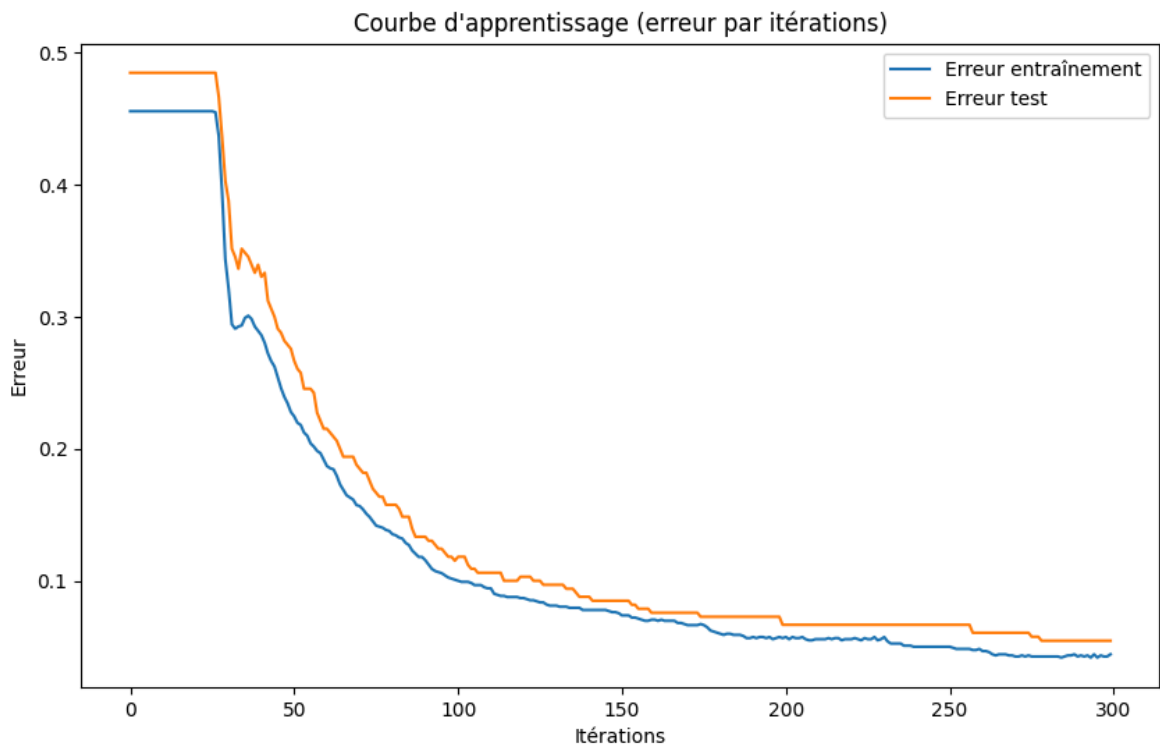
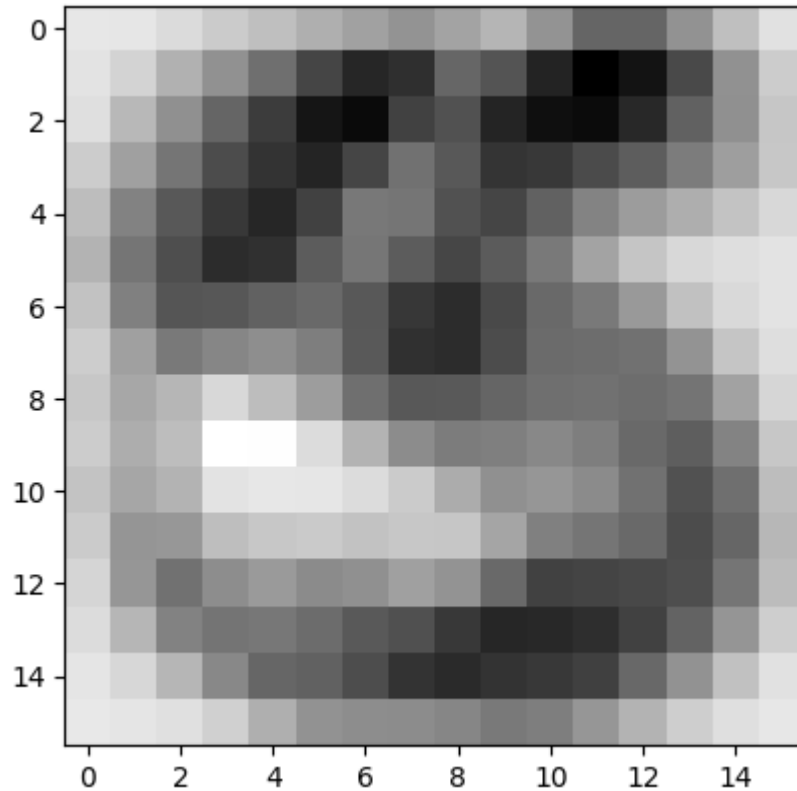
```

```

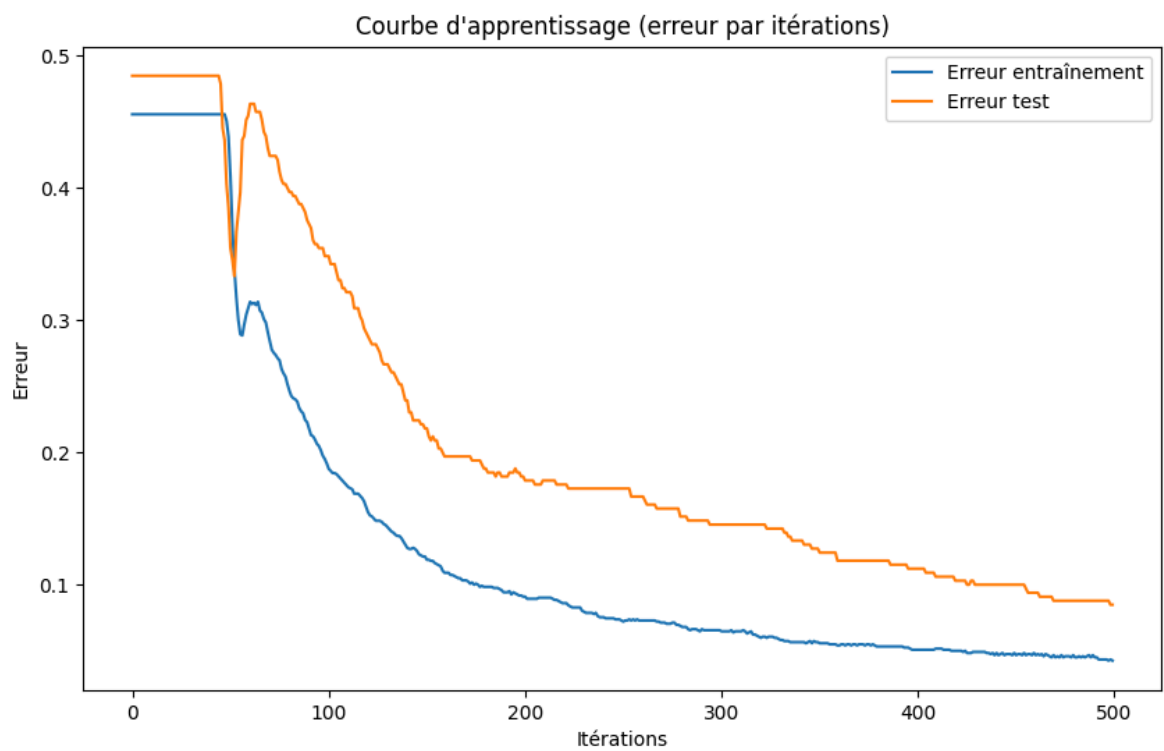
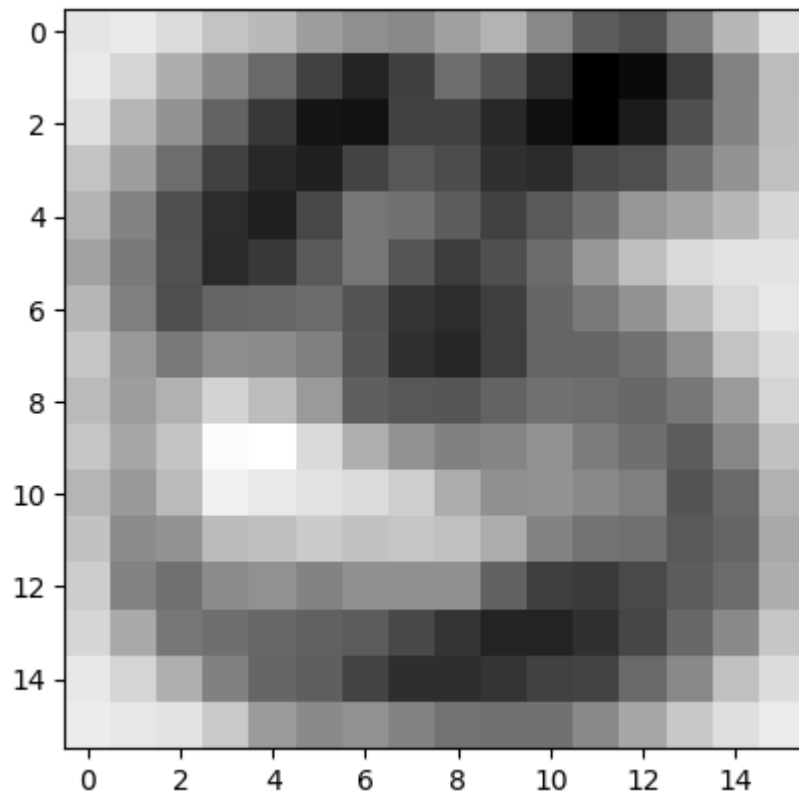
lin_batch_noised50 = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad, n_e
noisy_x_train = add_noise(datax, eps=0.5)
lin_batch_noised50.fit_batch(noisy_x_train, datay, testx, testy)
print("matrice de poids w après apprentissage par batch pour données bruitées (5
show_usps(lin_batch_noised50.w.reshape(16, 16))
display_error(lin_batch_noised50.train_errors, lin_batch_noised50.test_errors)
# données bruitées 70%

```

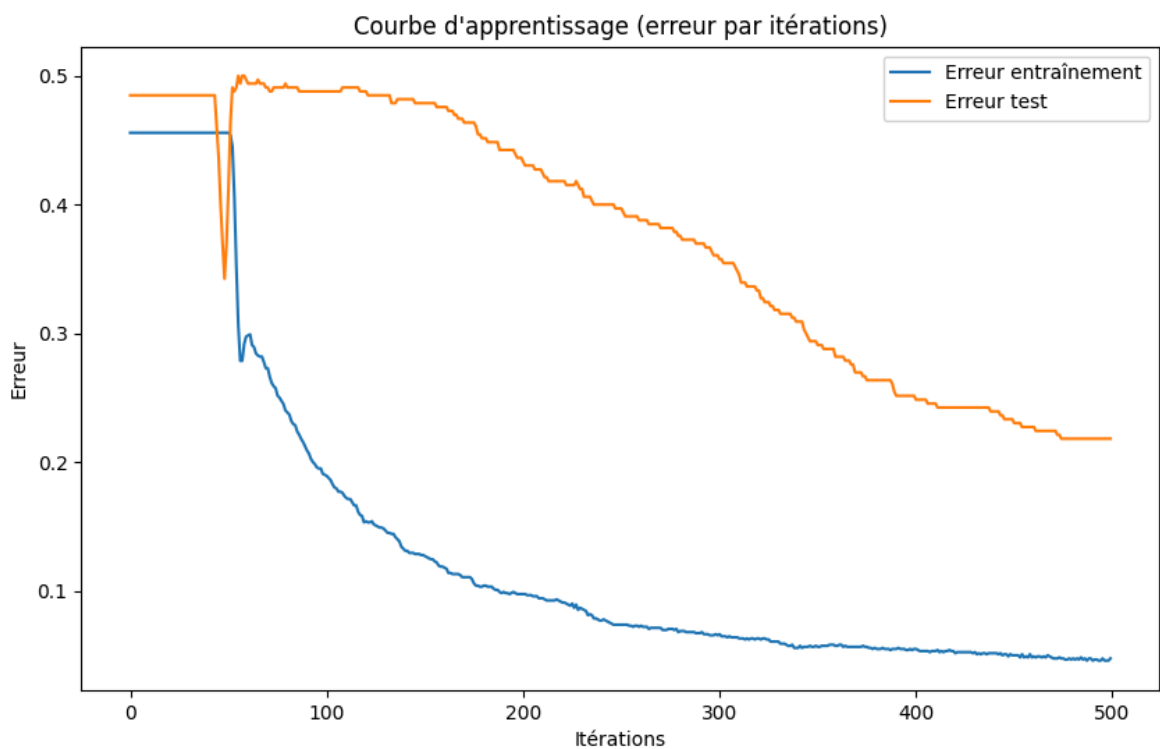
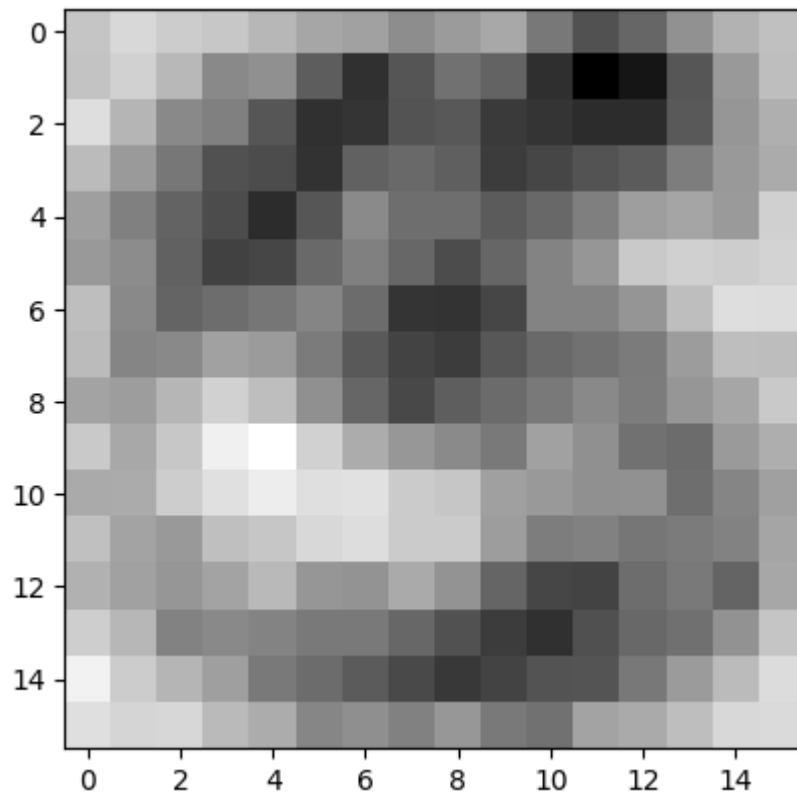
matrice de poids w après apprentissage par batch:



matrice de poids w après apprentissage par batch pour données bruitées (10%):



matrice de poids w après apprentissage par batch pour données bruitées (50%):



b) Apprentissage stochastique

```
In [30]: # Données non bruitées
lin_stoch = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch=300,
lin_stoch.fit_stochastic(datax, datay, testx, testy)
print("matrice de poids w après apprentissage stochastique: ")
show_usps(lin_stoch.w.reshape(16, 16))
display_error(lin_stoch.train_errors, lin_stoch.test_errors)

# Données bruitées 10%
lin_stoch_noised10 = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad, n_e
```

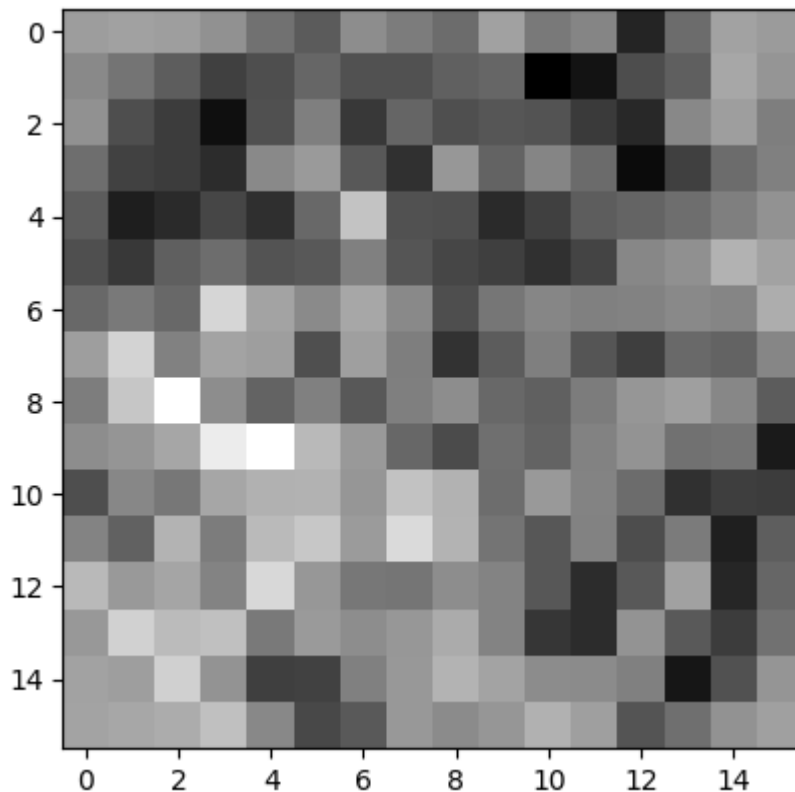
```

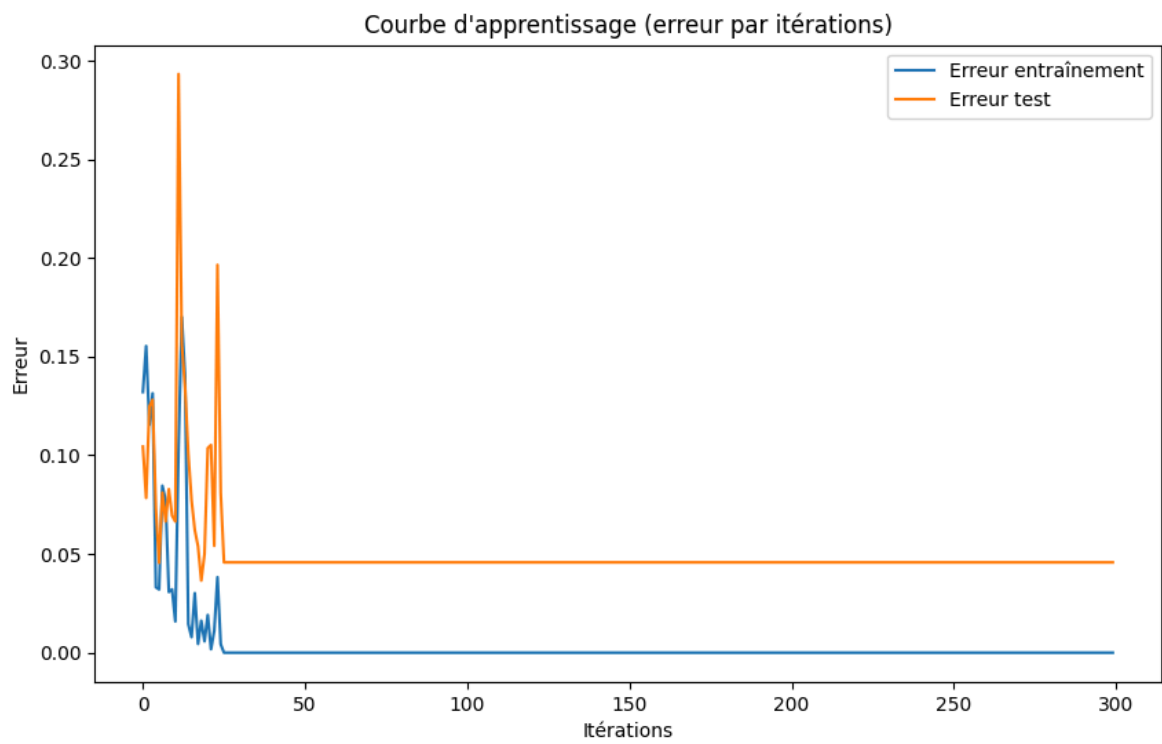
noisy_x_train = add_noise(datax, eps=0.1)
lin_stoch_noised10.fit_stochastic(noisy_x_train, datay, testx, testy)
print("matrice de poids w après apprentissage stochastique pour données bruitées")
show_usps(lin_stoch_noised10.w.reshape(16, 16))
display_error(lin_stoch_noised10.train_errors, lin_stoch_noised10.test_errors)

# Données bruitées 50%
lin_stoch_noised50 = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad, n_e
noisy_x_train = add_noise(datax, eps=0.5)
lin_stoch_noised50.fit_stochastic(noisy_x_train, datay, testx, testy)
print("matrice de poids w après apprentissage stochastique pour données bruitées")
show_usps(lin_stoch_noised50.w.reshape(16, 16))
display_error(lin_stoch_noised50.train_errors, lin_stoch_noised50.test_errors)

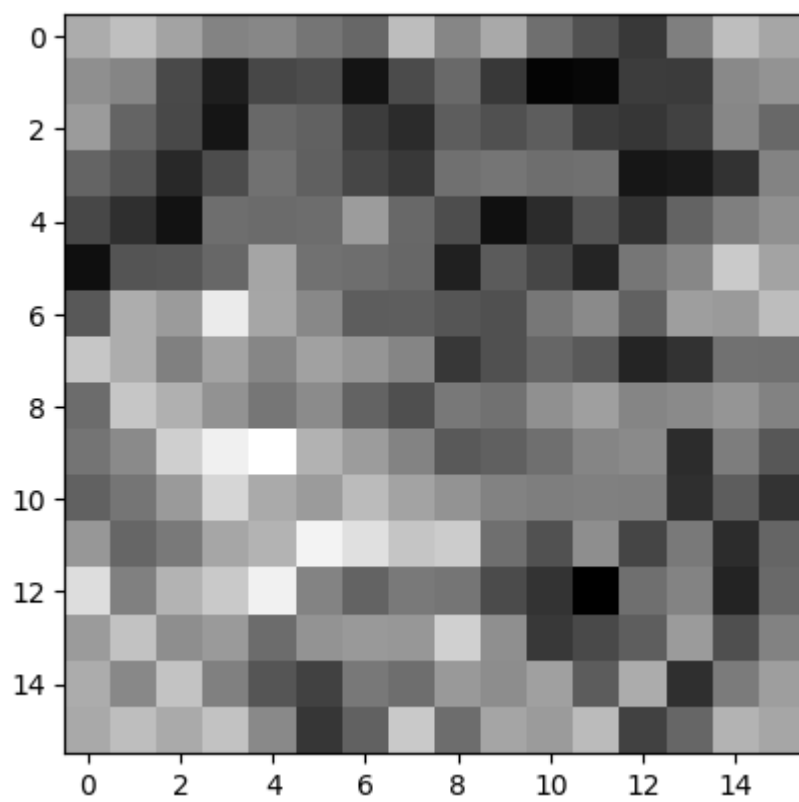
```

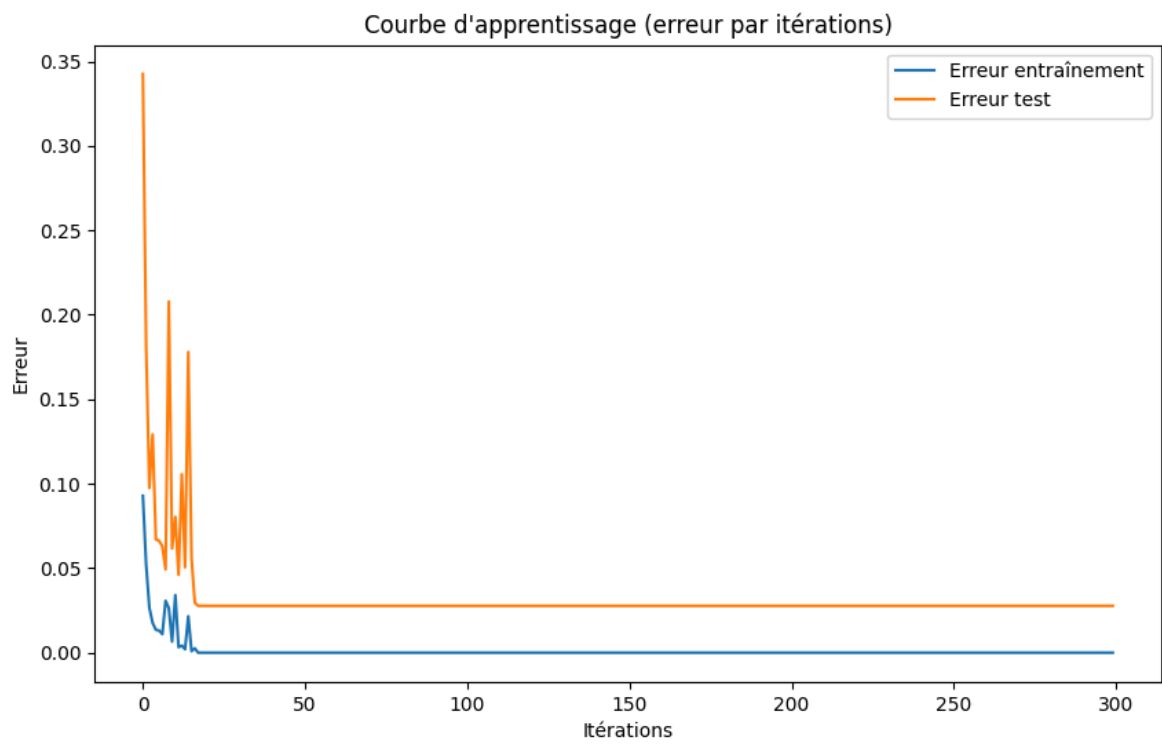
matrice de poids w après apprentissage stochastique:



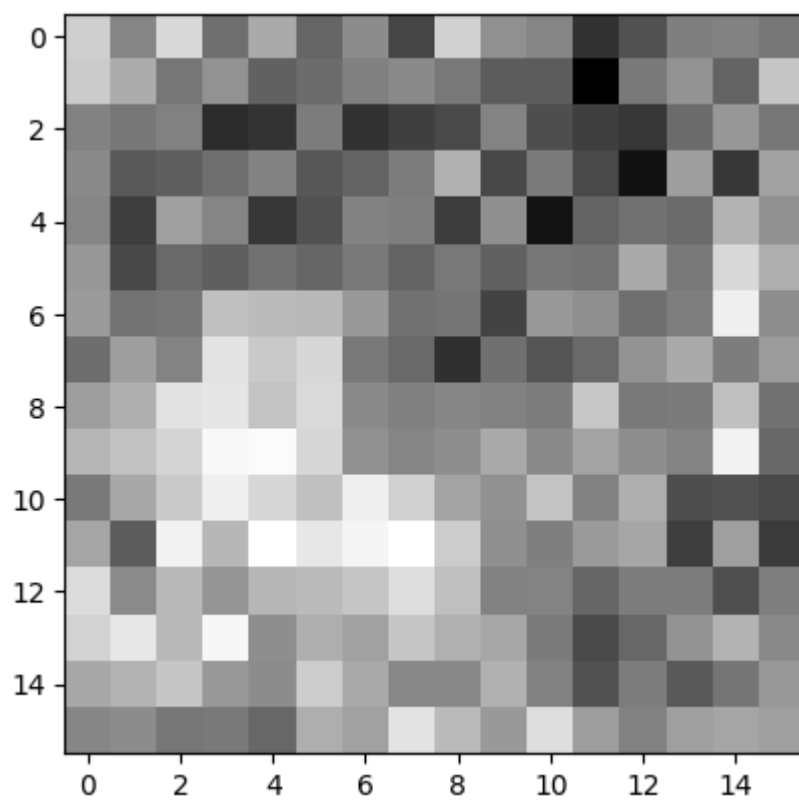


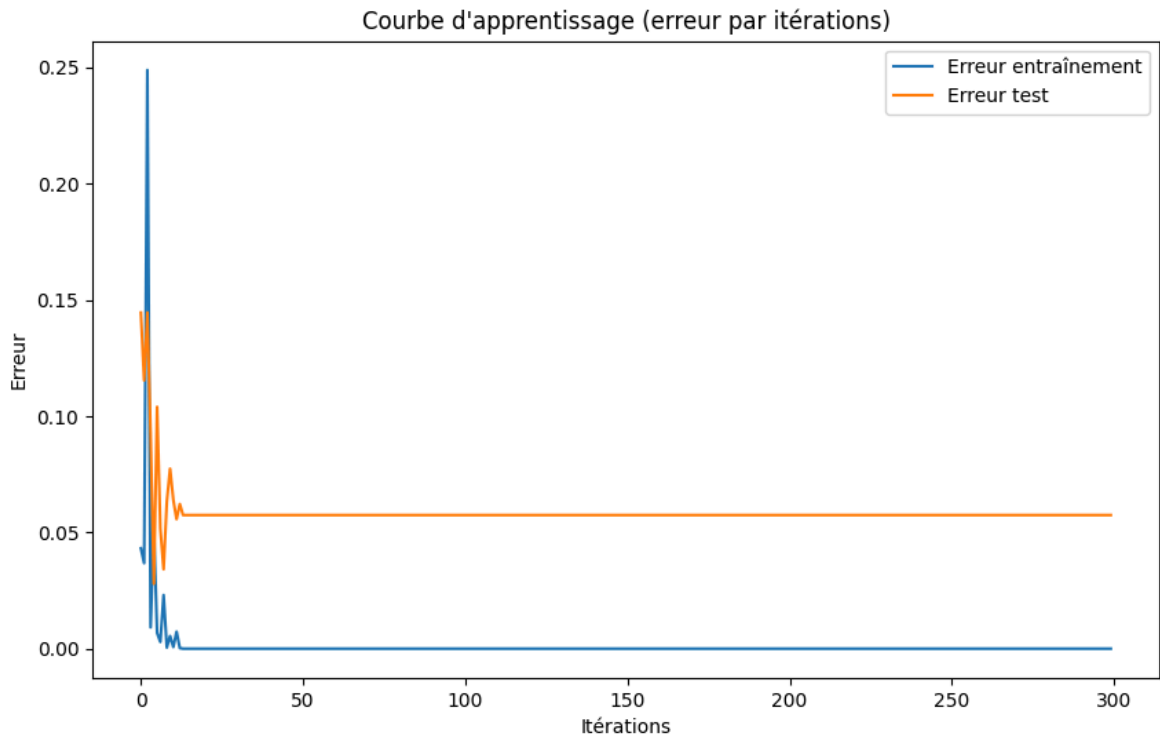
matrice de poids w après apprentissage stochastique pour données bruitées (10%):





matrice de poids w après apprentissage stochastique pour données bruitées (50%):





c) Apprentissage par mini-batch

```
In [31]: # Données non bruitées
lin_mbatch = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch=300)
lin_mbatch.fit_minibatch(datax, datay, testx, testy, batch_size=16)
print("matrice de poids w après apprentissage par minibatch: ")
show_usps(lin_mbatch.w.reshape(16, 16))
display_error(lin_mbatch.train_errors, lin_mbatch.test_errors)

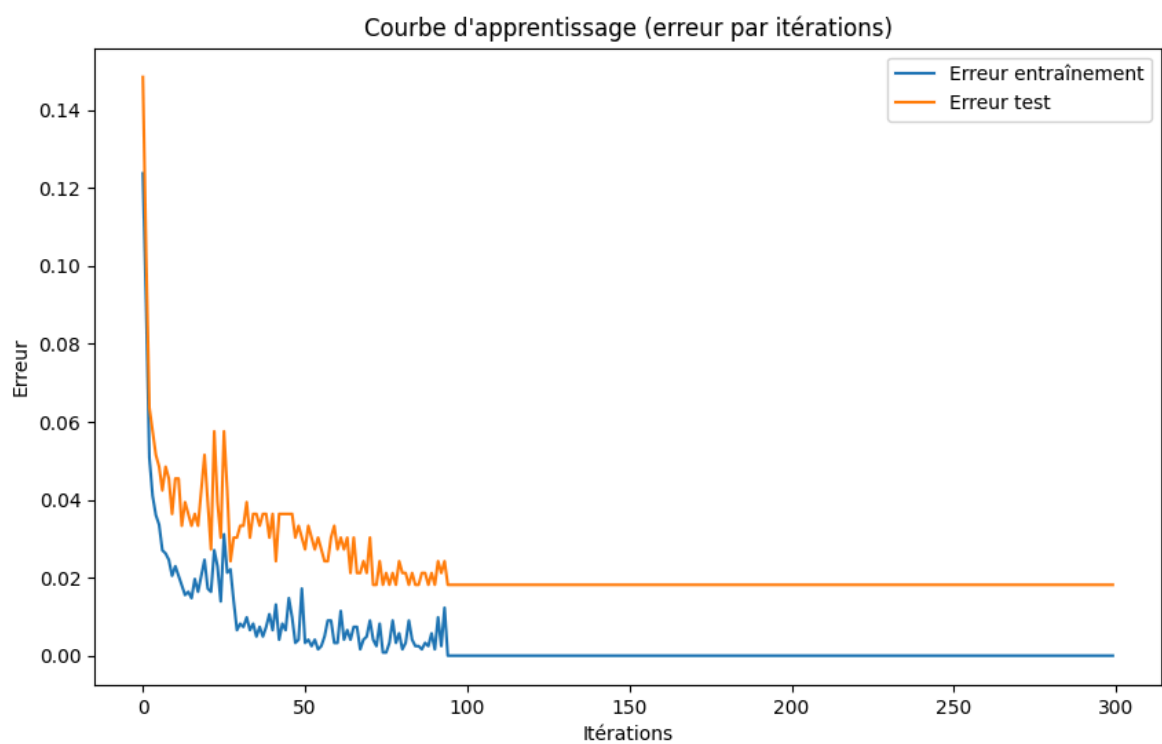
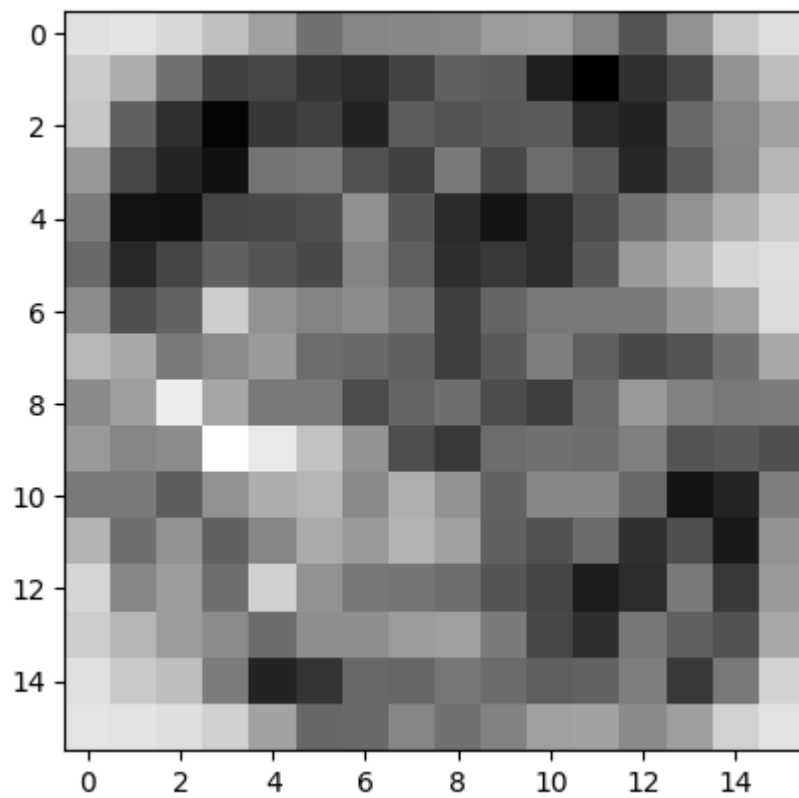
# Données bruitées 10%
lin_mbatch_noised10 = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad, n_
noisy_x_train = add_noise(datax, eps=0.1)
lin_mbatch_noised10.fit_minibatch(noisy_x_train, datay, testx, testy, batch_size

print("matrice de poids w après apprentissage par minibatch, pour des données br
show_usps(lin_mbatch_noised10.w.reshape(16, 16))
display_error(lin_mbatch_noised10.train_errors, lin_mbatch_noised10.test_errors)

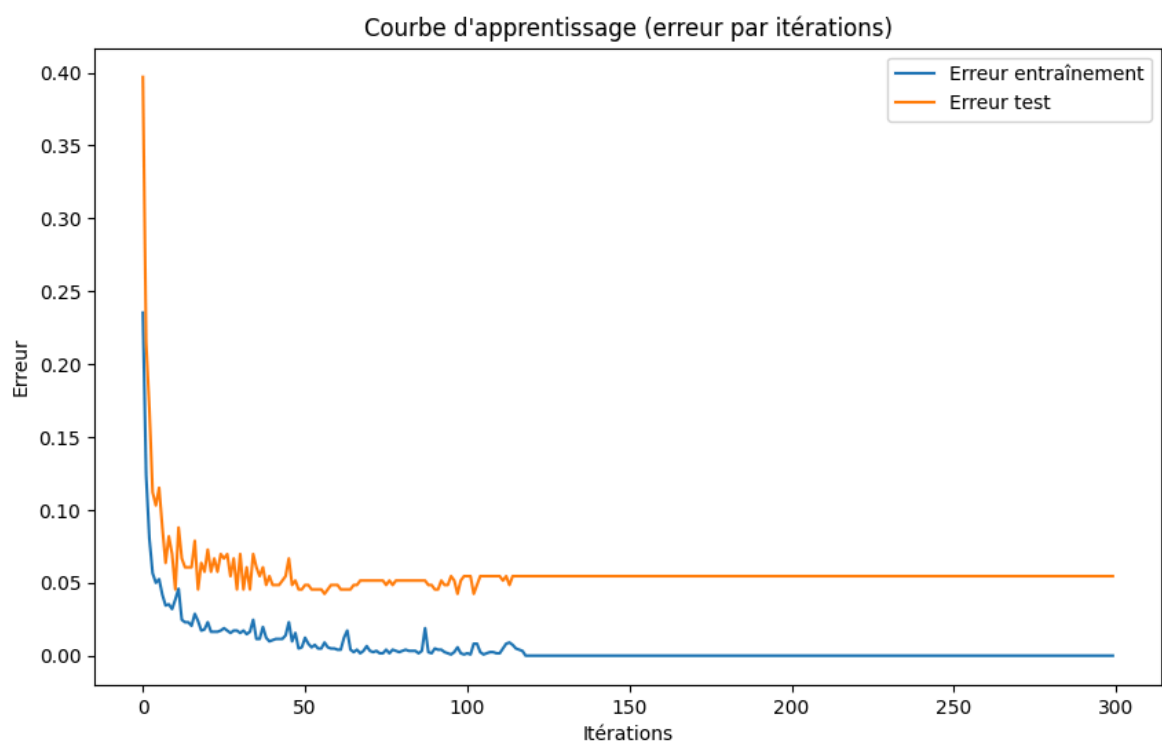
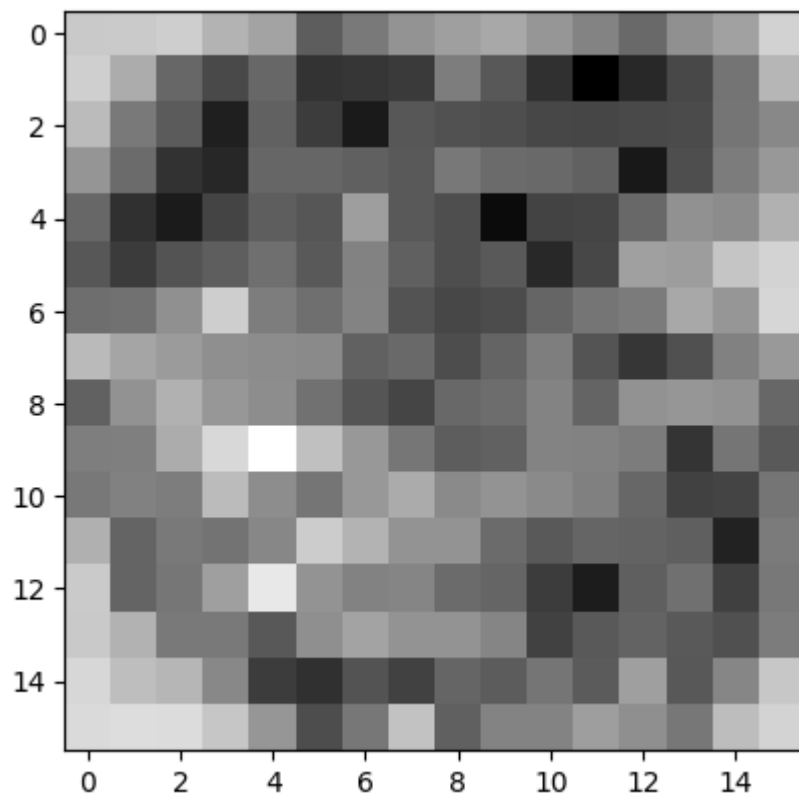
# Données bruitées 50%
lin_mbatch_noised50 = Lineaire2(loss=perceptron_loss, loss_g=perceptron_grad, n_
noisy_x_train = add_noise(datax, eps=0.5)
lin_mbatch_noised50.fit_minibatch(noisy_x_train, datay, testx, testy, batch_size

print("matrice de poids w après apprentissage par minibatch, pour des données br
show_usps(lin_mbatch_noised50.w.reshape(16, 16))
display_error(lin_mbatch_noised50.train_errors, lin_mbatch_noised50.test_errors)
```

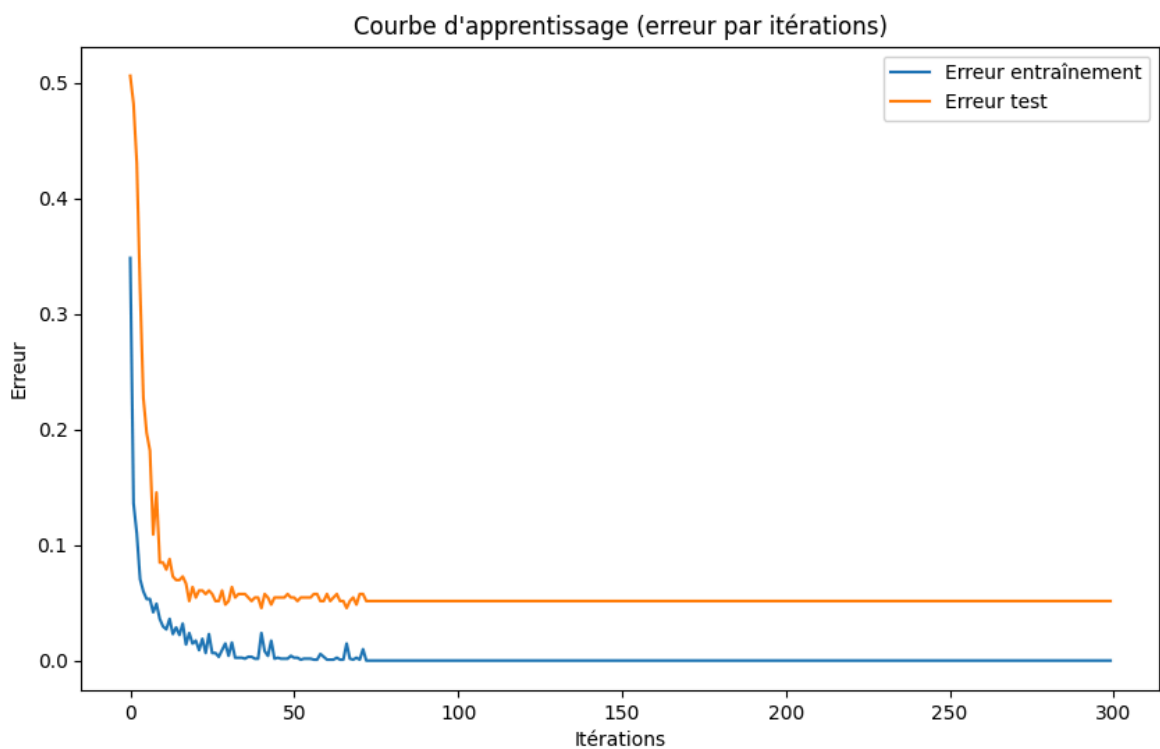
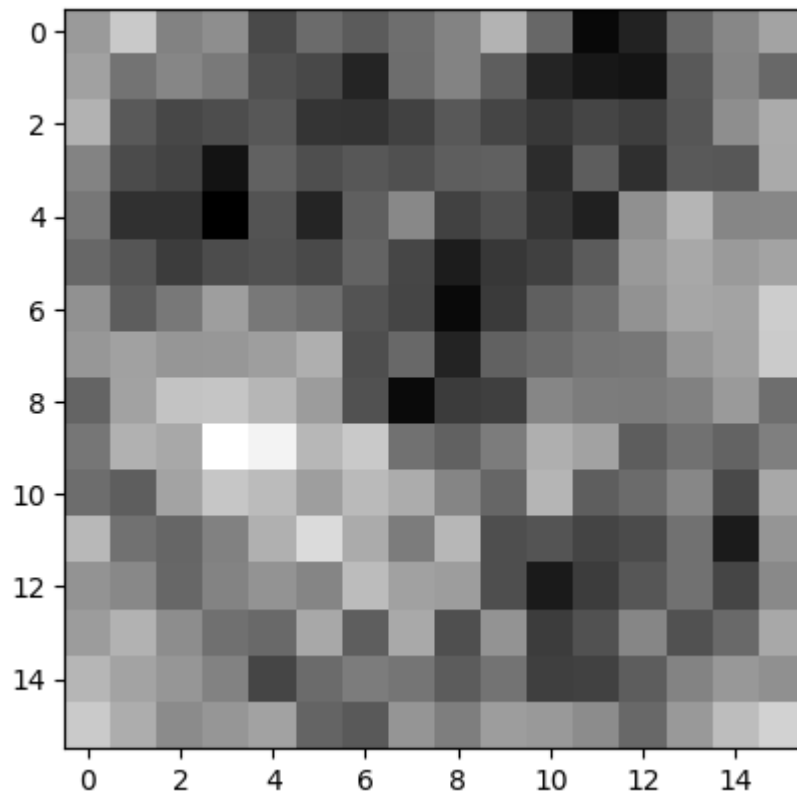
matrice de poids w après apprentissage par minibatch:



matrice de poids w après apprentissage par minibatch, pour des données bruitées (10%) :



matrice de poids w après apprentissage par minibatch, pour des données bruitées (50%) :



d) Comparaisons des vitesses de convergence selon la technique de descente de gradient (batch, mini-batch, stochastique) pour des données non bruitées

```
In [ ]: def compare_w(w_batch, w_stoch, w_mbatch) :
        """
        Compare the learned weights for batch, stochastic, and minibatch gradient de
        Params:
            w_batch (np.ndarray): learned weights from batch gradient descent
            w_stoch (np.ndarray): learned weights from stochastic gradient descent
```

```

w_mbatch (np.ndarray): learned weights from minibatch gradient descent
"""

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.title("Batch")
plt.imshow(w_batch.reshape(16, 16), cmap='gray')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.title("Stochastique")
plt.imshow(w_stoch.reshape(16, 16), cmap='gray')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.title("Mini-batch")
plt.imshow(w_mbatch.reshape(16, 16), cmap='gray')
plt.axis('off')

plt.suptitle("Comparaison des poids W appris")
plt.tight_layout()
plt.show()

```

```

In [ ]: def compare_learning_curves(error_batch_train, error_batch_test, error_stoch_train, error_stoch_test, error_mbatch_train, error_mbatch_test):
    """
    Compare the learning curves of batch, stochastic, and minibatch gradient descent.

    Params:
        error_batch_train (list): training error for batch gradient descent
        error_batch_test (list): test error for batch gradient descent
        error_stoch_train (list): training error for stochastic gradient descent
        error_stoch_test (list): test error for stochastic gradient descent
        error_mbatch_train (list): training error for minibatch gradient descent
        error_mbatch_test (list): test error for minibatch gradient descent
    """

    plt.figure(figsize=(10, 6))

    # Batch
    plt.plot(error_batch_train, label="Train - Batch", color='blue')
    plt.plot(error_batch_test, label="Test - Batch", color='skyblue')

    # Stochastique
    plt.plot(error_stoch_train, label="Train - Stochastique", color='green')
    plt.plot(error_stoch_test, label="Test - Stochastique", color='lightgreen')

    # Mini-batch
    plt.plot(error_mbatch_train, label="Train - Mini-batch", color='red')
    plt.plot(error_mbatch_test, label="Test - Mini-batch", color='orange')

    plt.title("Courbes d'erreur par méthode d'apprentissage")
    plt.xlabel("Itérations (epochs)")
    plt.ylabel("Erreur")
    plt.legend()
    plt.grid(True)
    plt.show()

```

```

In [32]: # Données non bruitées
print("Données non bruitées")

```

```

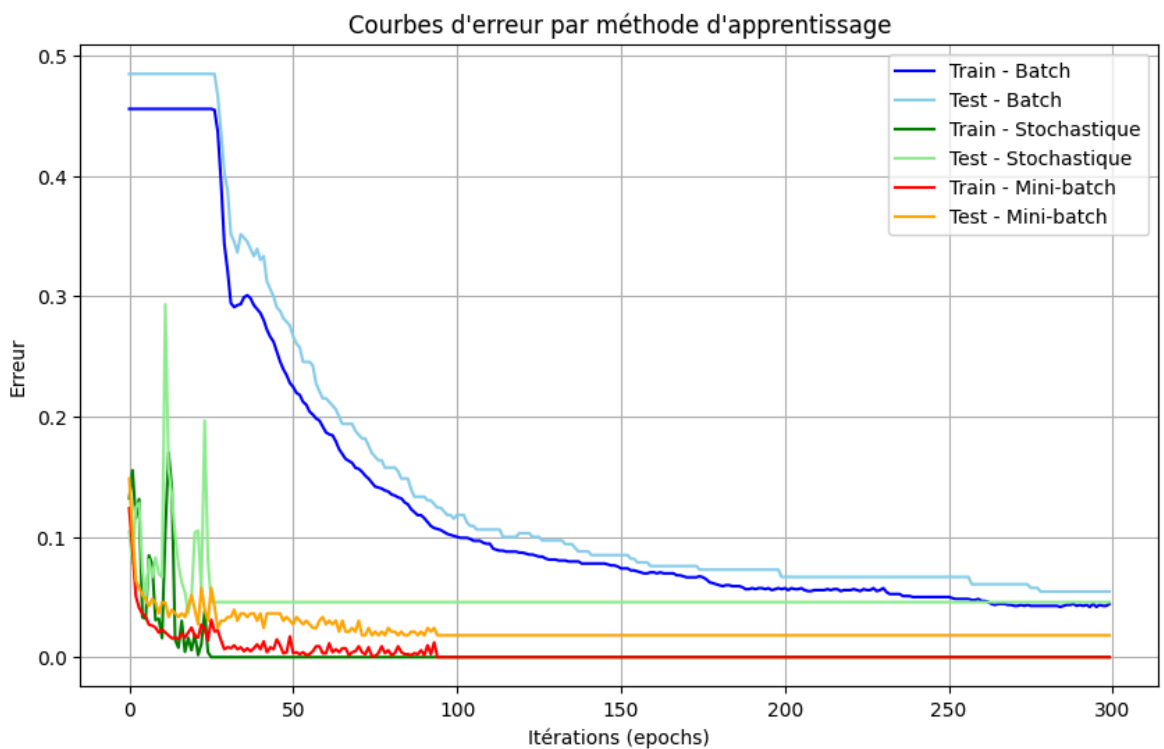
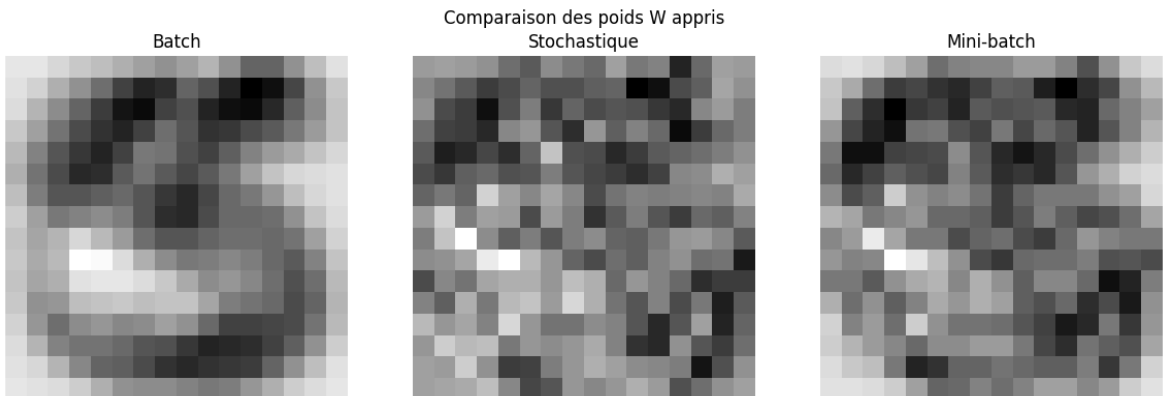
compare_w(lin_batch.w, lin_stoch.w, lin_mbatch.w)
compare_learning_curves(lin_batch.train_errors, lin_batch.test_errors, lin_stoch

# Données bruitées 10%
print("Données bruitées 10%")
compare_w(lin_batch_noised10.w, lin_stoch_noised10.w, lin_mbatch_noised10.w)
compare_learning_curves(lin_batch_noised10.train_errors, lin_batch_noised10.test

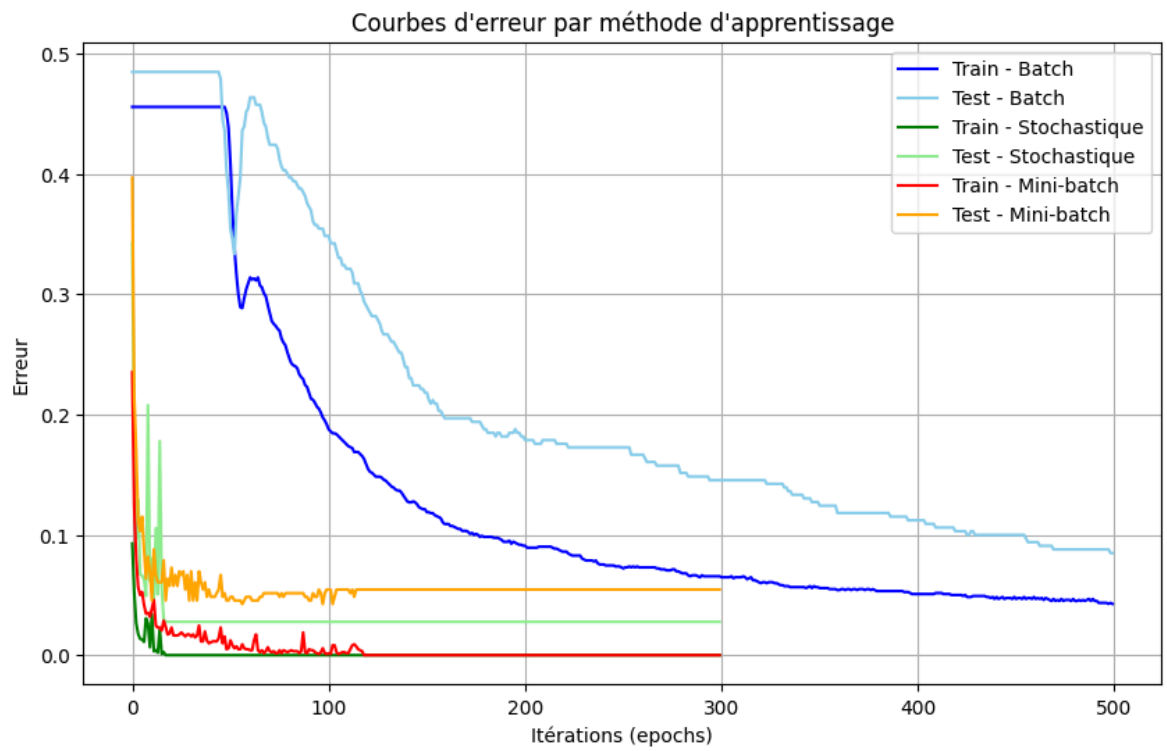
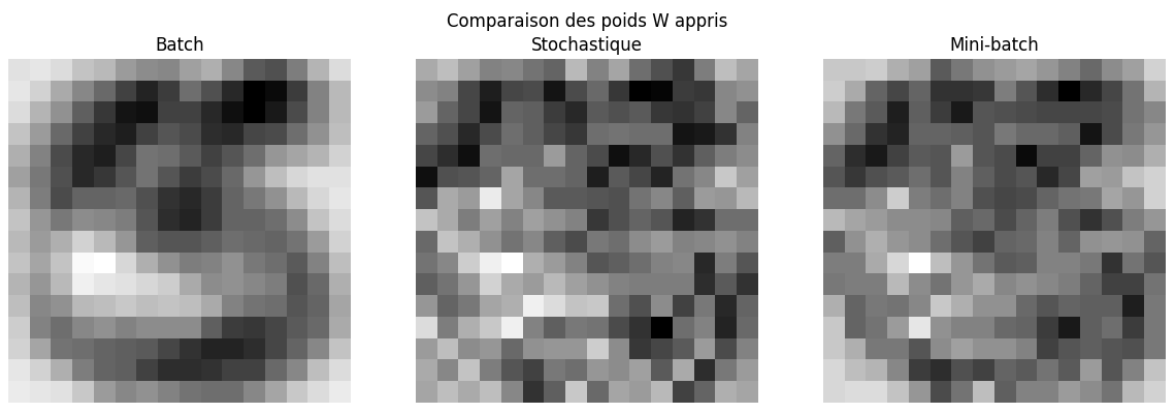
# Données bruitées 50%
print("Données bruitées 50%")
compare_w(lin_batch_noised50.w, lin_stoch_noised50.w, lin_mbatch_noised50.w)
compare_learning_curves(lin_batch_noised50.train_errors, lin_batch_noised50.test

```

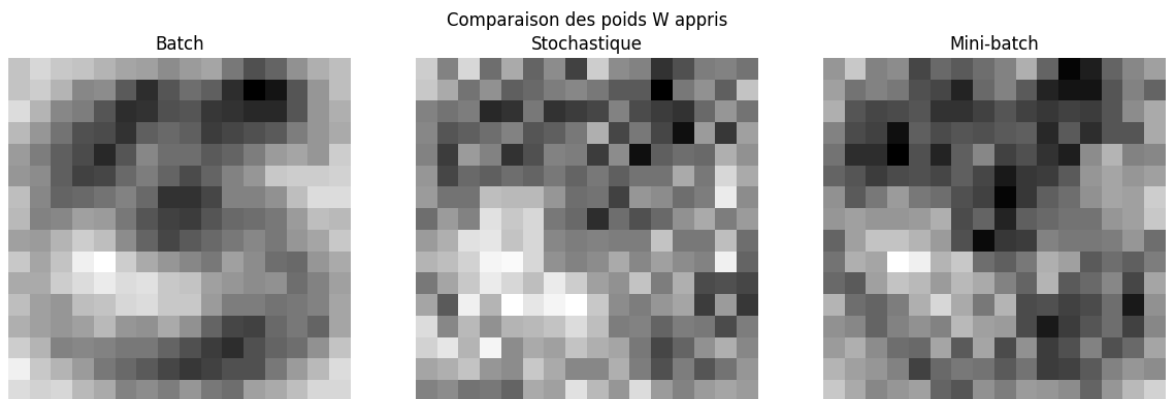
Données non bruitées

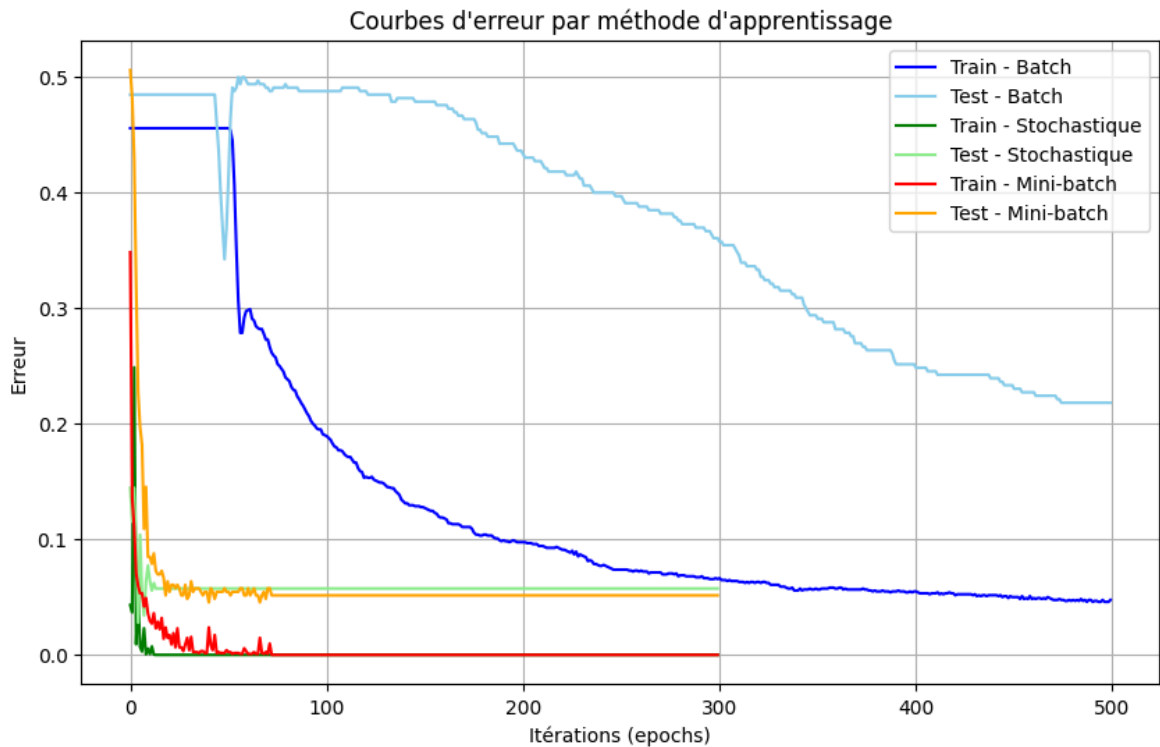


Données bruitées 10%



Données bruitées 50%





Interprétation des résultats

Les courbes d'erreurs pour les données train et tests nous permettent d'analyser les performances d'apprentissage des perceptrons linéaires.

En modifiant la technique de descente de gradient utilisée pour l'entraînement, on peut ainsi comparer les performances d'un même réseau de neurone pourvu d'une technique de descente de gradient différente (batch, mini-batch, stochastique) et donc élire potentiellement une technique de gradient plus optimale que les autres (par exemple).

Ici, on observe un apprentissage sur 300 itérations avec un pas d'apprentissage de $\epsilon = 0.07$.

- L'apprentissage par batch est celui qui met le plus d'itérations à converger (sur des données bruitées ou non). Il semble commencer à se stabiliser vraiment à 300 itérations pour les données non bruitées, et n'est toujours pas stabilisé à 300 itérations pour les données bruitées. Ce temps de convergence augmente proportionnellement à la proportion de bruit des données. La performance de test est plutôt proche de celle de train et sont inférieures à 0.1. D'un point de vue du temps d'exécution, l'apprentissage par batch est rapide.
- L'apprentissage stochastique est celui qui converge le plus rapidement. A 25 itérations, pour des données non bruitées il semble déjà stabilisé. Ce temps de convergence semble rester constant même en augmentant la proportion de bruit des données. Ses performances d'erreur sont bonnes ($< 0.1\%$ d'erreur) mais inférieures au mini batch. En terme de temps de calcul de l'algorithme, il est le plus long des trois algorithmes testés, à l'exécution.
- L'apprentissage par mini-batch semble être l'algorithme d'apprentissage le plus intermédiaire. Il est à la fois rapide en terme d'exécution, présente de bonnes performances d'apprentissage avec des erreurs tests et trains proches entre elles et

proche de 0 et converge assez rapidement (<100 itérations pour un bruit de 50% sur les données).

III - Projection et pénalisation

III - 1) Fonctions

a) Projections

```
In [ ]: def proj_poly(datax):
        """
        Project input data into polynomial features (linear, squared, and interaction terms).

        Params:
            datax (np.ndarray): input features

        Returns:
            np.ndarray: augmented feature set with bias, linear, squared, and interaction terms
        """

        n, d = datax.shape
        bias = np.ones((n, 1))
        lin = datax
        squares = datax**2
        interactions = []
        for i in range(d):
            for j in range(i+1, d):
                interactions.append((datax[:, i]*datax[:, j]).reshape(-1, 1))
        interactions = np.hstack(interactions) if interactions else np.empty((n, 0))
        return np.hstack([bias, lin, squares, interactions])
```

```
In [ ]: def proj_biais(datax):
        """
        Add a bias term (column of ones) to the input data.

        Params:
            datax (np.ndarray): input features

        Returns:
            np.ndarray: input data with an additional bias column
        """

        n = datax.shape[0]
        return np.hstack([np.ones((n, 1)), datax])
```

```
In [ ]: def proj_gauss(datax, base, sigma):
        """
        Project input data onto a Gaussian radial basis function (RBF) space.

        Params:
            datax (np.ndarray): input features
            base (np.ndarray): centers of the Gaussian basis functions
            sigma (float): standard deviation of the Gaussian function

        Returns:
```



```

        np.ndarray: projected data using Gaussian RBF kernels
    """

    n_samples = datax.shape[0]
    n_bases = base.shape[0]

    proj = np.zeros((n_samples, n_bases))
    for i in range(n_bases):
        distances = np.linalg.norm(datax - base[i], axis=1)
        proj[:, i] = np.exp(-(distances ** 2) / (2 * sigma ** 2))

    return proj

```

b) Apprentissage

```

In [ ]: class Lineaire3:
    def __init__(self, loss=perceptron_loss, loss_g=perceptron_grad, proj=None,
    """
        Linear classifier with optional projection (e.g., polynomial or Gaussian

        Params:
            loss (callable): loss function (default is perceptron_loss)
            loss_g (callable): gradient of the loss function (default is percept
            proj (callable, optional): projection function (e.g., for polynomial
            n_epoch (int): number of training epochs
            eps (float): learning rate
        """

        self.loss = loss
        self.loss_g = loss_g
        self.proj = proj
        self.n_epoch = n_epoch
        self.eps = eps
        self.w = None
        self.loss_history = []
        self.train_errors = []
        self.test_errors = []
        self.gauss_base = None

    def predict(self, datax):
        """
        Predict class labels after applying the projection function.

        Params:
            datax (np.ndarray): input features

        Returns:
            np.ndarray: predicted labels (+1 or -1)
        """

        datax = self.proj(datax)
        return np.sign(datax @ self.w)

    def score(self, datax, datay):
        """
        Compute accuracy score of the model after projection.

        Params:
            datax (np.ndarray): input features

```

```

        datay (np.ndarray): true labels

Returns:
    float: mean accuracy
"""

y_pred = self.predict(datax)
return (y_pred == datay).mean()

def fit(self, datax, datay, testx=None, testy=None):
    """
    Train the linear model with projection using gradient descent.

    Params:
        datax (np.ndarray): training input features
        datay (np.ndarray): training labels
        testx (np.ndarray, optional): test input features
        testy (np.ndarray, optional): test labels

    Returns:
        list: history of loss values during training
    """

    datax_proj = self.proj(datax)
    if testx is not None:
        testx_proj = self.proj(testx)
    else:
        testx_proj = None

    n, d = datax_proj.shape
    self.w = np.ones(d)
    self.loss_history = []
    self.train_errors = []
    self.test_errors = []

    for i in range(self.n_epoch):
        loss_val = self.loss(self.w, datax_proj, datay)
        self.loss_history.append(loss_val)
        grad = self.loss_g(self.w, datax_proj, datay)
        new_w = self.w - self.eps * grad
        self.w = new_w

        if testx is not None and testy is not None:
            train_error = 1 - self.score(datax, datay) # calcul de l'erreur
            test_error = 1 - self.score(testx, testy)
            self.train_errors.append(train_error) #stock it
            self.test_errors.append(test_error)

    return self.loss_history

```

III.2) Tests et résultats

a) générer des données

```

In [ ]: x0, y0 = gen_arti(epsilon=0.8)
        x0_train, x0_test, y0_train, y0_test = train_test_split(x0, y0, test_size=0.2, r
        x1, y1 = gen_arti(data_type=1, epsilon=0.8)
        x1_train, x1_test, y1_train, y1_test = train_test_split(x1, y1, test_size=0.2, r

```

```
x2, y2 = gen_arti(data_type=2, epsilon=0.8)
x2_train, x2_test, y2_train, y2_test = train_test_split(x2, y2, test_size=0.2, r
```

b) Projection polynomiales

In [202...

```
# apprentissage data 0
print("Apprentissage linéaire pour données 0 projetées poly")
pol0 = Lineaire3(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch=500, eps=
pol0.fit(x0_train, y0_train.ravel(), x0_test, y0_test.ravel())

plot_frontiere(x0, pol0.predict)
plot_data(x0, y0)
display_error(pol0.train_errors, pol0.test_errors)

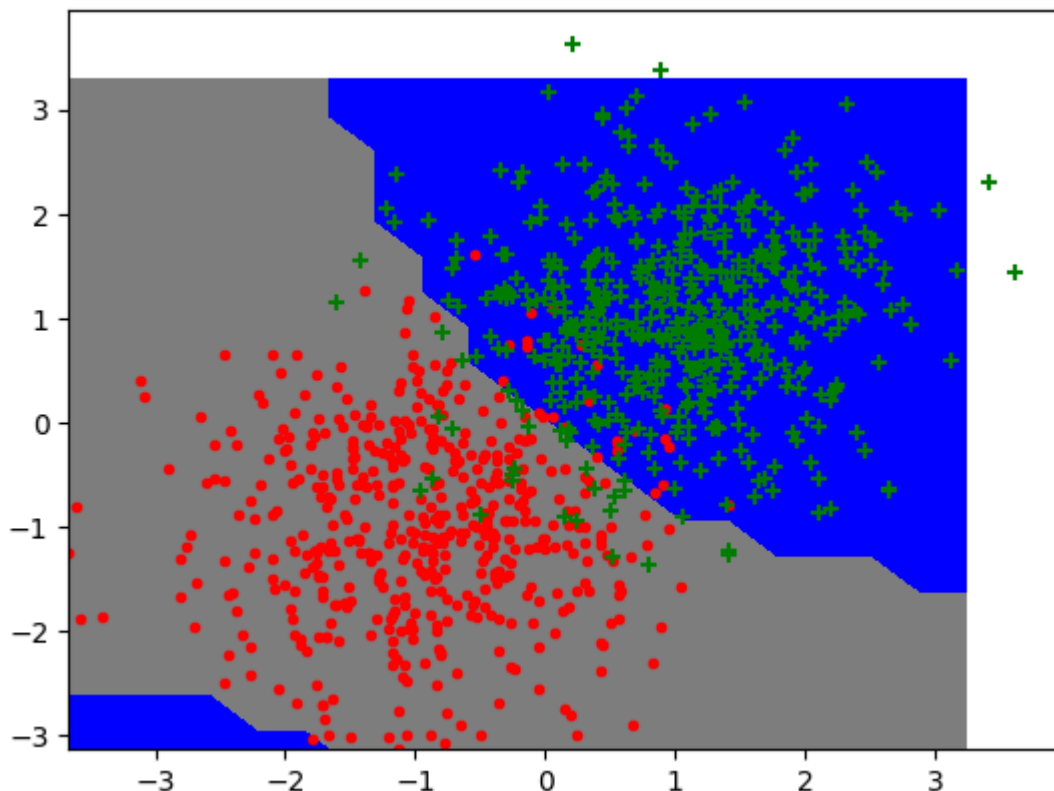
# apprentissage data 1
print("Apprentissage linéaire pour données 1 projetées poly")
pol1 = Lineaire3(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch=500, eps=
pol1.fit(x1_train, y1_train.ravel(), x1_test, y1_test.ravel())

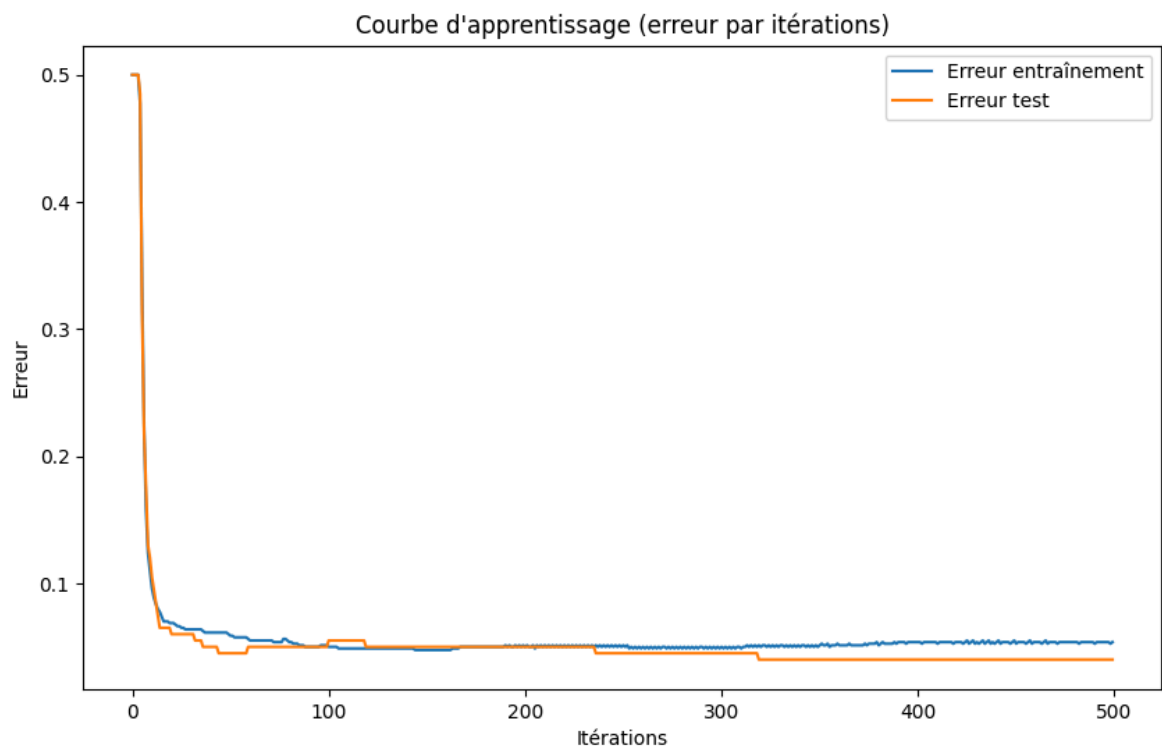
plot_frontiere(x1, pol1.predict)
plot_data(x1, y1)
display_error(pol1.train_errors, pol1.test_errors)

# apprentissage data 2
print("Apprentissage linéaire pour données 2 projetées poly")
pol2 = Lineaire3(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch=500, eps=
pol2.fit(x2_train, y2_train.ravel(), x2_test, y2_test.ravel())

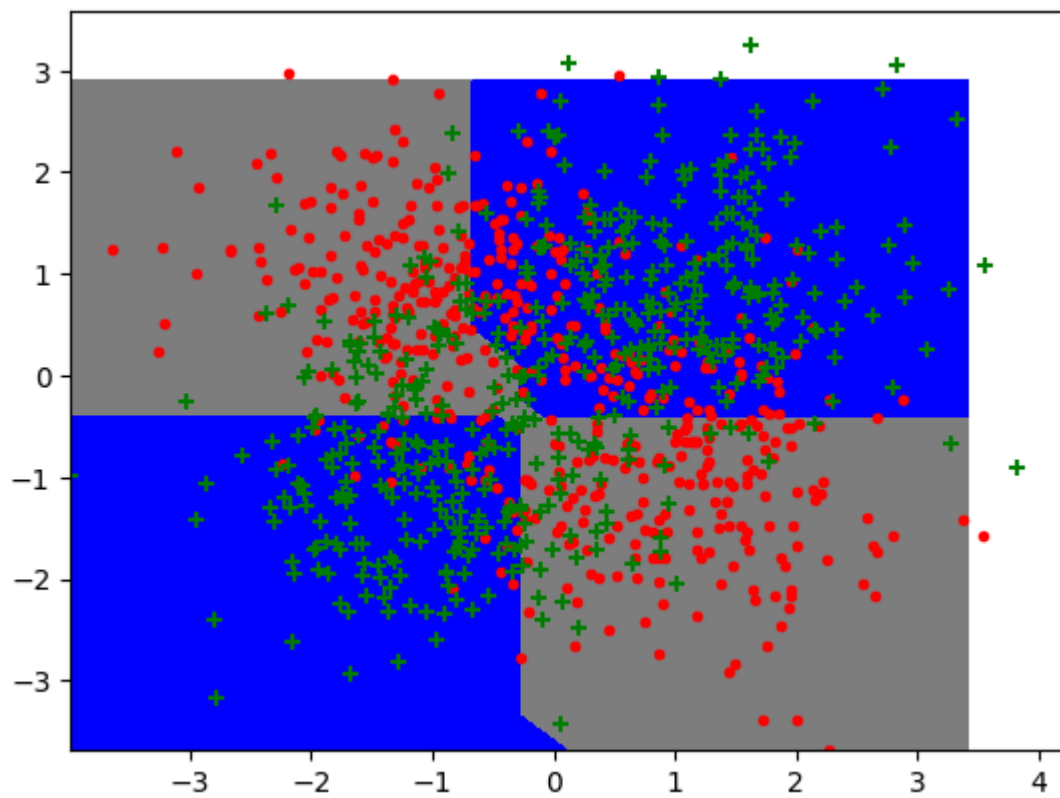
plot_frontiere(x2, pol2.predict)
plot_data(x2, y2)
display_error(pol2.train_errors, pol2.test_errors)
```

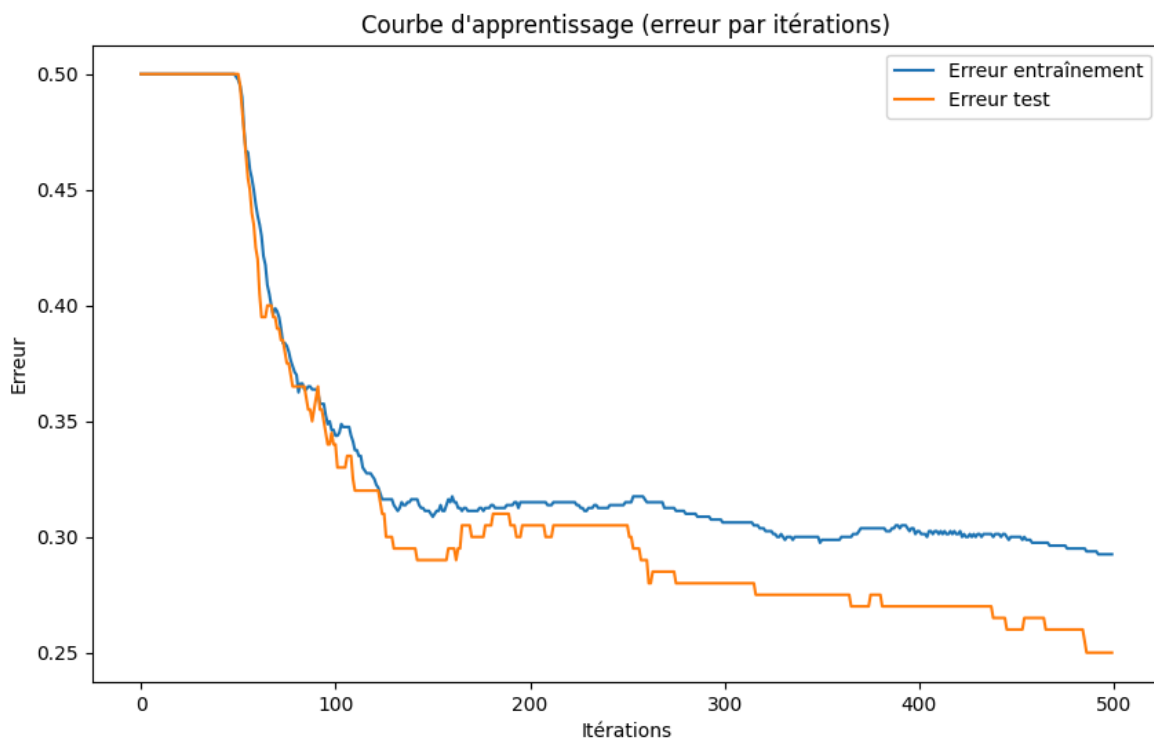
Apprentissage linéaire pour données 0 projetées poly



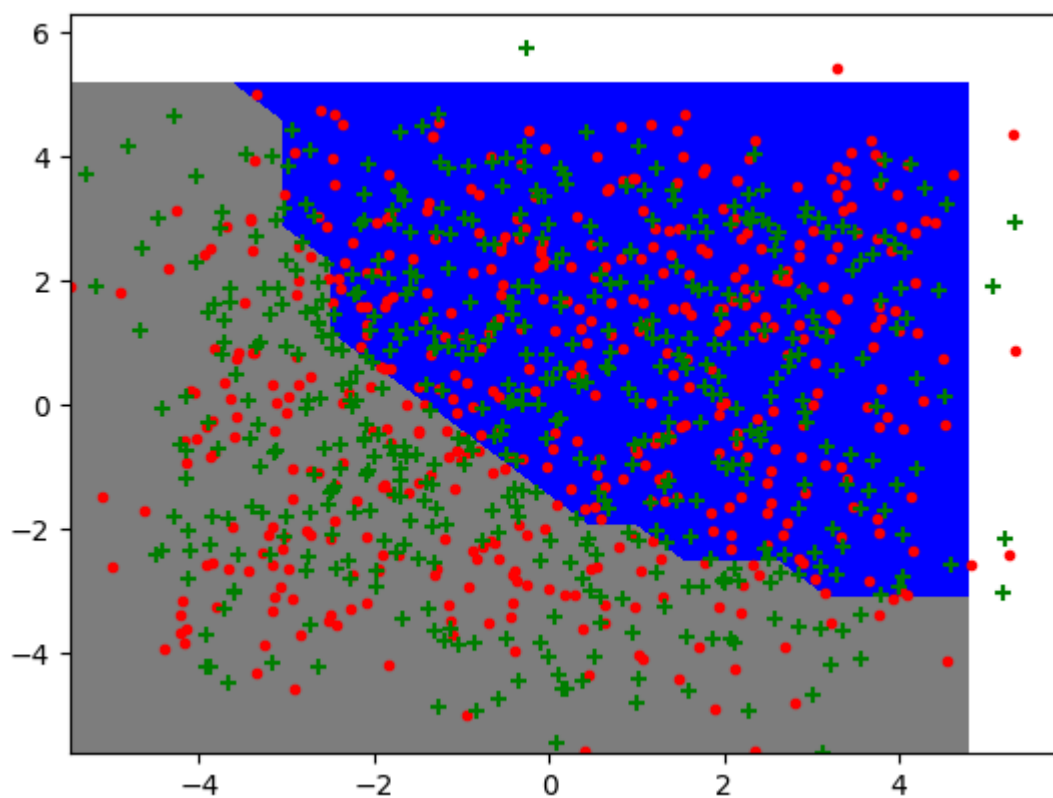


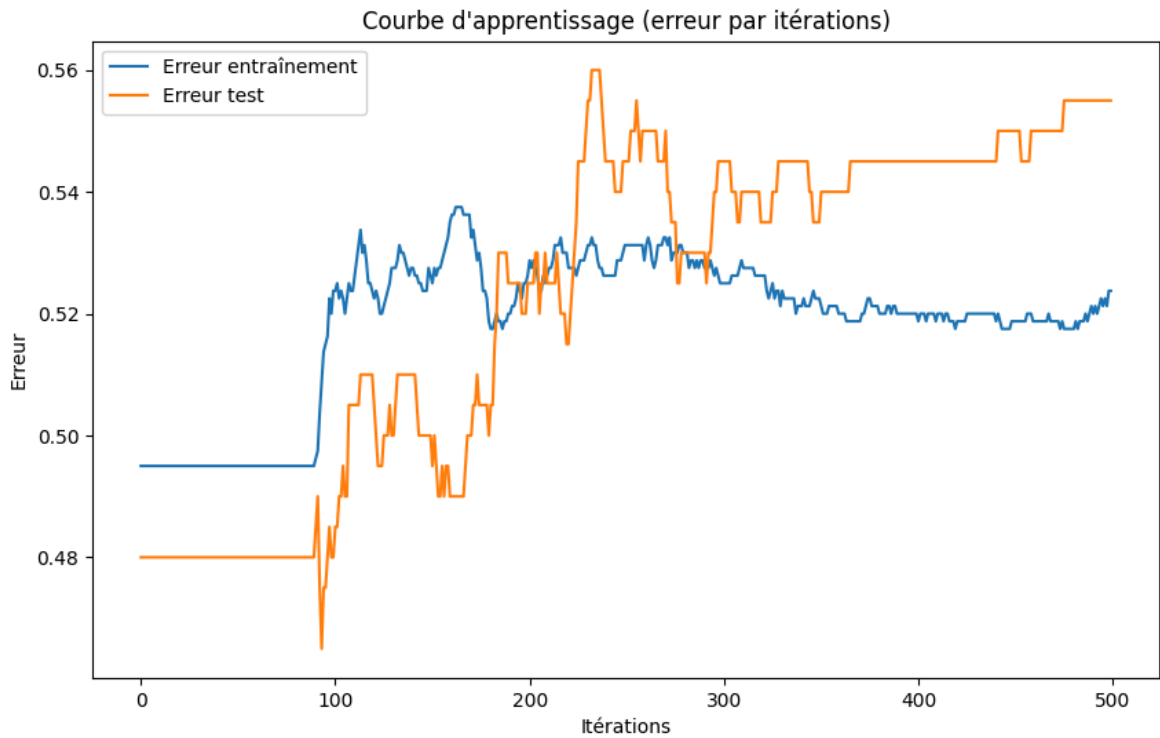
Apprentissage linéaire pour données 1 projetées poly





Apprentissage linéaire pour données 2 projetées poly





c) Projection gaussiennes

In [203...

```
# projection gaussienne
base_size = 100
index_b = np.random.choice(np.arange(1, datax.shape[0]), base_size)
base = datax[index_b]
for sigma in [0.5, 1, 1.5, 2] :
    print("Sigma = ", sigma)
    # apprentissage data 0
    gauss0 = Lineaire3(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch=500)
    gauss0.fit(x0_train, y0_train.ravel(), x0_test, y0_test.ravel())

    plot_frontiere(x0, gauss0.predict)
    plot_data(x0, y0)
    plt.show()
    display_error(gauss0.train_errors, gauss0.test_errors)

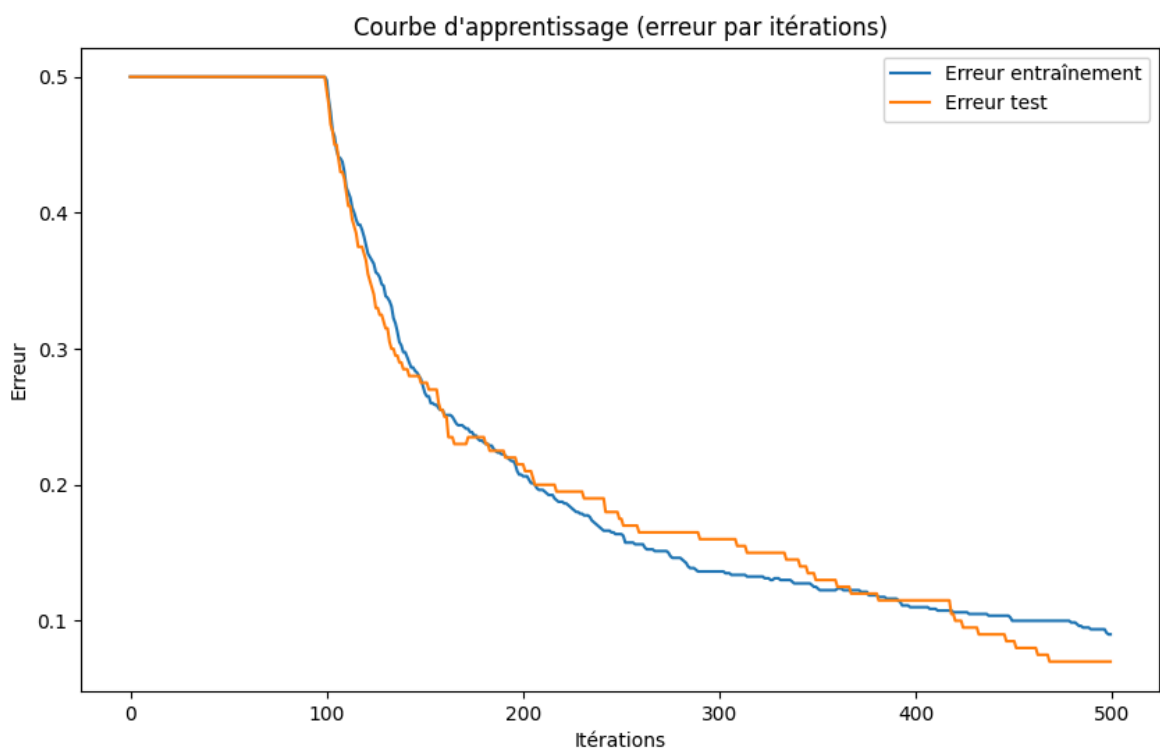
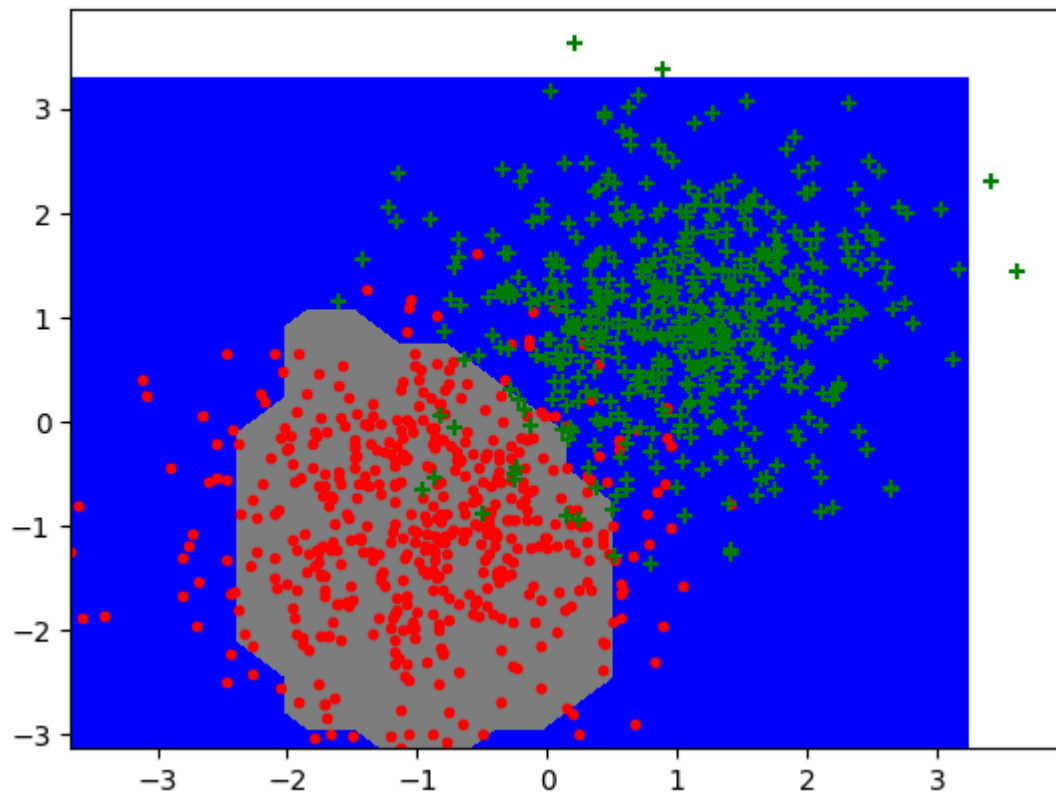
    # apprentissage data 1
    gauss1 = Lineaire3(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch=500)
    gauss1.fit(x1_train, y1_train.ravel(), x1_test, y1_test.ravel())

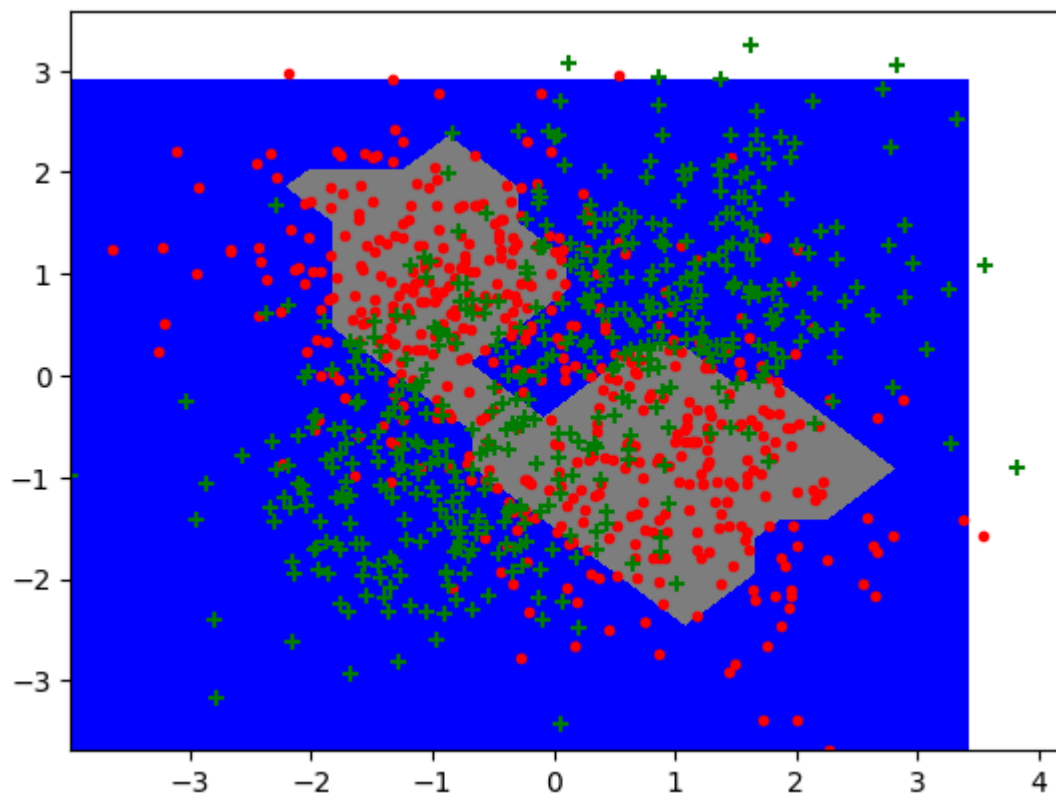
    plot_frontiere(x1, gauss1.predict)
    plot_data(x1, y1)
    plt.show()
    display_error(gauss1.train_errors, gauss1.test_errors)

    # apprentissage data 2
    gauss2 = Lineaire3(loss=perceptron_loss, loss_g=perceptron_grad, n_epoch=500)
    gauss2.fit(x2_train, y2_train.ravel(), x2_test, y2_test.ravel())

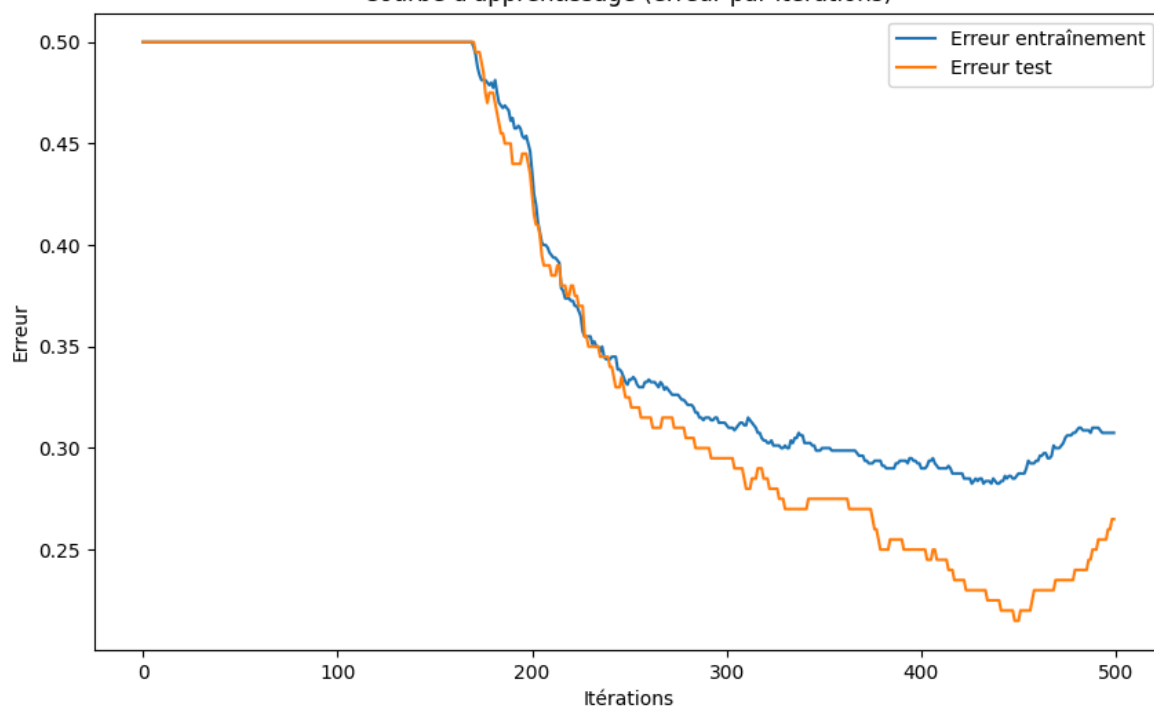
    plot_frontiere(x2, gauss2.predict)
    plot_data(x2, y2)
    plt.show()
    display_error(gauss2.train_errors, gauss2.test_errors)
```

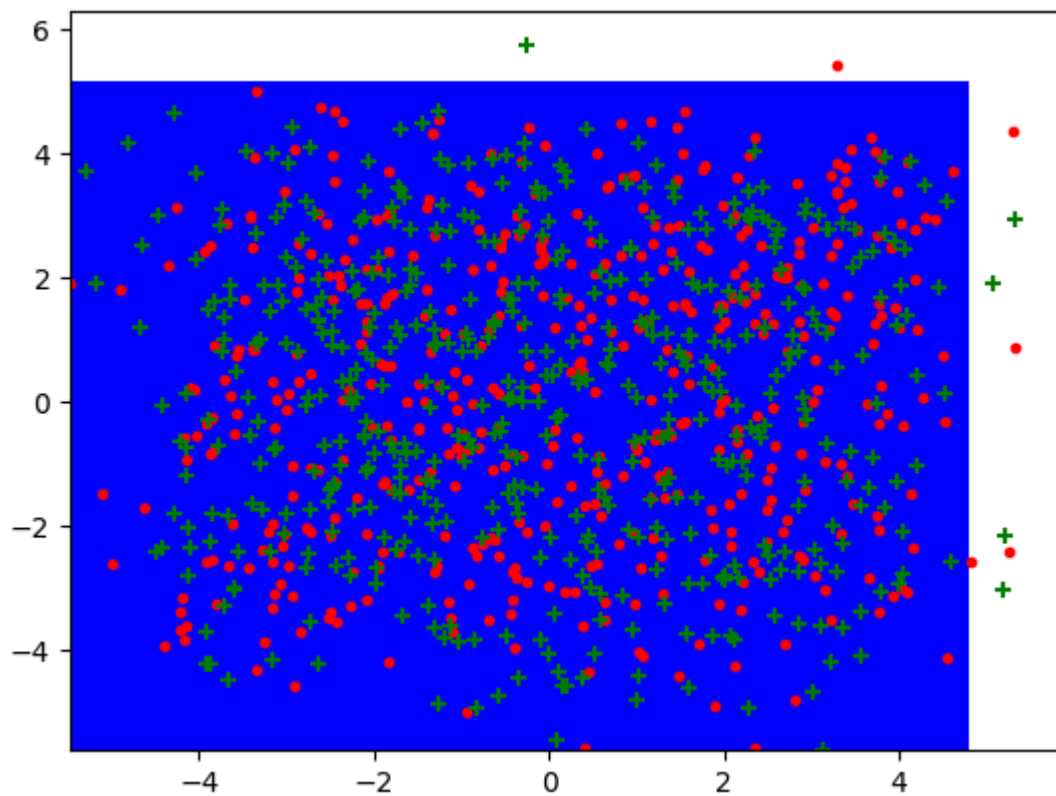
Sigma = 0.5



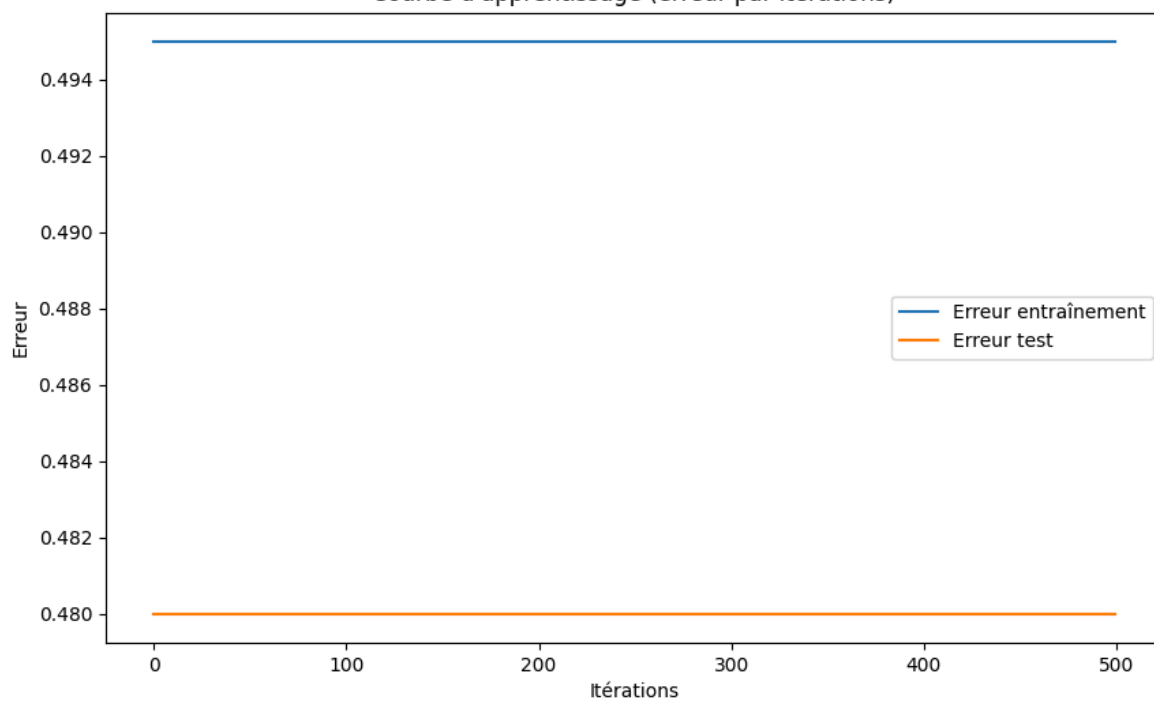


Courbe d'apprentissage (erreur par itérations)

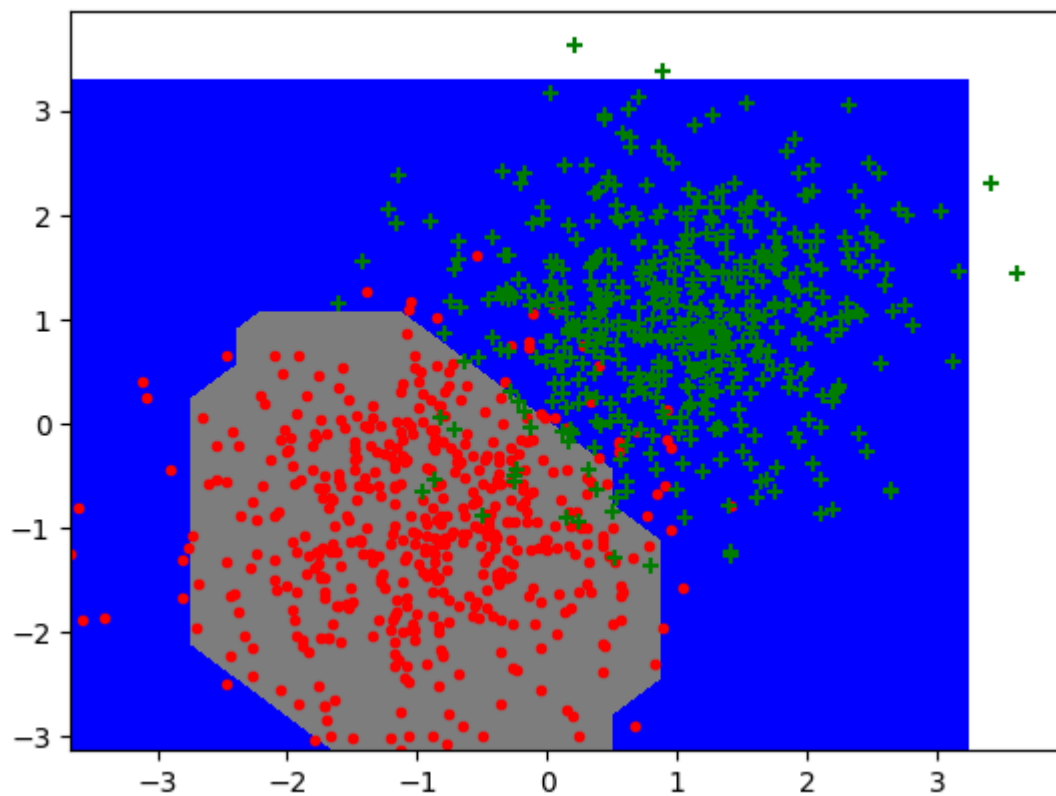




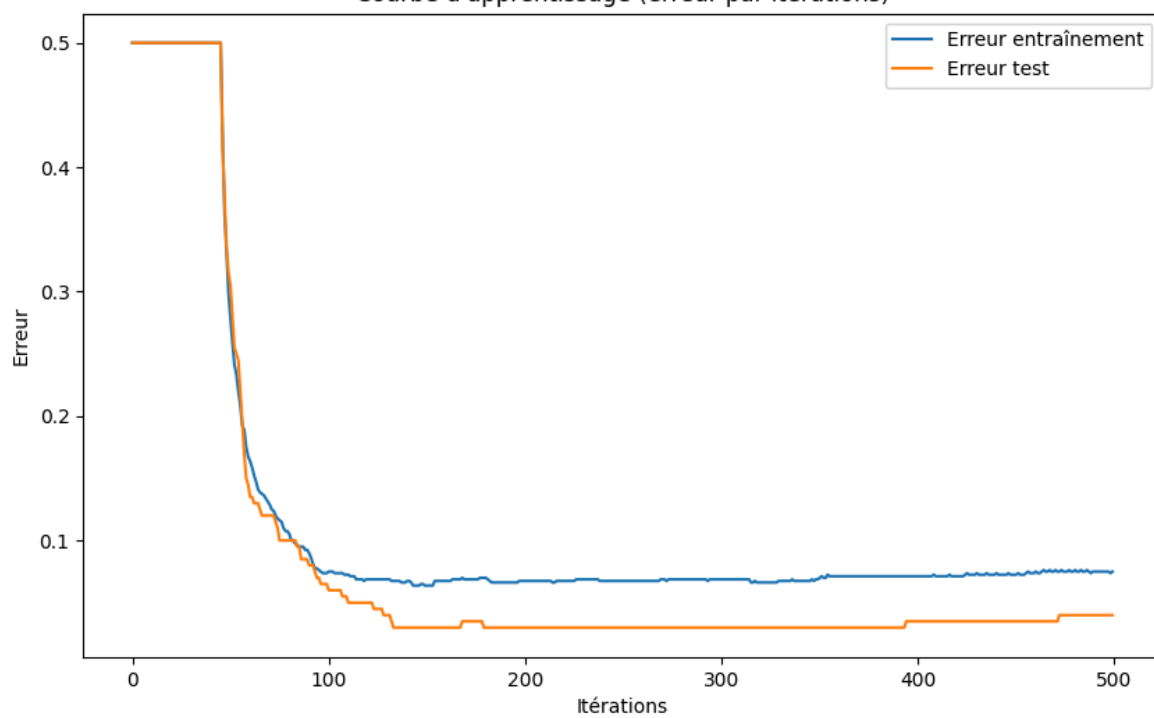
Courbe d'apprentissage (erreur par itérations)

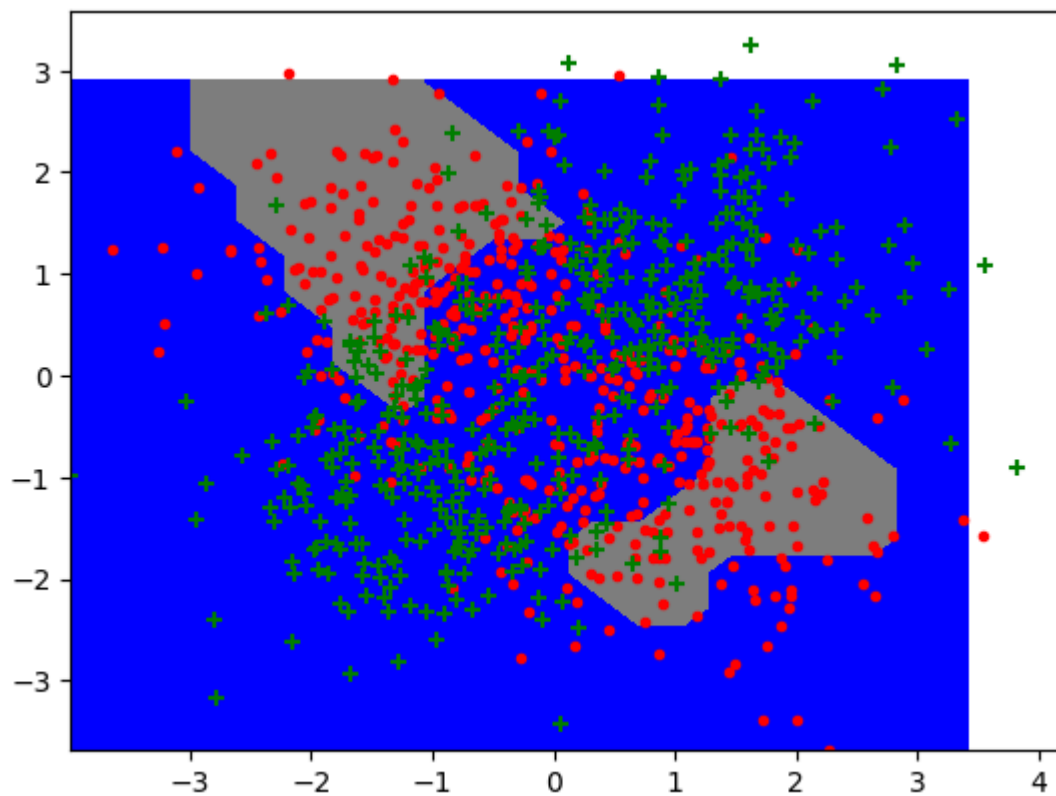


Sigma = 1

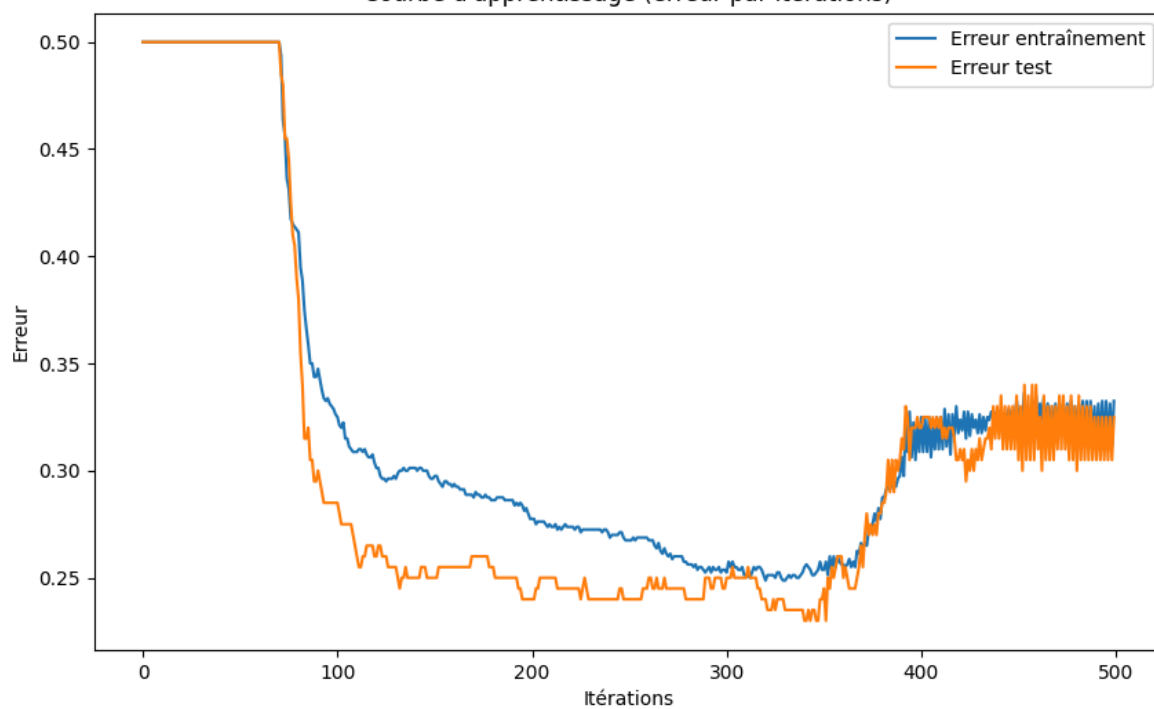


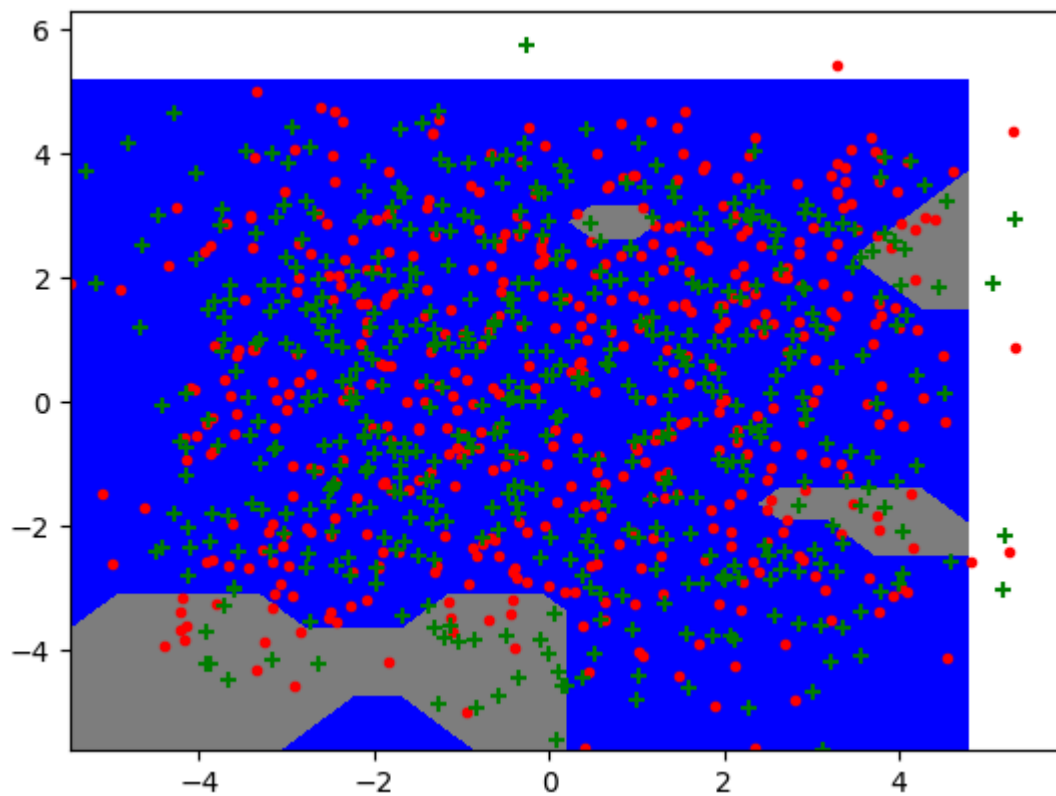
Courbe d'apprentissage (erreur par itérations)



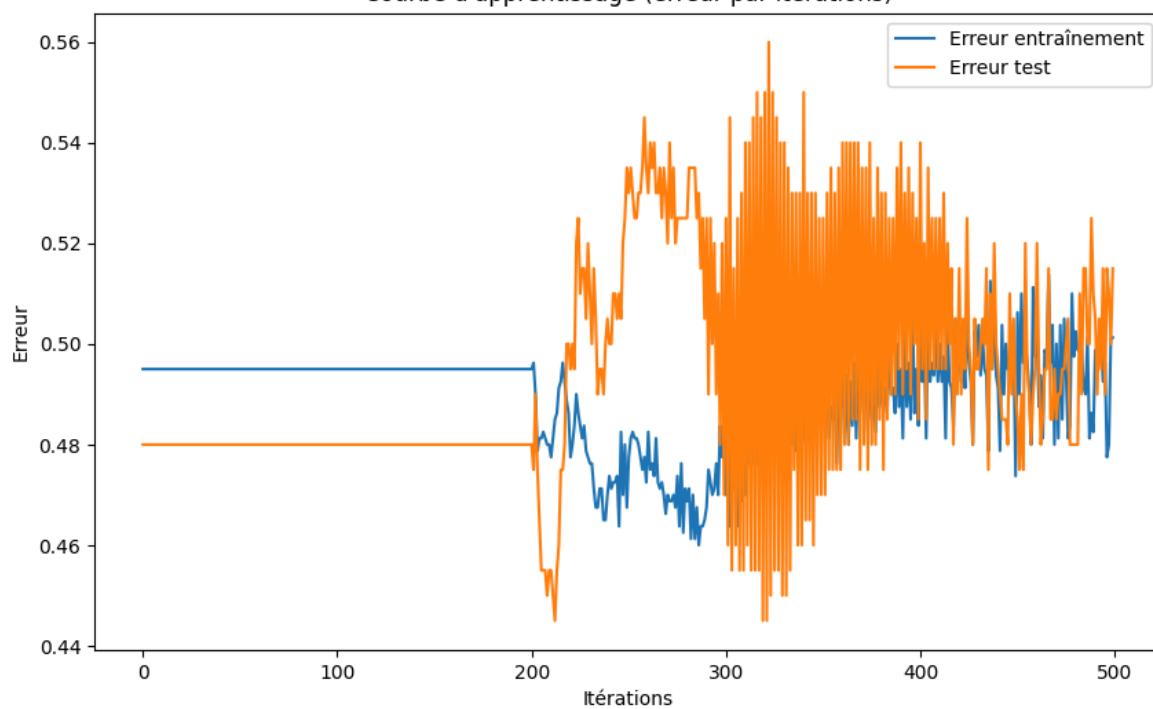


Courbe d'apprentissage (erreur par itérations)

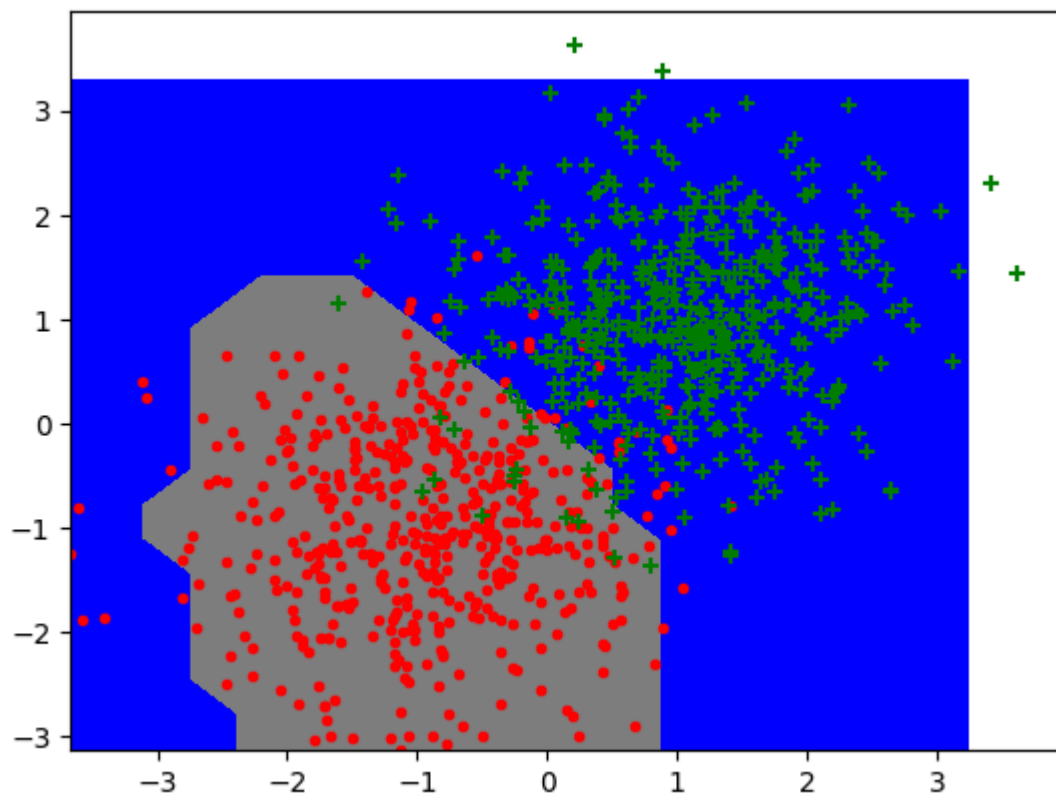




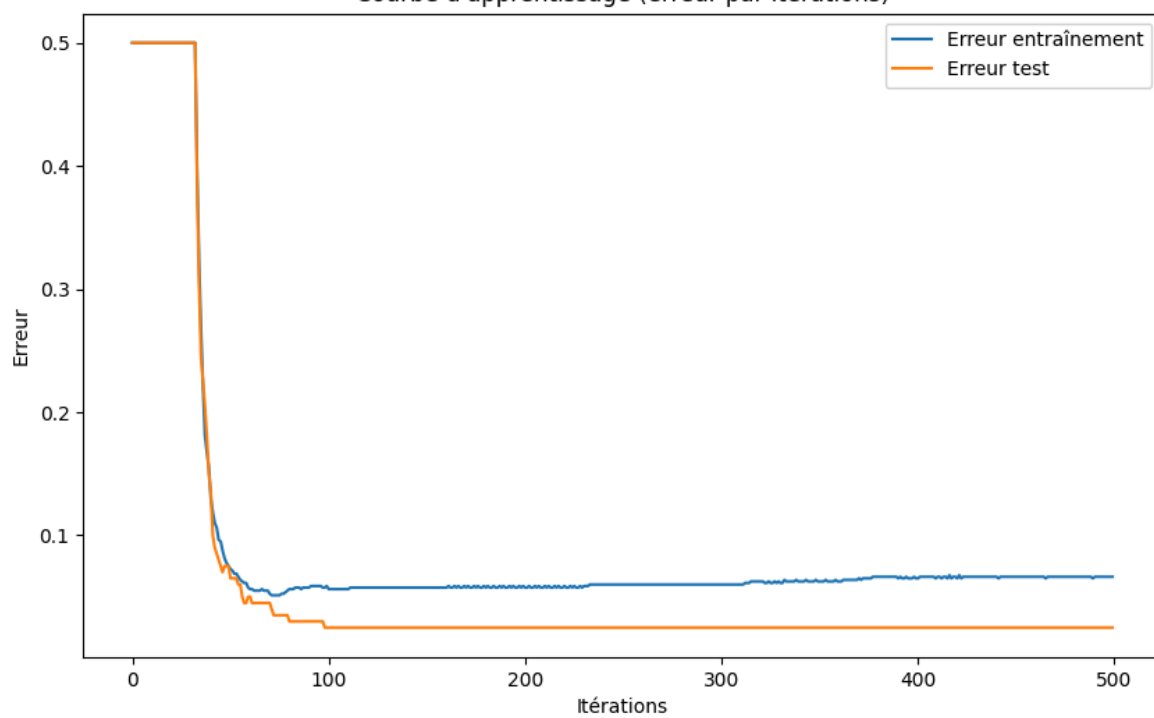
Courbe d'apprentissage (erreur par itérations)

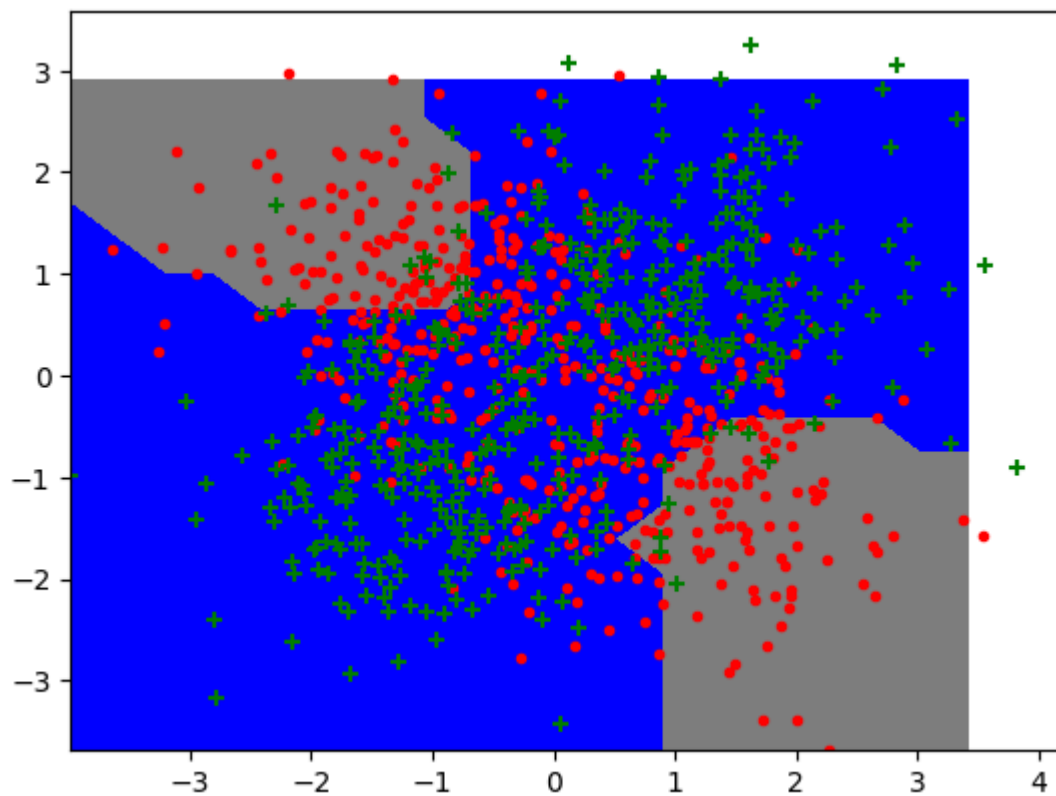


Sigma = 1.5

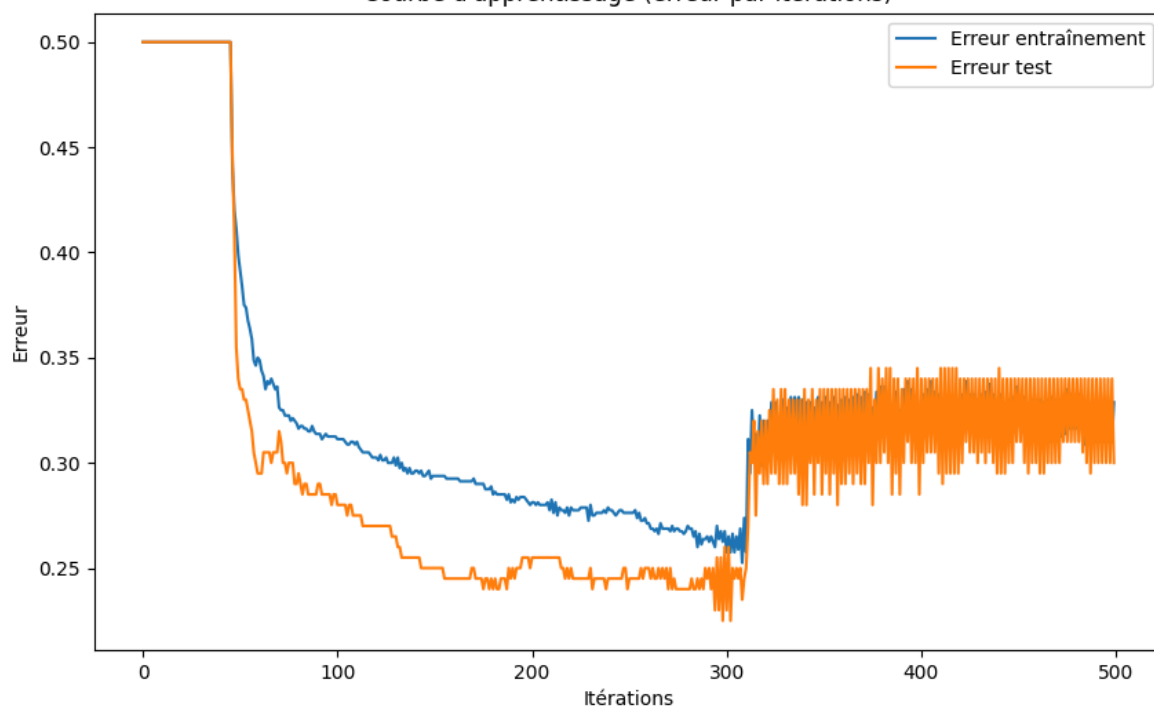


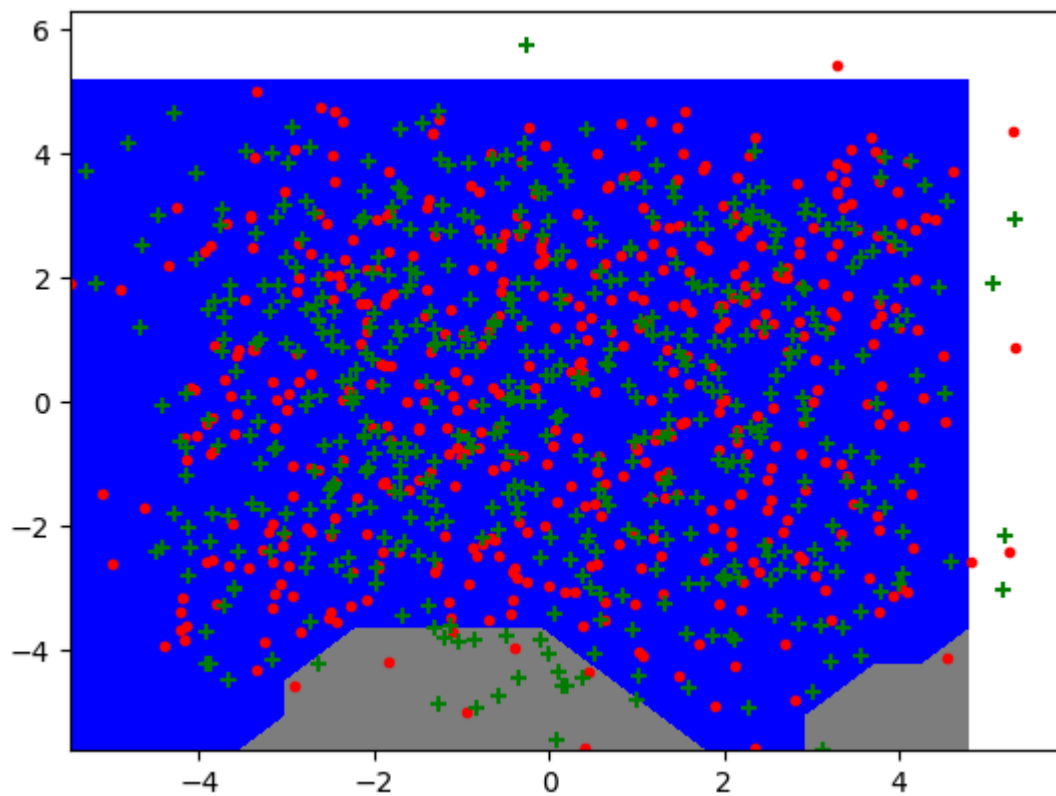
Courbe d'apprentissage (erreur par itérations)



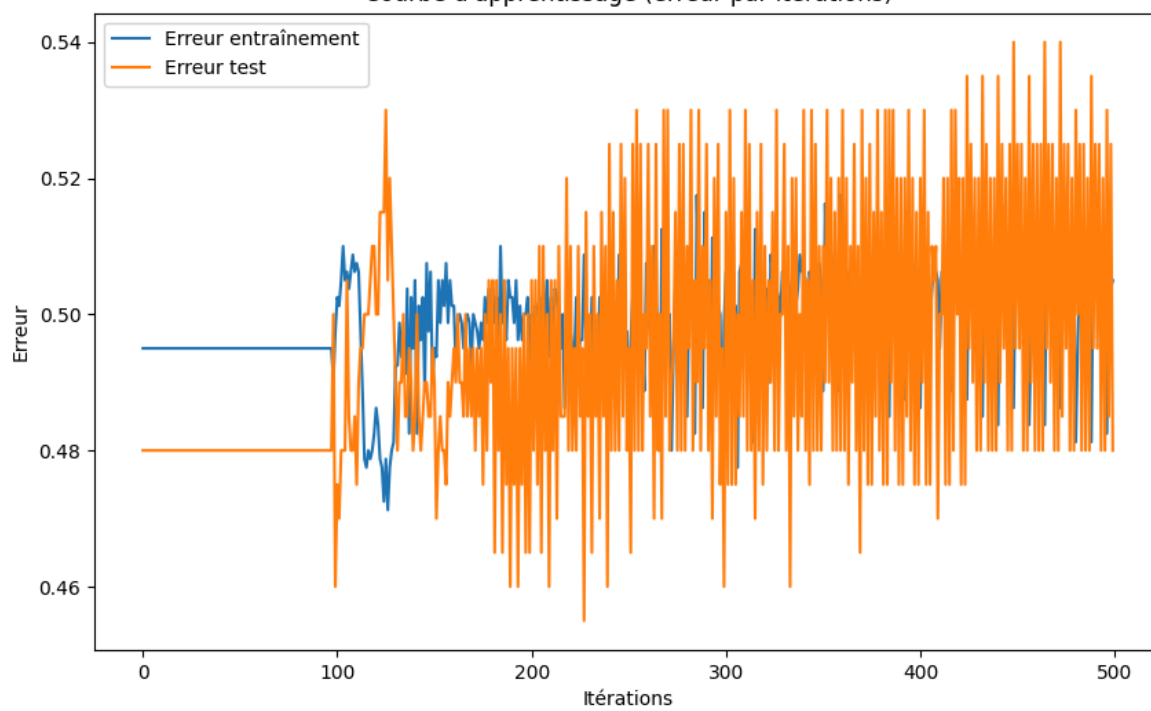


Courbe d'apprentissage (erreur par itérations)

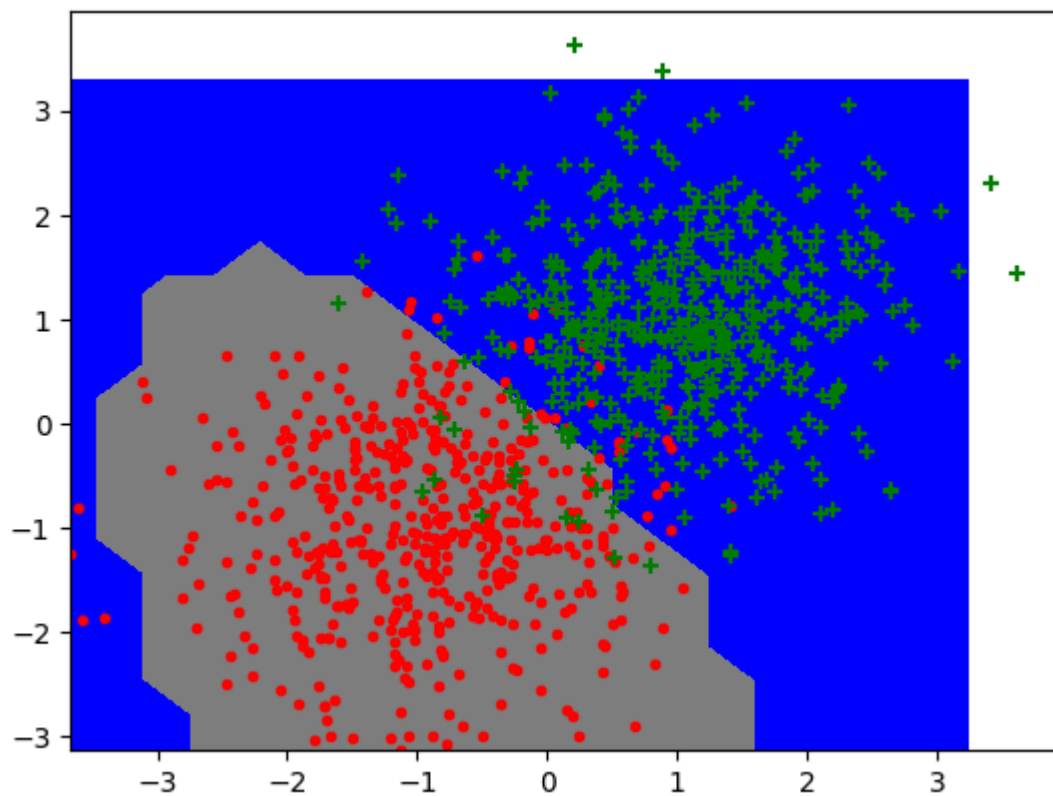




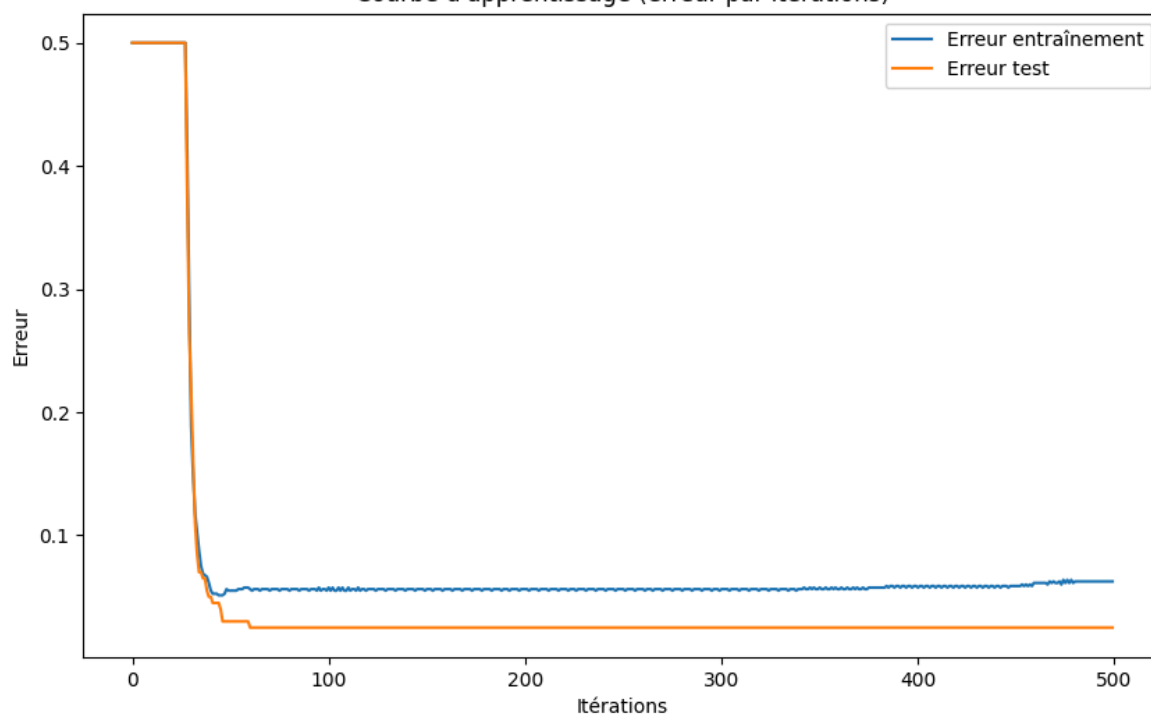
Courbe d'apprentissage (erreur par itérations)

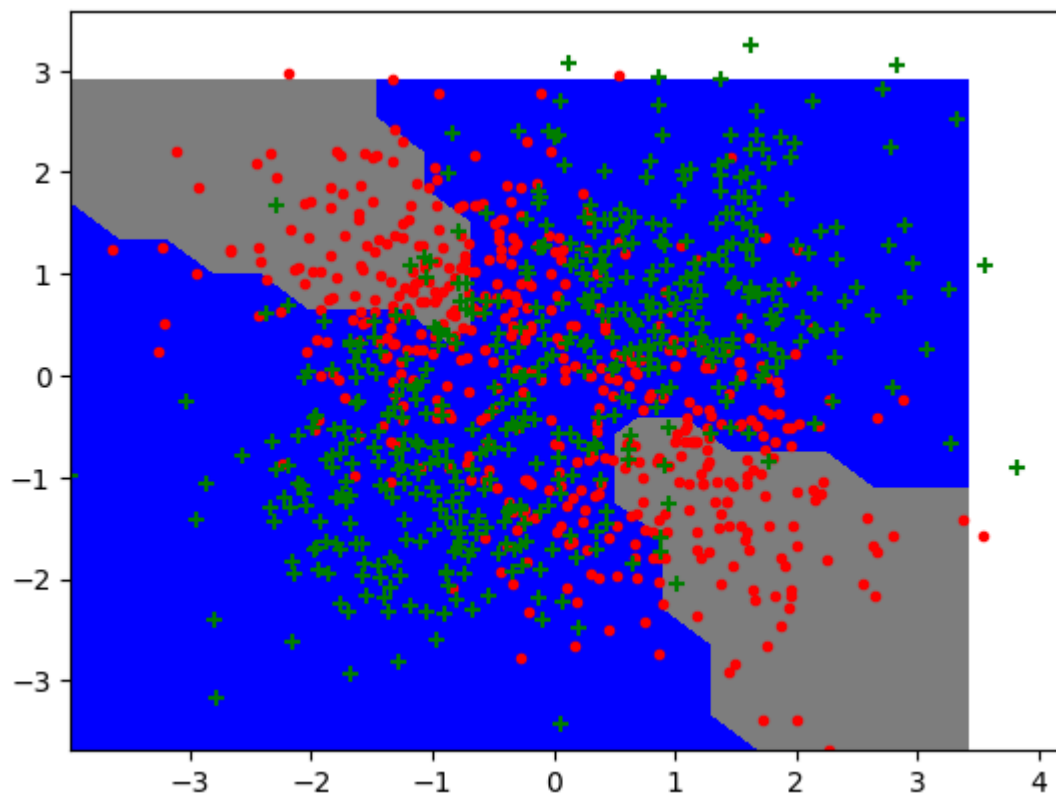


Sigma = 2

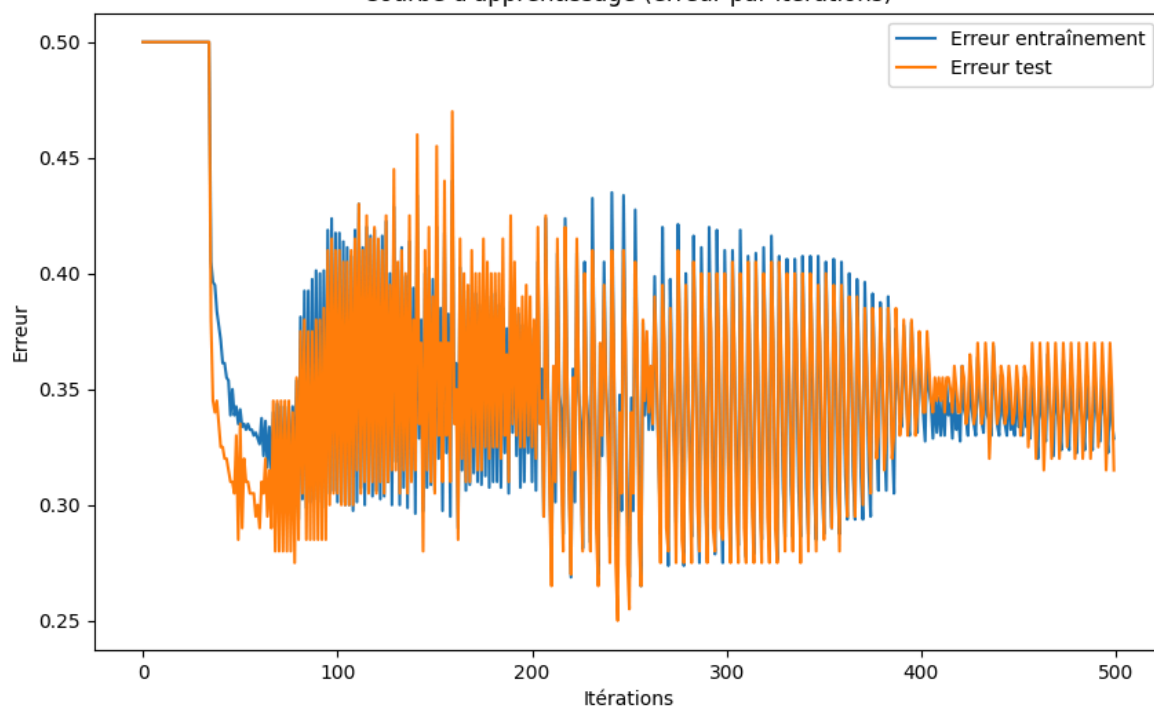


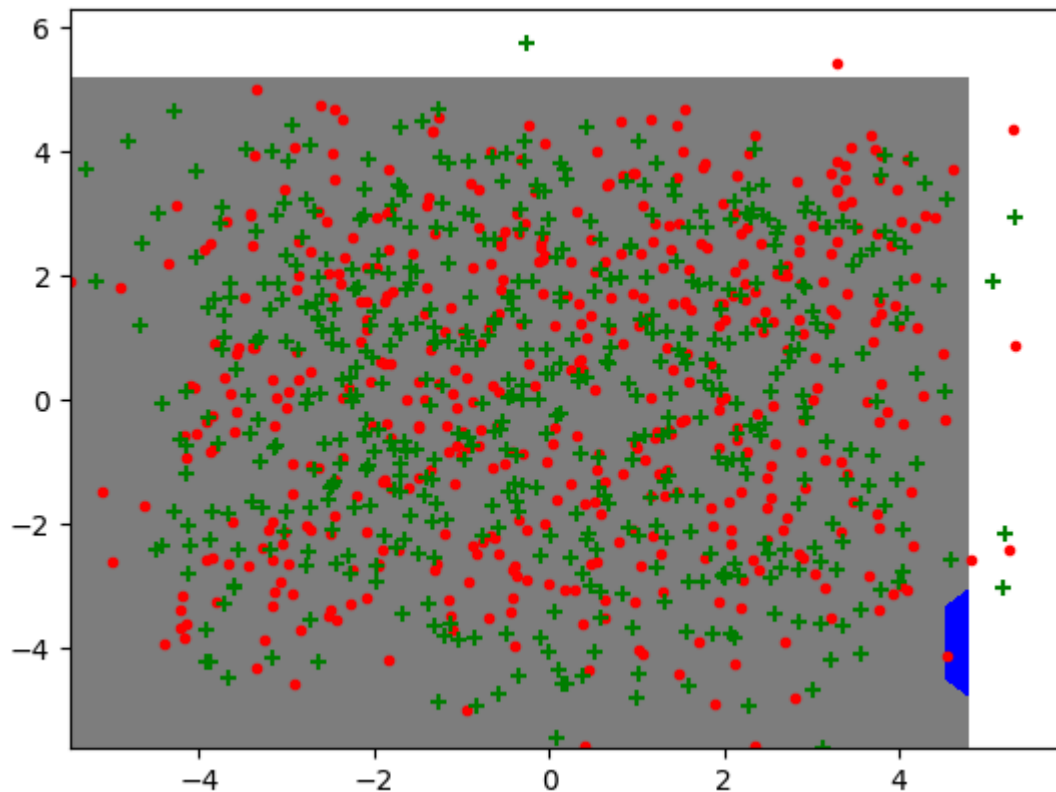
Courbe d'apprentissage (erreur par itérations)



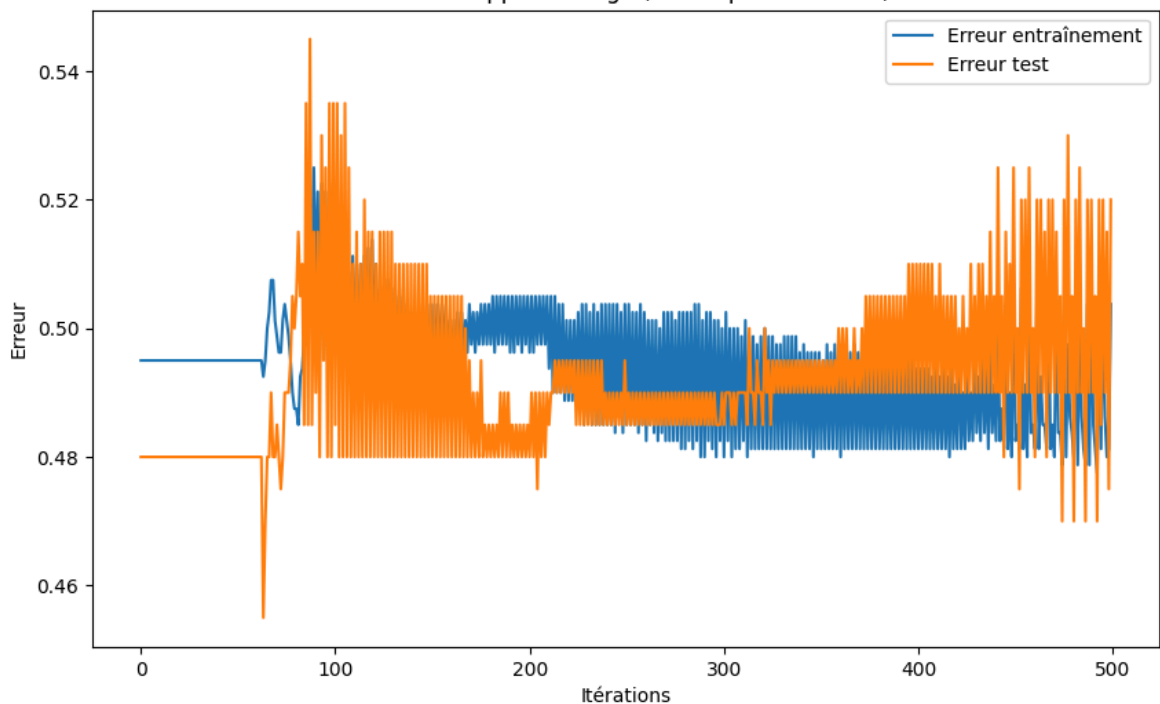


Courbe d'apprentissage (erreur par itérations)





Courbe d'apprentissage (erreur par itérations)



Interprétation

La taille de la base de projection ainsi que la taille de sigma sont deux facteurs importants pour la classification.

- Une taille de la base de projection petite est rapide à calculer mais peut ne pas être suffisante pour bien séparer les données. Une grande taille de base de projection capture les informations plus complexes mais plus la taille est grande plus il y a de risque d'overfitting et donc des mauvais résultats de classification. Ici on a testé avec une taille de 10, 100 et 200. La base de taille 10 ne capture pas les informations, avec

la base de taille 100, le réseau capture beaucoup mieux les informations avec une taille 200, le réseau commence à faire de l'overfitting.

- La taille de sigma dans une projection gaussienne est un paramètre qui influence la résolution de chaque base gaussienne. Une petite valeur de sigma rend la gaussienne très précise alors qu'une grande valeur de sigma est plus large, donnant un rendu plus lissé. Le sigma optimal semble être celui d'une valeur de 1.

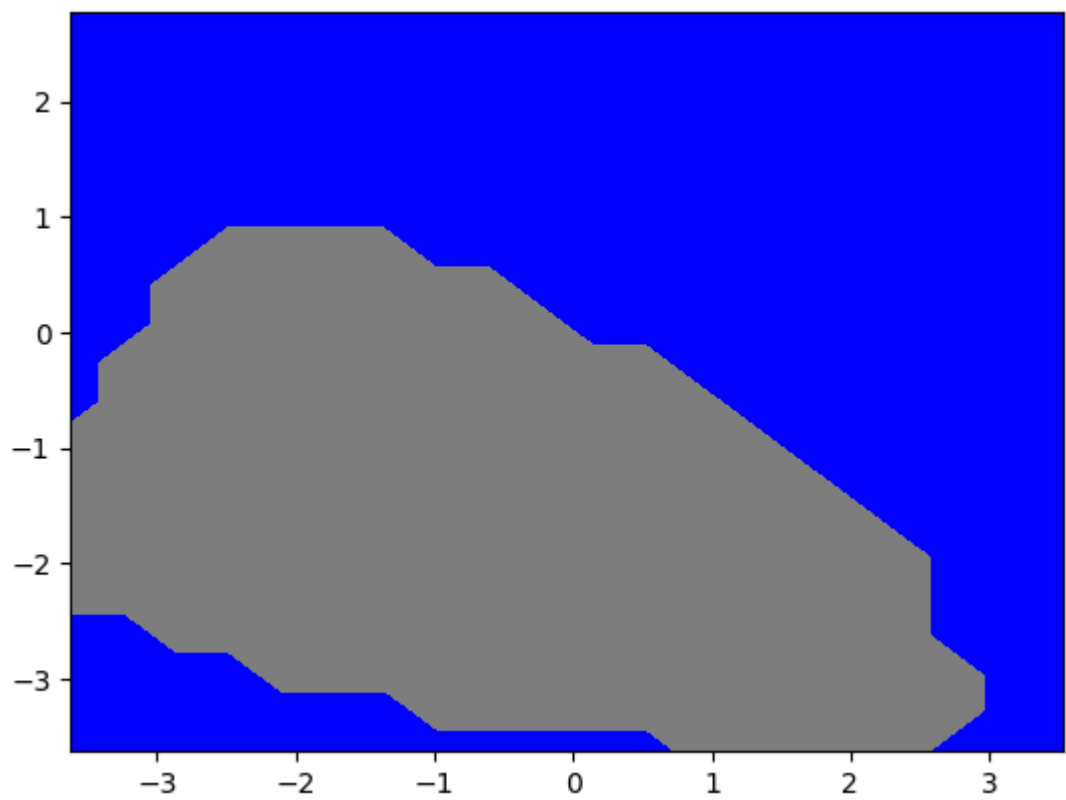
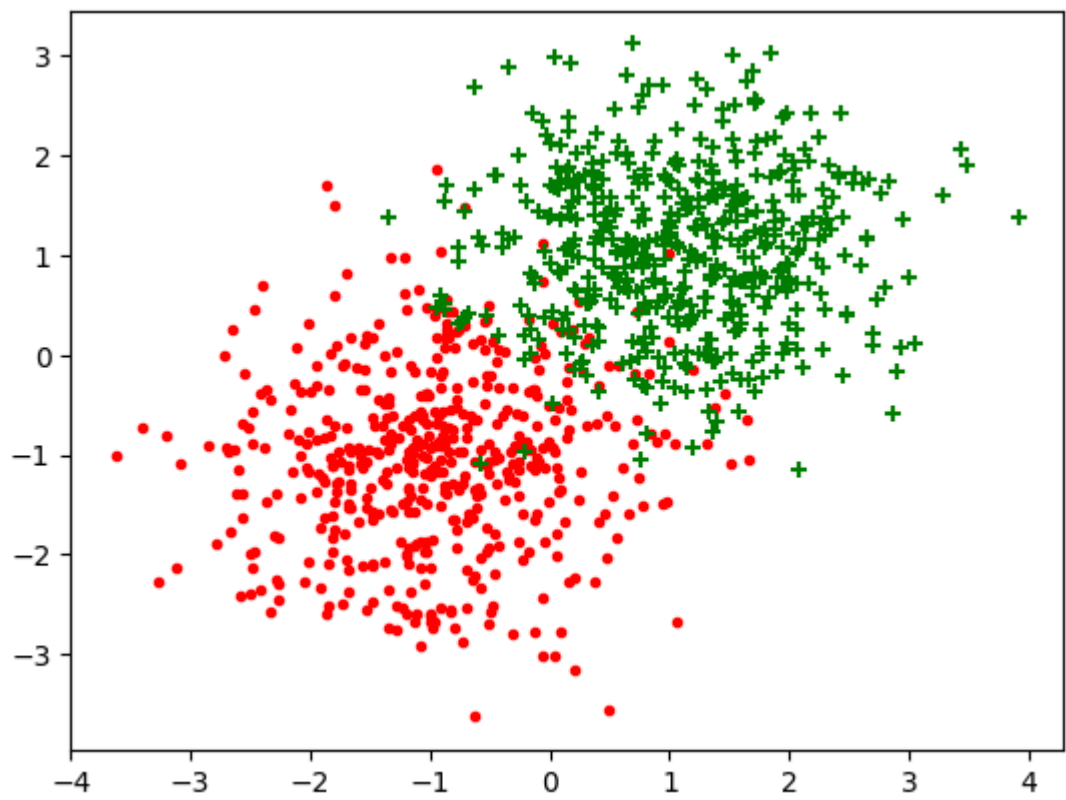
c) Hinge Loss

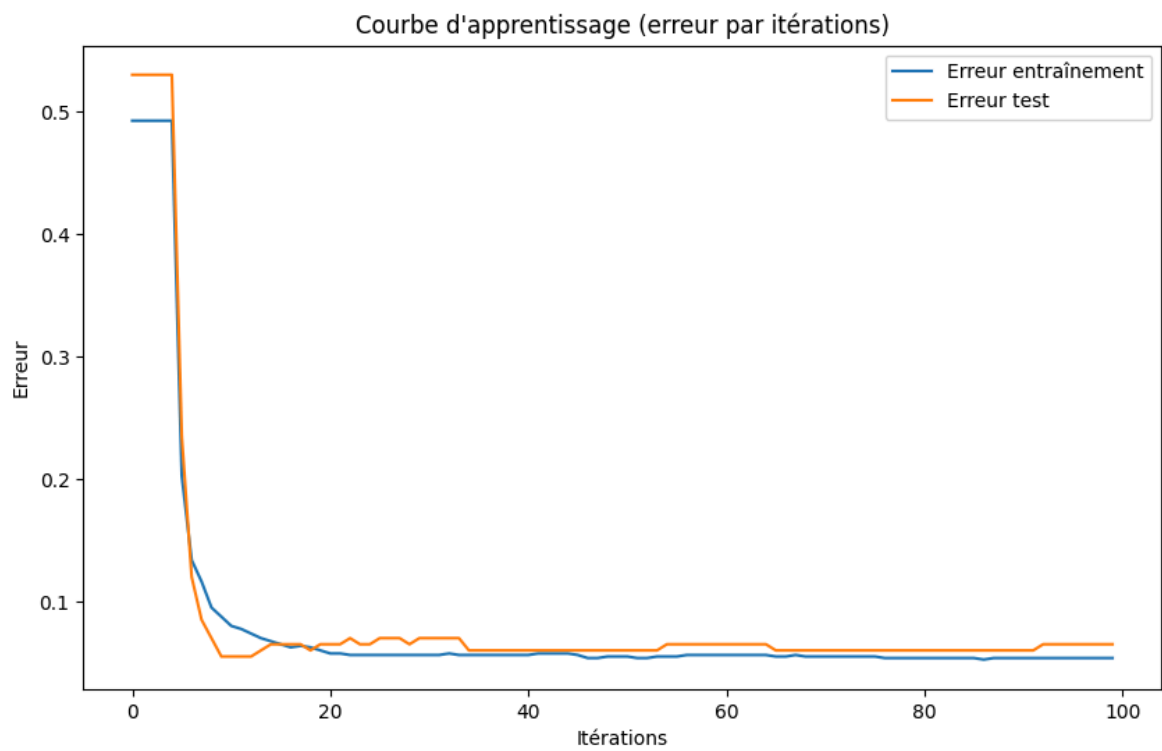
```
In [ ]: def hinge_loss(w, X, y, alpha=1.0, lambd=0.1):  
    margins = alpha - y * (X @ w)  
    hinge = np.maximum(0, margins)  
    loss = np.sum(hinge) + lambd * np.linalg.norm(w) ** 2  
    return loss
```

```
In [190... def hinge_loss_grad(w, X, y, alpha=1.0, lambd=0.1):  
    margins = alpha - y * (X @ w)  
    indicator = (margins > 0).astype(float)  
    grad = -np.sum((indicator * y)[: , np.newaxis] * X, axis=0)  
    grad += 2 * lambd * w  
    return grad
```

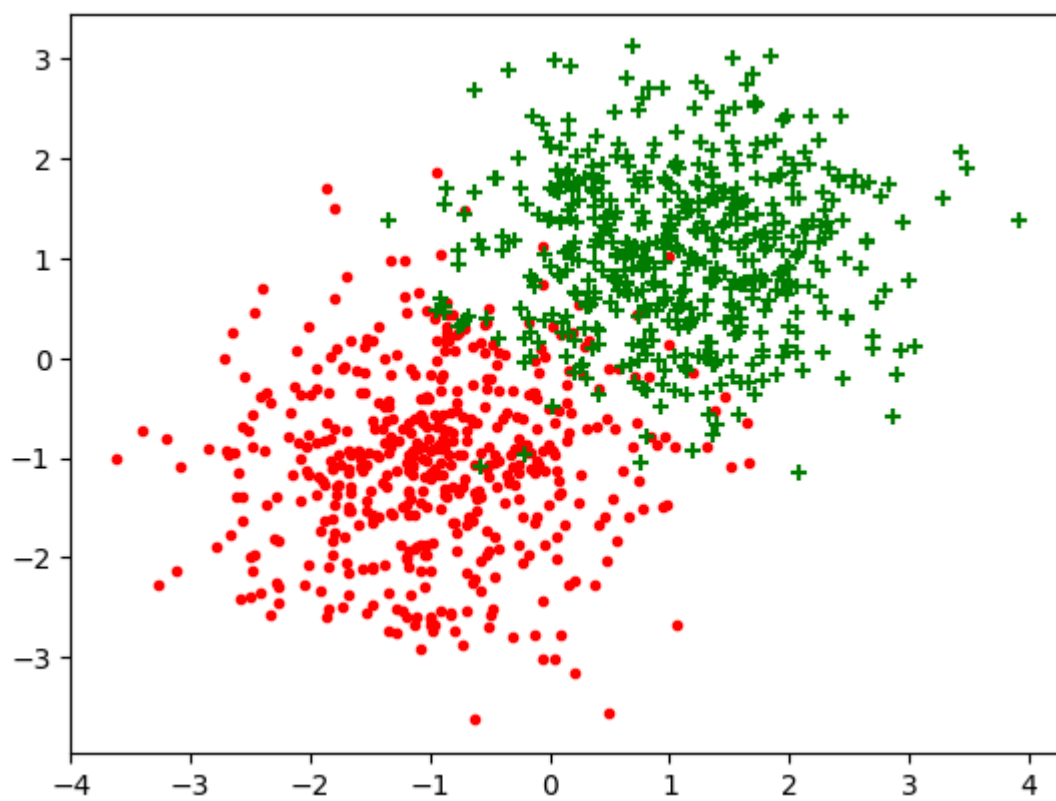
```
In [194... sigma = 1  
base_size = 100  
index_b = np.random.choice(np.arange(1, datax.shape[0]), base_size)  
base = datax[index_b]  
  
# apprentissage data 0  
for alpha in [0.5, 1, 1.5] : #taille de la marge  
    for lambd in [1e-2, 0.1, 1] :  
        print(f"alpha = {alpha}, lambda = {lambd}")  
        gauss0 = Lineaire3(loss=partial(hinge_loss, alpha=alpha, lambd=lambd), 1  
        gauss0.fit(x0_train, y0_train.ravel(), x0_test, y0_test.ravel())  
  
        plot_data(x0, y0)  
        plt.show()  
        plot_frontiere(x0, gauss0.predict)  
        display_error(gauss0.train_errors, gauss0.test_errors)
```

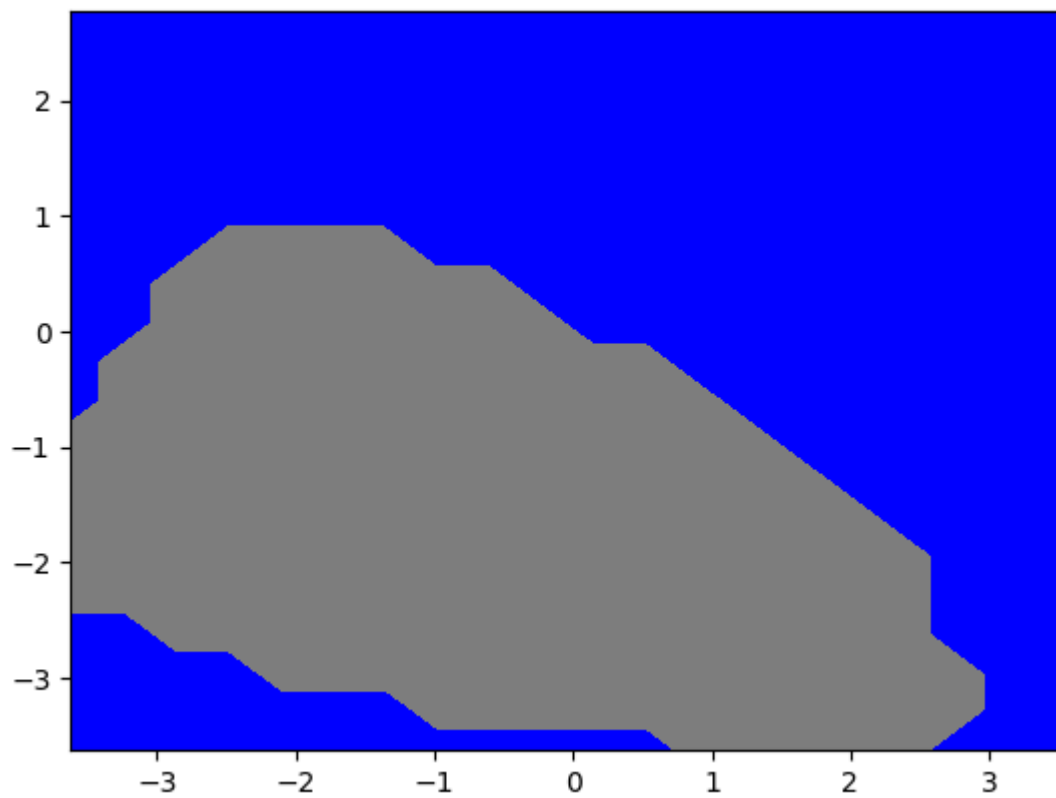
alpha = 0.5, lambda = 0.01



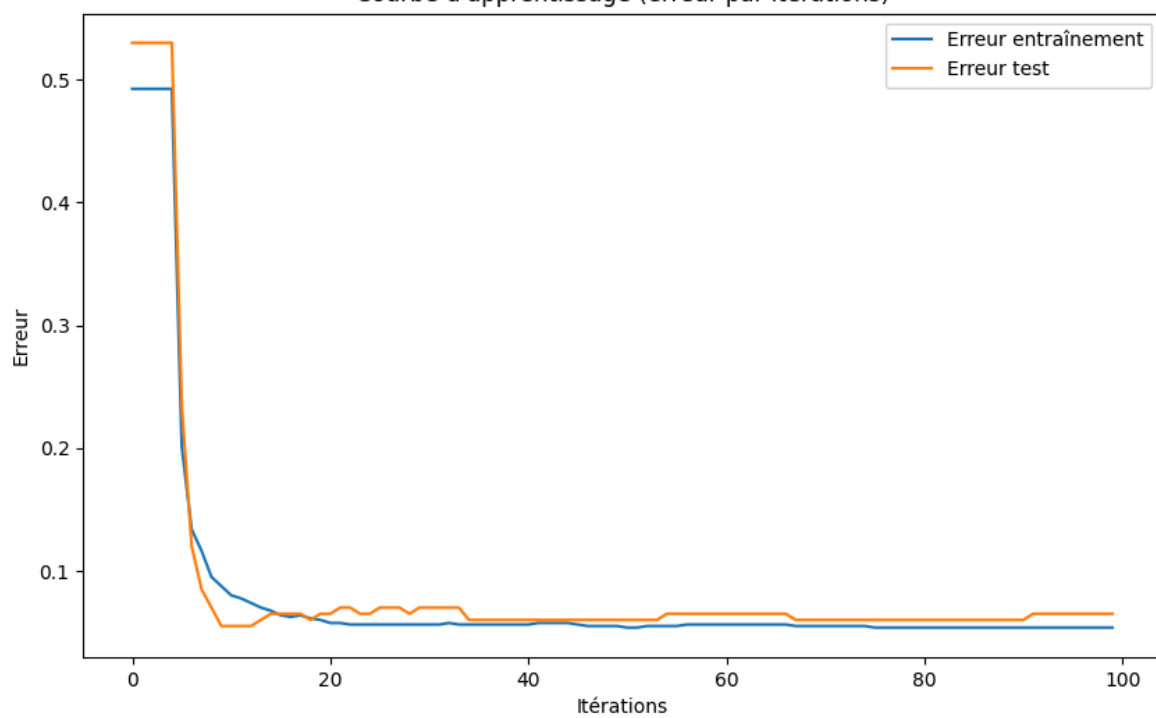


$\alpha = 0.5$, $\lambda = 0.1$

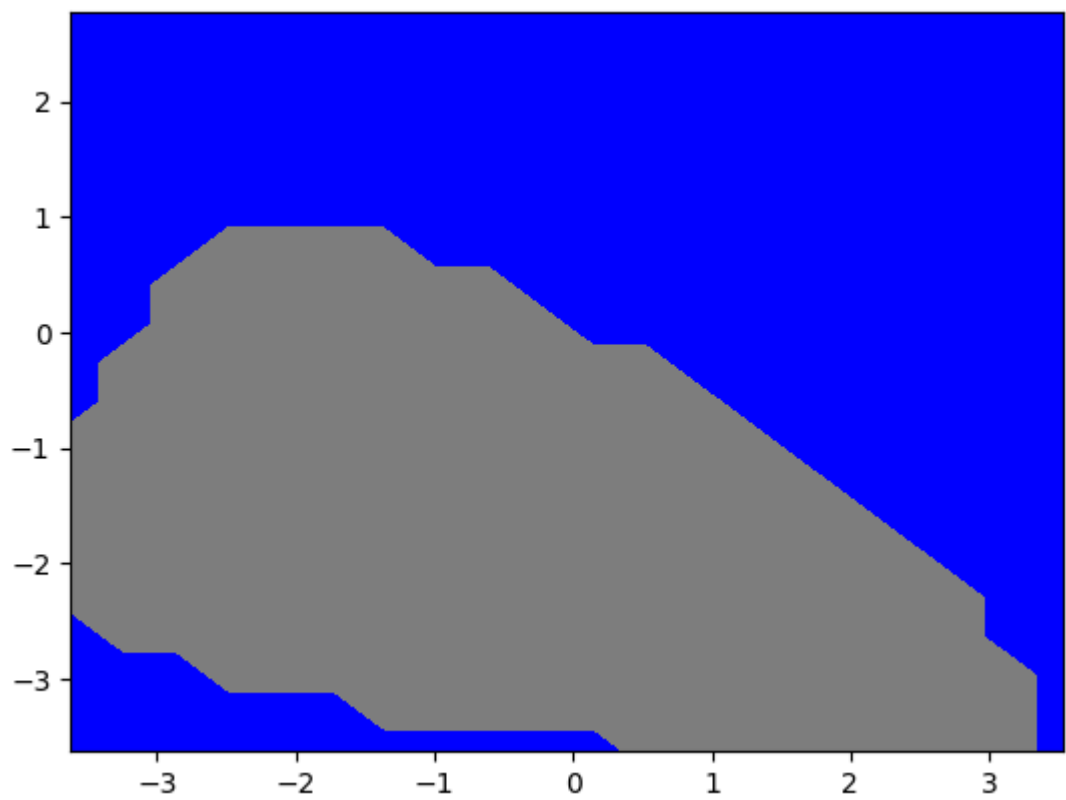
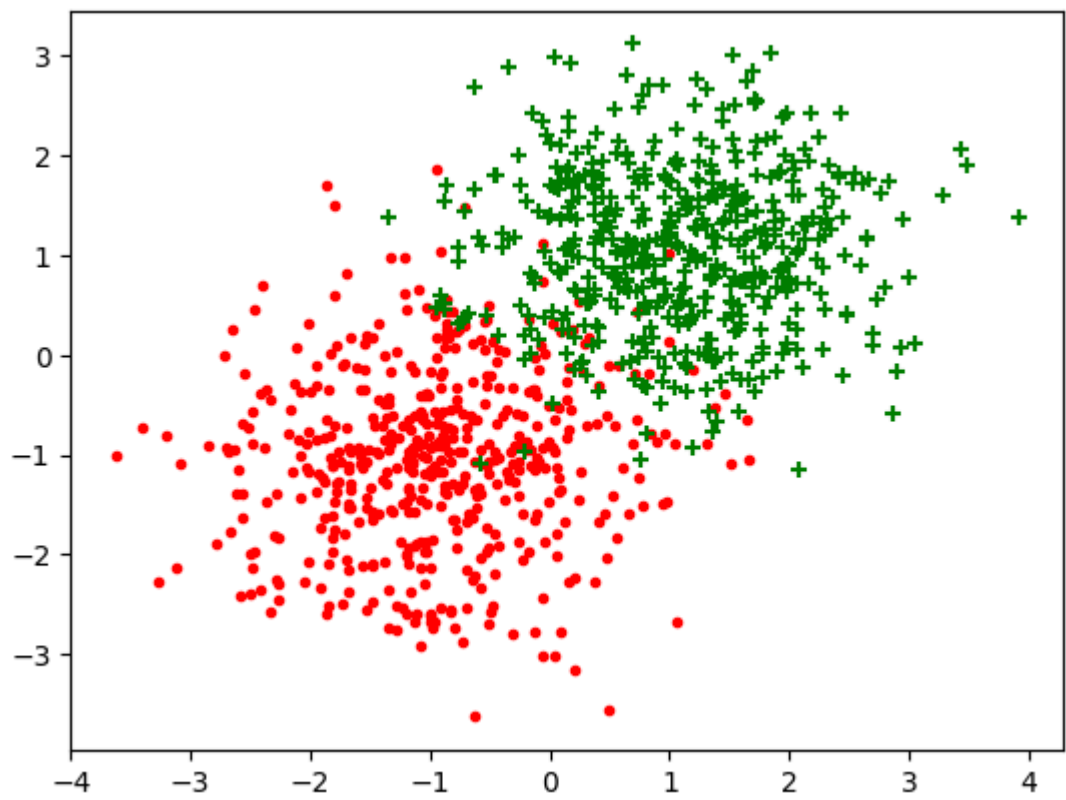


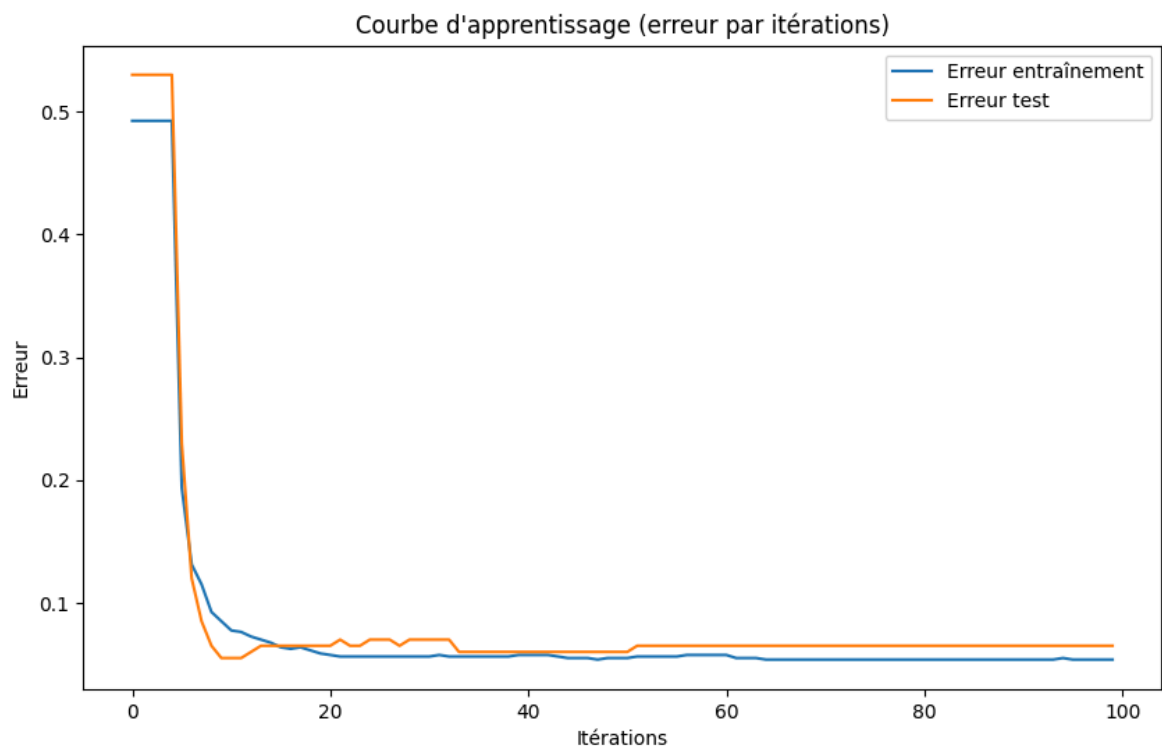


Courbe d'apprentissage (erreur par itérations)

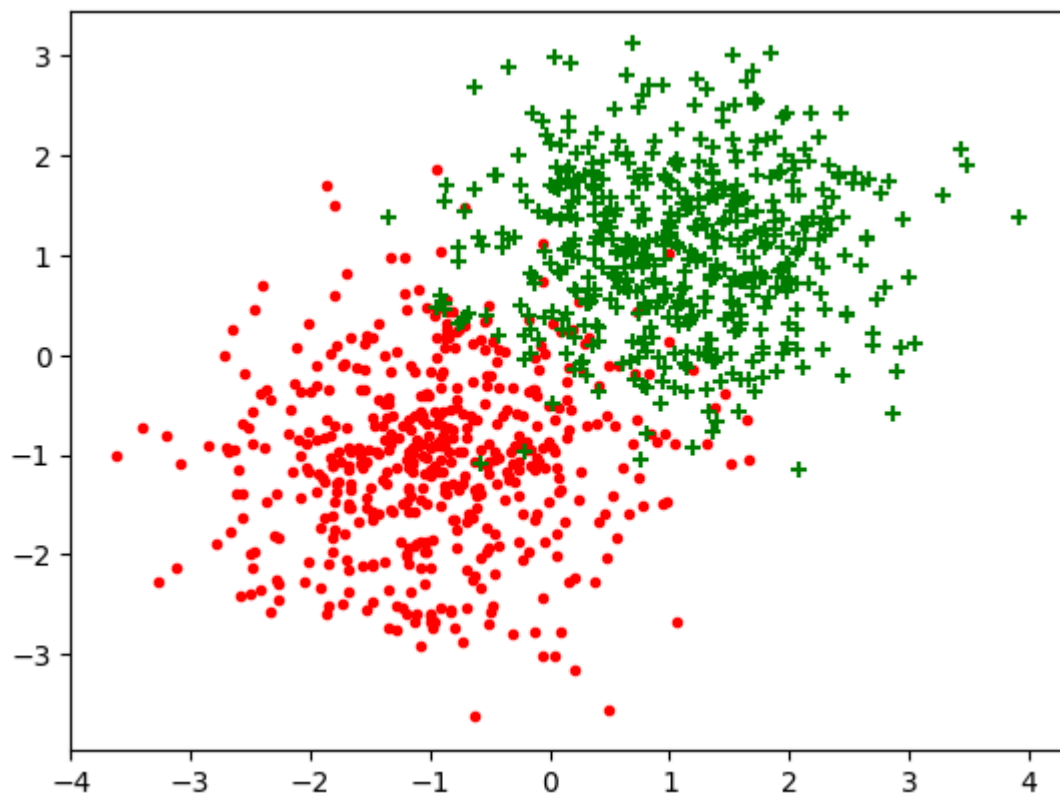


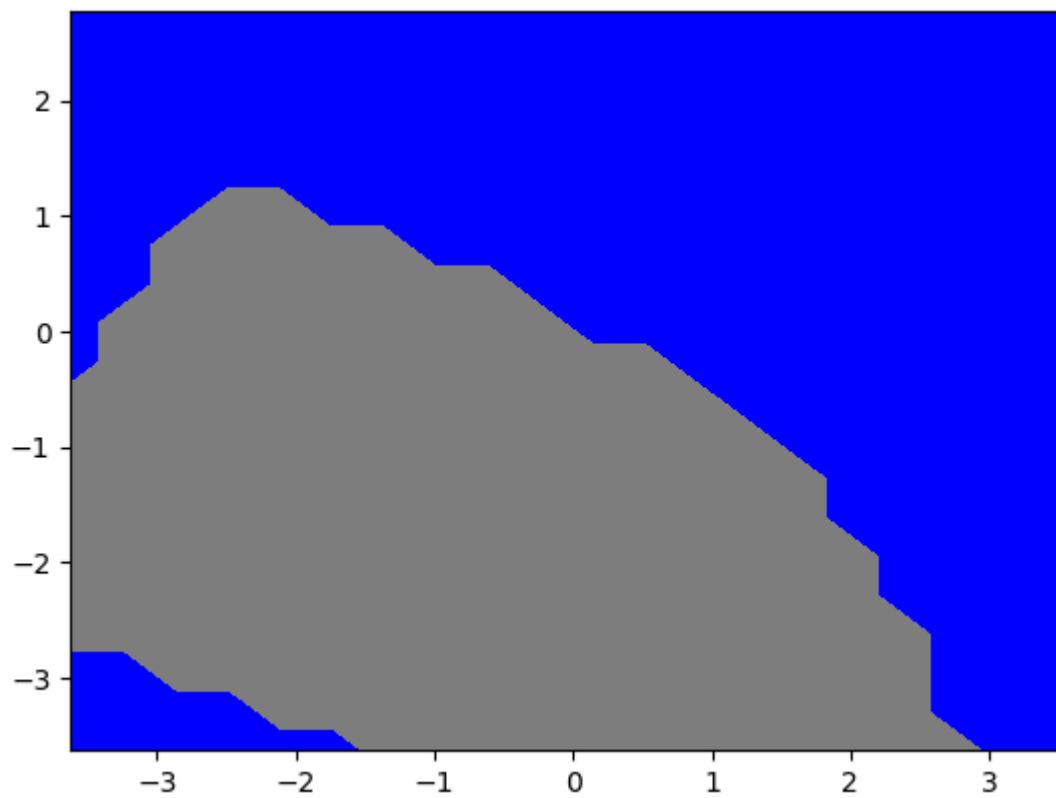
$\alpha = 0.5$, $\lambda = 1$



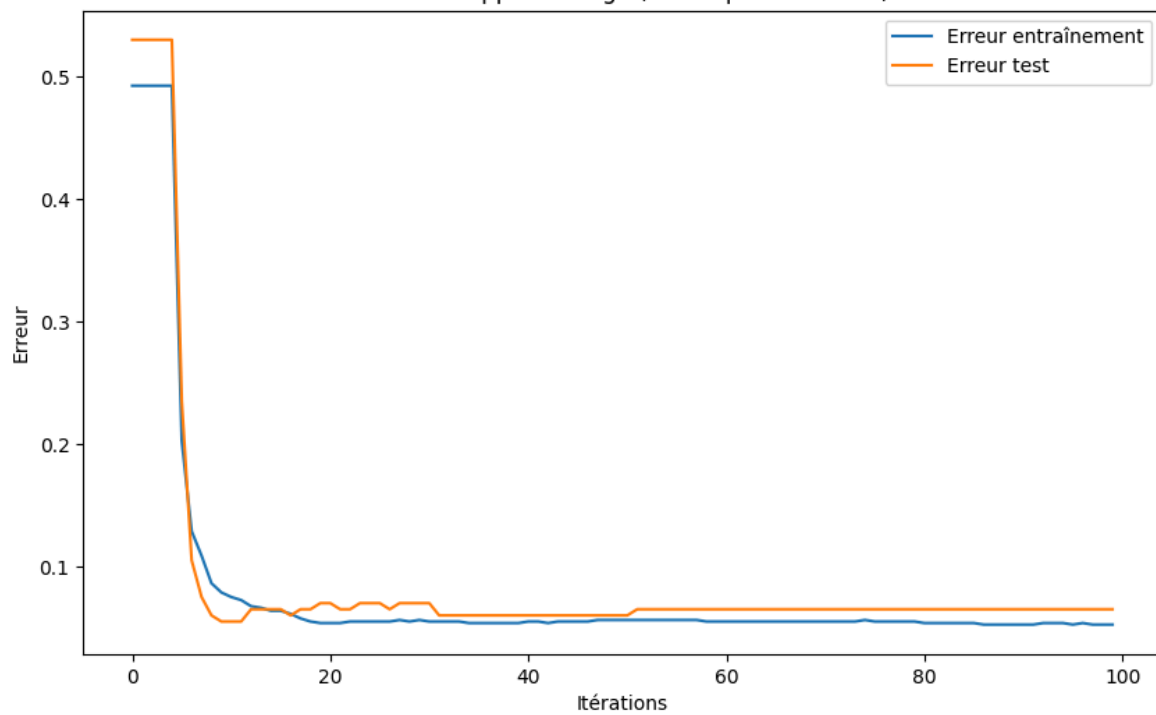


$\alpha = 1$, $\lambda = 0.01$

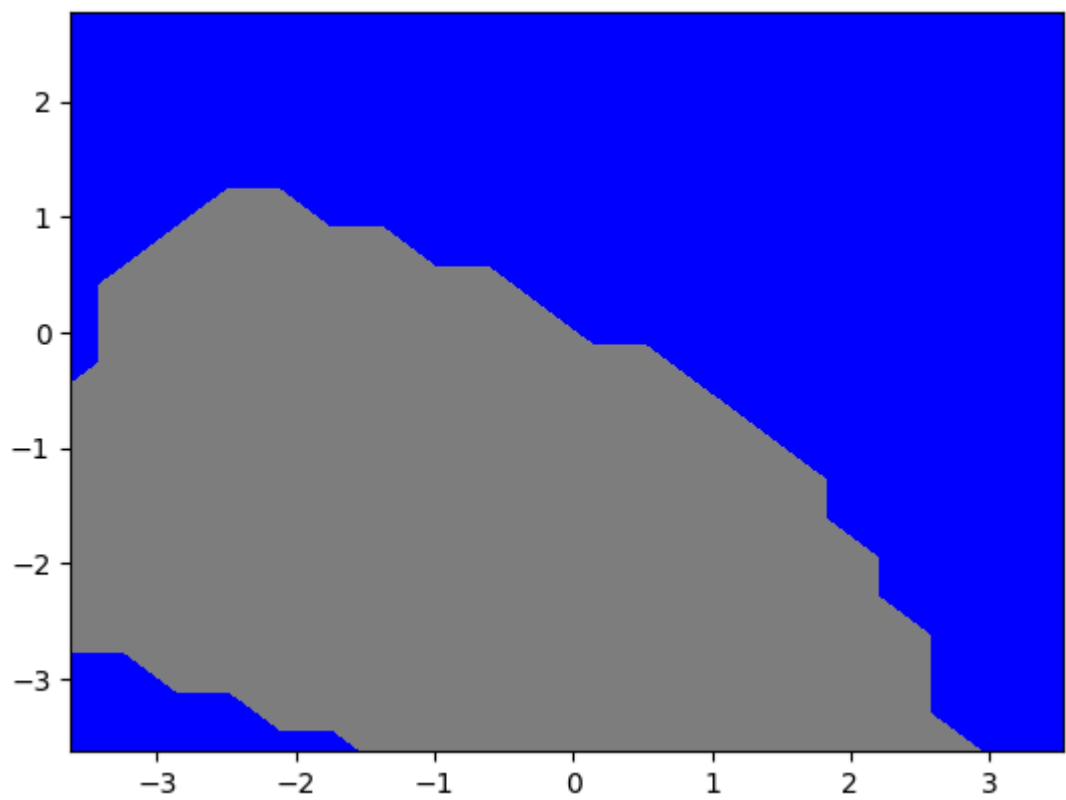
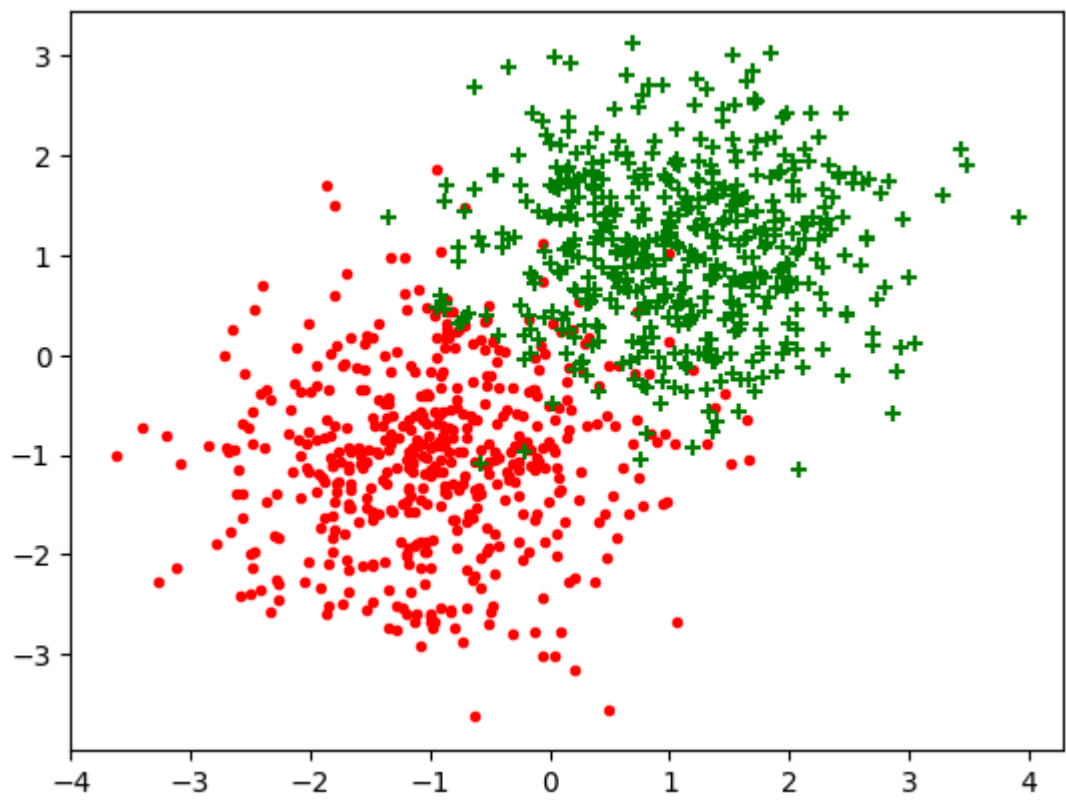


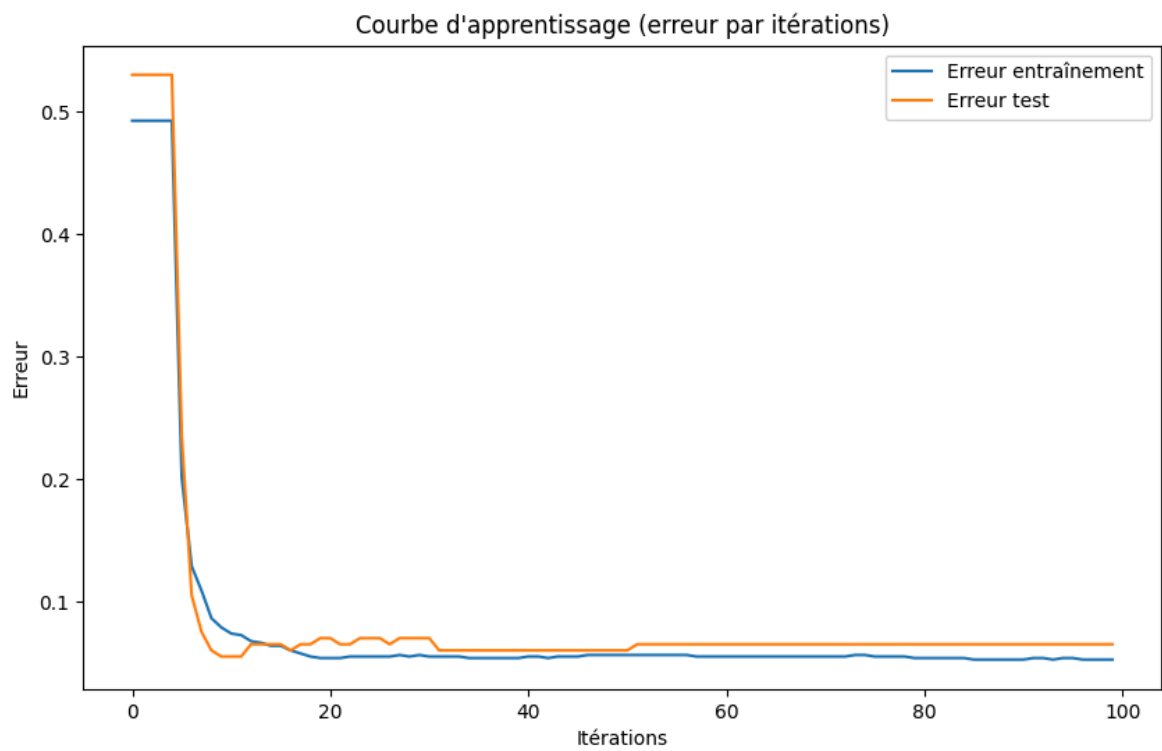


Courbe d'apprentissage (erreur par itérations)

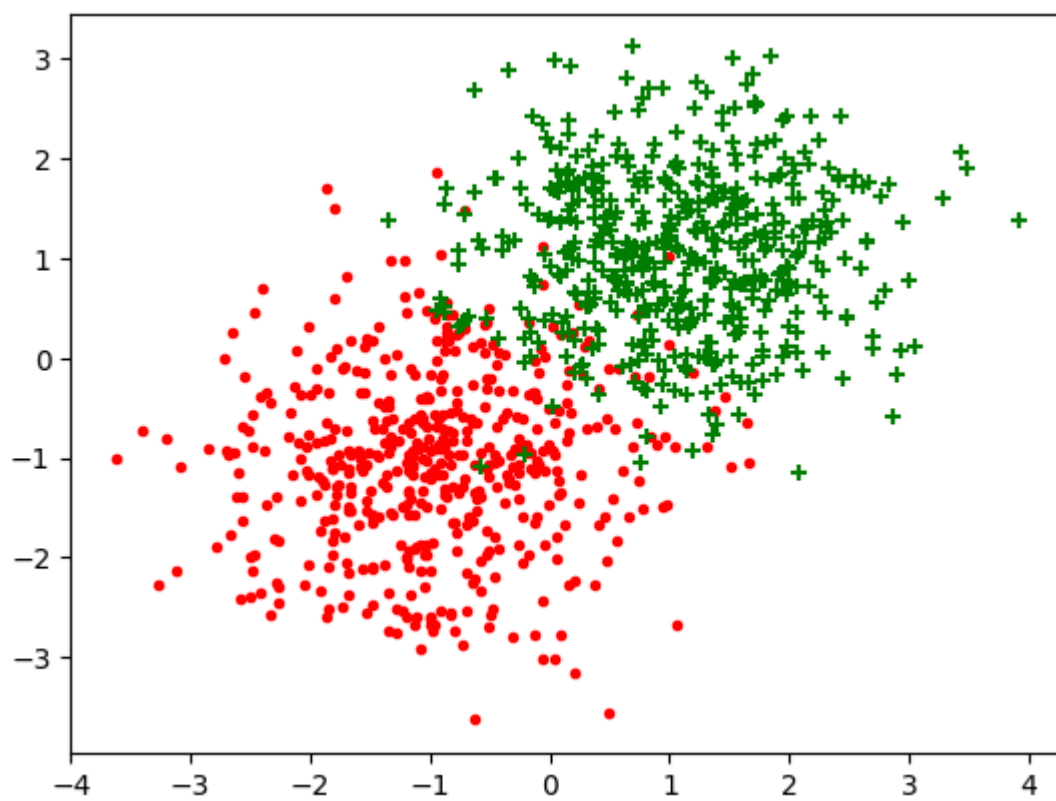


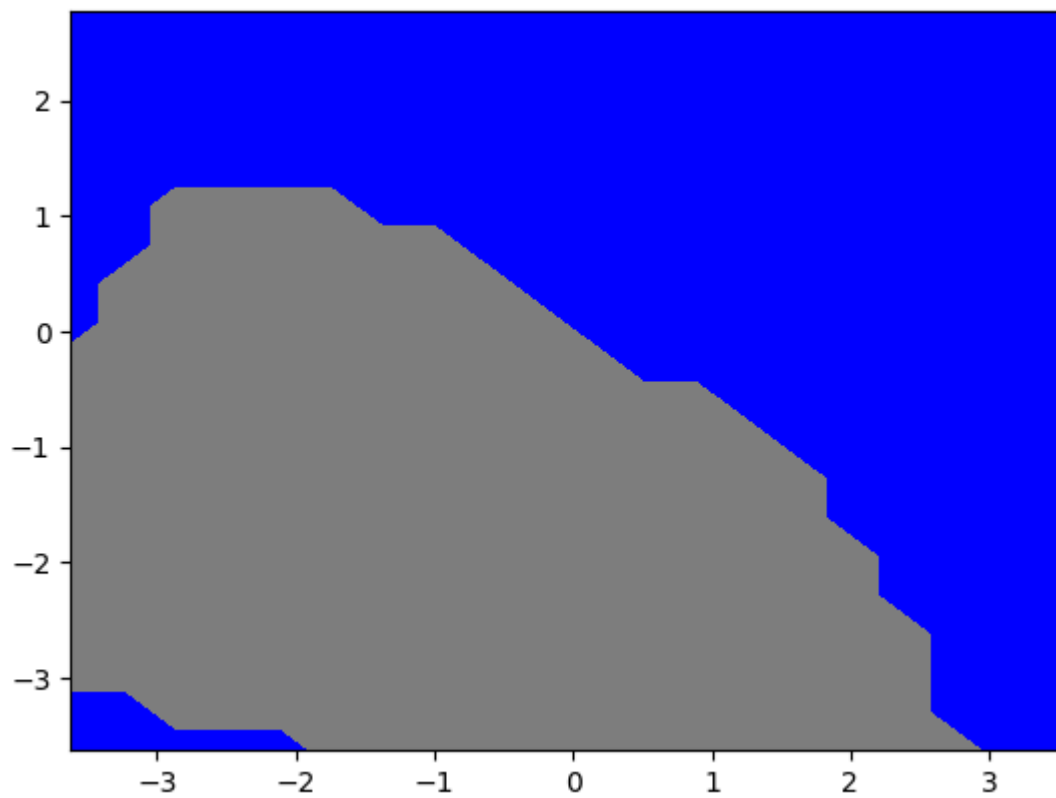
$\alpha = 1$, $\lambda = 0.1$



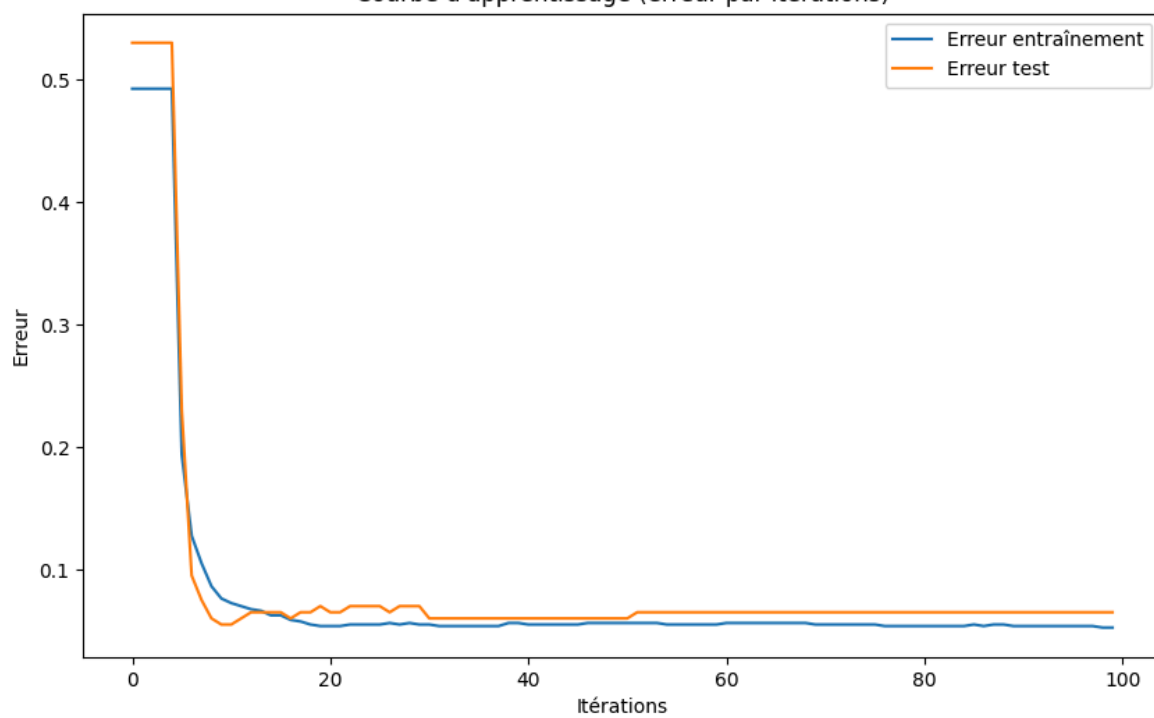


$\alpha = 1$, $\lambda = 1$

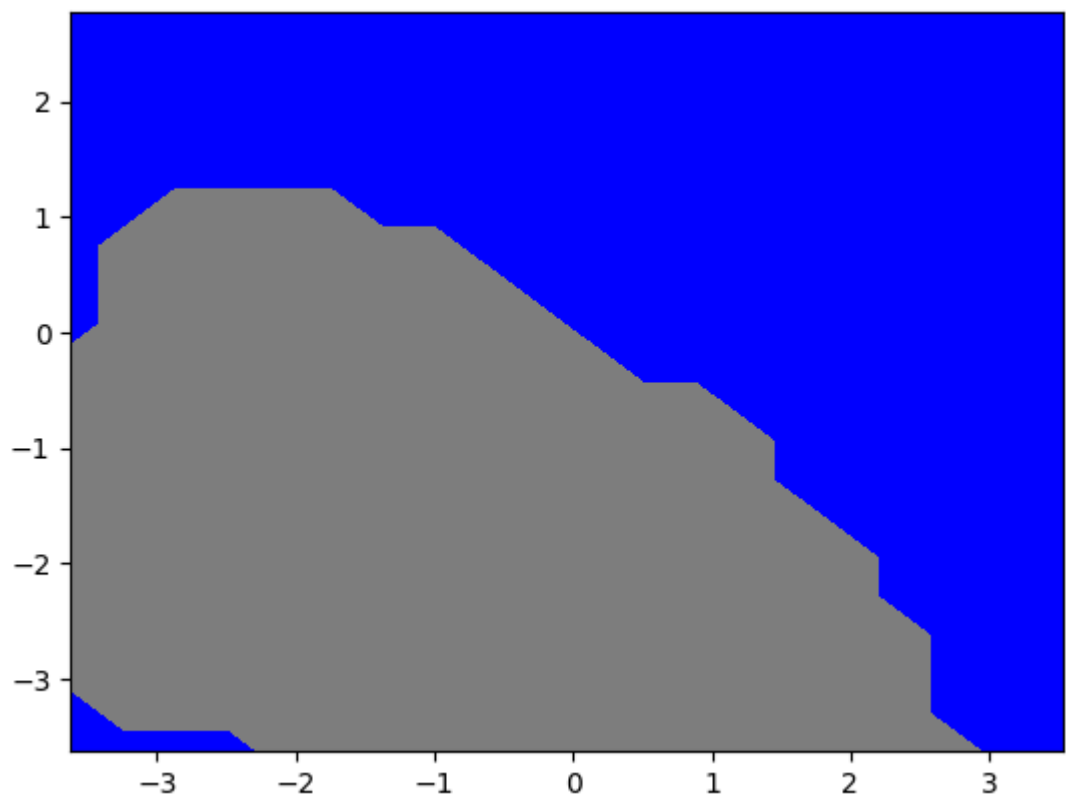
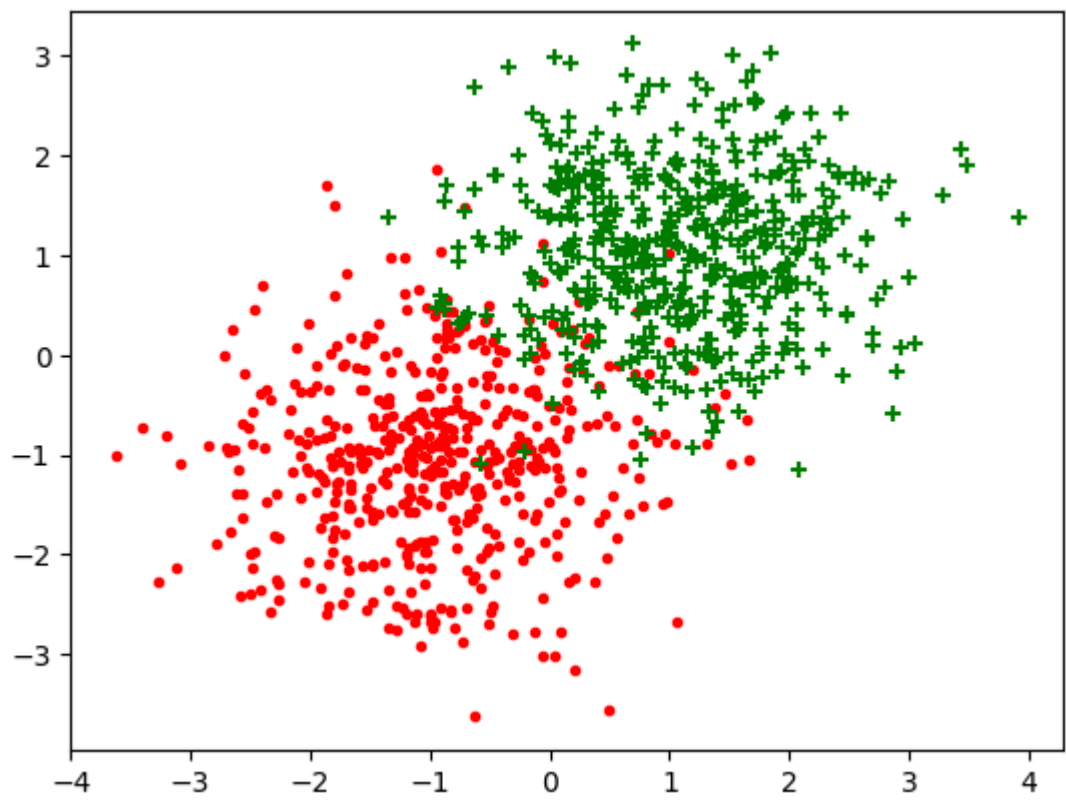


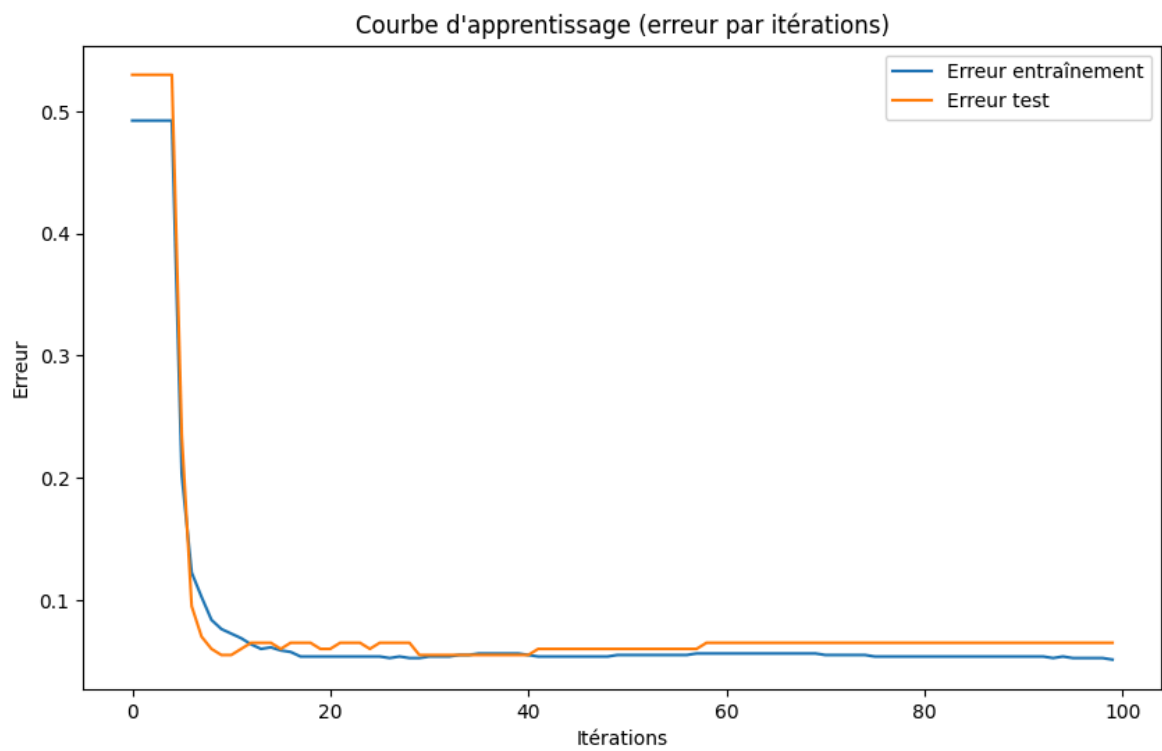


Courbe d'apprentissage (erreur par itérations)

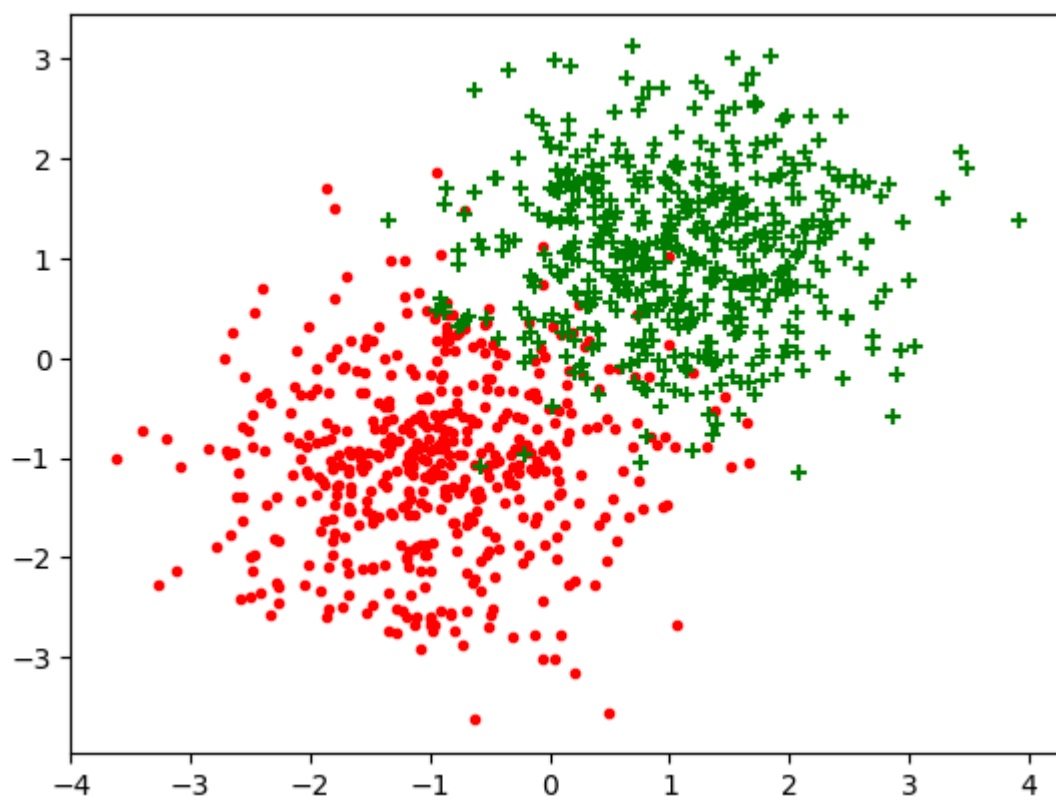


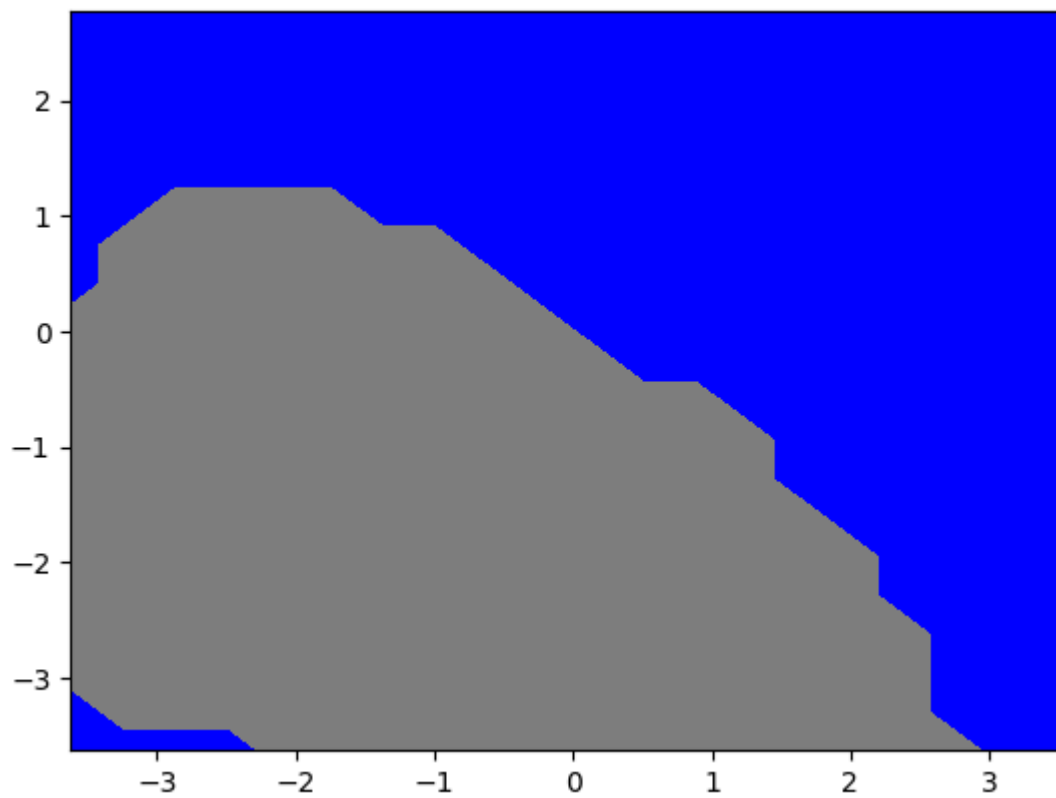
$\alpha = 1.5$, $\lambda = 0.01$



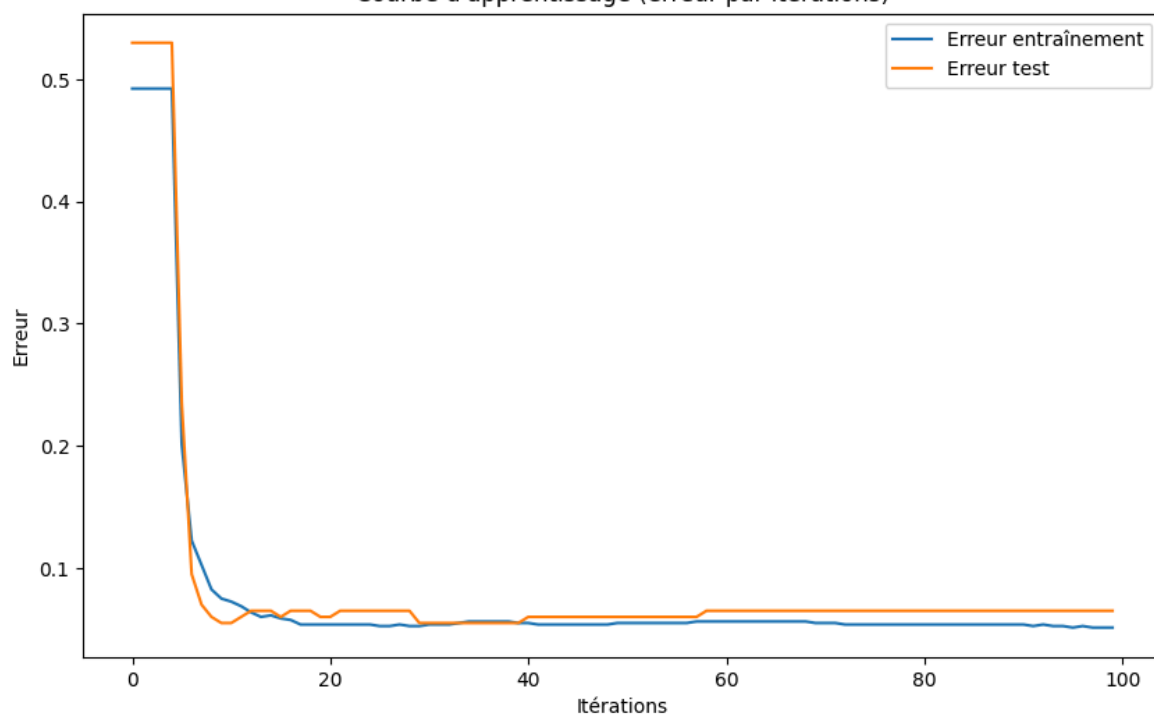


$\alpha = 1.5$, $\lambda = 0.1$

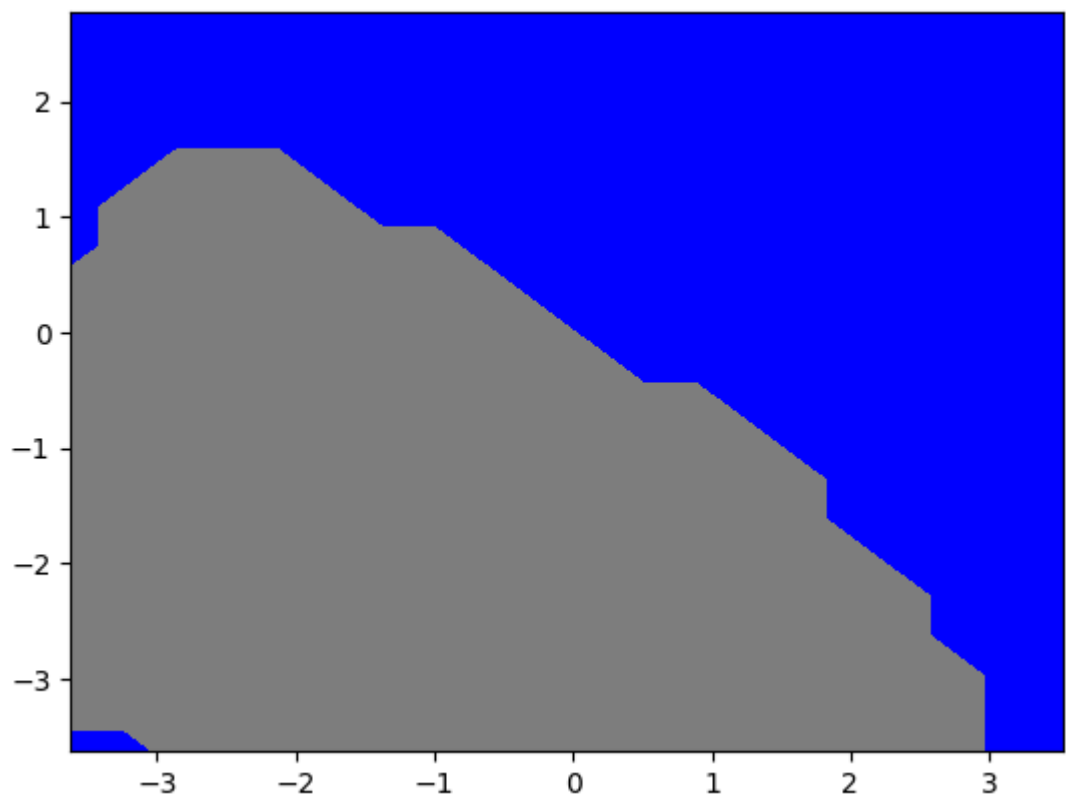
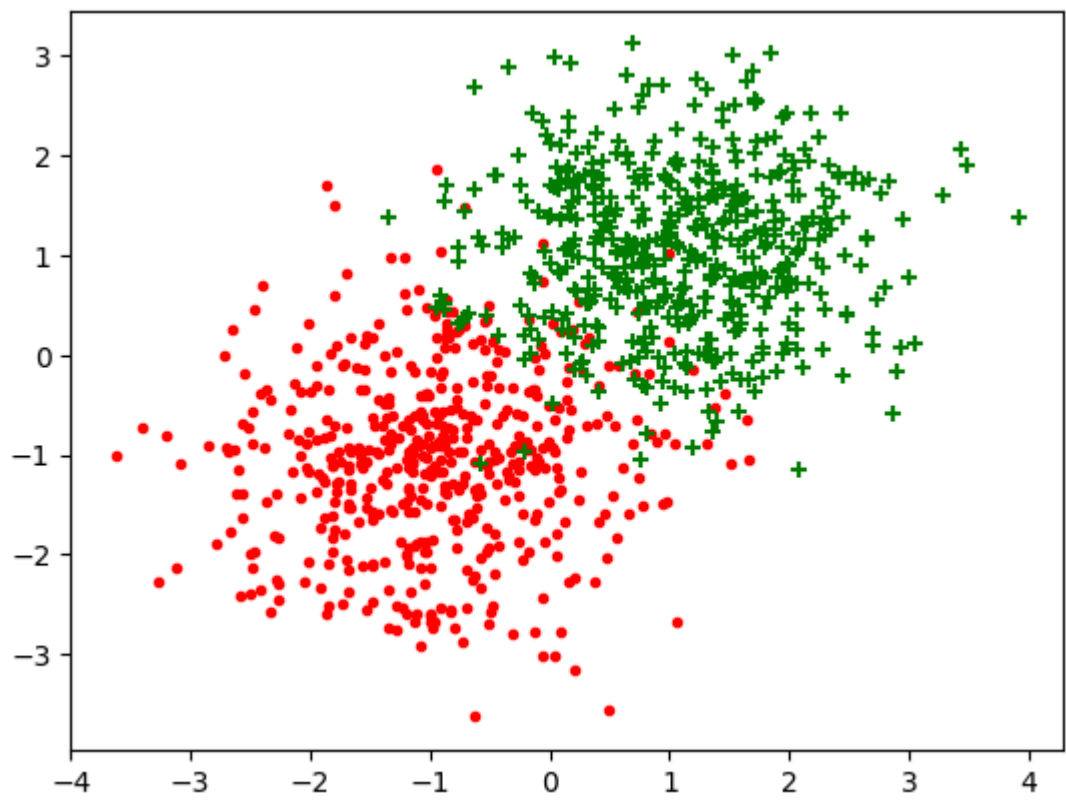


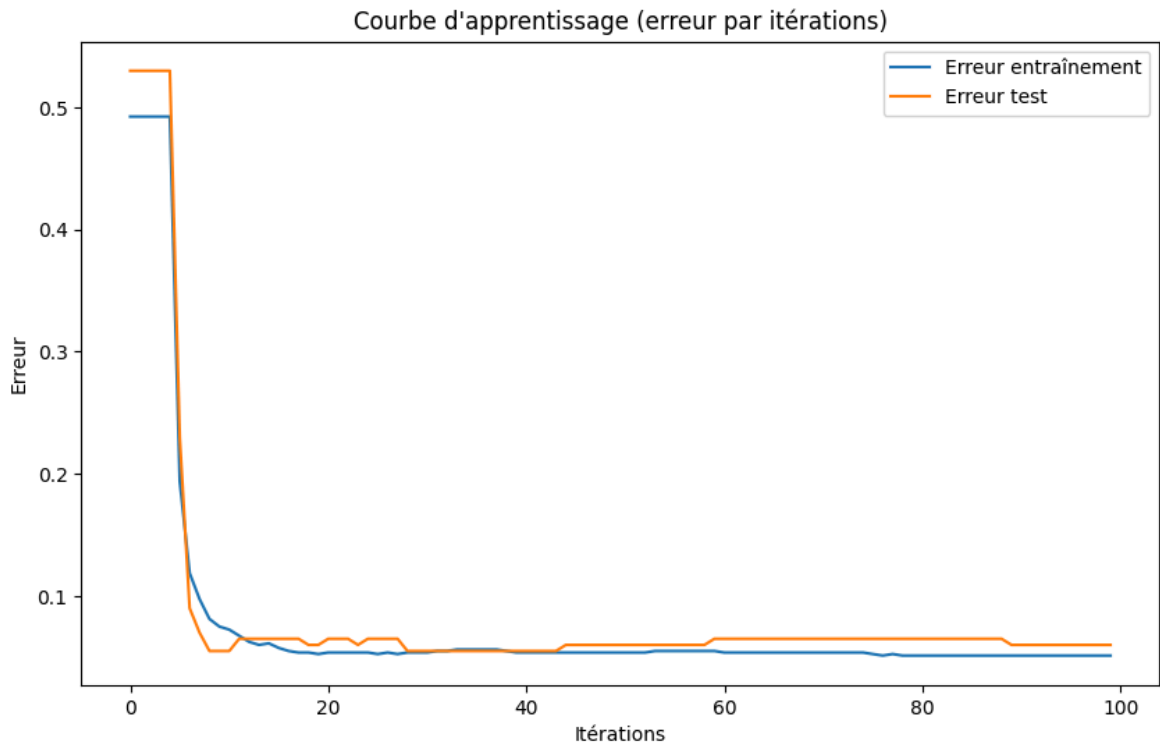


Courbe d'apprentissage (erreur par itérations)



$\alpha = 1.5$, $\lambda = 1$





Interprétation

- Plus les valeurs de alpha sont grandes, plus la marge est grande et moins le système est permissif. Ainsi on obtient une classification plus strictes avec un alpha plus grand.
- Plus lambda est grand plus le modèle est régularisé donc moins le modèle est complexe (limite l'overfitting). A l'inverse si lambda est trop faible alors le modèle est plus complexe mais avec un plus fort risque d'overfitting.
- Le lien avec SVM réside dans la mise en place d'une marge entre les données et la frontière de décision avec un certain coefficient de permissivité. Par ailleurs, SVM utilise la fonction de coût hinge loss et son gradient associé.

IV - SVM et grid search

```
In [2]: from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.linear_model import Perceptron
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
```

IV.1) Comparaison sklearn et les modèles mis en place dans les tme précédents

a) Récupérer les données

```
In [16]: x, y = gen_arti(epsilon=0.8)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_
y_train, y_test = y_train.ravel(), y_test.ravel())
```

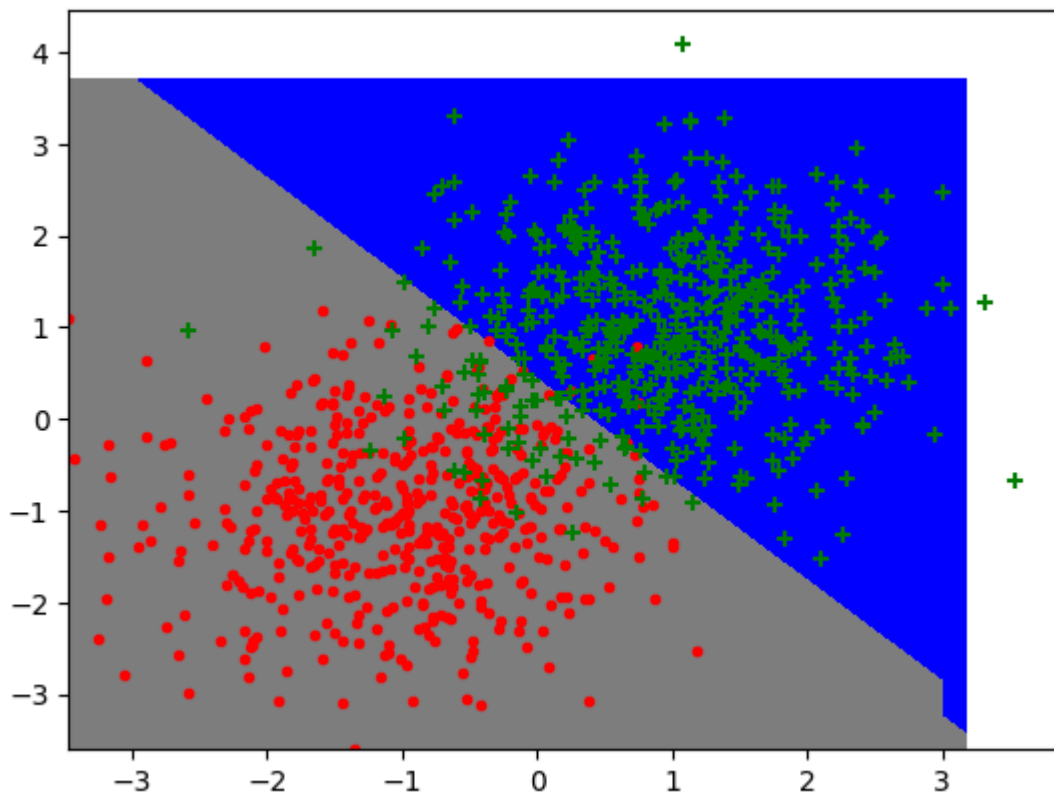
b) modèles sklearn

```
In [208... models = {
    "Perceptron": Perceptron(max_iter=1000, tol=1e-3),
    "k-NN": KNeighborsClassifier(n_neighbors=5),
    "Arbre de décision": DecisionTreeClassifier(max_depth=5),
    "SVM rbf": SVC(kernel='rbf', C=1.0), # Ajout du SVM linéaire
}

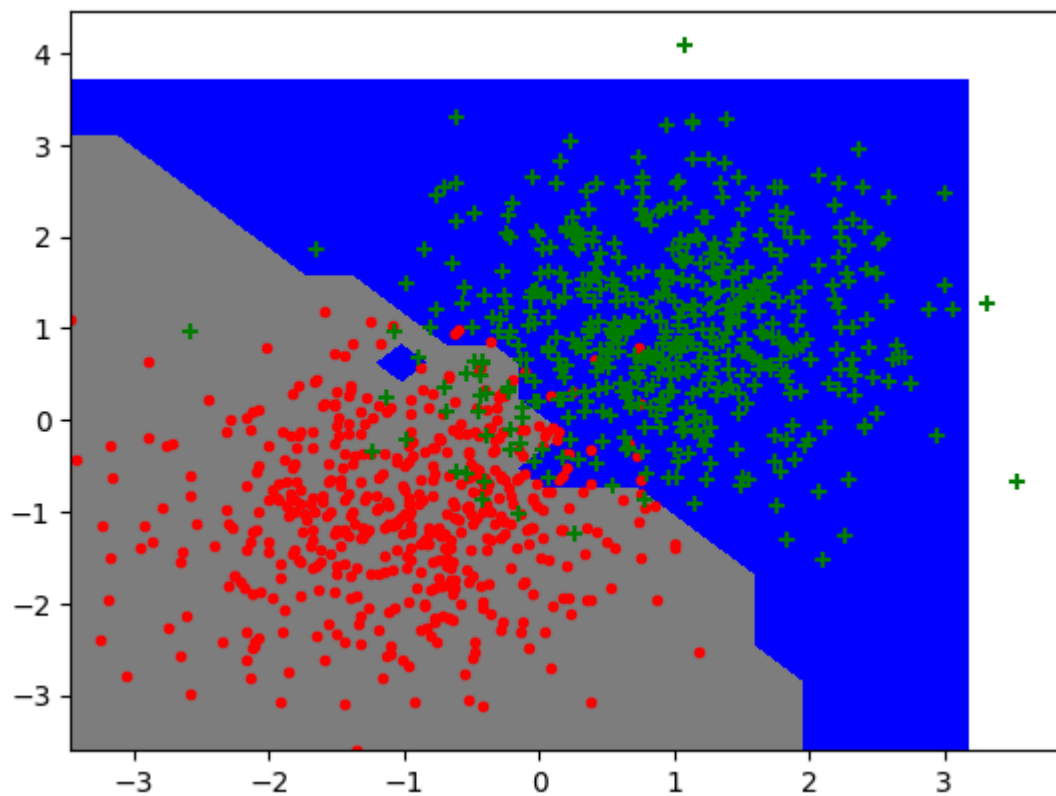
for name, model in models.items():
    model.fit(x_train, y_train)
    train_score = model.score(x_train, y_train)
    test_score = model.score(x_test, y_test)
    print(f"{name} - Train Acc: {train_score:.2f}, Test Acc: {test_score:.2f}")

    plot_frontiere(x, model.predict)
    plot_data(x, y)
    plt.show()
```

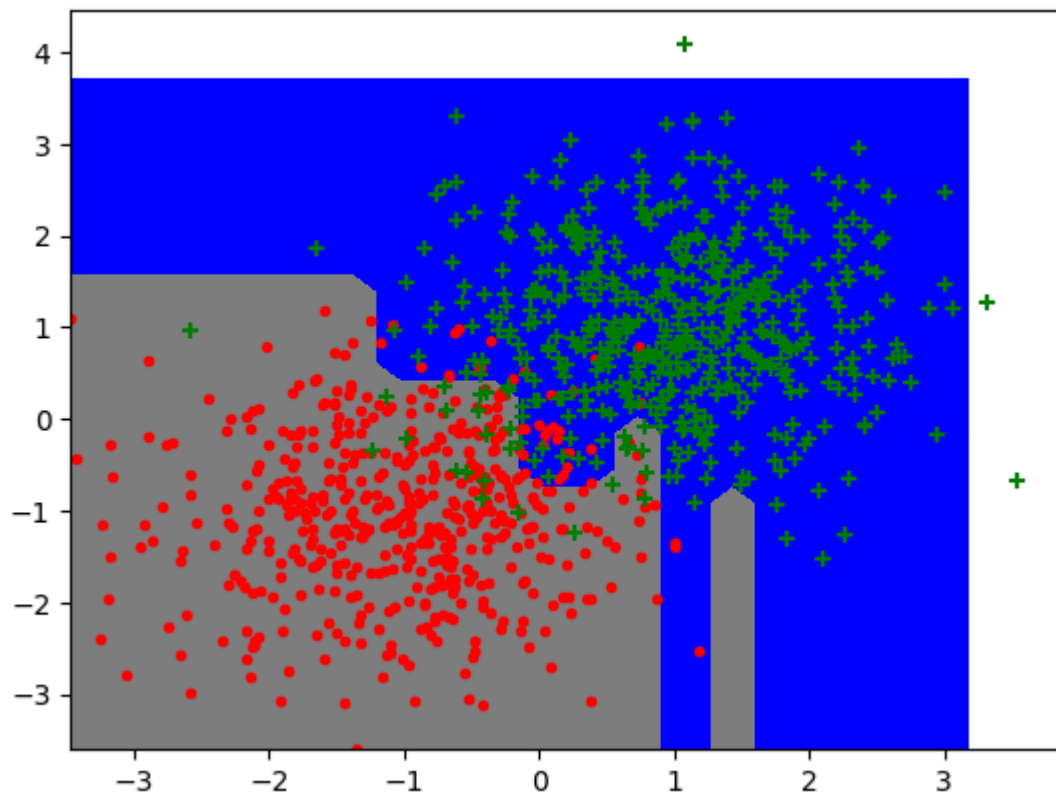
Perceptron - Train Acc: 0.94, Test Acc: 0.94



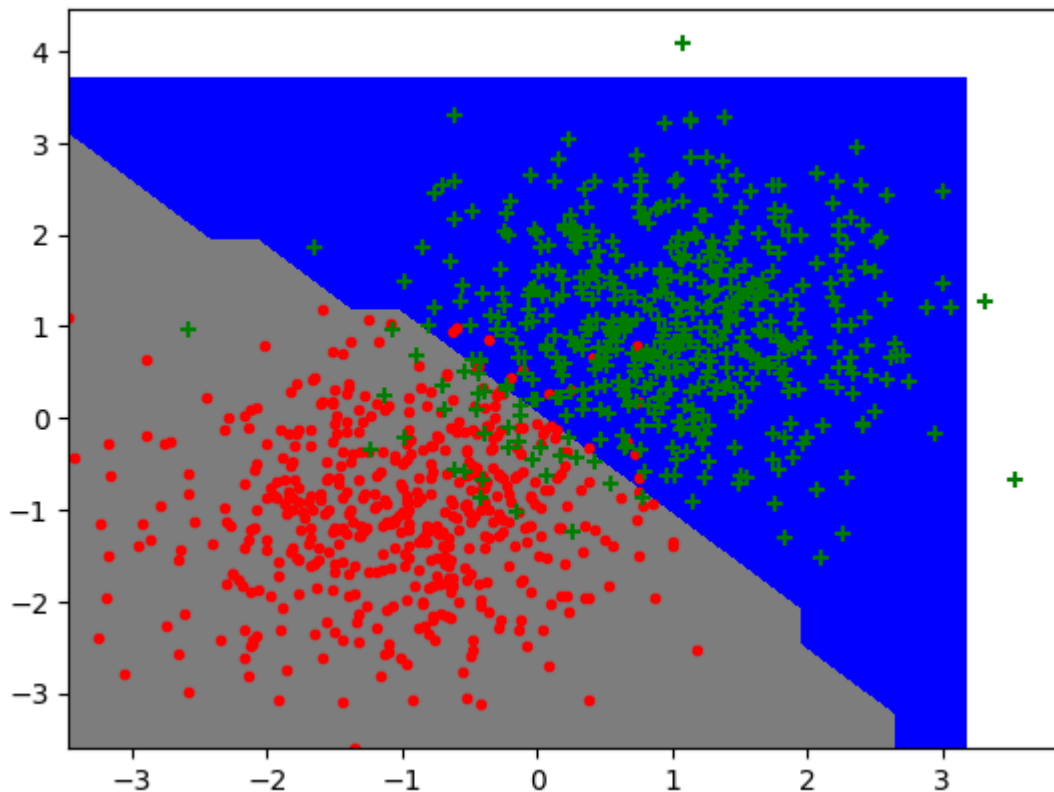
k-NN - Train Acc: 0.95, Test Acc: 0.94



Arbre de décision - Train Acc: 0.96, Test Acc: 0.93



SVM rbf - Train Acc: 0.95, Test Acc: 0.96



Tous les modèles sont performants (>90%). Le plus homogène semble être le k-NN avec des performances similaires en train et test.

c) Comparaison perceptron DIY vs perceptron Sklearn

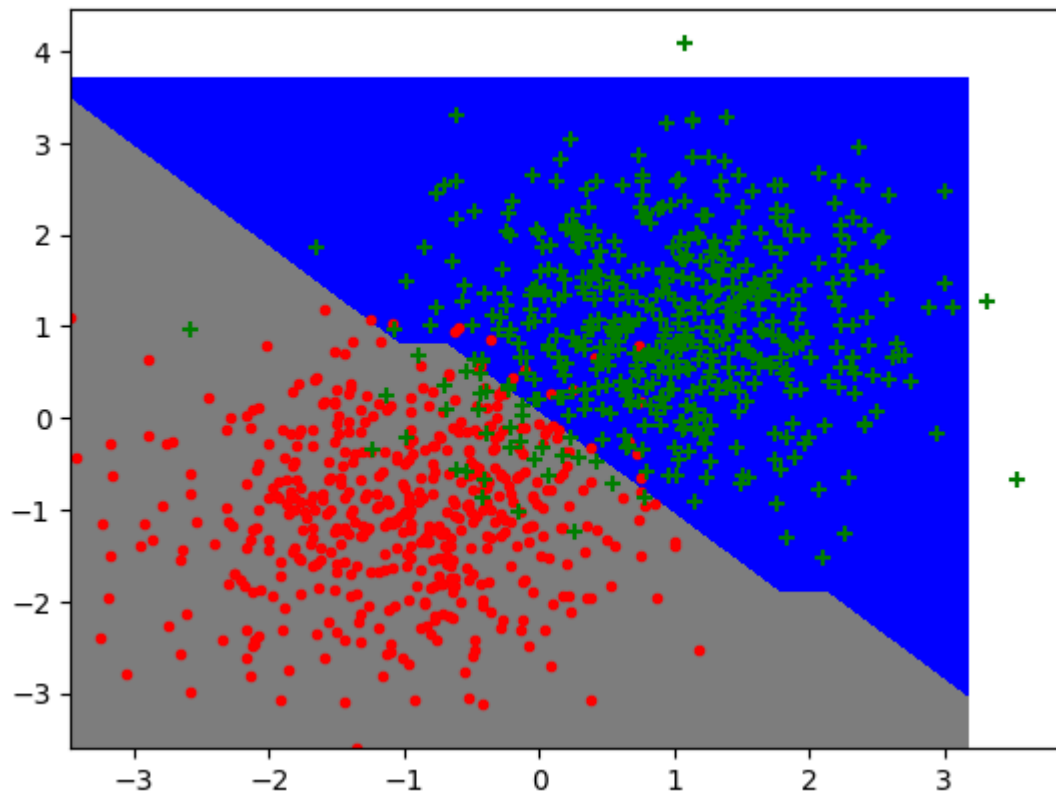
In [210...

```
models = {
    "DIY" : Lineaire(loss=perceptron_loss, loss_g=perceptron_grad, max_iter=300,
    "Perceptron": Perceptron(max_iter=300, tol=1e-3)
}

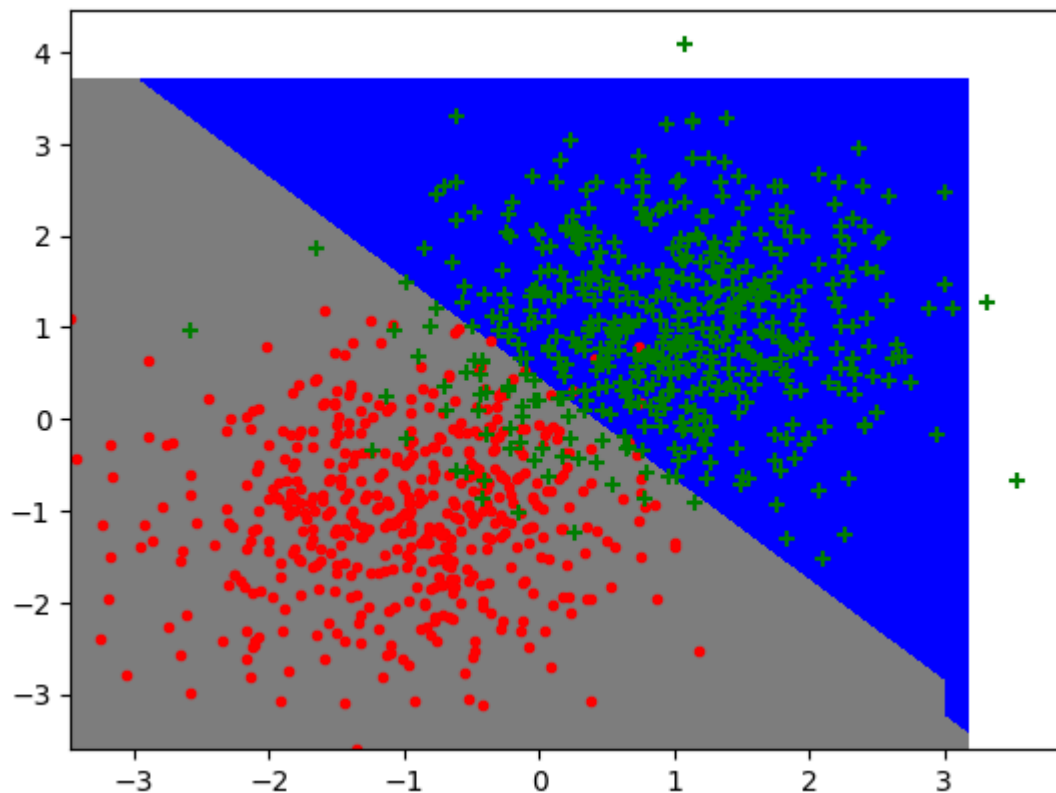
for name, model in models.items():
    model.fit(x_train, y_train)
    train_score = model.score(x_train, y_train)
    test_score = model.score(x_test, y_test)
    print(f"{name} - Train Acc: {train_score:.2f}, Test Acc: {test_score:.2f}")

    plot_frontiere(x, model.predict)
    plot_data(x, y)
    plt.show()
```

DIY - Train Acc: 0.94, Test Acc: 0.96



Perceptron - Train Acc: 0.94, Test Acc: 0.94



Le perceptron que nous avons produit semble avoir de meilleures performances que celui de sklearn.

IV.2) étude des différents svm (linéaires, rbf, poly)

```
In [ ]: def plot_frontiere_proba(data, f, step=20):
        """
```

Plot the decision boundary based on the probability function.

Params:

data (np.ndarray): input features to define the grid
f (callable): function that outputs probabilities or decision values
step (int): step size for grid resolution (default is 20)

Side Effects:

Displays the contour plot of the decision boundary

"""

```
grid, x, y = make_grid(data=data, step=step)
plt.contourf(x, y, f(grid).reshape(x.shape), 255)
```

In []: **def** plot_data_with_support_vectors(x, y, support_vectors):

"""

Plot data points and highlight the support vectors.

Params:

x (np.ndarray): input features
y (np.ndarray): labels (+1 or -1)
support_vectors (np.ndarray): coordinates of the support vectors

Side Effects:

Displays a scatter plot of data points with support vectors highlighted

"""

```
plt.scatter(x[:, 0], x[:, 1], c=y, cmap=plt.cm.Paired, edgecolors='k', label=
plt.scatter(support_vectors[:, 0], support_vectors[:, 1], facecolors='none',
```

In [239...

```
for s in ["linear", "rbf", "poly"]:
    for C in [0.1, 1, 10] :
        for gamma in [0.01, 0.1, 1] :
            print(f"Modèle de SVM : {s}, C = {C}, gamma = {gamma}")
            model = SVC(kernel=s, C=C, gamma=gamma, probability=True)
            model.fit(x_train, y_train)

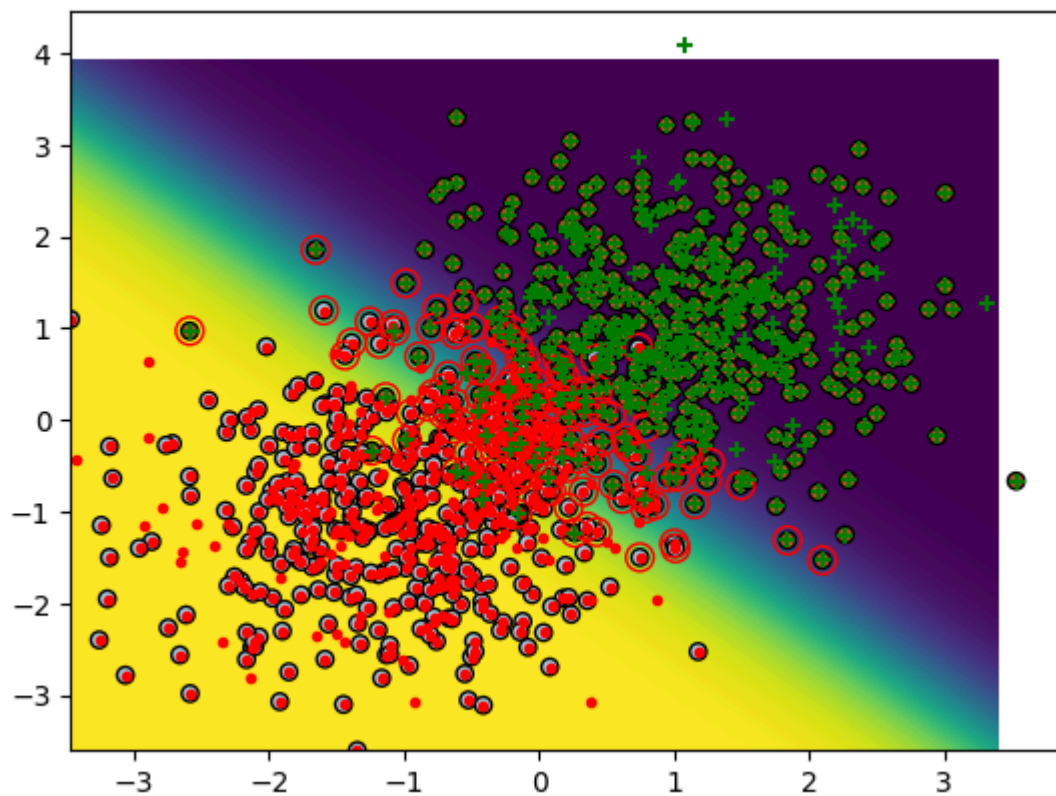
            support_vectors = model.support_vectors_

            train_score = model.score(x_train, y_train)
            test_score = model.score(x_test, y_test)
            print(f"{name} - Train Acc: {train_score:.2f}, Test Acc: {test_score:.2f}")

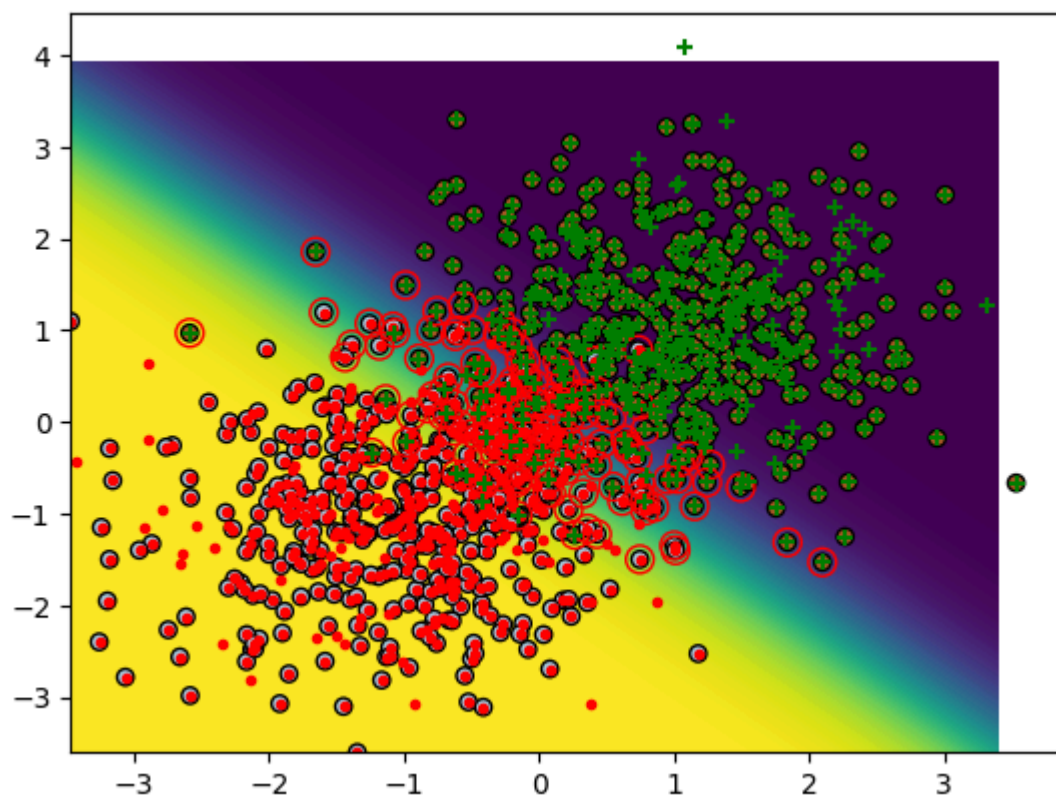
            plot_frontiere_proba(x, lambda x: model.predict_proba(x)[:, 0], step=20)
            plot_data_with_support_vectors(x_train, y_train, model.support_vectors_)
            plot_data(x, y)
            plt.show()
```

Modèle de SVM : linear, C = 0.1, gamma = 0.01

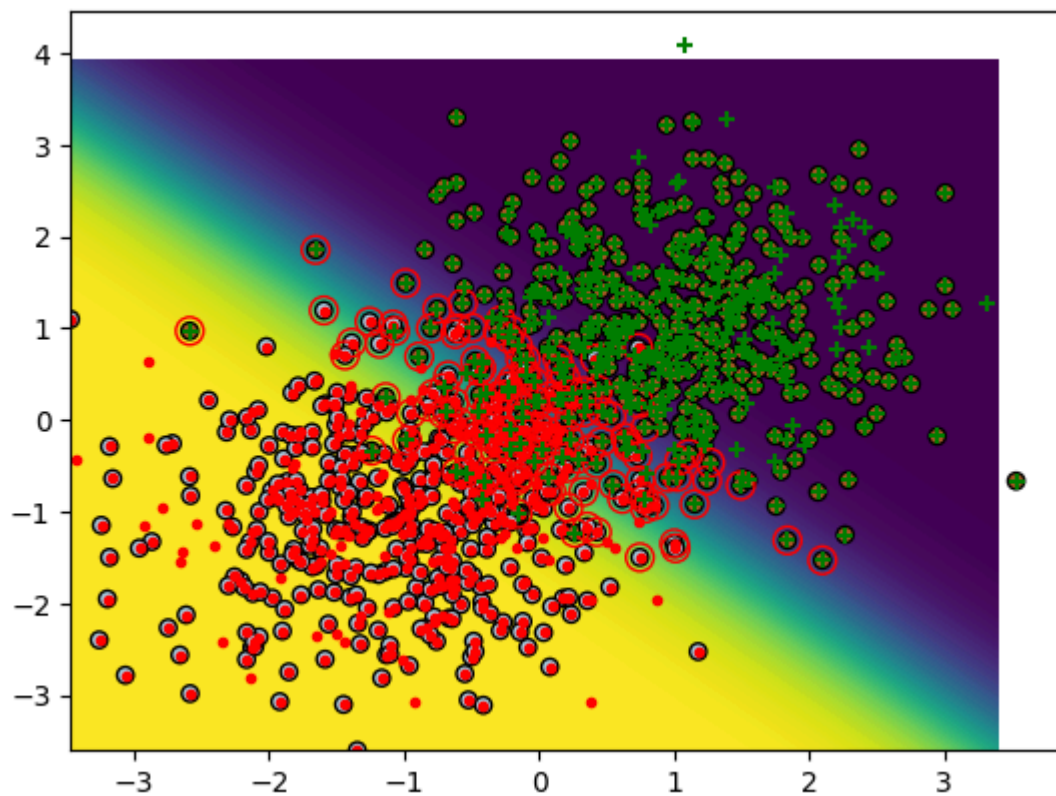
Perceptron - Train Acc: 0.94, Test Acc: 0.96



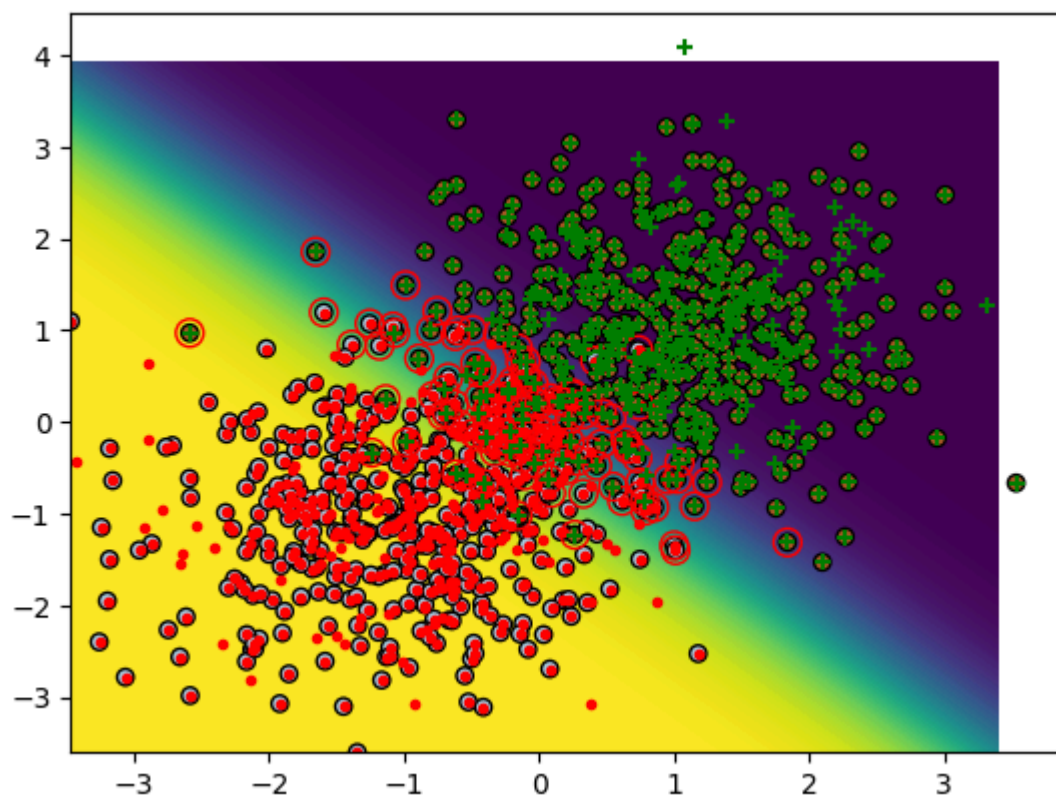
Modèle de SVM : linear, $C = 0.1$, $\gamma = 0.1$
 Perceptron - Train Acc: 0.94, Test Acc: 0.96



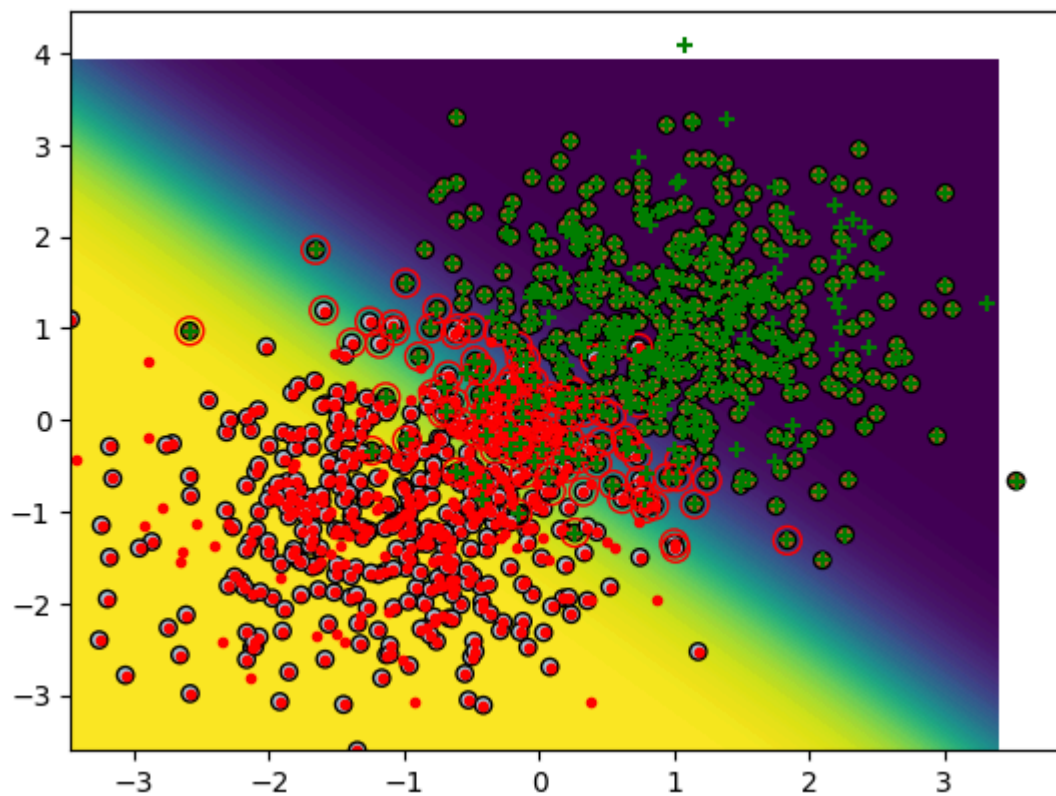
Modèle de SVM : linear, $C = 0.1$, $\gamma = 1$
 Perceptron - Train Acc: 0.94, Test Acc: 0.96



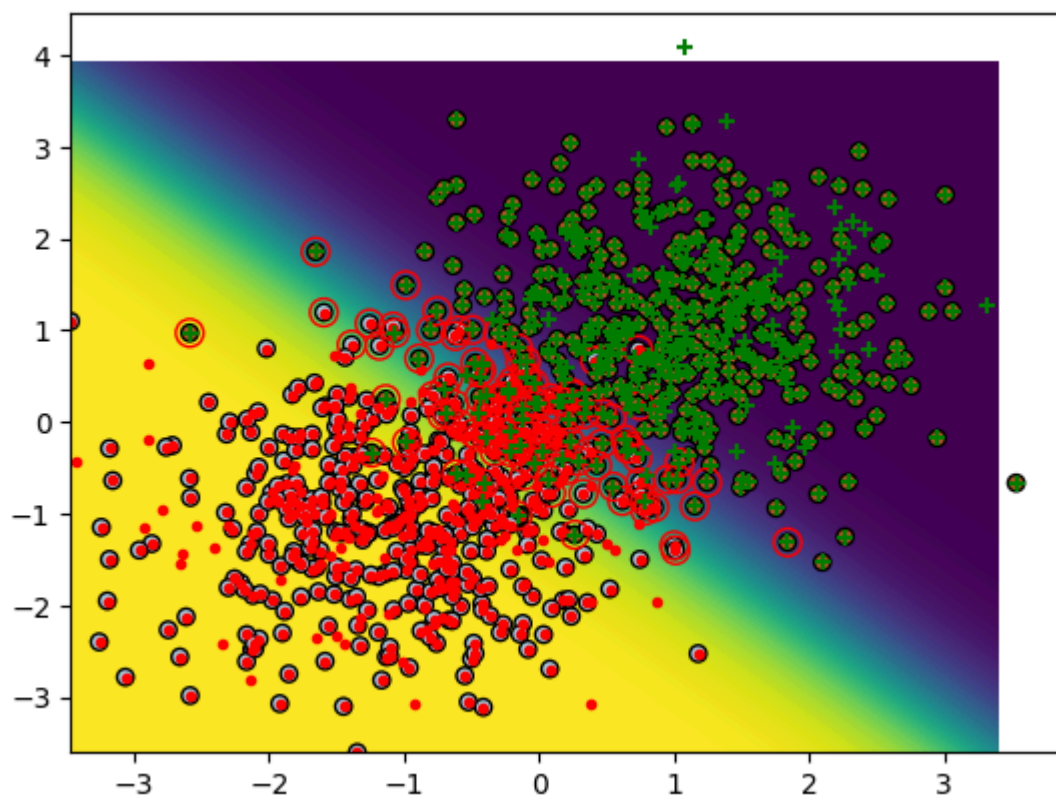
Modèle de SVM : linear, $C = 1$, $\gamma = 0.01$
 Perceptron - Train Acc: 0.94, Test Acc: 0.96



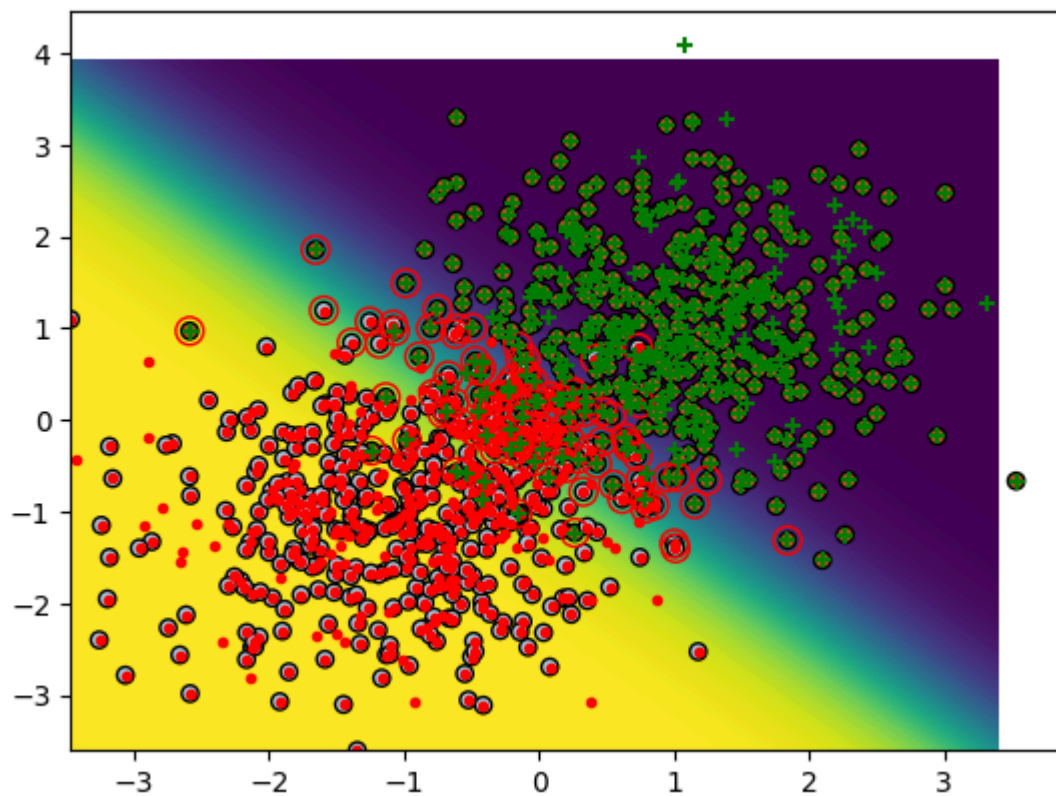
Modèle de SVM : linear, $C = 1$, $\gamma = 0.1$
 Perceptron - Train Acc: 0.94, Test Acc: 0.96



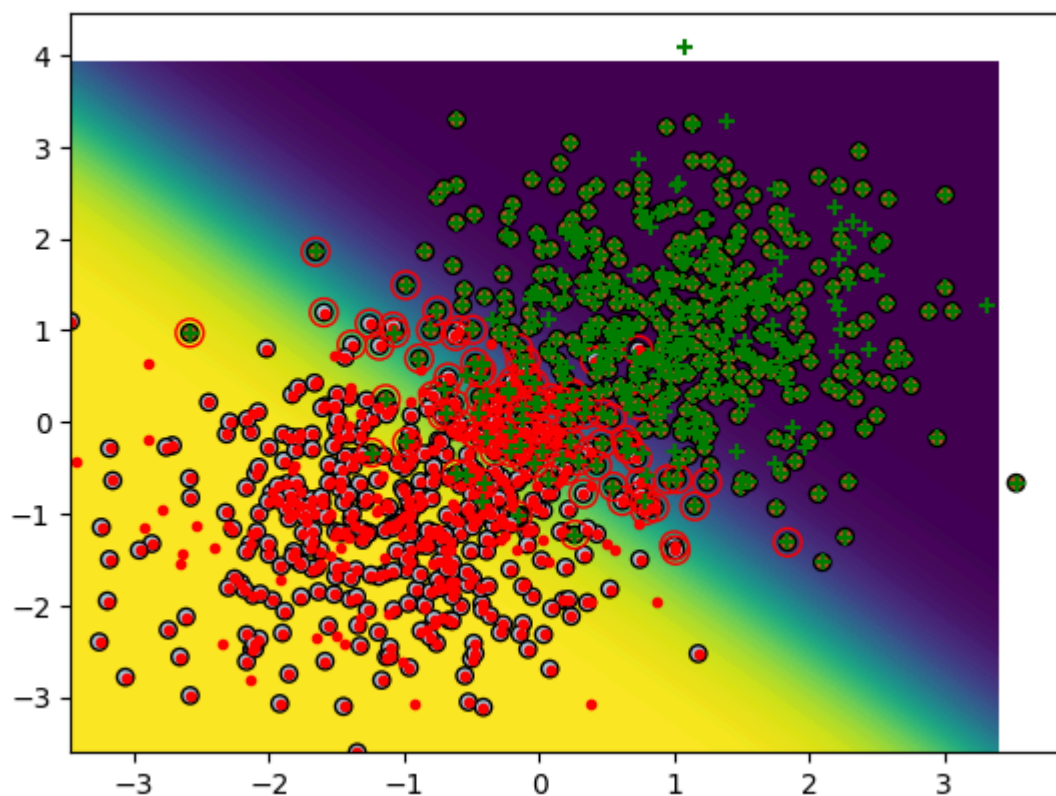
Modèle de SVM : linear, $C = 1$, $\gamma = 1$
 Perceptron - Train Acc: 0.94, Test Acc: 0.96



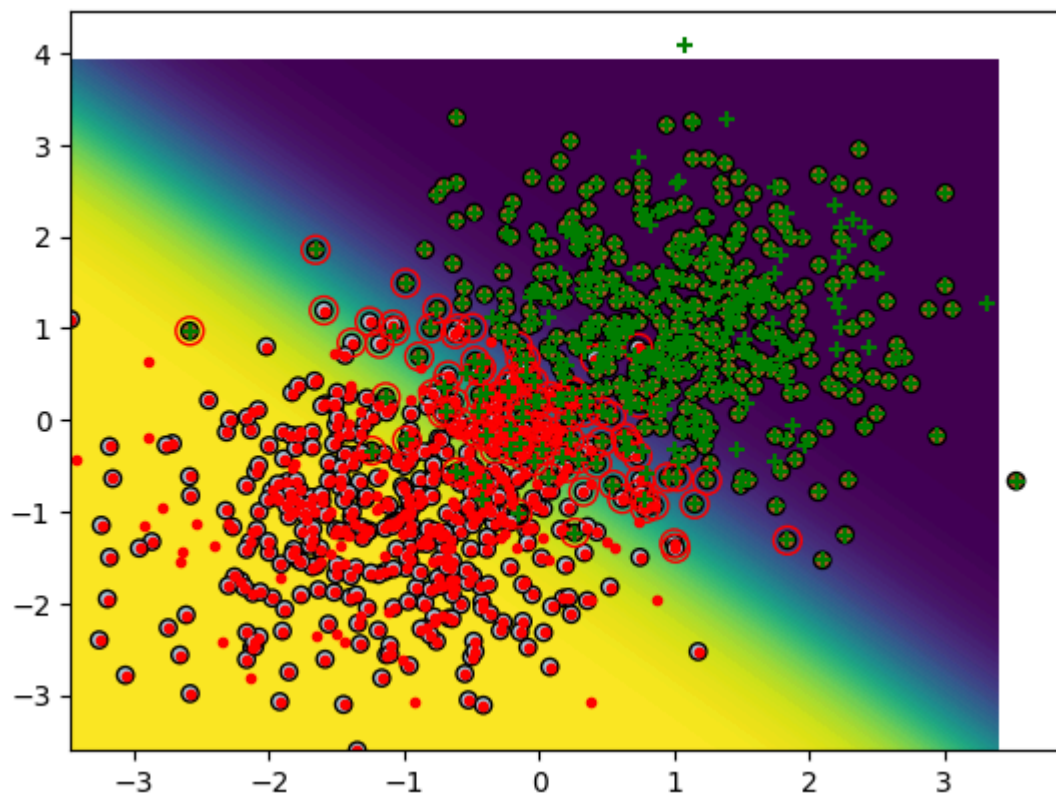
Modèle de SVM : linear, $C = 10$, $\gamma = 0.01$
 Perceptron - Train Acc: 0.95, Test Acc: 0.96



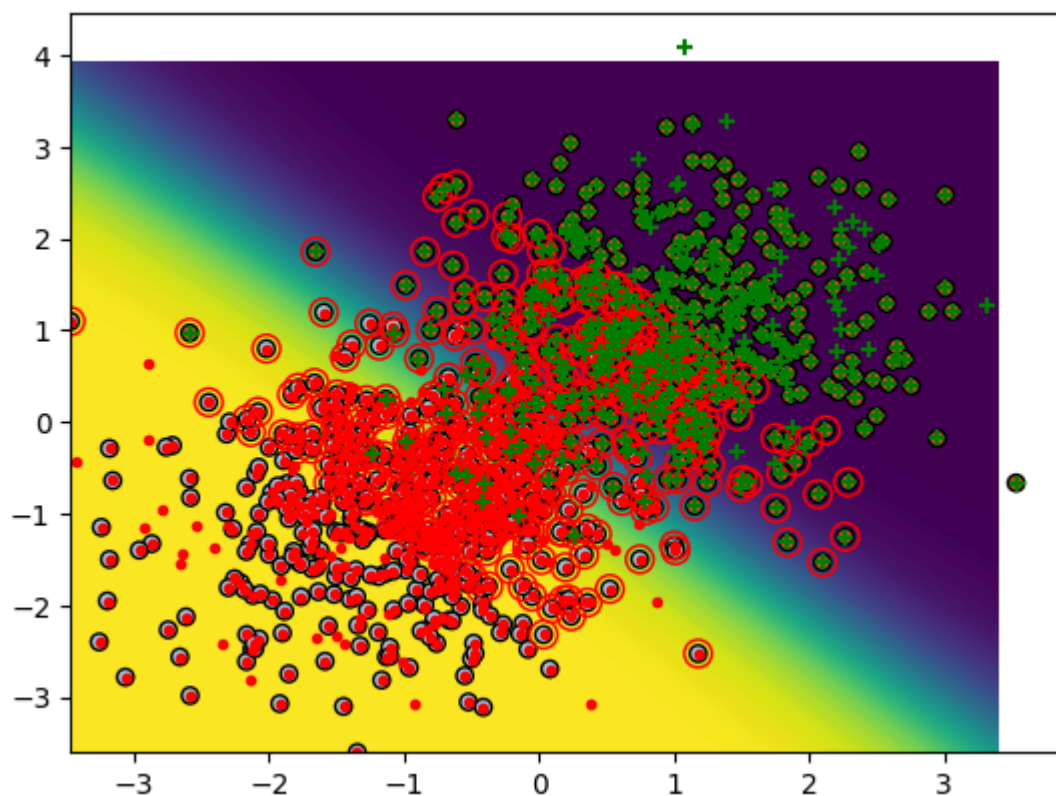
Modèle de SVM : linear, $C = 10$, $\gamma = 0.1$
 Perceptron - Train Acc: 0.95, Test Acc: 0.96



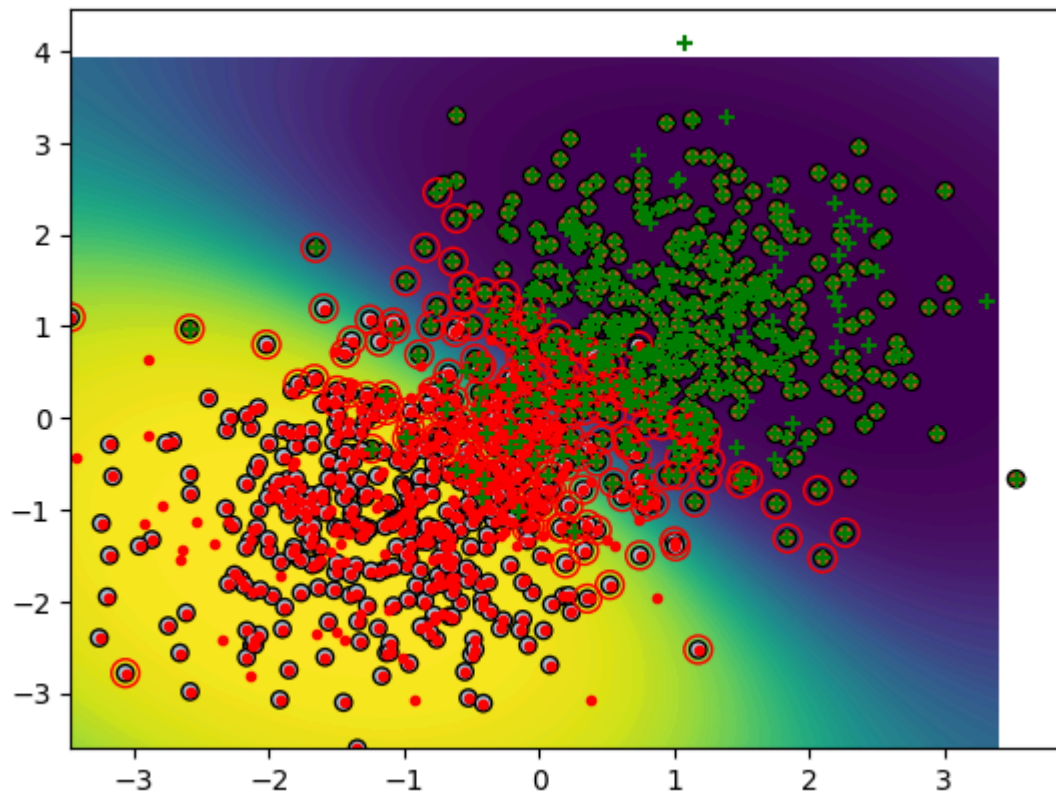
Modèle de SVM : linear, $C = 10$, $\gamma = 1$
 Perceptron - Train Acc: 0.95, Test Acc: 0.96



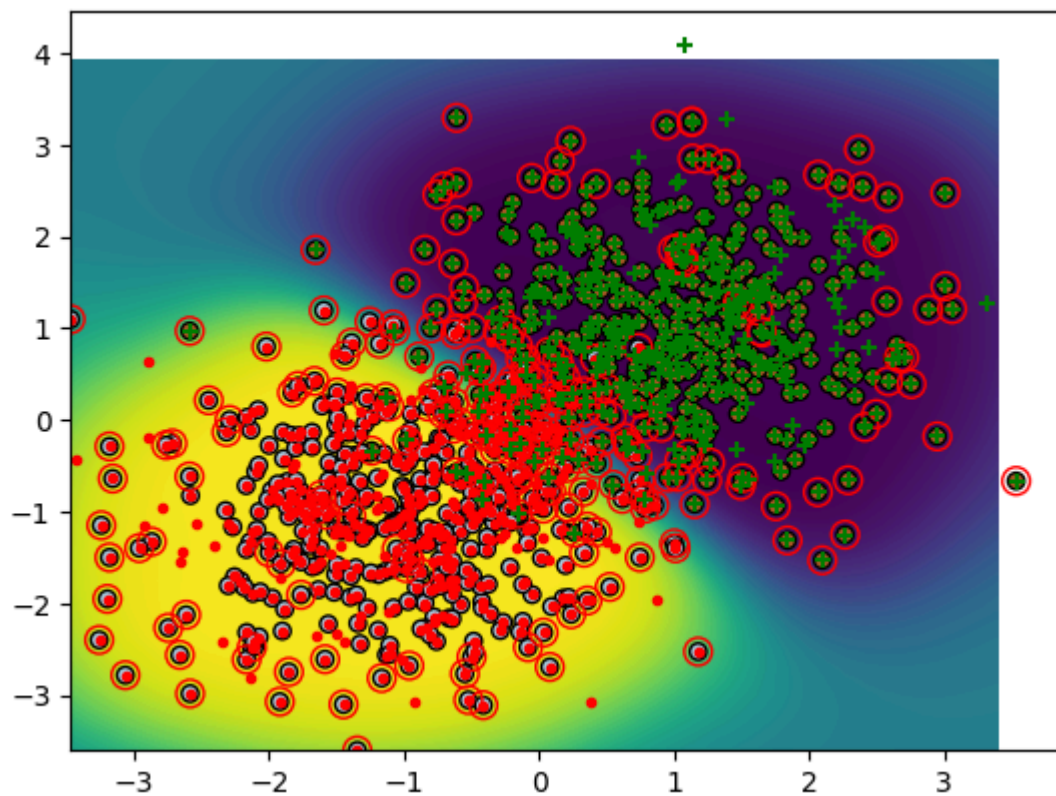
Modèle de SVM : rbf, $C = 0.1$, $\gamma = 0.01$
 Perceptron - Train Acc: 0.94, Test Acc: 0.96



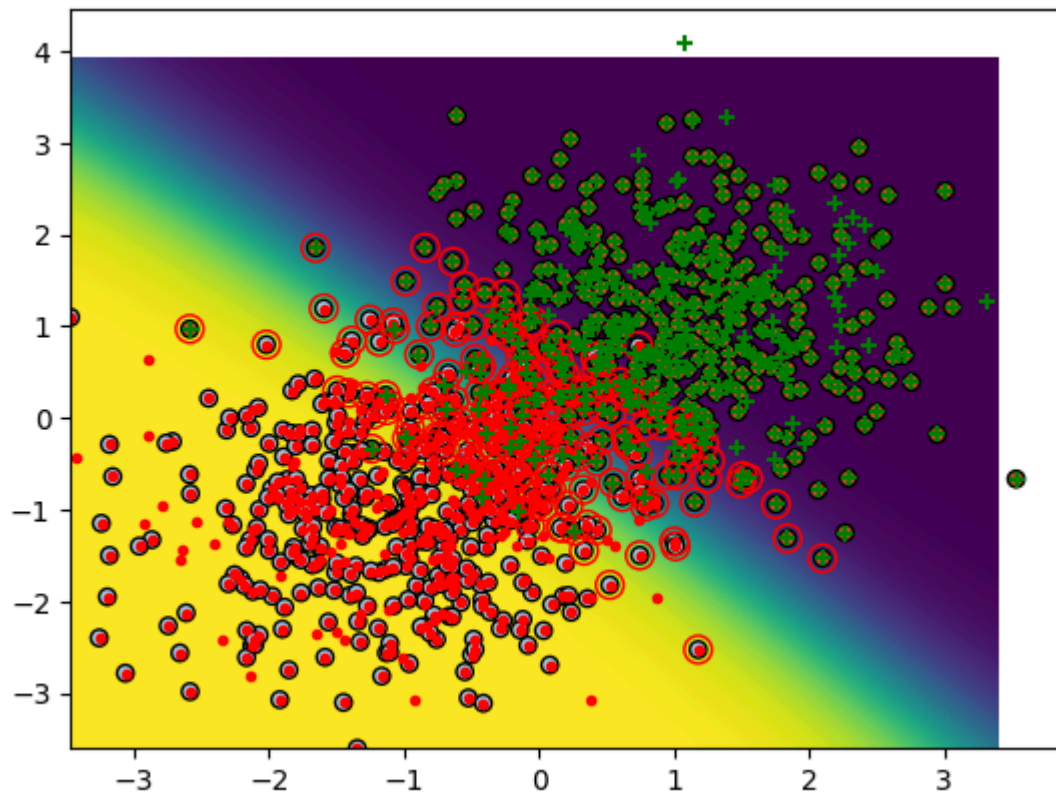
Modèle de SVM : rbf, $C = 0.1$, $\gamma = 0.1$
 Perceptron - Train Acc: 0.95, Test Acc: 0.96



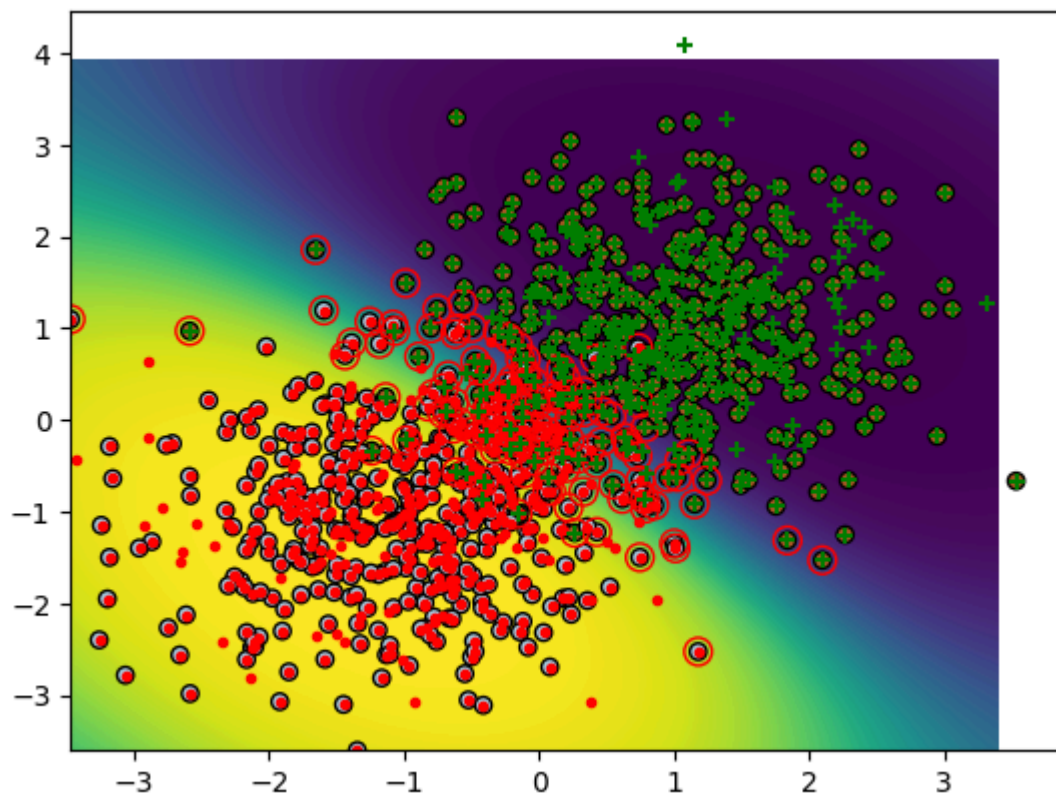
Modèle de SVM : rbf, $C = 0.1$, $\gamma = 1$
 Perceptron - Train Acc: 0.94, Test Acc: 0.97



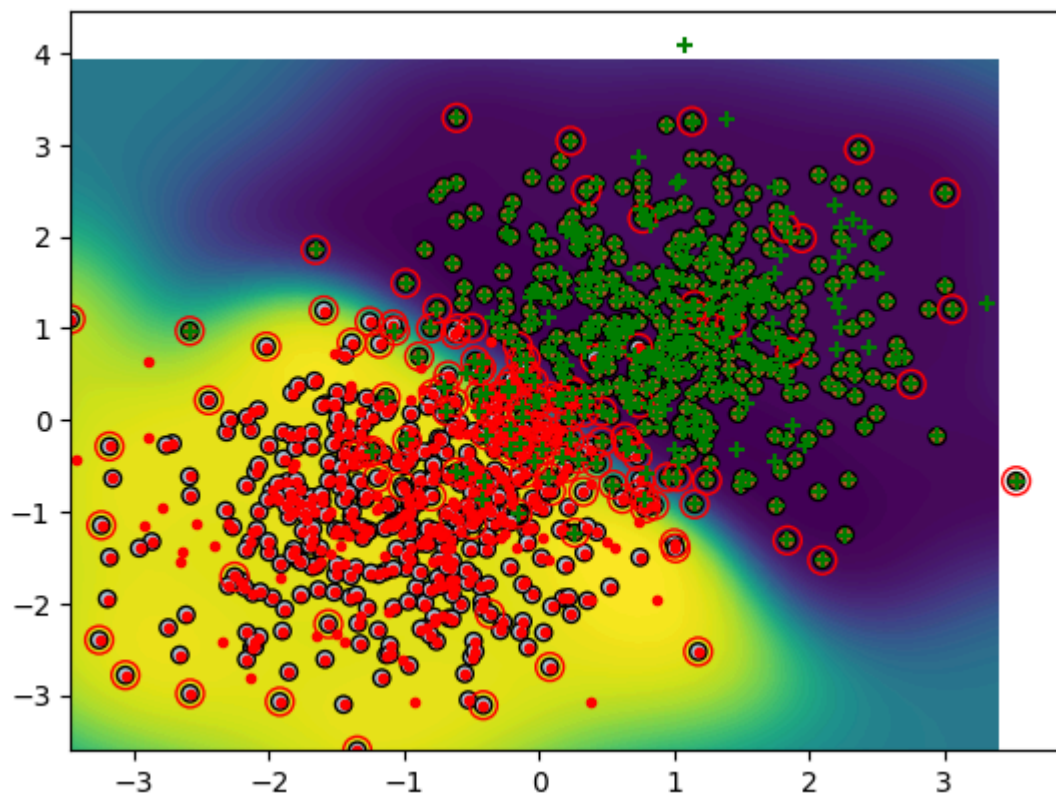
Modèle de SVM : rbf, $C = 1$, $\gamma = 0.01$
 Perceptron - Train Acc: 0.94, Test Acc: 0.96



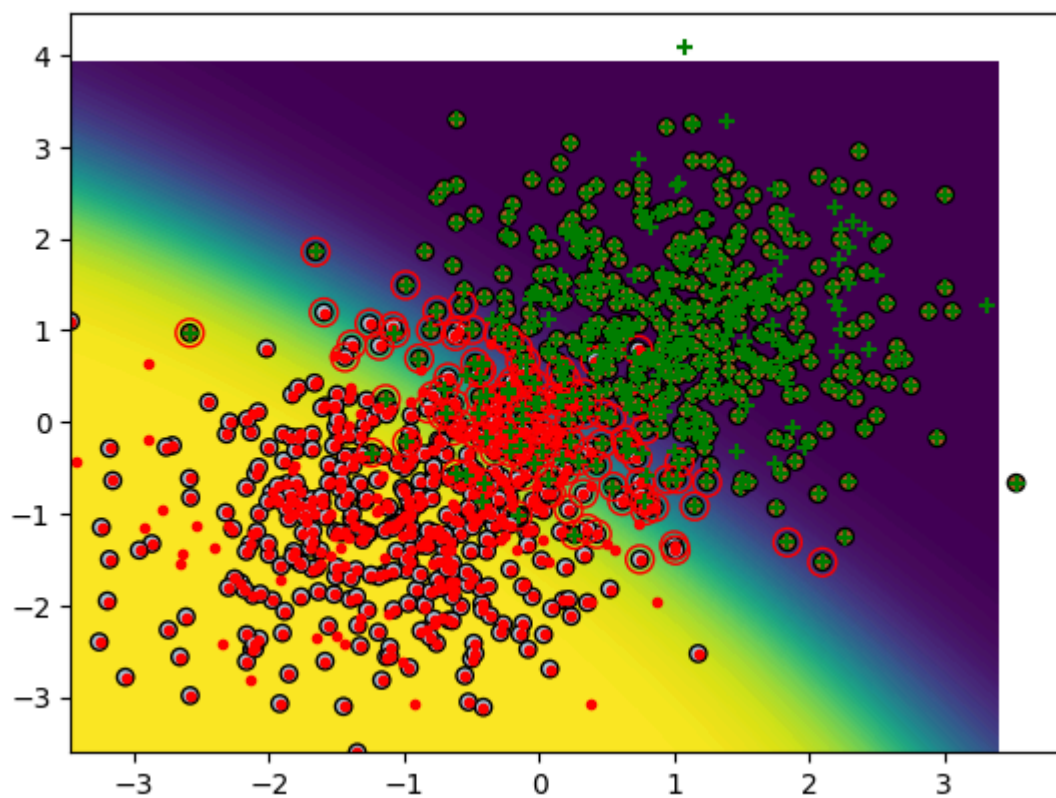
Modèle de SVM : rbf, $C = 1$, $\gamma = 0.1$
 Perceptron - Train Acc: 0.94, Test Acc: 0.96



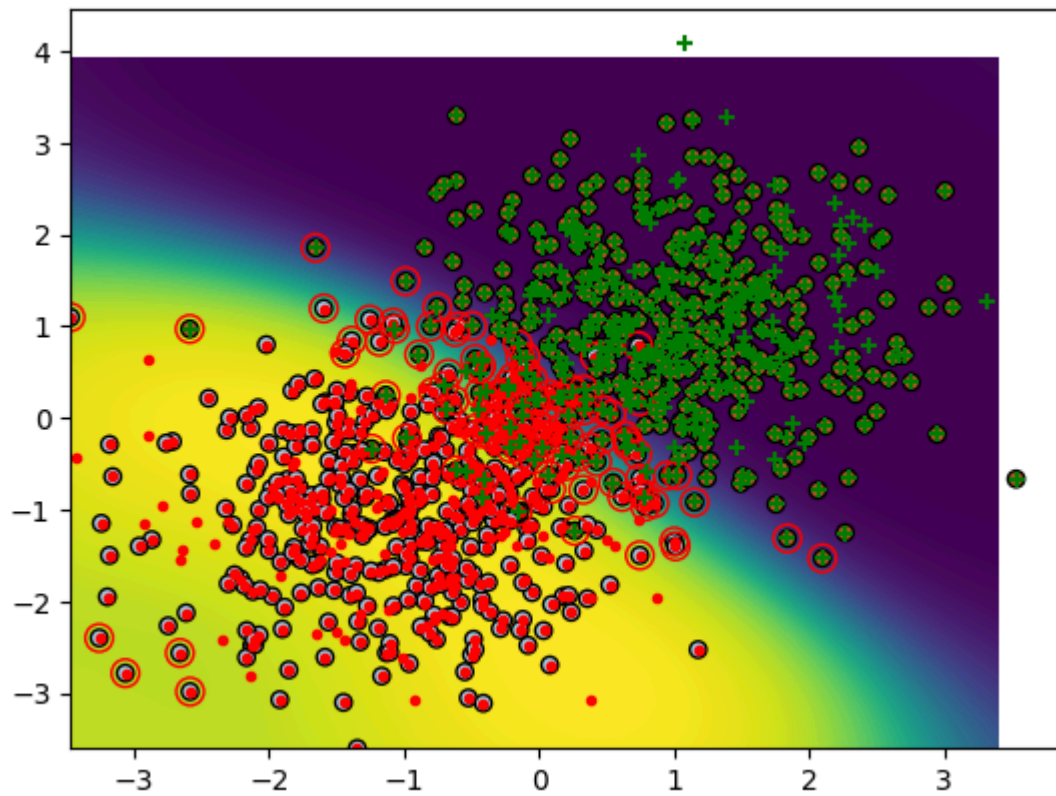
Modèle de SVM : rbf, $C = 1$, $\gamma = 1$
 Perceptron - Train Acc: 0.95, Test Acc: 0.96



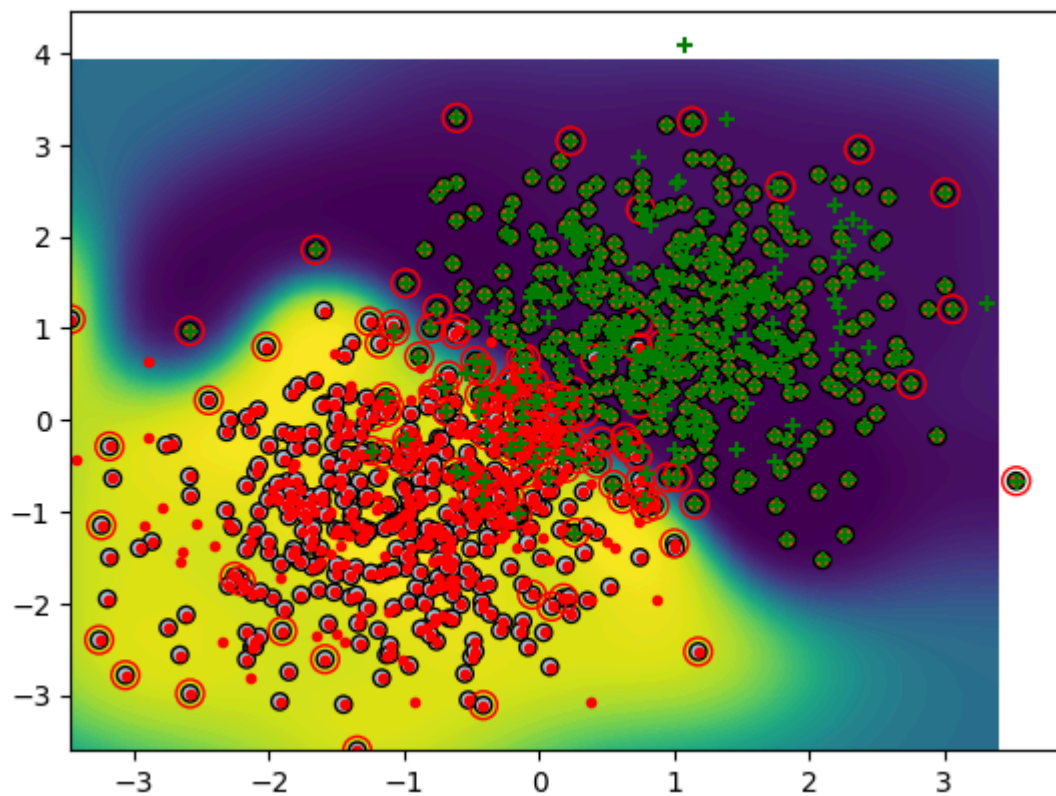
Modèle de SVM : rbf, $C = 10$, $\gamma = 0.01$
 Perceptron - Train Acc: 0.94, Test Acc: 0.96



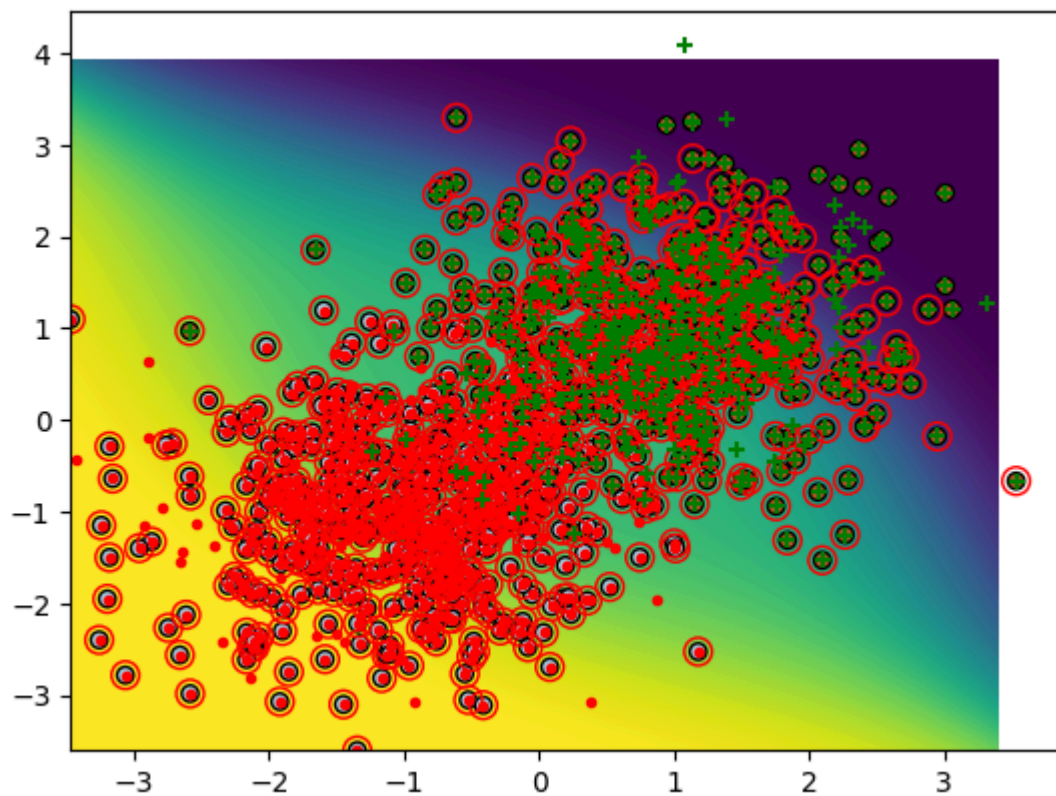
Modèle de SVM : rbf, $C = 10$, $\gamma = 0.1$
 Perceptron - Train Acc: 0.95, Test Acc: 0.96



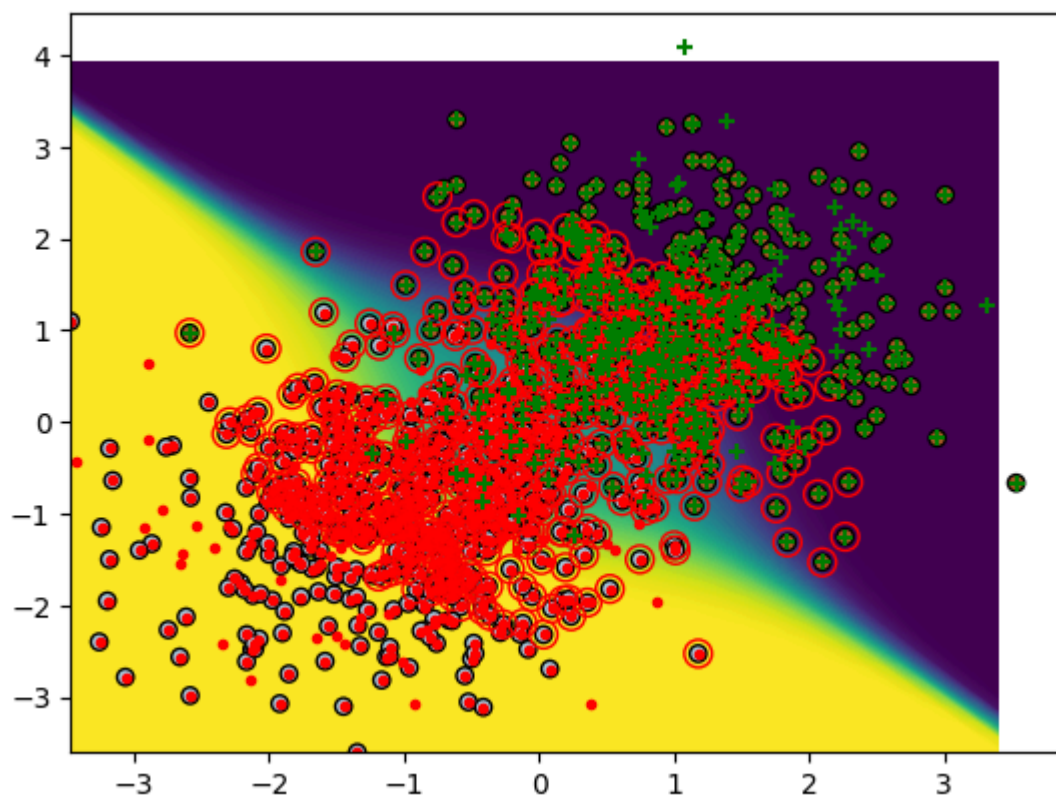
Modèle de SVM : rbf, $C = 10$, $\gamma = 1$
 Perceptron - Train Acc: 0.95, Test Acc: 0.95



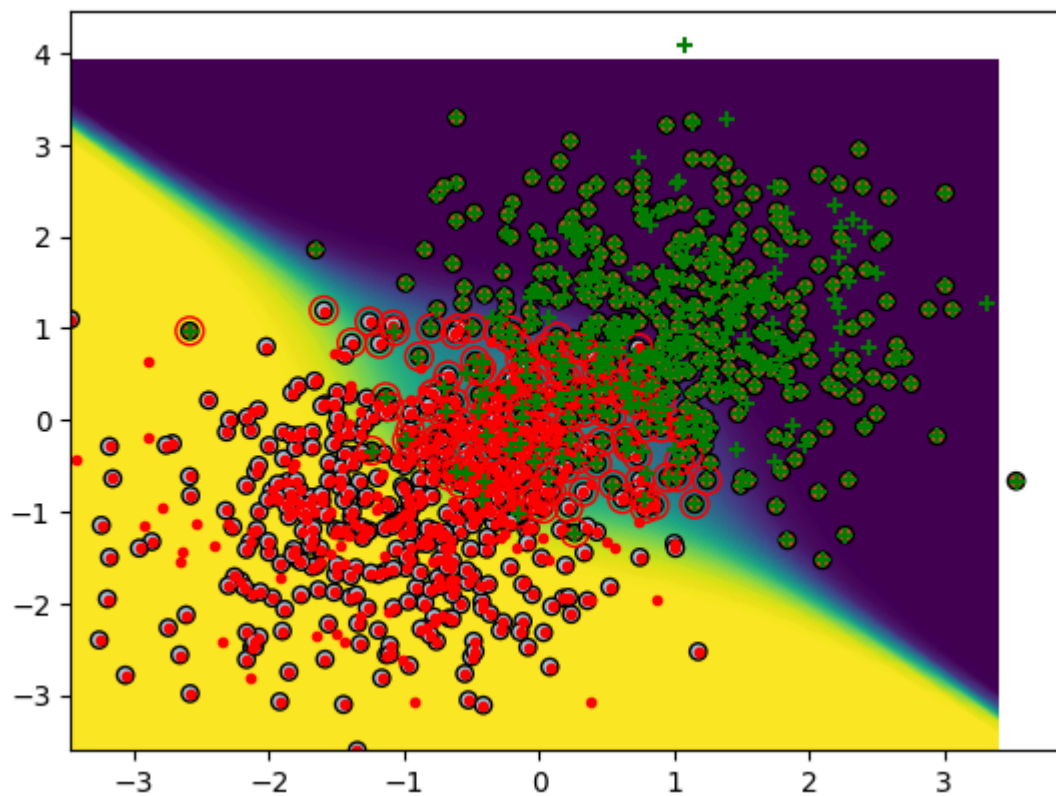
Modèle de SVM : poly, $C = 0.1$, $\gamma = 0.01$
 Perceptron - Train Acc: 0.51, Test Acc: 0.46



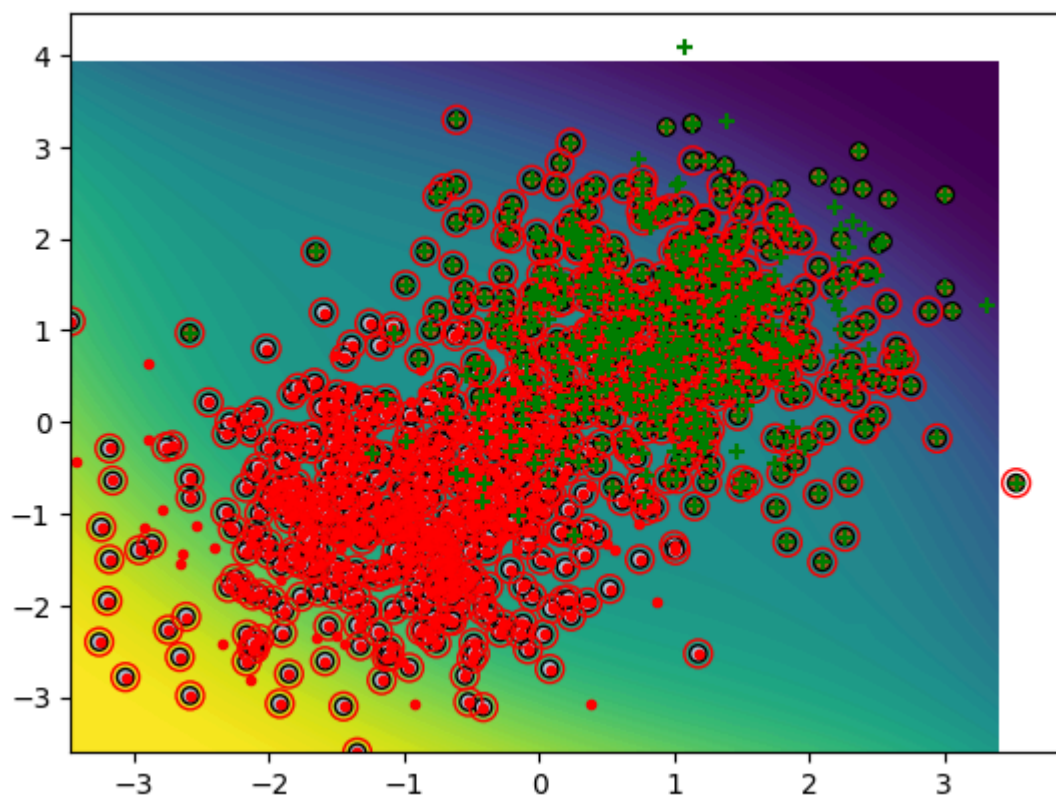
Modèle de SVM : poly, $C = 0.1$, $\gamma = 0.1$
 Perceptron - Train Acc: 0.91, Test Acc: 0.91



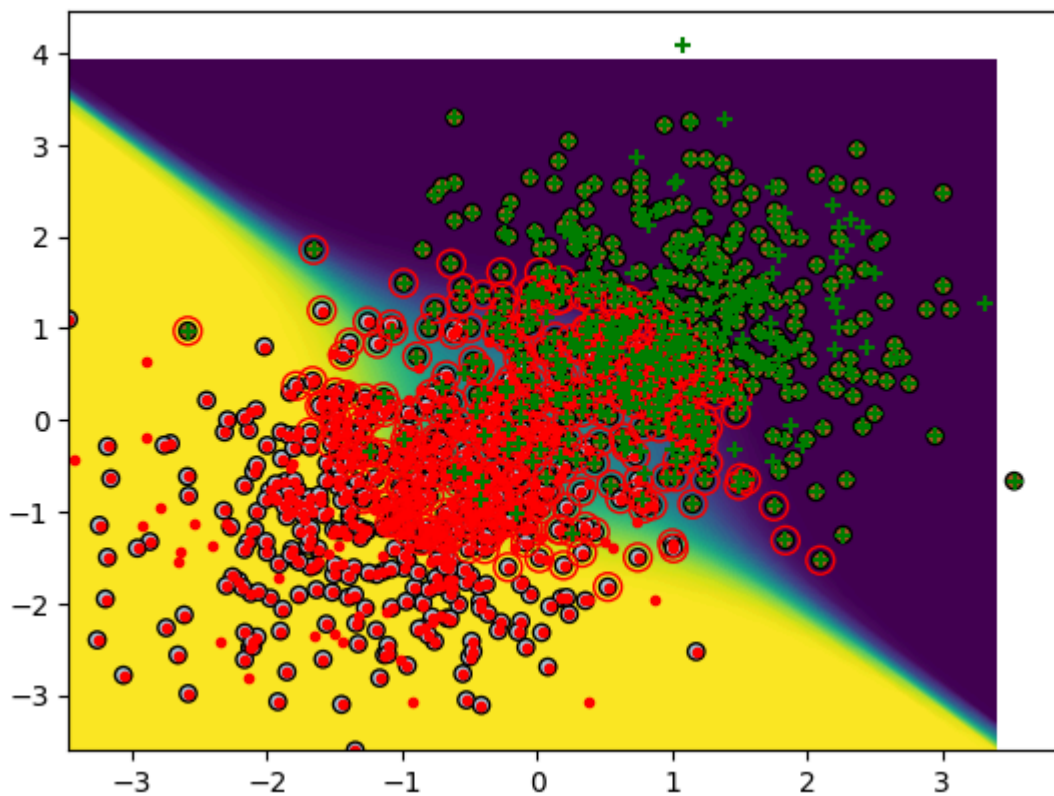
Modèle de SVM : poly, $C = 0.1$, $\gamma = 1$
 Perceptron - Train Acc: 0.93, Test Acc: 0.96



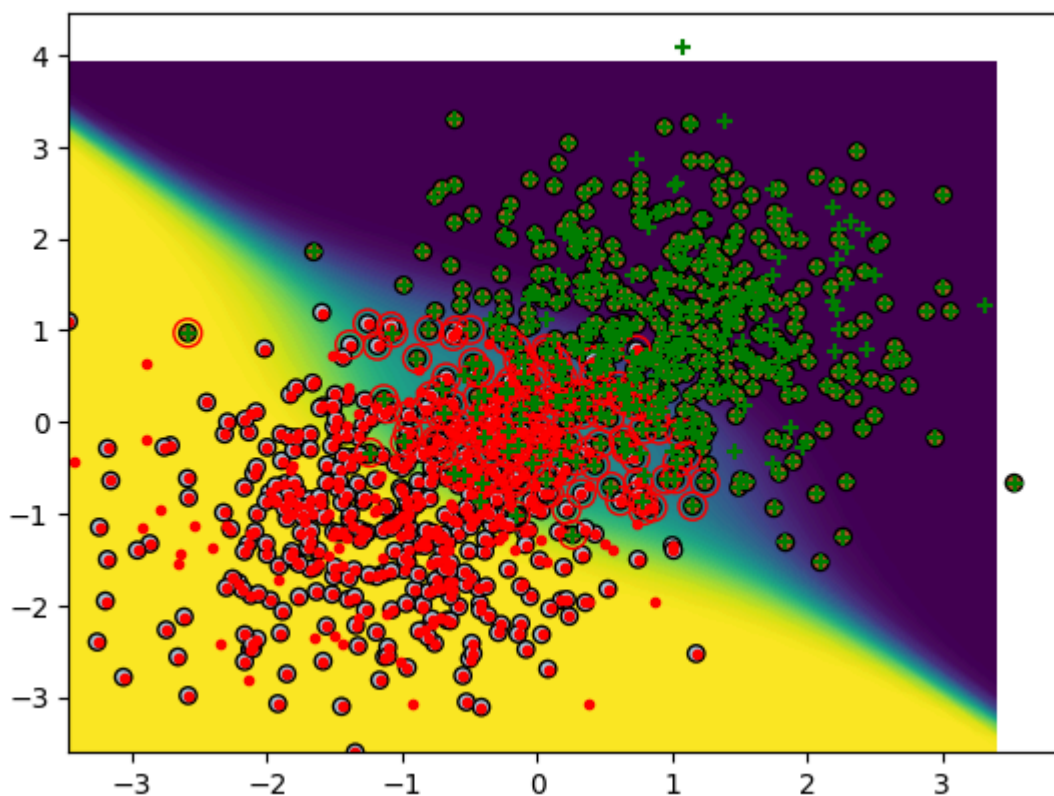
Modèle de SVM : poly, $C = 1$, $\gamma = 0.01$
 Perceptron - Train Acc: 0.51, Test Acc: 0.46



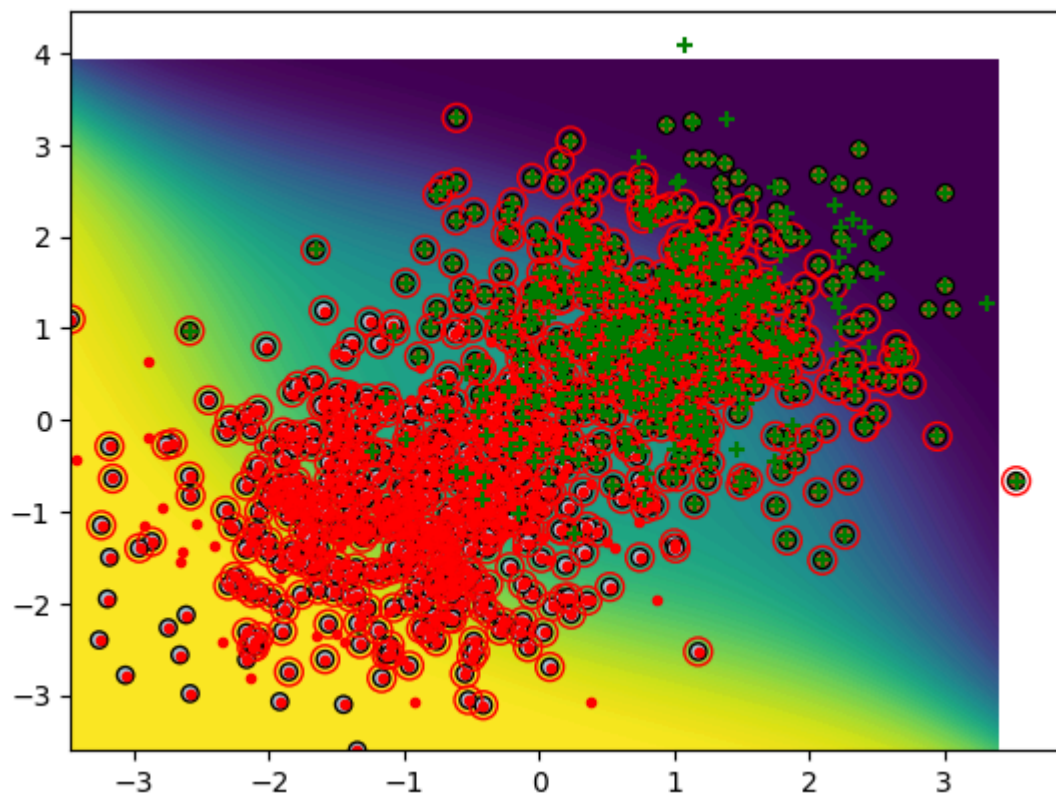
Modèle de SVM : poly, $C = 1$, $\gamma = 0.1$
 Perceptron - Train Acc: 0.91, Test Acc: 0.91



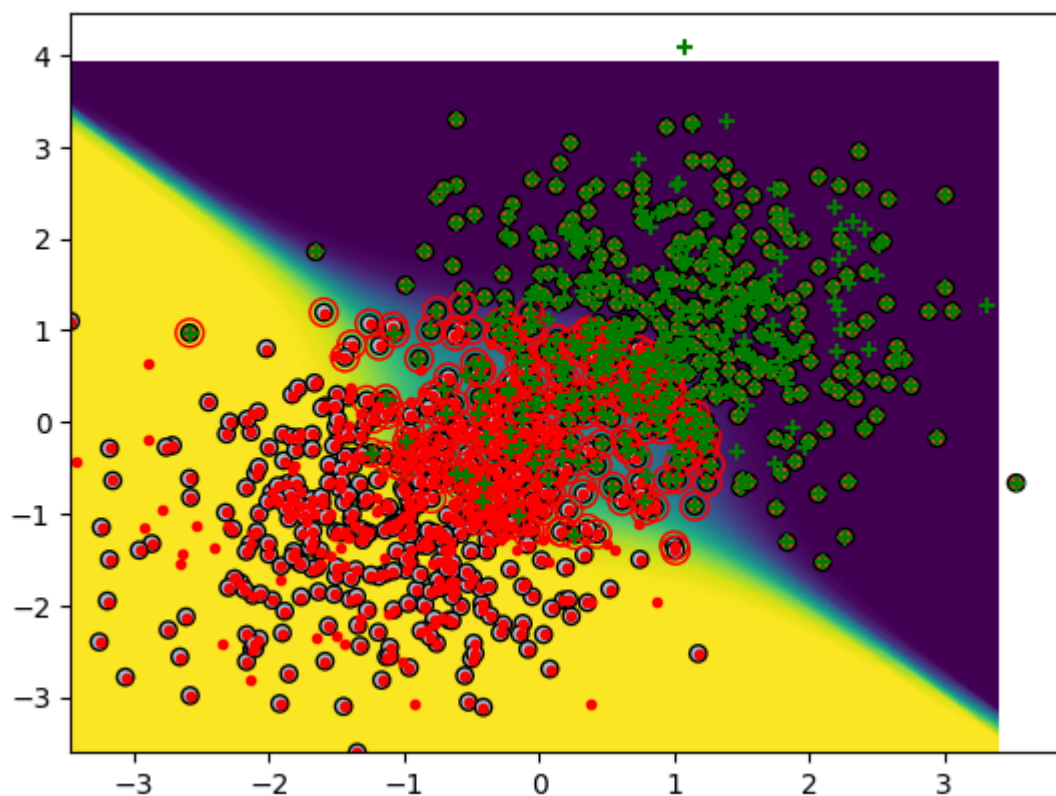
Modèle de SVM : poly, $C = 1$, $\gamma = 1$
 Perceptron - Train Acc: 0.94, Test Acc: 0.96



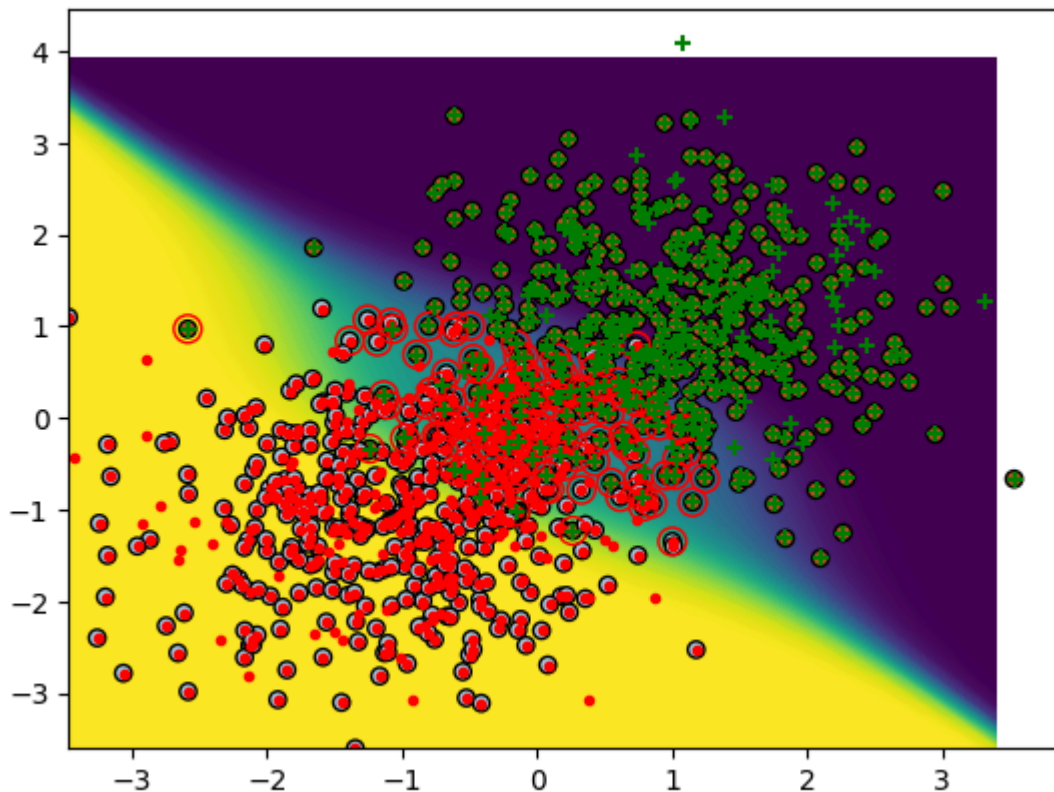
Modèle de SVM : poly, $C = 10$, $\gamma = 0.01$
 Perceptron - Train Acc: 0.64, Test Acc: 0.60



Modèle de SVM : poly, $C = 10$, $\gamma = 0.1$
 Perceptron - Train Acc: 0.93, Test Acc: 0.96



Modèle de SVM : poly, $C = 10$, $\gamma = 1$
 Perceptron - Train Acc: 0.93, Test Acc: 0.96



```
In [224... print("Avec sklearn on peut directement retrouver les paramètres optimaux :")

for s in ["linear", "rbf", "poly"]:
    params = {'C': [0.1, 1, 10], 'gamma': [0.01, 0.1, 1]}
    grid = GridSearchCV(SVC(kernel=s), params, cv=5)
    grid.fit(x_train, y_train)
    best_model = grid.best_estimator_
    best_params = grid.best_params_
    best_score = grid.best_score_
    print(f"Meilleurs paramètre pour le modèle SVM {s}:", best_params)

print("Ce qui s'observe sur les plots avec les frontières de décisions précédentes")
```

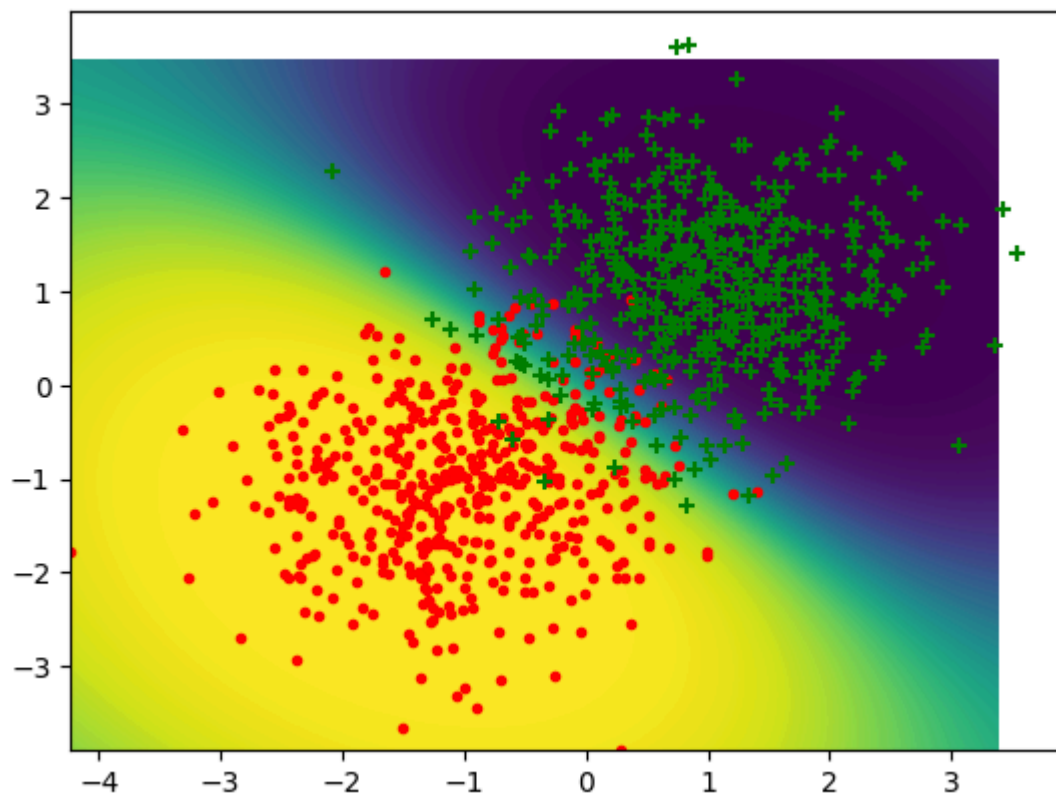
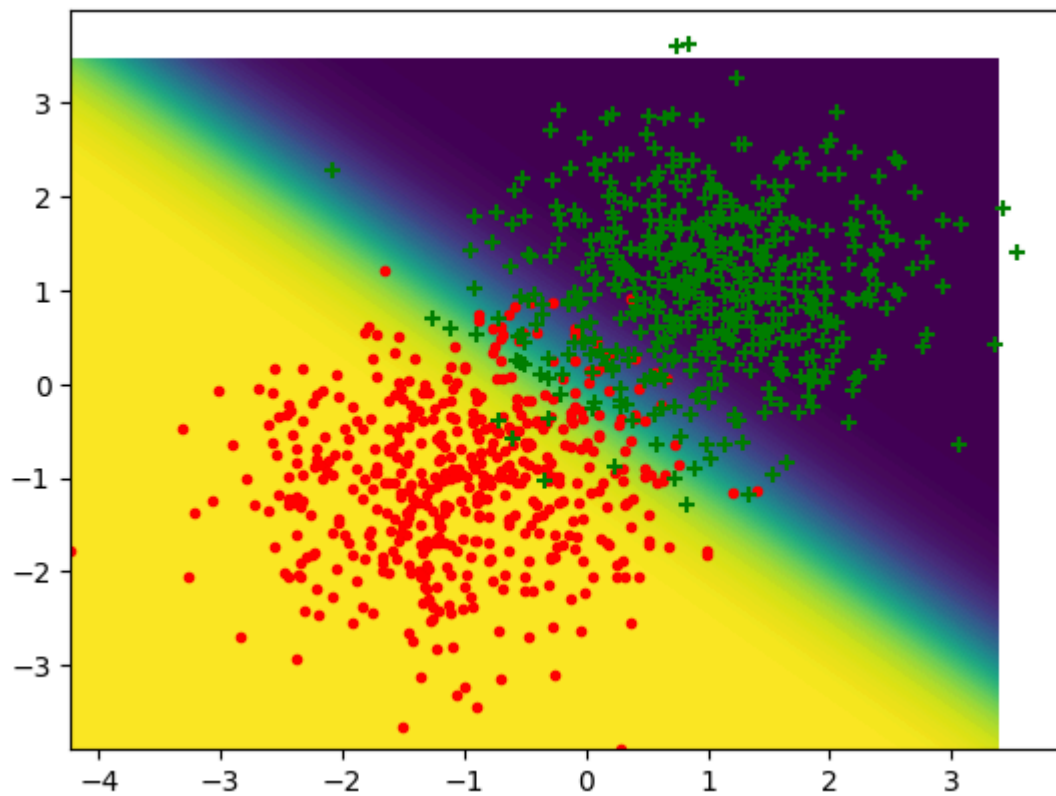
Avec sklearn on peut directement retrouver les paramètres optimaux :
 Meilleurs paramètre pour le modèle SVM linear: {'C': 1, 'gamma': 0.01}
 Meilleurs paramètre pour le modèle SVM rbf: {'C': 0.1, 'gamma': 0.1}
 Meilleurs paramètre pour le modèle SVM poly: {'C': 1, 'gamma': 1}
 Ce qui s'observe sur les plots avec les frontières de décisions précédentes.

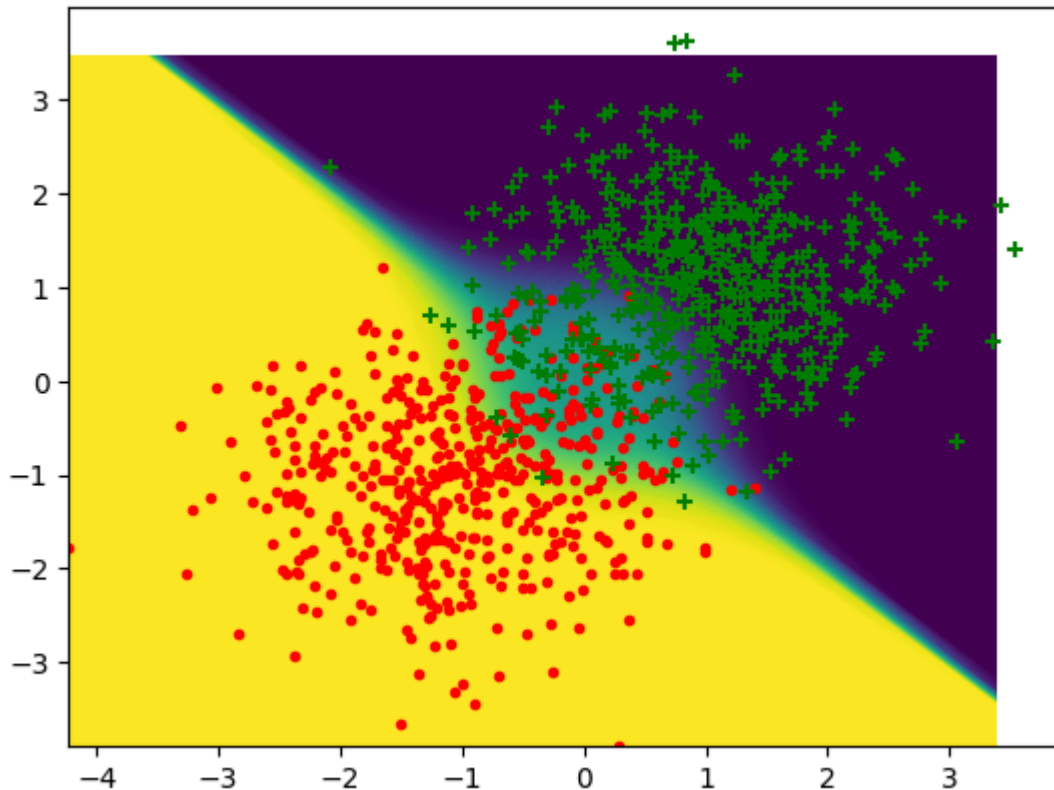
```
In [17]: # On utilise les paramètres optimaux
model = SVC(kernel='linear', C=1, gamma=0.01, probability=True)
model.fit(x_train, y_train)
plot_frontiere_proba(x, lambda x: model.predict_proba(x)[:, 0], step=50)
plot_data(x, y)
plt.show()

model = SVC(kernel='rbf', C=0.1, gamma=0.1, probability=True)
model.fit(x_train, y_train)
plot_frontiere_proba(x, lambda x: model.predict_proba(x)[:, 0], step=50)
plot_data(x, y)
plt.show()

model = SVC(kernel='poly', C=1, gamma=1, probability=True)
model.fit(x_train, y_train)
model.predict_proba(x_test)
```

```
plot_frontiere_proba(x, lambda x: model.predict_proba(x)[:, 0], step=50)  
plot_data(x, y)  
plt.show()
```





- frontières de décisions : Plus la frontière de décision est floue, plus les données sont difficilement séparables, les probabilités d'appartenances aux classes sont faibles. Une frontière nette indique une séparation puissante des données avec de fortes probabilités d'appartenance des points à la classe. Plus la marge est grande (C grand), plus le classifieur est tolérant aux erreurs et plus la frontière sera floue. À l'inverse, si C est faible, le classifieur sera plus strict et la frontière sera plus nette. Si gamma est faible, alors le classifieur est moins sensible aux points individuellement, la frontière sera plus large (et inversement pour gamma élevé) (C'est très visible sur la dernière image).
- Vecteurs supports : Les points de vecteurs supports sont proches de la frontière de décision. Plus il y a de vecteurs supports, plus le modèle est sensible à l'overfitting.

V - Kernels

```
In [ ]: def string_kernel(s, t, n, lambda_value=0.5):
        """
        Compute a string kernel based on common subsequences of length n.

        Params:
            s (str): first input string
            t (str): second input string
            n (int): length of the subsequences to compare
            lambda_value (float): weight factor for subsequences (default is 0.5)

        Returns:
            float: kernel value based on matching subsequences
        """

        similar_subsequences = []
```

```

for i in range(len(s) - n + 1):
    for j in range(len(t) - n + 1):
        if s[i:i+n] == t[j:j+n]:
            similar_subsequences.append((i, j))

kernel_value = 0
for i, j in similar_subsequences:
    length_s = i + n
    length_t = j + n
    kernel_value += lambda_value ** (length_s + length_t)

return kernel_value

```

```

In [ ]: def similarity_matrix(texts, n, lambda_value=0.5):
        """
        Compute a similarity matrix based on string kernels for a list of texts.

        Params:
            texts (list of str): list of text strings to compare
            n (int): length of subsequences for the string kernel
            lambda_value (float): weight factor for subsequences (default is 0.5)

        Returns:
            np.ndarray: matrix of pairwise similarity values between texts
        """

        num_texts = len(texts)
        similarity_matrix = np.zeros((num_texts, num_texts))

        for i in range(num_texts):
            for j in range(num_texts):
                similarity_matrix[i, j] = string_kernel(texts[i], texts[j], n, lambda_value)

        return similarity_matrix

```

```

In [ ]: def plot_similarity_matrix(similarity_matrix, texts):
        """
        Plot the similarity matrix as a heatmap.

        Params:
            similarity_matrix (np.ndarray): matrix of similarity values between text
            texts (list of str): list of text strings (used for labels on axes)

        Side Effects:
            Displays a heatmap of the similarity matrix
        """

        plt.figure(figsize=(10, 8))
        sns.heatmap(similarity_matrix, xticklabels=texts, yticklabels=texts, cmap="Blues")
        plt.title("similarity matrix between texts")
        plt.show()

```

```

In [19]: # Données
texts = [
    "Le soleil brillait sur la mer calme ce matin",
    "Une étrange créature surgit de la forêt profonde",
    "Le jeune garçon lisait un livre sous l'arbre",
    "Des éclairs illuminaient le ciel noir pendant l'orage",
    "Le chat dormait paisiblement sur le canapé",

```

```

    "Un dragon majestueux volait au-dessus des montagnes",
    "Elle écrivait dans son journal tous les soirs",
    "Une porte secrète menait à un monde oublié",
    "Les enfants jouaient dans le parc après l'école",
    "Un vieux grimoire contenait des sorts puissants",
    "Le vent soufflait fort dans les rues désertes",
    "La sorcière préparait une potion dans son chaudron",
    "Ils regardaient les étoiles dans la nuit silencieuse",
    "Une armée de squelettes sortit du sol poussiéreux",
    "La pluie tombait doucement sur les vitres embuées"
]

labels = [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]

indices = np.arange(len(texts))
train_idx, test_idx, y_train, y_test = train_test_split(indices, labels, test_si

similarity_matrix_result = similarity_matrix(texts, n=3, lambda_value=0.5)
plot_similarity_matrix(similarity_matrix_result, texts)

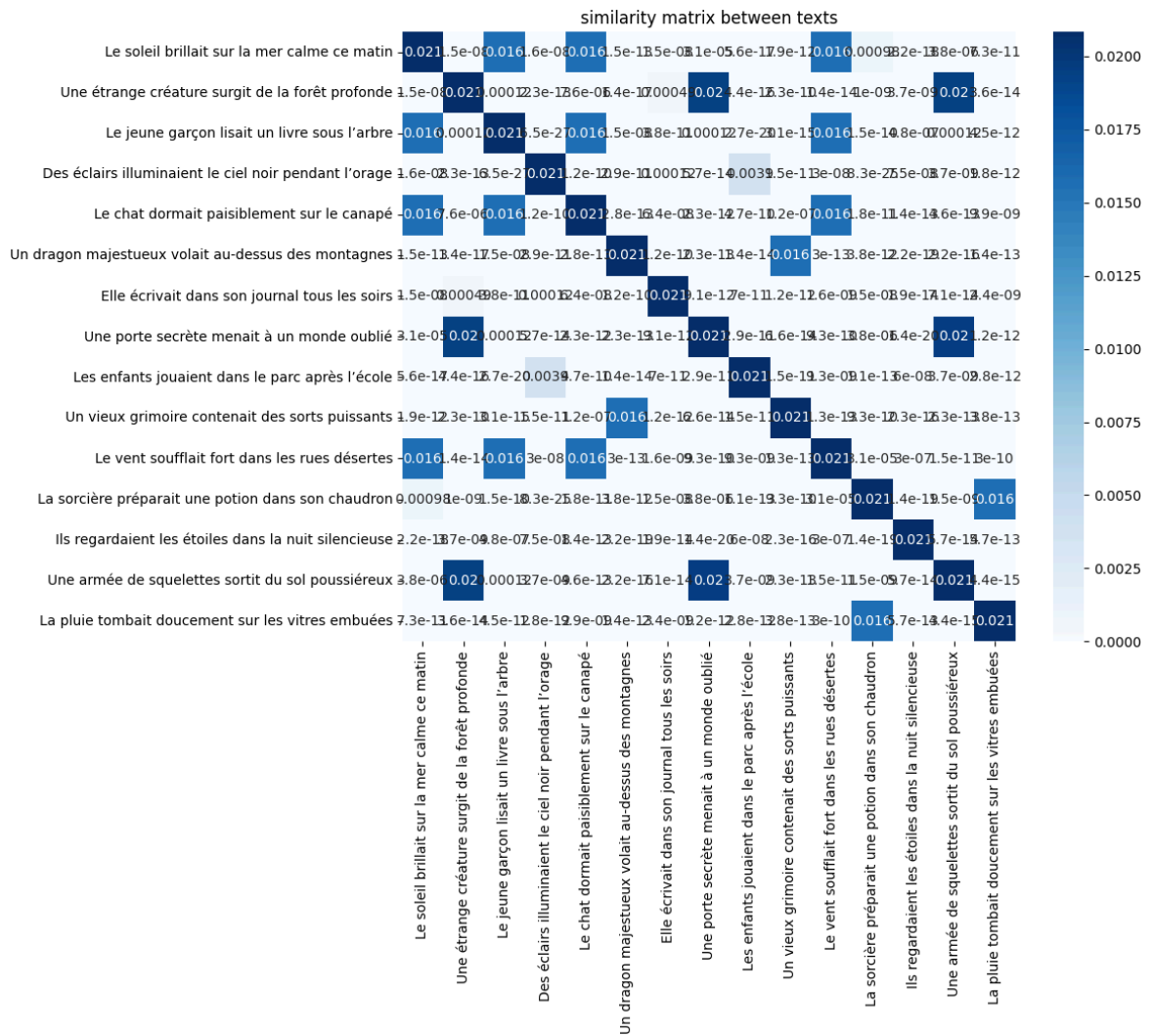
x_train = similarity_matrix_result[np.ix_(train_idx, train_idx)]
x_test = similarity_matrix_result[np.ix_(test_idx, train_idx)]

# apprentissage
svm_model = SVC(kernel='precomputed')
svm_model.fit(x_train, y_train)

train_score = svm_model.score(x_train, y_train)
test_score = svm_model.score(x_test, y_test)

print(f"Training score: {train_score:.2f}")
print(f"Test score: {test_score:.2f}")

```

Training score: 0.58

Test score: 0.33

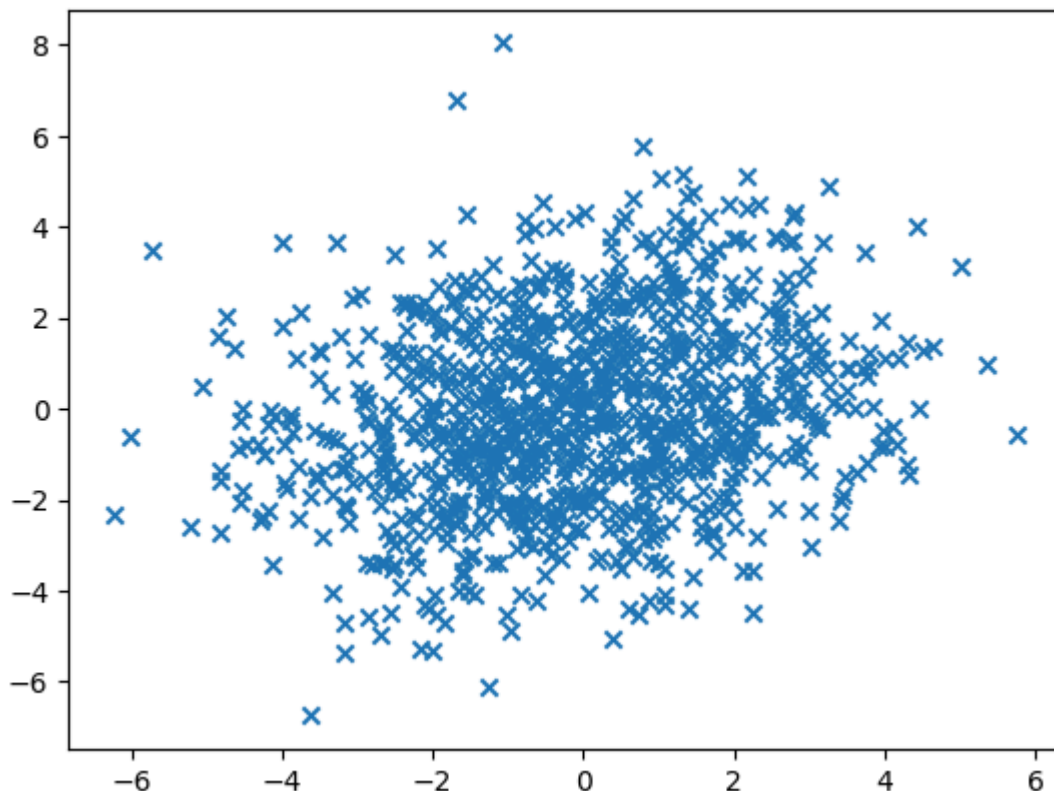
TME06 Bagging, Boosting

Imports

```
In [2]: import numpy as np
from mltools import plot_data, plot_frontiere, make_grid, gen_arti
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
import matplotlib.pyplot as plt
```

Forêts aléatoires (Bagging)

```
In [3]: # Nos données artificielles (2D)
np.random.seed(0)
data, y = gen_arti(centerx=1, centery=1, sigma=0.9, nbex=1000, data_type=0, epsi
plot_data(data)
```



```
In [4]: # Séparation en train/test (70% train, 30% test)
data_train, data_test, y_train, y_test = train_test_split(data, y, test_size=0.3
```

Nous allons tester différents paramètres (`n_estimators` : nombre d'arbres, `max_depth` : profondeur des arbres, `min_samples_split` : nombre minimum d'échantillons pour diviser un nœud, `min_samples_leaf` : nombre minimum d'échantillons par feuille) et étudier l'évolution des frontières de décision, de l'erreur en apprentissage et de l'erreur en test.

```

In [5]: n_estimators_list = [1, 5, 50]
max_depth_list = [None, 1, 5, 50, 100]
min_samples_split_list = [2, 3, 5, 10]
min_samples_leaf_list = [1, 3, 5, 10]

for min_samples_split in min_samples_split_list :
    for min_samples_leaf in min_samples_leaf_list :
        for max_depth in max_depth_list :
            for n_estimators in n_estimators_list :
                classifier = RandomForestClassifier(n_estimators=n_estimators,
                                                    max_depth=max_depth,
                                                    min_samples_split=min_samples_split,
                                                    min_samples_leaf=min_samples_leaf,
                                                    random_state=42)
                classifier.fit(data_train, y_train.ravel()) # On utilise .ravel() pour transformer un array de numpy en un vecteur
                y_pred_train = classifier.predict(data_train)
                y_pred_test = classifier.predict(data_test)

                train_acc = (y_pred_train == y_train.ravel()).mean() * 100
                test_acc = (y_pred_test == y_test.ravel()).mean() * 100

                print(f"Criterion: {classifier.criterion}, Min Split: {min_samples_split}, Max Depth: {max_depth}, N Estimators: {n_estimators}, Train Acc: {train_acc:.2f}, Test Acc: {test_acc:.2f}")

                plot_frontiere(data, lambda data: classifier.predict(data))

```

Criterion: gini, Min Split: 2, Min Leaf: 1, Max Depth: None, N Estimators: 1, Train Acc: 89.29, Test Acc: 68.67
Criterion: gini, Min Split: 2, Min Leaf: 1, Max Depth: None, N Estimators: 5, Train Acc: 97.00, Test Acc: 76.00
Criterion: gini, Min Split: 2, Min Leaf: 1, Max Depth: None, N Estimators: 50, Train Acc: 100.00, Test Acc: 77.67
Criterion: gini, Min Split: 2, Min Leaf: 1, Max Depth: 1, N Estimators: 1, Train Acc: 68.57, Test Acc: 70.00
Criterion: gini, Min Split: 2, Min Leaf: 1, Max Depth: 1, N Estimators: 5, Train Acc: 78.00, Test Acc: 79.00
Criterion: gini, Min Split: 2, Min Leaf: 1, Max Depth: 1, N Estimators: 50, Train Acc: 77.29, Test Acc: 75.67
Criterion: gini, Min Split: 2, Min Leaf: 1, Max Depth: 5, N Estimators: 1, Train Acc: 79.43, Test Acc: 77.67
Criterion: gini, Min Split: 2, Min Leaf: 1, Max Depth: 5, N Estimators: 5, Train Acc: 81.43, Test Acc: 81.00
Criterion: gini, Min Split: 2, Min Leaf: 1, Max Depth: 5, N Estimators: 50, Train Acc: 82.00, Test Acc: 80.67
Criterion: gini, Min Split: 2, Min Leaf: 1, Max Depth: 50, N Estimators: 1, Train Acc: 89.29, Test Acc: 68.67
Criterion: gini, Min Split: 2, Min Leaf: 1, Max Depth: 50, N Estimators: 5, Train Acc: 97.00, Test Acc: 76.00
Criterion: gini, Min Split: 2, Min Leaf: 1, Max Depth: 50, N Estimators: 50, Train Acc: 100.00, Test Acc: 77.67
Criterion: gini, Min Split: 2, Min Leaf: 1, Max Depth: 100, N Estimators: 1, Train Acc: 89.29, Test Acc: 68.67
Criterion: gini, Min Split: 2, Min Leaf: 1, Max Depth: 100, N Estimators: 5, Train Acc: 97.00, Test Acc: 76.00
Criterion: gini, Min Split: 2, Min Leaf: 1, Max Depth: 100, N Estimators: 50, Train Acc: 100.00, Test Acc: 77.67
Criterion: gini, Min Split: 2, Min Leaf: 3, Max Depth: None, N Estimators: 1, Train Acc: 82.71, Test Acc: 72.00
Criterion: gini, Min Split: 2, Min Leaf: 3, Max Depth: None, N Estimators: 5, Train Acc: 88.71, Test Acc: 78.00
Criterion: gini, Min Split: 2, Min Leaf: 3, Max Depth: None, N Estimators: 50, Train Acc: 90.14, Test Acc: 80.67
Criterion: gini, Min Split: 2, Min Leaf: 3, Max Depth: 1, N Estimators: 1, Train Acc: 68.57, Test Acc: 70.00
Criterion: gini, Min Split: 2, Min Leaf: 3, Max Depth: 1, N Estimators: 5, Train Acc: 78.00, Test Acc: 79.00
Criterion: gini, Min Split: 2, Min Leaf: 3, Max Depth: 1, N Estimators: 50, Train Acc: 77.29, Test Acc: 75.67
Criterion: gini, Min Split: 2, Min Leaf: 3, Max Depth: 5, N Estimators: 1, Train Acc: 79.00, Test Acc: 77.33
Criterion: gini, Min Split: 2, Min Leaf: 3, Max Depth: 5, N Estimators: 5, Train Acc: 81.00, Test Acc: 81.00
Criterion: gini, Min Split: 2, Min Leaf: 3, Max Depth: 5, N Estimators: 50, Train Acc: 81.86, Test Acc: 80.00
Criterion: gini, Min Split: 2, Min Leaf: 3, Max Depth: 50, N Estimators: 1, Train Acc: 82.71, Test Acc: 72.00
Criterion: gini, Min Split: 2, Min Leaf: 3, Max Depth: 50, N Estimators: 5, Train Acc: 88.71, Test Acc: 78.00
Criterion: gini, Min Split: 2, Min Leaf: 3, Max Depth: 50, N Estimators: 50, Train Acc: 90.14, Test Acc: 80.67
Criterion: gini, Min Split: 2, Min Leaf: 3, Max Depth: 100, N Estimators: 1, Train Acc: 82.71, Test Acc: 72.00
Criterion: gini, Min Split: 2, Min Leaf: 3, Max Depth: 100, N Estimators: 5, Train Acc: 88.71, Test Acc: 78.00
Criterion: gini, Min Split: 2, Min Leaf: 3, Max Depth: 100, N Estimators: 50, Train Acc: 90.14, Test Acc: 80.67

Criterion: gini, Min Split: 2, Min Leaf: 5, Max Depth: None, N Estimators: 1, Train Acc: 81.14, Test Acc: 74.33

Criterion: gini, Min Split: 2, Min Leaf: 5, Max Depth: None, N Estimators: 5, Train Acc: 84.14, Test Acc: 78.67

Criterion: gini, Min Split: 2, Min Leaf: 5, Max Depth: None, N Estimators: 50, Train Acc: 85.29, Test Acc: 80.33

Criterion: gini, Min Split: 2, Min Leaf: 5, Max Depth: 1, N Estimators: 1, Train Acc: 68.57, Test Acc: 70.00

Criterion: gini, Min Split: 2, Min Leaf: 5, Max Depth: 1, N Estimators: 5, Train Acc: 78.00, Test Acc: 79.00

Criterion: gini, Min Split: 2, Min Leaf: 5, Max Depth: 1, N Estimators: 50, Train Acc: 77.29, Test Acc: 75.67

Criterion: gini, Min Split: 2, Min Leaf: 5, Max Depth: 5, N Estimators: 1, Train Acc: 79.00, Test Acc: 79.33

Criterion: gini, Min Split: 2, Min Leaf: 5, Max Depth: 5, N Estimators: 5, Train Acc: 80.57, Test Acc: 82.00

Criterion: gini, Min Split: 2, Min Leaf: 5, Max Depth: 5, N Estimators: 50, Train Acc: 81.29, Test Acc: 80.00

Criterion: gini, Min Split: 2, Min Leaf: 5, Max Depth: 50, N Estimators: 1, Train Acc: 81.14, Test Acc: 74.33

Criterion: gini, Min Split: 2, Min Leaf: 5, Max Depth: 50, N Estimators: 5, Train Acc: 84.14, Test Acc: 78.67

Criterion: gini, Min Split: 2, Min Leaf: 5, Max Depth: 50, N Estimators: 50, Train Acc: 85.29, Test Acc: 80.33

Criterion: gini, Min Split: 2, Min Leaf: 5, Max Depth: 100, N Estimators: 1, Train Acc: 81.14, Test Acc: 74.33

Criterion: gini, Min Split: 2, Min Leaf: 5, Max Depth: 100, N Estimators: 5, Train Acc: 84.14, Test Acc: 78.67

Criterion: gini, Min Split: 2, Min Leaf: 5, Max Depth: 100, N Estimators: 50, Train Acc: 85.29, Test Acc: 80.33

Criterion: gini, Min Split: 2, Min Leaf: 10, Max Depth: None, N Estimators: 1, Train Acc: 77.86, Test Acc: 78.33

Criterion: gini, Min Split: 2, Min Leaf: 10, Max Depth: None, N Estimators: 5, Train Acc: 81.00, Test Acc: 81.33

Criterion: gini, Min Split: 2, Min Leaf: 10, Max Depth: None, N Estimators: 50, Train Acc: 82.14, Test Acc: 79.67

Criterion: gini, Min Split: 2, Min Leaf: 10, Max Depth: 1, N Estimators: 1, Train Acc: 68.57, Test Acc: 70.00

Criterion: gini, Min Split: 2, Min Leaf: 10, Max Depth: 1, N Estimators: 5, Train Acc: 78.00, Test Acc: 79.00

Criterion: gini, Min Split: 2, Min Leaf: 10, Max Depth: 1, N Estimators: 50, Train Acc: 77.29, Test Acc: 75.67

Criterion: gini, Min Split: 2, Min Leaf: 10, Max Depth: 5, N Estimators: 1, Train Acc: 78.29, Test Acc: 78.67

Criterion: gini, Min Split: 2, Min Leaf: 10, Max Depth: 5, N Estimators: 5, Train Acc: 79.86, Test Acc: 81.00

Criterion: gini, Min Split: 2, Min Leaf: 10, Max Depth: 5, N Estimators: 50, Train Acc: 80.86, Test Acc: 80.33

Criterion: gini, Min Split: 2, Min Leaf: 10, Max Depth: 50, N Estimators: 1, Train Acc: 77.86, Test Acc: 78.33

Criterion: gini, Min Split: 2, Min Leaf: 10, Max Depth: 50, N Estimators: 5, Train Acc: 81.00, Test Acc: 81.33

Criterion: gini, Min Split: 2, Min Leaf: 10, Max Depth: 50, N Estimators: 50, Train Acc: 82.14, Test Acc: 79.67

Criterion: gini, Min Split: 2, Min Leaf: 10, Max Depth: 100, N Estimators: 1, Train Acc: 77.86, Test Acc: 78.33

Criterion: gini, Min Split: 2, Min Leaf: 10, Max Depth: 100, N Estimators: 5, Train Acc: 81.00, Test Acc: 81.33

Criterion: gini, Min Split: 2, Min Leaf: 10, Max Depth: 100, N Estimators: 50, Train Acc: 82.14, Test Acc: 79.67

Criterion: gini, Min Split: 3, Min Leaf: 1, Max Depth: None, N Estimators: 1, Train Acc: 85.43, Test Acc: 73.00
Criterion: gini, Min Split: 3, Min Leaf: 1, Max Depth: None, N Estimators: 5, Train Acc: 94.57, Test Acc: 75.33
Criterion: gini, Min Split: 3, Min Leaf: 1, Max Depth: None, N Estimators: 50, Train Acc: 99.71, Test Acc: 78.00
Criterion: gini, Min Split: 3, Min Leaf: 1, Max Depth: 1, N Estimators: 1, Train Acc: 68.57, Test Acc: 70.00
Criterion: gini, Min Split: 3, Min Leaf: 1, Max Depth: 1, N Estimators: 5, Train Acc: 78.00, Test Acc: 79.00
Criterion: gini, Min Split: 3, Min Leaf: 1, Max Depth: 1, N Estimators: 50, Train Acc: 77.29, Test Acc: 75.67
Criterion: gini, Min Split: 3, Min Leaf: 1, Max Depth: 5, N Estimators: 1, Train Acc: 79.43, Test Acc: 77.67
Criterion: gini, Min Split: 3, Min Leaf: 1, Max Depth: 5, N Estimators: 5, Train Acc: 81.43, Test Acc: 81.00
Criterion: gini, Min Split: 3, Min Leaf: 1, Max Depth: 5, N Estimators: 50, Train Acc: 82.14, Test Acc: 80.67
Criterion: gini, Min Split: 3, Min Leaf: 1, Max Depth: 50, N Estimators: 1, Train Acc: 85.43, Test Acc: 73.00
Criterion: gini, Min Split: 3, Min Leaf: 1, Max Depth: 50, N Estimators: 5, Train Acc: 94.57, Test Acc: 75.33
Criterion: gini, Min Split: 3, Min Leaf: 1, Max Depth: 50, N Estimators: 50, Train Acc: 99.71, Test Acc: 78.00
Criterion: gini, Min Split: 3, Min Leaf: 1, Max Depth: 100, N Estimators: 1, Train Acc: 85.43, Test Acc: 73.00
Criterion: gini, Min Split: 3, Min Leaf: 1, Max Depth: 100, N Estimators: 5, Train Acc: 94.57, Test Acc: 75.33
Criterion: gini, Min Split: 3, Min Leaf: 1, Max Depth: 100, N Estimators: 50, Train Acc: 99.71, Test Acc: 78.00
Criterion: gini, Min Split: 3, Min Leaf: 3, Max Depth: None, N Estimators: 1, Train Acc: 82.71, Test Acc: 72.00
Criterion: gini, Min Split: 3, Min Leaf: 3, Max Depth: None, N Estimators: 5, Train Acc: 88.71, Test Acc: 78.00
Criterion: gini, Min Split: 3, Min Leaf: 3, Max Depth: None, N Estimators: 50, Train Acc: 90.14, Test Acc: 80.67
Criterion: gini, Min Split: 3, Min Leaf: 3, Max Depth: 1, N Estimators: 1, Train Acc: 68.57, Test Acc: 70.00
Criterion: gini, Min Split: 3, Min Leaf: 3, Max Depth: 1, N Estimators: 5, Train Acc: 78.00, Test Acc: 79.00
Criterion: gini, Min Split: 3, Min Leaf: 3, Max Depth: 1, N Estimators: 50, Train Acc: 77.29, Test Acc: 75.67
Criterion: gini, Min Split: 3, Min Leaf: 3, Max Depth: 5, N Estimators: 1, Train Acc: 79.00, Test Acc: 77.33
Criterion: gini, Min Split: 3, Min Leaf: 3, Max Depth: 5, N Estimators: 5, Train Acc: 81.00, Test Acc: 81.00
Criterion: gini, Min Split: 3, Min Leaf: 3, Max Depth: 5, N Estimators: 50, Train Acc: 81.86, Test Acc: 80.00
Criterion: gini, Min Split: 3, Min Leaf: 3, Max Depth: 50, N Estimators: 1, Train Acc: 82.71, Test Acc: 72.00
Criterion: gini, Min Split: 3, Min Leaf: 3, Max Depth: 50, N Estimators: 5, Train Acc: 88.71, Test Acc: 78.00
Criterion: gini, Min Split: 3, Min Leaf: 3, Max Depth: 50, N Estimators: 50, Train Acc: 90.14, Test Acc: 80.67
Criterion: gini, Min Split: 3, Min Leaf: 3, Max Depth: 100, N Estimators: 1, Train Acc: 82.71, Test Acc: 72.00
Criterion: gini, Min Split: 3, Min Leaf: 3, Max Depth: 100, N Estimators: 5, Train Acc: 88.71, Test Acc: 78.00
Criterion: gini, Min Split: 3, Min Leaf: 3, Max Depth: 100, N Estimators: 50, Train Acc: 90.14, Test Acc: 80.67

Criterion: gini, Min Split: 3, Min Leaf: 5, Max Depth: None, N Estimators: 1, Train Acc: 81.14, Test Acc: 74.33

Criterion: gini, Min Split: 3, Min Leaf: 5, Max Depth: None, N Estimators: 5, Train Acc: 84.14, Test Acc: 78.67

Criterion: gini, Min Split: 3, Min Leaf: 5, Max Depth: None, N Estimators: 50, Train Acc: 85.29, Test Acc: 80.33

Criterion: gini, Min Split: 3, Min Leaf: 5, Max Depth: 1, N Estimators: 1, Train Acc: 68.57, Test Acc: 70.00

Criterion: gini, Min Split: 3, Min Leaf: 5, Max Depth: 1, N Estimators: 5, Train Acc: 78.00, Test Acc: 79.00

Criterion: gini, Min Split: 3, Min Leaf: 5, Max Depth: 1, N Estimators: 50, Train Acc: 77.29, Test Acc: 75.67

Criterion: gini, Min Split: 3, Min Leaf: 5, Max Depth: 5, N Estimators: 1, Train Acc: 79.00, Test Acc: 79.33

Criterion: gini, Min Split: 3, Min Leaf: 5, Max Depth: 5, N Estimators: 5, Train Acc: 80.57, Test Acc: 82.00

Criterion: gini, Min Split: 3, Min Leaf: 5, Max Depth: 5, N Estimators: 50, Train Acc: 81.29, Test Acc: 80.00

Criterion: gini, Min Split: 3, Min Leaf: 5, Max Depth: 50, N Estimators: 1, Train Acc: 81.14, Test Acc: 74.33

Criterion: gini, Min Split: 3, Min Leaf: 5, Max Depth: 50, N Estimators: 5, Train Acc: 84.14, Test Acc: 78.67

Criterion: gini, Min Split: 3, Min Leaf: 5, Max Depth: 50, N Estimators: 50, Train Acc: 85.29, Test Acc: 80.33

Criterion: gini, Min Split: 3, Min Leaf: 5, Max Depth: 100, N Estimators: 1, Train Acc: 81.14, Test Acc: 74.33

Criterion: gini, Min Split: 3, Min Leaf: 5, Max Depth: 100, N Estimators: 5, Train Acc: 84.14, Test Acc: 78.67

Criterion: gini, Min Split: 3, Min Leaf: 5, Max Depth: 100, N Estimators: 50, Train Acc: 85.29, Test Acc: 80.33

Criterion: gini, Min Split: 3, Min Leaf: 10, Max Depth: None, N Estimators: 1, Train Acc: 77.86, Test Acc: 78.33

Criterion: gini, Min Split: 3, Min Leaf: 10, Max Depth: None, N Estimators: 5, Train Acc: 81.00, Test Acc: 81.33

Criterion: gini, Min Split: 3, Min Leaf: 10, Max Depth: None, N Estimators: 50, Train Acc: 82.14, Test Acc: 79.67

Criterion: gini, Min Split: 3, Min Leaf: 10, Max Depth: 1, N Estimators: 1, Train Acc: 68.57, Test Acc: 70.00

Criterion: gini, Min Split: 3, Min Leaf: 10, Max Depth: 1, N Estimators: 5, Train Acc: 78.00, Test Acc: 79.00

Criterion: gini, Min Split: 3, Min Leaf: 10, Max Depth: 1, N Estimators: 50, Train Acc: 77.29, Test Acc: 75.67

Criterion: gini, Min Split: 3, Min Leaf: 10, Max Depth: 5, N Estimators: 1, Train Acc: 78.29, Test Acc: 78.67

Criterion: gini, Min Split: 3, Min Leaf: 10, Max Depth: 5, N Estimators: 5, Train Acc: 79.86, Test Acc: 81.00

Criterion: gini, Min Split: 3, Min Leaf: 10, Max Depth: 5, N Estimators: 50, Train Acc: 80.86, Test Acc: 80.33

Criterion: gini, Min Split: 3, Min Leaf: 10, Max Depth: 50, N Estimators: 1, Train Acc: 77.86, Test Acc: 78.33

Criterion: gini, Min Split: 3, Min Leaf: 10, Max Depth: 50, N Estimators: 5, Train Acc: 81.00, Test Acc: 81.33

Criterion: gini, Min Split: 3, Min Leaf: 10, Max Depth: 50, N Estimators: 50, Train Acc: 82.14, Test Acc: 79.67

Criterion: gini, Min Split: 3, Min Leaf: 10, Max Depth: 100, N Estimators: 1, Train Acc: 77.86, Test Acc: 78.33

Criterion: gini, Min Split: 3, Min Leaf: 10, Max Depth: 100, N Estimators: 5, Train Acc: 81.00, Test Acc: 81.33

Criterion: gini, Min Split: 3, Min Leaf: 10, Max Depth: 100, N Estimators: 50, Train Acc: 82.14, Test Acc: 79.67

Criterion: gini, Min Split: 5, Min Leaf: 1, Max Depth: None, N Estimators: 1, Train Acc: 85.00, Test Acc: 68.67
Criterion: gini, Min Split: 5, Min Leaf: 1, Max Depth: None, N Estimators: 5, Train Acc: 92.29, Test Acc: 74.67
Criterion: gini, Min Split: 5, Min Leaf: 1, Max Depth: None, N Estimators: 50, Train Acc: 97.00, Test Acc: 77.67
Criterion: gini, Min Split: 5, Min Leaf: 1, Max Depth: 1, N Estimators: 1, Train Acc: 68.57, Test Acc: 70.00
Criterion: gini, Min Split: 5, Min Leaf: 1, Max Depth: 1, N Estimators: 5, Train Acc: 78.00, Test Acc: 79.00
Criterion: gini, Min Split: 5, Min Leaf: 1, Max Depth: 1, N Estimators: 50, Train Acc: 77.29, Test Acc: 75.67
Criterion: gini, Min Split: 5, Min Leaf: 1, Max Depth: 5, N Estimators: 1, Train Acc: 79.43, Test Acc: 77.67
Criterion: gini, Min Split: 5, Min Leaf: 1, Max Depth: 5, N Estimators: 5, Train Acc: 81.57, Test Acc: 80.67
Criterion: gini, Min Split: 5, Min Leaf: 1, Max Depth: 5, N Estimators: 50, Train Acc: 81.43, Test Acc: 79.33
Criterion: gini, Min Split: 5, Min Leaf: 1, Max Depth: 50, N Estimators: 1, Train Acc: 85.00, Test Acc: 68.67
Criterion: gini, Min Split: 5, Min Leaf: 1, Max Depth: 50, N Estimators: 5, Train Acc: 92.29, Test Acc: 74.67
Criterion: gini, Min Split: 5, Min Leaf: 1, Max Depth: 50, N Estimators: 50, Train Acc: 97.00, Test Acc: 77.67
Criterion: gini, Min Split: 5, Min Leaf: 1, Max Depth: 100, N Estimators: 1, Train Acc: 85.00, Test Acc: 68.67
Criterion: gini, Min Split: 5, Min Leaf: 1, Max Depth: 100, N Estimators: 5, Train Acc: 92.29, Test Acc: 74.67
Criterion: gini, Min Split: 5, Min Leaf: 1, Max Depth: 100, N Estimators: 50, Train Acc: 97.00, Test Acc: 77.67
Criterion: gini, Min Split: 5, Min Leaf: 3, Max Depth: None, N Estimators: 1, Train Acc: 82.71, Test Acc: 72.00
Criterion: gini, Min Split: 5, Min Leaf: 3, Max Depth: None, N Estimators: 5, Train Acc: 88.71, Test Acc: 78.00
Criterion: gini, Min Split: 5, Min Leaf: 3, Max Depth: None, N Estimators: 50, Train Acc: 90.14, Test Acc: 80.67
Criterion: gini, Min Split: 5, Min Leaf: 3, Max Depth: 1, N Estimators: 1, Train Acc: 68.57, Test Acc: 70.00
Criterion: gini, Min Split: 5, Min Leaf: 3, Max Depth: 1, N Estimators: 5, Train Acc: 78.00, Test Acc: 79.00
Criterion: gini, Min Split: 5, Min Leaf: 3, Max Depth: 1, N Estimators: 50, Train Acc: 77.29, Test Acc: 75.67
Criterion: gini, Min Split: 5, Min Leaf: 3, Max Depth: 5, N Estimators: 1, Train Acc: 79.00, Test Acc: 77.33
Criterion: gini, Min Split: 5, Min Leaf: 3, Max Depth: 5, N Estimators: 5, Train Acc: 81.00, Test Acc: 81.00
Criterion: gini, Min Split: 5, Min Leaf: 3, Max Depth: 5, N Estimators: 50, Train Acc: 81.86, Test Acc: 80.00
Criterion: gini, Min Split: 5, Min Leaf: 3, Max Depth: 50, N Estimators: 1, Train Acc: 82.71, Test Acc: 72.00
Criterion: gini, Min Split: 5, Min Leaf: 3, Max Depth: 50, N Estimators: 5, Train Acc: 88.71, Test Acc: 78.00
Criterion: gini, Min Split: 5, Min Leaf: 3, Max Depth: 50, N Estimators: 50, Train Acc: 90.14, Test Acc: 80.67
Criterion: gini, Min Split: 5, Min Leaf: 3, Max Depth: 100, N Estimators: 1, Train Acc: 82.71, Test Acc: 72.00
Criterion: gini, Min Split: 5, Min Leaf: 3, Max Depth: 100, N Estimators: 5, Train Acc: 88.71, Test Acc: 78.00
Criterion: gini, Min Split: 5, Min Leaf: 3, Max Depth: 100, N Estimators: 50, Train Acc: 90.14, Test Acc: 80.67

Criterion: gini, Min Split: 5, Min Leaf: 5, Max Depth: None, N Estimators: 1, Train Acc: 81.14, Test Acc: 74.33
 Criterion: gini, Min Split: 5, Min Leaf: 5, Max Depth: None, N Estimators: 5, Train Acc: 84.14, Test Acc: 78.67
 Criterion: gini, Min Split: 5, Min Leaf: 5, Max Depth: None, N Estimators: 50, Train Acc: 85.29, Test Acc: 80.33
 Criterion: gini, Min Split: 5, Min Leaf: 5, Max Depth: 1, N Estimators: 1, Train Acc: 68.57, Test Acc: 70.00
 Criterion: gini, Min Split: 5, Min Leaf: 5, Max Depth: 1, N Estimators: 5, Train Acc: 78.00, Test Acc: 79.00
 Criterion: gini, Min Split: 5, Min Leaf: 5, Max Depth: 1, N Estimators: 50, Train Acc: 77.29, Test Acc: 75.67
 Criterion: gini, Min Split: 5, Min Leaf: 5, Max Depth: 5, N Estimators: 1, Train Acc: 79.00, Test Acc: 79.33
 Criterion: gini, Min Split: 5, Min Leaf: 5, Max Depth: 5, N Estimators: 5, Train Acc: 80.57, Test Acc: 82.00
 Criterion: gini, Min Split: 5, Min Leaf: 5, Max Depth: 5, N Estimators: 50, Train Acc: 81.29, Test Acc: 80.00
 Criterion: gini, Min Split: 5, Min Leaf: 5, Max Depth: 50, N Estimators: 1, Train Acc: 81.14, Test Acc: 74.33
 Criterion: gini, Min Split: 5, Min Leaf: 5, Max Depth: 50, N Estimators: 5, Train Acc: 84.14, Test Acc: 78.67
 Criterion: gini, Min Split: 5, Min Leaf: 5, Max Depth: 50, N Estimators: 50, Train Acc: 85.29, Test Acc: 80.33
 Criterion: gini, Min Split: 5, Min Leaf: 5, Max Depth: 100, N Estimators: 1, Train Acc: 81.14, Test Acc: 74.33
 Criterion: gini, Min Split: 5, Min Leaf: 5, Max Depth: 100, N Estimators: 5, Train Acc: 84.14, Test Acc: 78.67
 Criterion: gini, Min Split: 5, Min Leaf: 5, Max Depth: 100, N Estimators: 50, Train Acc: 85.29, Test Acc: 80.33
 Criterion: gini, Min Split: 5, Min Leaf: 10, Max Depth: None, N Estimators: 1, Train Acc: 77.86, Test Acc: 78.33
 Criterion: gini, Min Split: 5, Min Leaf: 10, Max Depth: None, N Estimators: 5, Train Acc: 81.00, Test Acc: 81.33
 Criterion: gini, Min Split: 5, Min Leaf: 10, Max Depth: None, N Estimators: 50, Train Acc: 82.14, Test Acc: 79.67
 Criterion: gini, Min Split: 5, Min Leaf: 10, Max Depth: 1, N Estimators: 1, Train Acc: 68.57, Test Acc: 70.00
 Criterion: gini, Min Split: 5, Min Leaf: 10, Max Depth: 1, N Estimators: 5, Train Acc: 78.00, Test Acc: 79.00
 Criterion: gini, Min Split: 5, Min Leaf: 10, Max Depth: 1, N Estimators: 50, Train Acc: 77.29, Test Acc: 75.67
 Criterion: gini, Min Split: 5, Min Leaf: 10, Max Depth: 5, N Estimators: 1, Train Acc: 78.29, Test Acc: 78.67
 Criterion: gini, Min Split: 5, Min Leaf: 10, Max Depth: 5, N Estimators: 5, Train Acc: 79.86, Test Acc: 81.00
 Criterion: gini, Min Split: 5, Min Leaf: 10, Max Depth: 5, N Estimators: 50, Train Acc: 80.86, Test Acc: 80.33
 Criterion: gini, Min Split: 5, Min Leaf: 10, Max Depth: 50, N Estimators: 1, Train Acc: 77.86, Test Acc: 78.33
 Criterion: gini, Min Split: 5, Min Leaf: 10, Max Depth: 50, N Estimators: 5, Train Acc: 81.00, Test Acc: 81.33
 Criterion: gini, Min Split: 5, Min Leaf: 10, Max Depth: 50, N Estimators: 50, Train Acc: 82.14, Test Acc: 79.67
 Criterion: gini, Min Split: 5, Min Leaf: 10, Max Depth: 100, N Estimators: 1, Train Acc: 77.86, Test Acc: 78.33
 Criterion: gini, Min Split: 5, Min Leaf: 10, Max Depth: 100, N Estimators: 5, Train Acc: 81.00, Test Acc: 81.33
 Criterion: gini, Min Split: 5, Min Leaf: 10, Max Depth: 100, N Estimators: 50, Train Acc: 82.14, Test Acc: 79.67

Criterion: gini, Min Split: 10, Min Leaf: 1, Max Depth: None, N Estimators: 1, Train Acc: 81.43, Test Acc: 73.67
Criterion: gini, Min Split: 10, Min Leaf: 1, Max Depth: None, N Estimators: 5, Train Acc: 89.86, Test Acc: 77.67
Criterion: gini, Min Split: 10, Min Leaf: 1, Max Depth: None, N Estimators: 50, Train Acc: 90.43, Test Acc: 79.00
Criterion: gini, Min Split: 10, Min Leaf: 1, Max Depth: 1, N Estimators: 1, Train Acc: 68.57, Test Acc: 70.00
Criterion: gini, Min Split: 10, Min Leaf: 1, Max Depth: 1, N Estimators: 5, Train Acc: 78.00, Test Acc: 79.00
Criterion: gini, Min Split: 10, Min Leaf: 1, Max Depth: 1, N Estimators: 50, Train Acc: 77.29, Test Acc: 75.67
Criterion: gini, Min Split: 10, Min Leaf: 1, Max Depth: 5, N Estimators: 1, Train Acc: 78.71, Test Acc: 76.67
Criterion: gini, Min Split: 10, Min Leaf: 1, Max Depth: 5, N Estimators: 5, Train Acc: 82.29, Test Acc: 80.67
Criterion: gini, Min Split: 10, Min Leaf: 1, Max Depth: 5, N Estimators: 50, Train Acc: 81.29, Test Acc: 79.67
Criterion: gini, Min Split: 10, Min Leaf: 1, Max Depth: 50, N Estimators: 1, Train Acc: 81.43, Test Acc: 73.67
Criterion: gini, Min Split: 10, Min Leaf: 1, Max Depth: 50, N Estimators: 5, Train Acc: 89.86, Test Acc: 77.67
Criterion: gini, Min Split: 10, Min Leaf: 1, Max Depth: 50, N Estimators: 50, Train Acc: 90.43, Test Acc: 79.00
Criterion: gini, Min Split: 10, Min Leaf: 1, Max Depth: 100, N Estimators: 1, Train Acc: 81.43, Test Acc: 73.67
Criterion: gini, Min Split: 10, Min Leaf: 1, Max Depth: 100, N Estimators: 5, Train Acc: 89.86, Test Acc: 77.67
Criterion: gini, Min Split: 10, Min Leaf: 1, Max Depth: 100, N Estimators: 50, Train Acc: 90.43, Test Acc: 79.00
Criterion: gini, Min Split: 10, Min Leaf: 3, Max Depth: None, N Estimators: 1, Train Acc: 79.71, Test Acc: 75.33
Criterion: gini, Min Split: 10, Min Leaf: 3, Max Depth: None, N Estimators: 5, Train Acc: 85.86, Test Acc: 77.00
Criterion: gini, Min Split: 10, Min Leaf: 3, Max Depth: None, N Estimators: 50, Train Acc: 88.14, Test Acc: 80.67
Criterion: gini, Min Split: 10, Min Leaf: 3, Max Depth: 1, N Estimators: 1, Train Acc: 68.57, Test Acc: 70.00
Criterion: gini, Min Split: 10, Min Leaf: 3, Max Depth: 1, N Estimators: 5, Train Acc: 78.00, Test Acc: 79.00
Criterion: gini, Min Split: 10, Min Leaf: 3, Max Depth: 1, N Estimators: 50, Train Acc: 77.29, Test Acc: 75.67
Criterion: gini, Min Split: 10, Min Leaf: 3, Max Depth: 5, N Estimators: 1, Train Acc: 78.43, Test Acc: 76.33
Criterion: gini, Min Split: 10, Min Leaf: 3, Max Depth: 5, N Estimators: 5, Train Acc: 80.71, Test Acc: 81.00
Criterion: gini, Min Split: 10, Min Leaf: 3, Max Depth: 5, N Estimators: 50, Train Acc: 81.57, Test Acc: 79.67
Criterion: gini, Min Split: 10, Min Leaf: 3, Max Depth: 50, N Estimators: 1, Train Acc: 79.71, Test Acc: 75.33
Criterion: gini, Min Split: 10, Min Leaf: 3, Max Depth: 50, N Estimators: 5, Train Acc: 85.86, Test Acc: 77.00
Criterion: gini, Min Split: 10, Min Leaf: 3, Max Depth: 50, N Estimators: 50, Train Acc: 88.14, Test Acc: 80.67
Criterion: gini, Min Split: 10, Min Leaf: 3, Max Depth: 100, N Estimators: 1, Train Acc: 79.71, Test Acc: 75.33
Criterion: gini, Min Split: 10, Min Leaf: 3, Max Depth: 100, N Estimators: 5, Train Acc: 85.86, Test Acc: 77.00
Criterion: gini, Min Split: 10, Min Leaf: 3, Max Depth: 100, N Estimators: 50, Train Acc: 88.14, Test Acc: 80.67

Criterion: gini, Min Split: 10, Min Leaf: 5, Max Depth: None, N Estimators: 1, Train Acc: 81.14, Test Acc: 74.33

Criterion: gini, Min Split: 10, Min Leaf: 5, Max Depth: None, N Estimators: 5, Train Acc: 84.14, Test Acc: 78.67

Criterion: gini, Min Split: 10, Min Leaf: 5, Max Depth: None, N Estimators: 50, Train Acc: 85.29, Test Acc: 80.33

Criterion: gini, Min Split: 10, Min Leaf: 5, Max Depth: 1, N Estimators: 1, Train Acc: 68.57, Test Acc: 70.00

Criterion: gini, Min Split: 10, Min Leaf: 5, Max Depth: 1, N Estimators: 5, Train Acc: 78.00, Test Acc: 79.00

Criterion: gini, Min Split: 10, Min Leaf: 5, Max Depth: 1, N Estimators: 50, Train Acc: 77.29, Test Acc: 75.67

Criterion: gini, Min Split: 10, Min Leaf: 5, Max Depth: 5, N Estimators: 1, Train Acc: 79.00, Test Acc: 79.33

Criterion: gini, Min Split: 10, Min Leaf: 5, Max Depth: 5, N Estimators: 5, Train Acc: 80.57, Test Acc: 82.00

Criterion: gini, Min Split: 10, Min Leaf: 5, Max Depth: 5, N Estimators: 50, Train Acc: 81.29, Test Acc: 80.00

Criterion: gini, Min Split: 10, Min Leaf: 5, Max Depth: 50, N Estimators: 1, Train Acc: 81.14, Test Acc: 74.33

Criterion: gini, Min Split: 10, Min Leaf: 5, Max Depth: 50, N Estimators: 5, Train Acc: 84.14, Test Acc: 78.67

Criterion: gini, Min Split: 10, Min Leaf: 5, Max Depth: 50, N Estimators: 50, Train Acc: 85.29, Test Acc: 80.33

Criterion: gini, Min Split: 10, Min Leaf: 5, Max Depth: 100, N Estimators: 1, Train Acc: 81.14, Test Acc: 74.33

Criterion: gini, Min Split: 10, Min Leaf: 5, Max Depth: 100, N Estimators: 5, Train Acc: 84.14, Test Acc: 78.67

Criterion: gini, Min Split: 10, Min Leaf: 5, Max Depth: 100, N Estimators: 50, Train Acc: 85.29, Test Acc: 80.33

Criterion: gini, Min Split: 10, Min Leaf: 10, Max Depth: None, N Estimators: 1, Train Acc: 77.86, Test Acc: 78.33

Criterion: gini, Min Split: 10, Min Leaf: 10, Max Depth: None, N Estimators: 5, Train Acc: 81.00, Test Acc: 81.33

Criterion: gini, Min Split: 10, Min Leaf: 10, Max Depth: None, N Estimators: 50, Train Acc: 82.14, Test Acc: 79.67

Criterion: gini, Min Split: 10, Min Leaf: 10, Max Depth: 1, N Estimators: 1, Train Acc: 68.57, Test Acc: 70.00

Criterion: gini, Min Split: 10, Min Leaf: 10, Max Depth: 1, N Estimators: 5, Train Acc: 78.00, Test Acc: 79.00

Criterion: gini, Min Split: 10, Min Leaf: 10, Max Depth: 1, N Estimators: 50, Train Acc: 77.29, Test Acc: 75.67

Criterion: gini, Min Split: 10, Min Leaf: 10, Max Depth: 5, N Estimators: 1, Train Acc: 78.29, Test Acc: 78.67

Criterion: gini, Min Split: 10, Min Leaf: 10, Max Depth: 5, N Estimators: 5, Train Acc: 79.86, Test Acc: 81.00

Criterion: gini, Min Split: 10, Min Leaf: 10, Max Depth: 5, N Estimators: 50, Train Acc: 80.86, Test Acc: 80.33

Criterion: gini, Min Split: 10, Min Leaf: 10, Max Depth: 50, N Estimators: 1, Train Acc: 77.86, Test Acc: 78.33

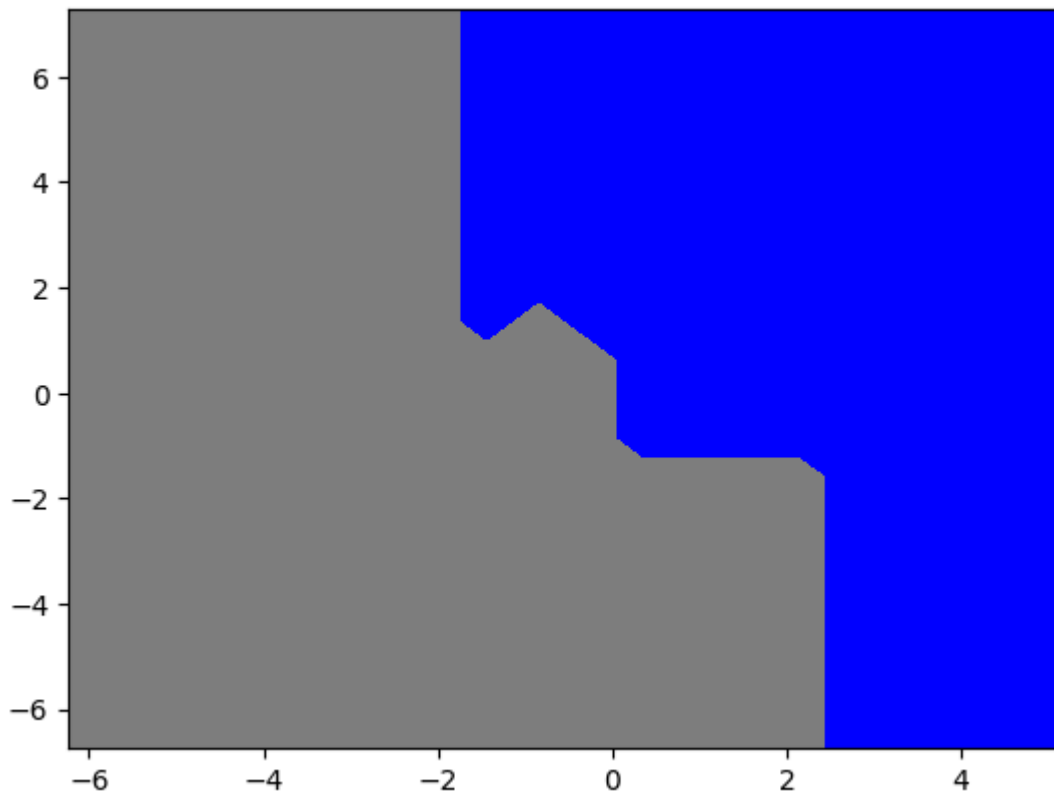
Criterion: gini, Min Split: 10, Min Leaf: 10, Max Depth: 50, N Estimators: 5, Train Acc: 81.00, Test Acc: 81.33

Criterion: gini, Min Split: 10, Min Leaf: 10, Max Depth: 50, N Estimators: 50, Train Acc: 82.14, Test Acc: 79.67

Criterion: gini, Min Split: 10, Min Leaf: 10, Max Depth: 100, N Estimators: 1, Train Acc: 77.86, Test Acc: 78.33

Criterion: gini, Min Split: 10, Min Leaf: 10, Max Depth: 100, N Estimators: 5, Train Acc: 81.00, Test Acc: 81.33

Criterion: gini, Min Split: 10, Min Leaf: 10, Max Depth: 100, N Estimators: 50, Train Acc: 82.14, Test Acc: 79.67



On observe que sur des données bruitées, plus la profondeur des arbres et le nombre d'arbres des forêts aléatoires augmentent, plus il y a du surapprentissage. Par exemple, pour un modèle avec un nombre minimum d'échantillons par feuille égal à 1, une profondeur de 100 et 50 arbres, l'accuracy sur le train atteint 100%, tandis que sur le test, elle tombe à 77.67%. Cela montre un écart important entre les données d'entraînement et les données de test, typique du surapprentissage : le modèle est trop ajusté aux données d'entraînement et n'arrive pas à généraliser correctement aux nouvelles données.

Cependant, ce problème de surapprentissage peut être atténué en augmentant le nombre minimum d'échantillons par feuille ou le nombre minimum d'échantillons requis pour diviser un nœud. En faisant cela, on voit que l'accuracy devient plus stable/similaire entre les données d'entraînement et de test. Cela signifie que le modèle ne surapprend plus et devient plus généraliste. On note cependant que l'accuracy (sur train et test) se situe dans une plage de 70-80%. Cela montre que le modèle Random Forest est relativement robuste au surapprentissage grâce au bagging (sous-échantillonnage). En effet, le modèle combine les résultats de plusieurs arbres, ce qui permet de lisser les erreurs : même si un arbre surapprend, la moyenne des prédictions des différents arbres permet de réduire cet effet de surapprentissage.

AdaBoost (Boosting)

Nous allons maintenant passer à l'implémentation d'AdaBoost et observer l'évolution de l'erreur ϵ_t , des poids α_t et de Z_t , le cumul des fonctions de normalisation Z_t .

```
In [6]: def AdaBoost_algorithm(X, Y, T=50, max_depth=1):
        N, _ = X.shape
```

```

D = np.ones(N) / N
classifiers = []
alphas = []
epsilons = []
Z_values = []

for t in range(T):
    # Apprentissage d'un arbre faible
    stump = DecisionTreeClassifier(max_depth=max_depth)
    stump.fit(X, Y, sample_weight=D)
    pred = stump.predict(X)

    # Calcul de l'erreur epsilon_t
    Y = Y.flatten()
    miss = (pred != Y)
    epsilon_t = max(np.dot(D, miss), 1e-10)

    # Fixation de alpha_t
    alpha_t = 0.5 * np.log((1 - epsilon_t) / epsilon_t)

    # Mise à jour des poids
    D = D * np.exp(-alpha_t * Y * pred)
    Z_t = np.sum(D)
    D /= Z_t

    classifiers.append(stump)
    alphas.append(alpha_t)
    epsilons.append(epsilon_t)
    Z_values.append(Z_t)

    print(f"Iteration {t+1}: epsilon_t = {epsilon_t:.4f}, alpha_t = {alpha_t:.4f}")

return classifiers, alphas, epsilons, Z_values

```

```

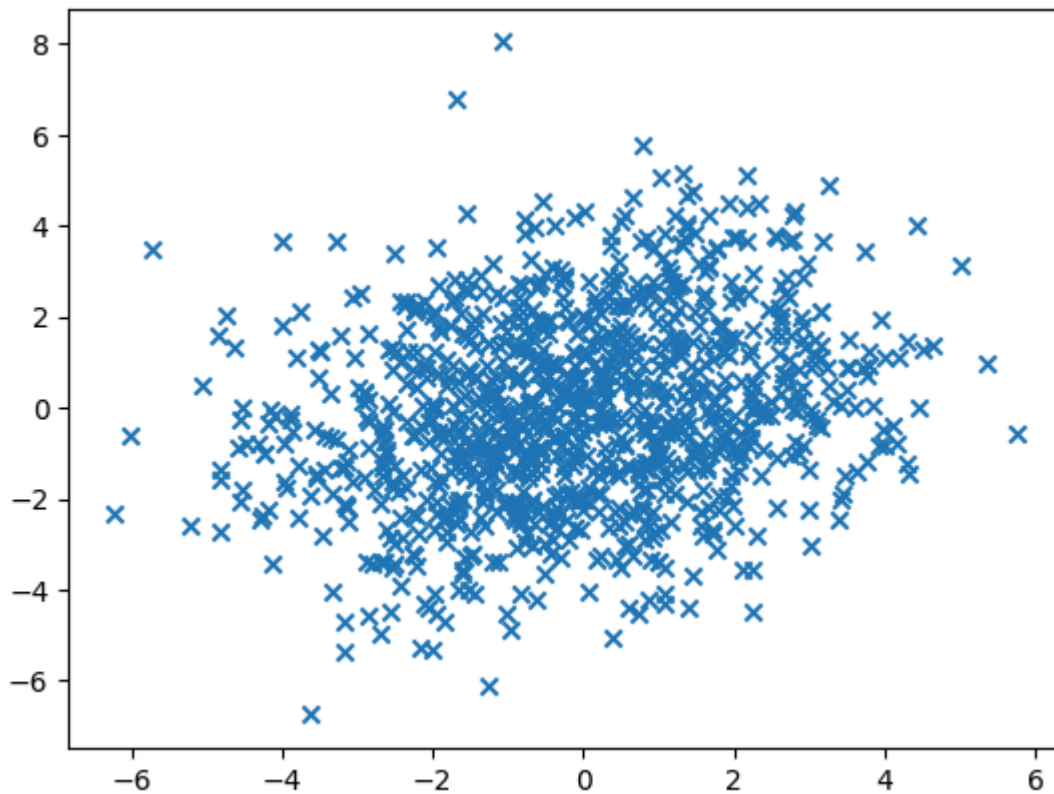
In [7]: def AdaBoost_classifier(X, classifiers, alphas):
        clf_preds = np.array([alpha * classifier.predict(X) for classifier, alpha in
                               zip(alphas, classifiers)])
        return np.sign(np.sum(clf_preds, axis=0))

```

```

In [15]: # Nos données artificielles (2D)
np.random.seed(0)
data, y = gen_arti(centerx=1, centery=1, sigma=0.9, nbex=1000, data_type=0, epsilon=0.1)
plot_data(data)

```



```
In [16]: # Séparation en train/test (70% train, 30% test)
data_train, data_test, y_train, y_test = train_test_split(data, y, test_size=0.3
```

```
In [17]: # MAX_DEPTH = 1

# Entraînement sur train
classifiers, alphas, epsilons, Z_values = AdaBoost_algorithm(data_train, y_train

# Prédiction sur train
y_pred_train = AdaBoost_classifient(data_train, classifiers, alphas)
train_acc = (y_pred_train == y_train.ravel()).mean() * 100
print("Accuracy sur les données train:", train_acc)

# Prédiction sur test
y_pred_test = AdaBoost_classifient(data_test, classifiers, alphas)
test_acc = (y_pred_test == y_test.ravel()).mean() * 100
print("Accuracy sur les données test:", test_acc)
```


Iteration 1: epsilon_t = 0.2629, alpha_t = 0.5156, Z_t = 0.8804
Iteration 2: epsilon_t = 0.3101, alpha_t = 0.3997, Z_t = 0.9251
Iteration 3: epsilon_t = 0.4384, alpha_t = 0.1238, Z_t = 0.9924
Iteration 4: epsilon_t = 0.3891, alpha_t = 0.2255, Z_t = 0.9751
Iteration 5: epsilon_t = 0.4657, alpha_t = 0.0687, Z_t = 0.9976
Iteration 6: epsilon_t = 0.4538, alpha_t = 0.0926, Z_t = 0.9957
Iteration 7: epsilon_t = 0.4235, alpha_t = 0.1543, Z_t = 0.9882
Iteration 8: epsilon_t = 0.4262, alpha_t = 0.1487, Z_t = 0.9891
Iteration 9: epsilon_t = 0.4282, alpha_t = 0.1446, Z_t = 0.9896
Iteration 10: epsilon_t = 0.4372, alpha_t = 0.1263, Z_t = 0.9921
Iteration 11: epsilon_t = 0.4406, alpha_t = 0.1194, Z_t = 0.9929
Iteration 12: epsilon_t = 0.4391, alpha_t = 0.1225, Z_t = 0.9925
Iteration 13: epsilon_t = 0.4747, alpha_t = 0.0507, Z_t = 0.9987
Iteration 14: epsilon_t = 0.4860, alpha_t = 0.0279, Z_t = 0.9996
Iteration 15: epsilon_t = 0.4864, alpha_t = 0.0272, Z_t = 0.9996
Iteration 16: epsilon_t = 0.4747, alpha_t = 0.0507, Z_t = 0.9987
Iteration 17: epsilon_t = 0.4759, alpha_t = 0.0482, Z_t = 0.9988
Iteration 18: epsilon_t = 0.4432, alpha_t = 0.1140, Z_t = 0.9935
Iteration 19: epsilon_t = 0.4803, alpha_t = 0.0395, Z_t = 0.9992
Iteration 20: epsilon_t = 0.4776, alpha_t = 0.0449, Z_t = 0.9990
Iteration 21: epsilon_t = 0.4778, alpha_t = 0.0444, Z_t = 0.9990
Iteration 22: epsilon_t = 0.4788, alpha_t = 0.0425, Z_t = 0.9991
Iteration 23: epsilon_t = 0.4794, alpha_t = 0.0413, Z_t = 0.9991
Iteration 24: epsilon_t = 0.4898, alpha_t = 0.0205, Z_t = 0.9998
Iteration 25: epsilon_t = 0.4894, alpha_t = 0.0212, Z_t = 0.9998
Iteration 26: epsilon_t = 0.4800, alpha_t = 0.0400, Z_t = 0.9992
Iteration 27: epsilon_t = 0.4808, alpha_t = 0.0384, Z_t = 0.9993
Iteration 28: epsilon_t = 0.4883, alpha_t = 0.0233, Z_t = 0.9997
Iteration 29: epsilon_t = 0.4476, alpha_t = 0.1051, Z_t = 0.9945
Iteration 30: epsilon_t = 0.4599, alpha_t = 0.0804, Z_t = 0.9968
Iteration 31: epsilon_t = 0.4684, alpha_t = 0.0633, Z_t = 0.9980
Iteration 32: epsilon_t = 0.4828, alpha_t = 0.0344, Z_t = 0.9994
Iteration 33: epsilon_t = 0.4913, alpha_t = 0.0174, Z_t = 0.9998
Iteration 34: epsilon_t = 0.4915, alpha_t = 0.0171, Z_t = 0.9999
Iteration 35: epsilon_t = 0.4916, alpha_t = 0.0168, Z_t = 0.9999
Iteration 36: epsilon_t = 0.4916, alpha_t = 0.0168, Z_t = 0.9999
Iteration 37: epsilon_t = 0.4838, alpha_t = 0.0325, Z_t = 0.9995
Iteration 38: epsilon_t = 0.4842, alpha_t = 0.0317, Z_t = 0.9995
Iteration 39: epsilon_t = 0.4921, alpha_t = 0.0158, Z_t = 0.9999
Iteration 40: epsilon_t = 0.4919, alpha_t = 0.0163, Z_t = 0.9999
Iteration 41: epsilon_t = 0.4920, alpha_t = 0.0160, Z_t = 0.9999
Iteration 42: epsilon_t = 0.4921, alpha_t = 0.0158, Z_t = 0.9999
Iteration 43: epsilon_t = 0.4848, alpha_t = 0.0305, Z_t = 0.9995
Iteration 44: epsilon_t = 0.4851, alpha_t = 0.0298, Z_t = 0.9996
Iteration 45: epsilon_t = 0.4926, alpha_t = 0.0149, Z_t = 0.9999
Iteration 46: epsilon_t = 0.4923, alpha_t = 0.0153, Z_t = 0.9999
Iteration 47: epsilon_t = 0.4925, alpha_t = 0.0151, Z_t = 0.9999
Iteration 48: epsilon_t = 0.4926, alpha_t = 0.0149, Z_t = 0.9999
Iteration 49: epsilon_t = 0.4856, alpha_t = 0.0288, Z_t = 0.9996
Iteration 50: epsilon_t = 0.4859, alpha_t = 0.0282, Z_t = 0.9996
Iteration 51: epsilon_t = 0.4859, alpha_t = 0.0282, Z_t = 0.9996
Iteration 52: epsilon_t = 0.4860, alpha_t = 0.0279, Z_t = 0.9996
Iteration 53: epsilon_t = 0.4864, alpha_t = 0.0272, Z_t = 0.9996
Iteration 54: epsilon_t = 0.4866, alpha_t = 0.0267, Z_t = 0.9996
Iteration 55: epsilon_t = 0.4866, alpha_t = 0.0267, Z_t = 0.9996
Iteration 56: epsilon_t = 0.4868, alpha_t = 0.0265, Z_t = 0.9996
Iteration 57: epsilon_t = 0.4590, alpha_t = 0.0821, Z_t = 0.9966
Iteration 58: epsilon_t = 0.4724, alpha_t = 0.0553, Z_t = 0.9985
Iteration 59: epsilon_t = 0.4772, alpha_t = 0.0457, Z_t = 0.9990
Iteration 60: epsilon_t = 0.4870, alpha_t = 0.0260, Z_t = 0.9997

```

Iteration 61: epsilon_t = 0.4872, alpha_t = 0.0257, Z_t = 0.9997
Iteration 62: epsilon_t = 0.4874, alpha_t = 0.0251, Z_t = 0.9997
Iteration 63: epsilon_t = 0.4876, alpha_t = 0.0249, Z_t = 0.9997
Iteration 64: epsilon_t = 0.4876, alpha_t = 0.0248, Z_t = 0.9997
Iteration 65: epsilon_t = 0.4878, alpha_t = 0.0244, Z_t = 0.9997
Iteration 66: epsilon_t = 0.4880, alpha_t = 0.0239, Z_t = 0.9997
Iteration 67: epsilon_t = 0.4881, alpha_t = 0.0237, Z_t = 0.9997
Iteration 68: epsilon_t = 0.4882, alpha_t = 0.0236, Z_t = 0.9997
Iteration 69: epsilon_t = 0.4884, alpha_t = 0.0233, Z_t = 0.9997
Iteration 70: epsilon_t = 0.4886, alpha_t = 0.0229, Z_t = 0.9997
Iteration 71: epsilon_t = 0.4887, alpha_t = 0.0227, Z_t = 0.9997
Iteration 72: epsilon_t = 0.4887, alpha_t = 0.0226, Z_t = 0.9997
Iteration 73: epsilon_t = 0.4889, alpha_t = 0.0223, Z_t = 0.9998
Iteration 74: epsilon_t = 0.4891, alpha_t = 0.0219, Z_t = 0.9998
Iteration 75: epsilon_t = 0.4892, alpha_t = 0.0217, Z_t = 0.9998
Iteration 76: epsilon_t = 0.4892, alpha_t = 0.0216, Z_t = 0.9998
Iteration 77: epsilon_t = 0.4893, alpha_t = 0.0213, Z_t = 0.9998
Iteration 78: epsilon_t = 0.4895, alpha_t = 0.0210, Z_t = 0.9998
Iteration 79: epsilon_t = 0.4896, alpha_t = 0.0208, Z_t = 0.9998
Iteration 80: epsilon_t = 0.4896, alpha_t = 0.0207, Z_t = 0.9998
Iteration 81: epsilon_t = 0.4898, alpha_t = 0.0205, Z_t = 0.9998
Iteration 82: epsilon_t = 0.4899, alpha_t = 0.0201, Z_t = 0.9998
Iteration 83: epsilon_t = 0.4900, alpha_t = 0.0200, Z_t = 0.9998
Iteration 84: epsilon_t = 0.4900, alpha_t = 0.0199, Z_t = 0.9998
Iteration 85: epsilon_t = 0.4902, alpha_t = 0.0197, Z_t = 0.9998
Iteration 86: epsilon_t = 0.4780, alpha_t = 0.0440, Z_t = 0.9990
Iteration 87: epsilon_t = 0.4751, alpha_t = 0.0497, Z_t = 0.9988
Iteration 88: epsilon_t = 0.4909, alpha_t = 0.0182, Z_t = 0.9998
Iteration 89: epsilon_t = 0.4906, alpha_t = 0.0187, Z_t = 0.9998
Iteration 90: epsilon_t = 0.4908, alpha_t = 0.0184, Z_t = 0.9998
Iteration 91: epsilon_t = 0.4910, alpha_t = 0.0181, Z_t = 0.9998
Iteration 92: epsilon_t = 0.4910, alpha_t = 0.0181, Z_t = 0.9998
Iteration 93: epsilon_t = 0.4911, alpha_t = 0.0178, Z_t = 0.9998
Iteration 94: epsilon_t = 0.4913, alpha_t = 0.0174, Z_t = 0.9998
Iteration 95: epsilon_t = 0.4914, alpha_t = 0.0173, Z_t = 0.9999
Iteration 96: epsilon_t = 0.4914, alpha_t = 0.0173, Z_t = 0.9999
Iteration 97: epsilon_t = 0.4914, alpha_t = 0.0172, Z_t = 0.9999
Iteration 98: epsilon_t = 0.4916, alpha_t = 0.0169, Z_t = 0.9999
Iteration 99: epsilon_t = 0.4916, alpha_t = 0.0167, Z_t = 0.9999
Iteration 100: epsilon_t = 0.4916, alpha_t = 0.0167, Z_t = 0.9999
Accuracy sur les données train: 78.0
Accuracy sur les données test: 79.0

```

```

In [18]: Z = np.cumprod(Z_values)

plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.plot(epsilons)
plt.title("Erreur  $\epsilon_t$  par itération")
plt.xlabel("Itération t")
plt.ylabel(" $\epsilon_t$ ")

plt.subplot(2, 2, 2)
plt.plot(alphas)
plt.title("Poids  $\alpha_t$  par itération")
plt.xlabel("Itération t")
plt.ylabel(" $\alpha_t$ ")

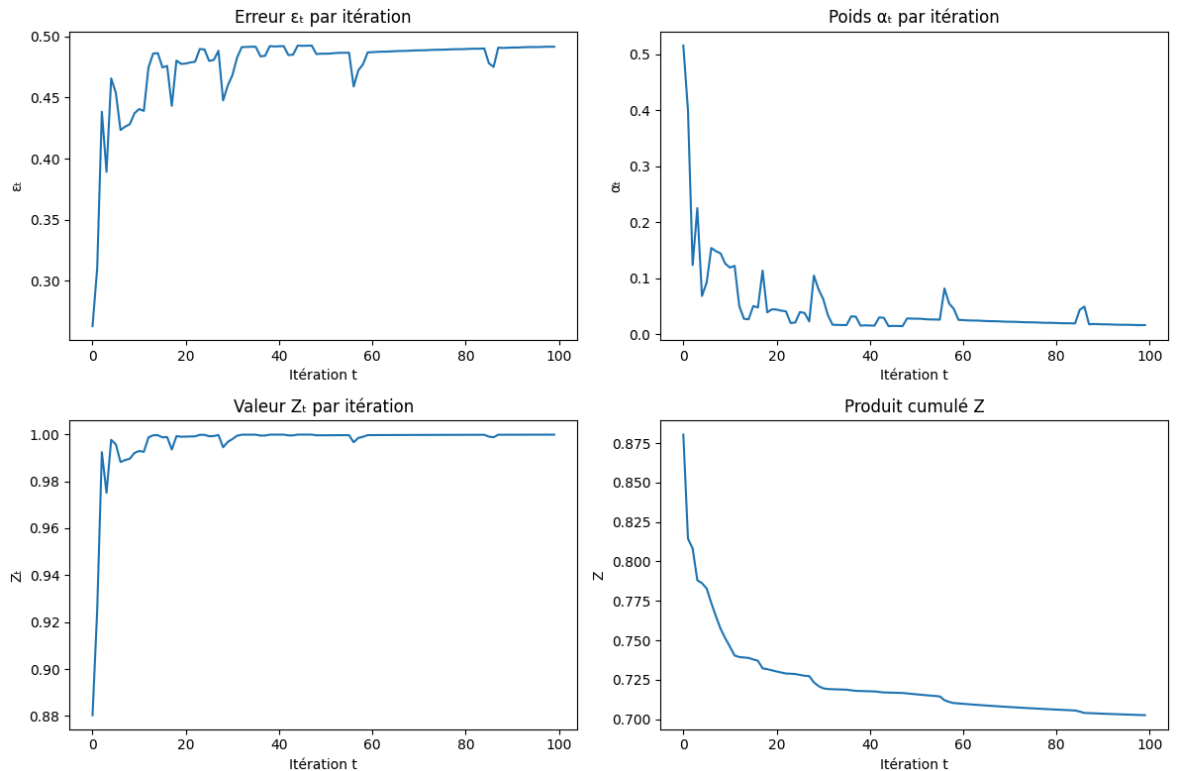
plt.subplot(2, 2, 3)

```

```
plt.plot(Z_values)
plt.title("Valeur  $Z_t$  par itération")
plt.xlabel("Itération  $t$ ")
plt.ylabel(" $Z_t$ ")

plt.subplot(2, 2, 4)
plt.plot(Z)
plt.title("Produit cumulé  $Z$ ")
plt.xlabel("Itération  $t$ ")
plt.ylabel(" $Z$ ")

plt.tight_layout()
plt.show()
```



```
In [19]: # MAX_DEPTH = 5

# Entraînement sur train
classifiers, alphas, epsilons, Z_values = AdaBoost_algorithm(data_train, y_train)

# Prédiction sur train
y_pred_train = AdaBoost_classifier(data_train, classifiers, alphas)
train_acc = (y_pred_train == y_train.ravel()).mean() * 100
print("Accuracy sur les données train:", train_acc)

# Prédiction sur test
y_pred_test = AdaBoost_classifier(data_test, classifiers, alphas)
test_acc = (y_pred_test == y_test.ravel()).mean() * 100
print("Accuracy sur les données test:", test_acc)
```

Iteration 1: epsilon_t = 0.1814, alpha_t = 0.7533, Z_t = 0.7707
Iteration 2: epsilon_t = 0.2527, alpha_t = 0.5421, Z_t = 0.8691
Iteration 3: epsilon_t = 0.2508, alpha_t = 0.5471, Z_t = 0.8670
Iteration 4: epsilon_t = 0.3464, alpha_t = 0.3174, Z_t = 0.9517
Iteration 5: epsilon_t = 0.3375, alpha_t = 0.3371, Z_t = 0.9457
Iteration 6: epsilon_t = 0.3562, alpha_t = 0.2960, Z_t = 0.9577
Iteration 7: epsilon_t = 0.4032, alpha_t = 0.1961, Z_t = 0.9811
Iteration 8: epsilon_t = 0.4356, alpha_t = 0.1296, Z_t = 0.9917
Iteration 9: epsilon_t = 0.4340, alpha_t = 0.1328, Z_t = 0.9912
Iteration 10: epsilon_t = 0.3934, alpha_t = 0.2166, Z_t = 0.9770
Iteration 11: epsilon_t = 0.4145, alpha_t = 0.1727, Z_t = 0.9853
Iteration 12: epsilon_t = 0.4064, alpha_t = 0.1894, Z_t = 0.9823
Iteration 13: epsilon_t = 0.4017, alpha_t = 0.1991, Z_t = 0.9805
Iteration 14: epsilon_t = 0.3910, alpha_t = 0.2216, Z_t = 0.9759
Iteration 15: epsilon_t = 0.4113, alpha_t = 0.1793, Z_t = 0.9841
Iteration 16: epsilon_t = 0.4345, alpha_t = 0.1318, Z_t = 0.9914
Iteration 17: epsilon_t = 0.3966, alpha_t = 0.2099, Z_t = 0.9784
Iteration 18: epsilon_t = 0.4143, alpha_t = 0.1731, Z_t = 0.9852
Iteration 19: epsilon_t = 0.4017, alpha_t = 0.1992, Z_t = 0.9805
Iteration 20: epsilon_t = 0.3491, alpha_t = 0.3115, Z_t = 0.9534
Iteration 21: epsilon_t = 0.3690, alpha_t = 0.2682, Z_t = 0.9651
Iteration 22: epsilon_t = 0.3542, alpha_t = 0.3003, Z_t = 0.9566
Iteration 23: epsilon_t = 0.3436, alpha_t = 0.3236, Z_t = 0.9498
Iteration 24: epsilon_t = 0.3559, alpha_t = 0.2967, Z_t = 0.9575
Iteration 25: epsilon_t = 0.3793, alpha_t = 0.2463, Z_t = 0.9704
Iteration 26: epsilon_t = 0.3810, alpha_t = 0.2427, Z_t = 0.9712
Iteration 27: epsilon_t = 0.3580, alpha_t = 0.2920, Z_t = 0.9588
Iteration 28: epsilon_t = 0.3412, alpha_t = 0.3290, Z_t = 0.9482
Iteration 29: epsilon_t = 0.3520, alpha_t = 0.3051, Z_t = 0.9552
Iteration 30: epsilon_t = 0.3975, alpha_t = 0.2079, Z_t = 0.9788
Iteration 31: epsilon_t = 0.3548, alpha_t = 0.2989, Z_t = 0.9569
Iteration 32: epsilon_t = 0.3552, alpha_t = 0.2981, Z_t = 0.9572
Iteration 33: epsilon_t = 0.4293, alpha_t = 0.1423, Z_t = 0.9900
Iteration 34: epsilon_t = 0.4012, alpha_t = 0.2002, Z_t = 0.9803
Iteration 35: epsilon_t = 0.4086, alpha_t = 0.1848, Z_t = 0.9832
Iteration 36: epsilon_t = 0.4104, alpha_t = 0.1812, Z_t = 0.9838
Iteration 37: epsilon_t = 0.4226, alpha_t = 0.1560, Z_t = 0.9880
Iteration 38: epsilon_t = 0.4178, alpha_t = 0.1659, Z_t = 0.9864
Iteration 39: epsilon_t = 0.3876, alpha_t = 0.2286, Z_t = 0.9744
Iteration 40: epsilon_t = 0.3969, alpha_t = 0.2092, Z_t = 0.9785
Iteration 41: epsilon_t = 0.3887, alpha_t = 0.2264, Z_t = 0.9749
Iteration 42: epsilon_t = 0.4175, alpha_t = 0.1666, Z_t = 0.9863
Iteration 43: epsilon_t = 0.4122, alpha_t = 0.1775, Z_t = 0.9844
Iteration 44: epsilon_t = 0.4278, alpha_t = 0.1453, Z_t = 0.9895
Iteration 45: epsilon_t = 0.4226, alpha_t = 0.1561, Z_t = 0.9879
Iteration 46: epsilon_t = 0.4085, alpha_t = 0.1850, Z_t = 0.9831
Iteration 47: epsilon_t = 0.4019, alpha_t = 0.1987, Z_t = 0.9806
Iteration 48: epsilon_t = 0.4311, alpha_t = 0.1387, Z_t = 0.9905
Iteration 49: epsilon_t = 0.4302, alpha_t = 0.1405, Z_t = 0.9902
Iteration 50: epsilon_t = 0.4156, alpha_t = 0.1704, Z_t = 0.9857
Iteration 51: epsilon_t = 0.3708, alpha_t = 0.2643, Z_t = 0.9661
Iteration 52: epsilon_t = 0.4052, alpha_t = 0.1919, Z_t = 0.9819
Iteration 53: epsilon_t = 0.3977, alpha_t = 0.2076, Z_t = 0.9788
Iteration 54: epsilon_t = 0.4189, alpha_t = 0.1636, Z_t = 0.9868
Iteration 55: epsilon_t = 0.3965, alpha_t = 0.2101, Z_t = 0.9783
Iteration 56: epsilon_t = 0.4093, alpha_t = 0.1835, Z_t = 0.9834
Iteration 57: epsilon_t = 0.3954, alpha_t = 0.2124, Z_t = 0.9779
Iteration 58: epsilon_t = 0.4179, alpha_t = 0.1658, Z_t = 0.9864
Iteration 59: epsilon_t = 0.4219, alpha_t = 0.1574, Z_t = 0.9877
Iteration 60: epsilon_t = 0.4378, alpha_t = 0.1251, Z_t = 0.9922

```

Iteration 61: epsilon_t = 0.4090, alpha_t = 0.1840, Z_t = 0.9833
Iteration 62: epsilon_t = 0.4055, alpha_t = 0.1913, Z_t = 0.9820
Iteration 63: epsilon_t = 0.4002, alpha_t = 0.2023, Z_t = 0.9799
Iteration 64: epsilon_t = 0.4077, alpha_t = 0.1868, Z_t = 0.9828
Iteration 65: epsilon_t = 0.4107, alpha_t = 0.1804, Z_t = 0.9839
Iteration 66: epsilon_t = 0.4236, alpha_t = 0.1539, Z_t = 0.9883
Iteration 67: epsilon_t = 0.4444, alpha_t = 0.1117, Z_t = 0.9938
Iteration 68: epsilon_t = 0.4436, alpha_t = 0.1133, Z_t = 0.9936
Iteration 69: epsilon_t = 0.4437, alpha_t = 0.1131, Z_t = 0.9936
Iteration 70: epsilon_t = 0.4518, alpha_t = 0.0968, Z_t = 0.9953
Iteration 71: epsilon_t = 0.4254, alpha_t = 0.1504, Z_t = 0.9888
Iteration 72: epsilon_t = 0.4519, alpha_t = 0.0966, Z_t = 0.9954
Iteration 73: epsilon_t = 0.4620, alpha_t = 0.0761, Z_t = 0.9971
Iteration 74: epsilon_t = 0.4451, alpha_t = 0.1101, Z_t = 0.9940
Iteration 75: epsilon_t = 0.4165, alpha_t = 0.1686, Z_t = 0.9860
Iteration 76: epsilon_t = 0.4387, alpha_t = 0.1232, Z_t = 0.9925
Iteration 77: epsilon_t = 0.4424, alpha_t = 0.1158, Z_t = 0.9933
Iteration 78: epsilon_t = 0.4410, alpha_t = 0.1185, Z_t = 0.9930
Iteration 79: epsilon_t = 0.4593, alpha_t = 0.0816, Z_t = 0.9967
Iteration 80: epsilon_t = 0.4287, alpha_t = 0.1435, Z_t = 0.9898
Iteration 81: epsilon_t = 0.4384, alpha_t = 0.1238, Z_t = 0.9924
Iteration 82: epsilon_t = 0.4551, alpha_t = 0.0899, Z_t = 0.9960
Iteration 83: epsilon_t = 0.4400, alpha_t = 0.1206, Z_t = 0.9928
Iteration 84: epsilon_t = 0.4337, alpha_t = 0.1334, Z_t = 0.9912
Iteration 85: epsilon_t = 0.4024, alpha_t = 0.1978, Z_t = 0.9808
Iteration 86: epsilon_t = 0.4304, alpha_t = 0.1400, Z_t = 0.9903
Iteration 87: epsilon_t = 0.4249, alpha_t = 0.1514, Z_t = 0.9886
Iteration 88: epsilon_t = 0.4047, alpha_t = 0.1930, Z_t = 0.9817
Iteration 89: epsilon_t = 0.4139, alpha_t = 0.1739, Z_t = 0.9851
Iteration 90: epsilon_t = 0.4257, alpha_t = 0.1497, Z_t = 0.9889
Iteration 91: epsilon_t = 0.4369, alpha_t = 0.1270, Z_t = 0.9920
Iteration 92: epsilon_t = 0.4195, alpha_t = 0.1624, Z_t = 0.9870
Iteration 93: epsilon_t = 0.4092, alpha_t = 0.1836, Z_t = 0.9834
Iteration 94: epsilon_t = 0.3909, alpha_t = 0.2217, Z_t = 0.9759
Iteration 95: epsilon_t = 0.4227, alpha_t = 0.1558, Z_t = 0.9880
Iteration 96: epsilon_t = 0.4257, alpha_t = 0.1497, Z_t = 0.9889
Iteration 97: epsilon_t = 0.4297, alpha_t = 0.1416, Z_t = 0.9901
Iteration 98: epsilon_t = 0.4221, alpha_t = 0.1571, Z_t = 0.9878
Iteration 99: epsilon_t = 0.4179, alpha_t = 0.1657, Z_t = 0.9864
Iteration 100: epsilon_t = 0.4343, alpha_t = 0.1321, Z_t = 0.9913
Accuracy sur les données train: 100.0
Accuracy sur les données test: 76.33333333333333

```

```

In [20]: Z = np.cumprod(Z_values)

plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.plot(epsilons)
plt.title("Erreur  $\epsilon_t$  par itération")
plt.xlabel("Itération t")
plt.ylabel(" $\epsilon_t$ ")

plt.subplot(2, 2, 2)
plt.plot(alphas)
plt.title("Poids  $\alpha_t$  par itération")
plt.xlabel("Itération t")
plt.ylabel(" $\alpha_t$ ")

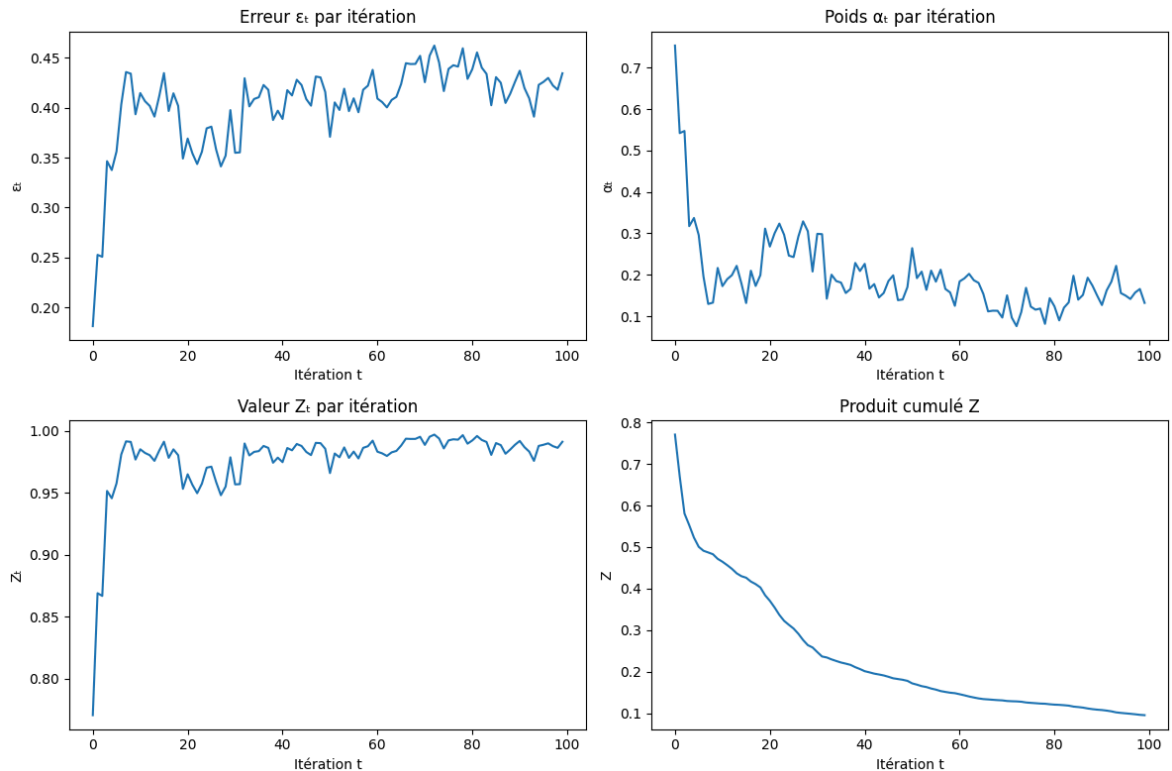
plt.subplot(2, 2, 3)

```

```
plt.plot(Z_values)
plt.title("Valeur  $Z_t$  par itération")
plt.xlabel("Itération  $t$ ")
plt.ylabel(" $Z_t$ ")

plt.subplot(2, 2, 4)
plt.plot(Z)
plt.title("Produit cumulé  $Z$ ")
plt.xlabel("Itération  $t$ ")
plt.ylabel(" $Z$ ")

plt.tight_layout()
plt.show()
```



Avec des arbres de décision de profondeur 1, on observe des résultats similaires entre les données d'entraînement et de test (78% et 79% d'accuracy). Cela montre que le modèle ne surapprend pas et qu'il reste assez stable au fil des itérations. À chaque itération, un nouvel arbre est ajouté au modèle, mais on remarque que le nombre d'itérations n'a pas un impact fort sur la précision tant que les classifieurs faibles restent simples. Cependant, lorsqu'on augmente la profondeur des arbres à 5, on observe du surapprentissage : l'accuracy atteint 99.85% sur le train, mais chute à 74.33% sur le test. Cela indique que les arbres profonds s'ajustent trop fortement aux données d'entraînement, ce qui empêche la généralisation.

En ce qui concerne l'erreur ϵ_t , on observe qu'elle tend à rester proche de 0.5 au fil des itérations. Cela signifie que les nouveaux classifieurs n'apportent plus d'amélioration significative, et que leurs performances sont proches du hasard (erreur de 50%). Pour les poids α_t , on voit une décroissance au cours des itérations : les premiers classifieurs ont des poids plus élevés car ils sont efficaces sur les données encore équilibrées uniformément. Plus on avance dans les itérations, plus les erreurs deviennent spécifiques, et les nouveaux arbres ont donc un poids plus faible car ils corrigent des cas

plus isolés.

Concernant le produit cumulé des normalisations Z , on observe aussi une diminution au fil des itérations. Cela signifie que le modèle global s'améliore : plus le modèle est bon, plus la somme pondérée des erreurs exponentielles diminue, ce qui entraîne une baisse de Z .

Conclusion :

Le bagging vise à réduire la variance du modèle en moyennant les prédictions de plusieurs arbres indépendants, tandis que le boosting cherche à réduire le biais en se concentrant sur les erreurs des itérations précédentes. Ainsi, le bagging est généralement plus robuste au surapprentissage, même avec des arbres profonds, grâce à l'effet de moyenne. Le boosting, lui, est plus sensible au surapprentissage si les arbres sont trop complexes ou trop nombreux, car il s'ajuste très finement aux données d'entraînement.